

SHIFT

v1.0

Generated by Doxygen 1.16.1

1 SHIFT (Secure Hardware Interface for File Transfer)	1
1.1 Project Structure	1
1.2 Documentation & API Reference	1
1.2.1 Generating the Doxygen Website	2
1.2.2 Exporting documentation to PDF	2
2 Directory Documentation	3
2.1 firmware Directory Reference	3
2.1.1 Detailed Description	3
2.1.2 SHIFT Firmware	3
2.1.3 Architecture Overview	4
2.1.4 Source Layout	4
2.1.5 Building & the Doxygen Setup	4
2.2 firmware/include Directory Reference	4
2.3 firmware/src Directory Reference	5
2.4 firmware/utls Directory Reference	6
2.4.1 Detailed Description	6
2.4.2 Firmware Utilities	6
2.4.3 <code>derive_secrets.py</code>	6
2.4.3.1 How It Works	7
3 Namespace Documentation	9
3.1 <code>derive_secrets</code> Namespace Reference	9
3.1.1 Function Documentation	10
3.1.1.1 <code>arg_parse()</code>	10
3.1.1.2 <code>generateCaseList()</code>	10
3.1.1.3 <code>generateSecrets()</code>	11
3.1.1.4 <code>interrogate_as_array()</code>	11
3.1.1.5 <code>key_as_array()</code>	11
3.1.2 Variable Documentation	12
3.1.2.1 <code>_</code>	12
3.1.2.2 <code>args</code>	12
3.1.2.3 <code>CASELIST</code>	12
3.1.2.4 <code>DEFINITIONS</code>	12
3.1.2.5 <code>DISCLAIMER</code>	12
3.1.2.6 <code>GEN_CASE_SMT</code>	13
3.1.2.7 <code>GEN_CASE_STUB</code>	13
3.1.2.8 <code>GEN_INTERROGATEKEY</code>	13
3.1.2.9 <code>GEN_READKEY</code>	13
3.1.2.10 <code>GEN_RECVKEY</code>	14
3.1.2.11 <code>GEN_REPLYKEY</code>	14
3.1.2.12 <code>GEN_SENDKEY</code>	14
3.1.2.13 <code>GEN_STUB</code>	14

3.1.2.14 GEN_WRITEKEY	15
3.1.2.15 HEADER_CONTENT	15
3.1.2.16 permissions	15
3.1.2.17 RECV_GROUPS	15
3.1.2.18 secrets	15
3.1.2.19 VALID_PERMS	15
4 Data Structure Documentation	17
4.1 EccPoint Struct Reference	17
4.1.1 Detailed Description	17
4.1.2 Field Documentation	17
4.1.2.1 x	17
4.1.2.2 y	17
4.2 FAT_TableTypeFlash Union Reference	18
4.2.1 Detailed Description	18
4.2.2 Field Documentation	18
4.2.2.1 asBytes	18
4.2.2.2 entry	18
4.3 FS_FAT Struct Reference	19
4.3.1 Detailed Description	19
4.3.2 Field Documentation	19
4.3.2.1 numEntries	19
4.3.2.2 slots	19
4.4 FS_File Struct Reference	20
4.4.1 Detailed Description	20
4.4.2 Field Documentation	20
4.4.2.1 content	20
4.4.2.2 entry	20
4.4.2.3 metadata	21
4.5 FS_FiledataFlash Struct Reference	21
4.5.1 Detailed Description	21
4.5.2 Field Documentation	21
4.5.2.1 data	21
4.6 FS_FileEntry Struct Reference	21
4.6.1 Detailed Description	22
4.6.2 Field Documentation	22
4.6.2.1 _pad	22
4.6.2.2 fileaddr	22
4.6.2.3 filesize	22
4.6.2.4 uuid	22
4.7 FS_Metadata Struct Reference	22
4.7.1 Detailed Description	23

4.7.2 Field Documentation	23
4.7.2.1 fileid	23
4.7.2.2 filename	23
4.7.2.3 groupid	23
4.7.2.4 key	23
4.7.2.5 nonce	24
4.7.2.6 tag	24
4.8 FS_MetadataEntryFlash Union Reference	24
4.8.1 Detailed Description	25
4.8.2 Field Documentation	25
4.8.2.1 _padding	25
4.8.2.2 asBytes	25
4.8.2.3 data	25
4.8.2.4 id	25
4.8.2.5 magic	25
4.8.2.6 [struct]	26
4.9 FS_Preamble Struct Reference	26
4.9.1 Detailed Description	26
4.9.2 Field Documentation	26
4.9.2.1 key	26
4.9.2.2 mesgid	26
4.9.2.3 nonce	27
4.9.2.4 tag	27
4.10 FS_Slot Struct Reference	27
4.10.1 Detailed Description	27
4.10.2 Field Documentation	27
4.10.2.1 filename	27
4.10.2.2 groupid	27
4.10.2.3 slot	28
4.11 FS_Stage Struct Reference	28
4.11.1 Detailed Description	28
4.11.2 Field Documentation	29
4.11.2.1 [union]	29
4.11.2.2 fat	29
4.11.2.3 file	29
4.11.2.4 preamble	29
4.12 FS_SystemStatusFlash Struct Reference	29
4.12.1 Detailed Description	29
4.12.2 Field Documentation	30
4.12.2.1 timeout	30
4.13 derive_secrets.Permission Class Reference	30
4.13.1 Detailed Description	30

4.13.2 Member Function Documentation	30
4.13.2.1 deserialize()	30
4.13.2.2 serialize()	31
4.13.3 Field Documentation	31
4.13.3.1 group_id	31
4.13.3.2 read	31
4.13.3.3 receive	31
4.13.3.4 write	31
4.14 derive_secrets.PermissionList Class Reference	32
4.14.1 Detailed Description	32
4.14.2 Constructor & Destructor Documentation	33
4.14.2.1 __init__()	33
4.14.3 Member Function Documentation	33
4.14.3.1 deserialize()	33
4.14.3.2 serialize()	34
4.15 UART_Channel Struct Reference	34
4.15.1 Detailed Description	34
4.15.2 Field Documentation	35
4.15.2.1 rx	35
4.15.2.2 tx	35
4.16 UART_RingBuffer Struct Reference	35
4.16.1 Detailed Description	35
4.16.2 Field Documentation	36
4.16.2.1 count	36
4.16.2.2 data	36
4.16.2.3 dirty	36
4.16.2.4 head	36
4.16.2.5 progress	36
4.16.2.6 tail	37
4.17 uECC_HashContext Struct Reference	37
4.17.1 Detailed Description	37
4.17.2 Field Documentation	37
4.17.2.1 block_size	37
4.17.2.2 finish_hash	38
4.17.2.3 init_hash	38
4.17.2.4 result_size	38
4.17.2.5 tmp	38
4.17.2.6 update_hash	38
5 File Documentation	39
5.1 firmware/include/dl_config.h File Reference	39
5.1.1 Detailed Description	41

5.1.2 Macro Definition Documentation	41
5.1.2.1 CONFIG_MSPM0L2228	41
5.1.2.2 CONFIG_MSPM0L222X	41
5.1.2.3 CPUCLK_FREQ	41
5.1.2.4 GPIO_UART_0_IOMUX_RX	41
5.1.2.5 GPIO_UART_0_IOMUX_RX_FUNC	42
5.1.2.6 GPIO_UART_0_IOMUX_TX	42
5.1.2.7 GPIO_UART_0_IOMUX_TX_FUNC	42
5.1.2.8 GPIO_UART_0_RX_PIN	42
5.1.2.9 GPIO_UART_0_RX_PORT	42
5.1.2.10 GPIO_UART_0_TX_PIN	42
5.1.2.11 GPIO_UART_0_TX_PORT	42
5.1.2.12 GPIO_UART_1_IOMUX_RX	43
5.1.2.13 GPIO_UART_1_IOMUX_RX_FUNC	43
5.1.2.14 GPIO_UART_1_IOMUX_TX	43
5.1.2.15 GPIO_UART_1_IOMUX_TX_FUNC	43
5.1.2.16 GPIO_UART_1_RX_PIN	43
5.1.2.17 GPIO_UART_1_RX_PORT	43
5.1.2.18 GPIO_UART_1_TX_PIN	43
5.1.2.19 GPIO_UART_1_TX_PORT	44
5.1.2.20 LEDS_PORT	44
5.1.2.21 LEDS_STATUS_LED_IOMUX	44
5.1.2.22 LEDS_STATUS_LED_PIN	44
5.1.2.23 POWER_STARTUP_DELAY	44
5.1.2.24 SYSCONFIG_WEAK	44
5.1.2.25 UART_0_BAUD_RATE	45
5.1.2.26 UART_0_FBRD_32_MHZ_115200_BAUD	45
5.1.2.27 UART_0_IBRD_32_MHZ_115200_BAUD	45
5.1.2.28 UART_0_INST	45
5.1.2.29 UART_0_INST_FREQUENCY	45
5.1.2.30 UART_0_INST_INT_IRQN	45
5.1.2.31 UART_0_INST_IRQHandler	45
5.1.2.32 UART_1_BAUD_RATE	46
5.1.2.33 UART_1_FBRD_32_MHZ_115200_BAUD	46
5.1.2.34 UART_1_IBRD_32_MHZ_115200_BAUD	46
5.1.2.35 UART_1_INST	46
5.1.2.36 UART_1_INST_FREQUENCY	46
5.1.2.37 UART_1_INST_INT_IRQN	46
5.1.2.38 UART_1_INST_IRQHandler	46
5.1.3 Function Documentation	47
5.1.3.1 SYSCFG_DL_GPIO_init()	47
5.1.3.2 SYSCFG_DL_init()	47

5.1.3.3 SYSCFG_DL_initPower()	48
5.1.3.4 SYSCFG_DL_SYSCTL_init()	49
5.1.3.5 SYSCFG_DL_TRNG_init()	49
5.1.3.6 SYSCFG_DL_UART_0_init()	50
5.1.3.7 SYSCFG_DL_UART_1_init()	50
5.2 dl_config.h	51
5.3 firmware/include/filesystem.h File Reference	52
5.3.1 Detailed Description	53
5.3.2 Macro Definition Documentation	53
5.3.2.1 MAX_FILE_SIZE	53
5.3.2.2 NUM_SLOTS	54
5.3.2.3 RAMFUNC	54
5.3.3 Typedef Documentation	54
5.3.3.1 FS_FileData	54
5.3.4 Function Documentation	54
5.3.4.1 LoadFile()	54
5.3.4.2 StoreFile()	55
5.3.5 Variable Documentation	55
5.3.5.1 FAT_Table	55
5.3.5.2 Filedata_Table	56
5.3.5.3 Metadata_Table	56
5.3.5.4 stage	56
5.3.5.5 SystemStatus	56
5.4 filesystem.h	56
5.5 firmware/include/flash.h File Reference	58
5.5.1 Detailed Description	59
5.5.2 Macro Definition Documentation	59
5.5.2.1 FLASH_PAGE_SIZE	59
5.5.2.2 RAMFUNC	59
5.5.3 Function Documentation	60
5.5.3.1 FLASH_Erase()	60
5.5.3.2 FLASH_Write()	60
5.6 flash.h	61
5.7 firmware/include/handler.h File Reference	62
5.7.1 Detailed Description	63
5.7.2 Function Documentation	63
5.7.2.1 InterrogateHandler()	63
5.7.2.2 ListHandler()	64
5.7.2.3 ReadHandler()	64
5.7.2.4 ReceiveHandler()	65
5.7.2.5 ReplyHandler()	66
5.7.2.6 SendHandler()	67

5.7.2.7 WriteHandler()	68
5.8 handler.h	68
5.9 firmware/include/parser.h File Reference	69
5.9.1 Detailed Description	70
5.9.2 Function Documentation	70
5.9.2.1 InterrogateParser()	70
5.9.2.2 ListenParser()	71
5.9.2.3 ListParser()	71
5.9.2.4 ReadParser()	72
5.9.2.5 ReceiveParser()	73
5.9.2.6 WriteParser()	74
5.10 parser.h	75
5.11 firmware/include/randombytes.h File Reference	75
5.11.1 Detailed Description	76
5.11.2 Function Documentation	77
5.11.2.1 getrandom32()	77
5.11.2.2 randombytes()	77
5.12 randombytes.h	78
5.13 firmware/include/status.h File Reference	79
5.13.1 Detailed Description	79
5.13.2 Enumeration Type Documentation	79
5.13.2.1 StatusCode	79
5.14 status.h	80
5.15 firmware/include/stubs.h File Reference	81
5.15.1 Detailed Description	81
5.15.2 Function Documentation	82
5.15.2.1 checkpin()	82
5.15.2.2 memclear()	83
5.15.2.3 securecmp()	84
5.16 stubs.h	85
5.17 firmware/include/uart.h File Reference	85
5.17.1 Detailed Description	87
5.17.2 Macro Definition Documentation	87
5.17.2.1 MAX_DRAIN_SIZE	87
5.17.2.2 UART_BUFFER_SIZE	87
5.17.2.3 UART_HOST	87
5.17.2.4 UART_PEER	88
5.17.3 Function Documentation	88
5.17.3.1 UART0_IRQHandler()	88
5.17.3.2 UART1_IRQHandler()	88
5.17.3.3 UART_init()	89
5.17.3.4 UART_RecvBytes()	90

5.17.3.5 UART_RecvUntil()	91
5.17.3.6 UART_SendBytes()	92
5.17.4 Variable Documentation	93
5.17.4.1 host	93
5.17.4.2 peer	93
5.18 uart.h	93
5.19 firmware/include/uECC.h File Reference	94
5.19.1 Detailed Description	97
5.19.2 Macro Definition Documentation	97
5.19.2.1 uECC_arch_other	97
5.19.2.2 uECC_arm	97
5.19.2.3 uECC_arm_thumb	97
5.19.2.4 uECC_arm_thumb2	98
5.19.2.5 uECC_ASM	98
5.19.2.6 uECC_asm_fast	98
5.19.2.7 uECC_asm_none	98
5.19.2.8 uECC_asm_small	98
5.19.2.9 uECC_avr	98
5.19.2.10 uECC_BYTES	99
5.19.2.11 uECC_CONCAT	99
5.19.2.12 uECC_CONCAT1	99
5.19.2.13 uECC_CURVE	99
5.19.2.14 uECC_secp160r1	99
5.19.2.15 uECC_secp192r1	100
5.19.2.16 uECC_secp224r1	100
5.19.2.17 uECC_secp256k1	100
5.19.2.18 uECC_secp256r1	100
5.19.2.19 uECC_size_1	100
5.19.2.20 uECC_size_2	100
5.19.2.21 uECC_size_3	101
5.19.2.22 uECC_size_4	101
5.19.2.23 uECC_size_5	101
5.19.2.24 uECC_SQUARE_FUNC	101
5.19.2.25 uECC_x86	101
5.19.2.26 uECC_x86_64	101
5.19.3 Typedef Documentation	102
5.19.3.1 uECC_HashContext	102
5.19.3.2 uECC_RNG_Function	102
5.19.4 Function Documentation	102
5.19.4.1 uECC_bytes()	102
5.19.4.2 uECC_compress()	102
5.19.4.3 uECC_compute_public_key()	103

5.19.4.4 uECC_curve()	104
5.19.4.5 uECC_decompress()	104
5.19.4.6 uECC_make_key()	105
5.19.4.7 uECC_set_rng()	105
5.19.4.8 uECC_shared_secret()	106
5.19.4.9 uECC_sign()	107
5.19.4.10 uECC_sign_deterministic()	108
5.19.4.11 uECC_valid_public_key()	109
5.19.4.12 uECC_verify()	111
5.20 uECC.h	113
5.21 firmware/src/dl_config.c File Reference	114
5.21.1 Detailed Description	115
5.21.2 Function Documentation	116
5.21.2.1 SYSCFG_DL_GPIO_init()	116
5.21.2.2 SYSCFG_DL_init()	116
5.21.2.3 SYSCFG_DL_initPower()	117
5.21.2.4 SYSCFG_DL_SYSCTL_init()	118
5.21.2.5 SYSCFG_DL_TRNG_init()	118
5.21.2.6 SYSCFG_DL_UART_0_init()	119
5.21.2.7 SYSCFG_DL_UART_1_init()	119
5.21.2.8 TRNG_SecureWarmup()	120
5.21.3 Variable Documentation	120
5.21.3.1 gUART_0ClockConfig	120
5.21.3.2 gUART_0Config	121
5.21.3.3 gUART_1ClockConfig	121
5.21.3.4 gUART_1Config	121
5.22 dl_config.c	122
5.23 firmware/src/filesystem.c File Reference	124
5.23.1 Detailed Description	124
5.23.2 Function Documentation	125
5.23.2.1 __attribute__()	125
5.23.2.2 StoreFile()	125
5.23.3 Variable Documentation	126
5.23.3.1 stage	126
5.24 filesystem.c	127
5.25 firmware/src/flash.c File Reference	127
5.25.1 Detailed Description	128
5.25.2 Function Documentation	129
5.25.2.1 __attribute__()	129
5.25.2.2 FLASH_ProcessSector()	129
5.25.2.3 FLASH_ReadSector()	130
5.25.2.4 FLASH_Write()	131

5.26 flash.c	132
5.27 firmware/src/handler.c File Reference	133
5.27.1 Detailed Description	134
5.27.2 Function Documentation	134
5.27.2.1 InterrogateHandler()	134
5.27.2.2 isReceivable()	135
5.27.2.3 ListHandler()	135
5.27.2.4 ReadHandler()	136
5.27.2.5 ReceiveHandler()	137
5.27.2.6 ReplyHandler()	137
5.27.2.7 SendHandler()	138
5.27.2.8 WriteHandler()	139
5.28 handler.c	140
5.29 firmware/src/hsm.c File Reference	143
5.29.1 Detailed Description	144
5.29.2 Macro Definition Documentation	144
5.29.2.1 HSM_PIN_SIZE	144
5.29.2.2 INTERROGATE_METADATA_SIZE	145
5.29.2.3 LIST_METADATA_SIZE	145
5.29.2.4 LISTEN_METADATA_SIZE	145
5.29.2.5 OP_ERROR	145
5.29.2.6 OP_INTERROGATE	145
5.29.2.7 OP_LIST	145
5.29.2.8 OP_LISTEN	146
5.29.2.9 OP_READ	146
5.29.2.10 OP_RECEIVE	146
5.29.2.11 OP_WRITE	146
5.29.2.12 READ_METADATA_SIZE	146
5.29.2.13 RECEIVE_METADATA_SIZE	146
5.29.2.14 WRITE_METADATA_SIZE	147
5.29.3 Function Documentation	147
5.29.3.1 BlinkLED()	147
5.29.3.2 ecc_rng()	147
5.29.3.3 HandleRequest()	148
5.29.3.4 init()	149
5.29.3.5 main()	150
5.29.3.6 ParseOpcode()	151
5.29.3.7 SendError()	152
5.29.3.8 SendHeader()	153
5.29.3.9 SendResponse()	153
5.29.3.10 ValidateBodySize()	154
5.29.4 Variable Documentation	155

5.29.4.1 host_magic_byte	155
5.30 hsm.c	155
5.31 firmware/src/parser.c File Reference	158
5.31.1 Detailed Description	159
5.31.2 Macro Definition Documentation	160
5.31.2.1 OP_ERROR	160
5.31.2.2 OP_INTERROGATE	160
5.31.2.3 OP_RECEIVE	160
5.31.3 Function Documentation	160
5.31.3.1 InterrogateParser()	160
5.31.3.2 ListenParser()	161
5.31.3.3 ListParser()	162
5.31.3.4 ReadParser()	162
5.31.3.5 ReceiveParser()	163
5.31.3.6 ReplyParser()	164
5.31.3.7 ResolveError()	165
5.31.3.8 SendParser()	166
5.31.3.9 WriteParser()	166
5.31.4 Variable Documentation	167
5.31.4.1 peer_magic_byte	167
5.32 parser.c	168
5.33 firmware/src/randombytes.c File Reference	172
5.33.1 Detailed Description	173
5.33.2 Function Documentation	173
5.33.2.1 getRandom32()	173
5.33.2.2 randombytes()	174
5.34 randombytes.c	174
5.35 firmware/src/startup_msp0l222x_ticlang.c File Reference	175
5.35.1 Detailed Description	176
5.35.2 Function Documentation	176
5.35.2.1 __PROGRAM_START()	176
5.35.2.2 Default_Handler()	176
5.35.2.3 Reset_Handler()	177
5.35.3 Variable Documentation	177
5.35.3.1 __STACK_END	177
5.36 startup_msp0l222x_ticlang.c	177
5.37 firmware/src/stubs.c File Reference	179
5.37.1 Detailed Description	180
5.37.2 Macro Definition Documentation	180
5.37.2.1 RAMFUNC	180
5.37.3 Function Documentation	180
5.37.3.1 checkpin()	180

5.37.3.2 memclear()	181
5.37.3.3 securecmp()	182
5.38 stubs.c	183
5.39 firmware/src/uart.c File Reference	184
5.39.1 Detailed Description	185
5.39.2 Function Documentation	186
5.39.2.1 DrainPushBytes()	186
5.39.2.2 IRQ_Handler()	186
5.39.2.3 SendHostACK()	187
5.39.2.4 SendPeerACK()	187
5.39.2.5 UART0_IRQHandler()	188
5.39.2.6 UART1_IRQHandler()	188
5.39.2.7 UART_ClearBuffer()	189
5.39.2.8 UART_ClearChannel()	189
5.39.2.9 UART_init()	190
5.39.2.10 UART_PopByte()	191
5.39.2.11 UART_PushByte()	191
5.39.2.12 UART_RecvBytes()	192
5.39.2.13 UART_RecvUntil()	193
5.39.2.14 UART_SendBytes()	194
5.39.2.15 UART_TransmitAll()	195
5.39.2.16 WaitForHostACK()	196
5.39.2.17 WaitForPeerACK()	197
5.39.3 Variable Documentation	198
5.39.3.1 host	198
5.39.3.2 peer	198
5.40 uart.c	198
5.41 firmware/src/uECC.c File Reference	201
5.41.1 Detailed Description	204
5.41.2 Macro Definition Documentation	204
5.41.2.1 EVEN	204
5.41.2.2 MAX_TRIES	205
5.41.2.3 muladd_exists [1/2]	205
5.41.2.4 muladd_exists [2/2]	205
5.41.2.5 RESTRICT	205
5.41.2.6 SUPPORTS_INT128	205
5.41.2.7 uECC_N_WORDS	205
5.41.2.8 uECC_PLATFORM	206
5.41.2.9 uECC_WORD_SIZE	206
5.41.2.10 uECC_WORDS	206
5.41.2.11 vli_modSquare_fast	206
5.41.2.12 vli_modSub_fast	206

5.41.2.13 vli_square	207
5.41.3 Typedef Documentation	207
5.41.3.1 EccPoint	207
5.41.4 Function Documentation	207
5.41.4.1 __attribute__()	207
5.41.4.2 apply_z()	207
5.41.4.3 curve_x_side()	208
5.41.4.4 default_RNG()	209
5.41.4.5 EccPoint_compute_public_key()	209
5.41.4.6 EccPoint_double_jacobian()	210
5.41.4.7 EccPoint_isZero()	211
5.41.4.8 EccPoint_mult()	212
5.41.4.9 HMAC_finish()	213
5.41.4.10 HMAC_init()	213
5.41.4.11 HMAC_update()	214
5.41.4.12 mod_sqrt()	214
5.41.4.13 mod_sqrt_secp224r1_rm()	215
5.41.4.14 mod_sqrt_secp224r1_rp()	216
5.41.4.15 mod_sqrt_secp224r1_rs()	217
5.41.4.16 mod_sqrt_secp224r1_rss()	218
5.41.4.17 muladd()	218
5.41.4.18 omega_mult()	219
5.41.4.19 smax()	219
5.41.4.20 uECC_bytes()	220
5.41.4.21 uECC_compress()	220
5.41.4.22 uECC_compute_public_key()	221
5.41.4.23 uECC_curve()	222
5.41.4.24 uECC_decompress()	222
5.41.4.25 uECC_make_key()	223
5.41.4.26 uECC_mod_mult()	223
5.41.4.27 uECC_shared_secret()	224
5.41.4.28 uECC_sign()	225
5.41.4.29 uECC_sign_deterministic()	226
5.41.4.30 uECC_sign_with_k()	227
5.41.4.31 uECC_valid_public_key()	229
5.41.4.32 uECC_verify()	230
5.41.4.33 update_V()	232
5.41.4.34 vli2_rshift1_n()	233
5.41.4.35 vli2_sub_n()	233
5.41.4.36 vli_add()	234
5.41.4.37 vli_add_n()	234
5.41.4.38 vli_blindScalar()	235

5.41.4.39 vli_bytesToNative()	235
5.41.4.40 vli_clear()	236
5.41.4.41 vli_clear_n()	237
5.41.4.42 vli_cmp()	237
5.41.4.43 vli_cmp_n()	238
5.41.4.44 vli_equal()	239
5.41.4.45 vli_isZero()	239
5.41.4.46 vli_isZero_n()	240
5.41.4.47 vli_mmod_fast()	240
5.41.4.48 vli_modAdd()	241
5.41.4.49 vli_modAdd_n()	242
5.41.4.50 vli_modInv()	243
5.41.4.51 vli_modInv_n()	244
5.41.4.52 vli_modMult_fast()	245
5.41.4.53 vli_modMult_n()	246
5.41.4.54 vli_modSub()	247
5.41.4.55 vli_mult()	248
5.41.4.56 vli_mult_n()	249
5.41.4.57 vli_nativeToBytes()	250
5.41.4.58 vli_numBits()	250
5.41.4.59 vli_numDigits()	251
5.41.4.60 vli_rshift1()	251
5.41.4.61 vli_rshift1_n()	252
5.41.4.62 vli_set()	252
5.41.4.63 vli_set_n()	253
5.41.4.64 vli_sub()	254
5.41.4.65 vli_sub_n()	254
5.41.4.66 vli_testBit()	255
5.41.4.67 XYcZ_add()	255
5.41.4.68 XYcZ_addC()	256
5.41.4.69 XYcZ_initial_double()	257
5.41.5 Variable Documentation	258
5.41.5.1 curve_b	258
5.41.5.2 curve_G	258
5.41.5.3 curve_n	259
5.41.5.4 curve_p	259
5.41.5.5 g_rng_function	259
5.42 uECC.c	259
5.43 firmware/utils/derive_secrets.py File Reference	298
5.43.1 Detailed Description	299
5.44 derive_secrets.py	299
5.45 firmware/README.md File Reference	308

5.46 firmware/utls/README.md File Reference	308
5.47 README.md File Reference	308
Index	309

Chapter 1

SHIFT (Secure Hardware Interface for File Transfer)

Welcome to the **SHIFT** repository! This project implements an advanced, cryptographically verifiable append-only filesystem and Host Security Module (HSM) designed specifically for the **ECTF 2026** competition.

1.1 Project Structure

The project is divided into several key directories:

- **firmware/**: Contains the core C codebase for the MSPM0L2228 microcontroller. This is where the primary HSM logic, cryptographic handlers, and flash driver reside.
 - **firmware/README.md**: Start here for an architectural overview of how the firmware operates, handles UART communications, and secures data in flash.
- **firmware/utls/**: Contains the Python tooling required to derive and provision device-specific cryptographic identities ([derive_secrets.py](#)).
 - **firmware/utls/README.md**: Read this to understand how the asymmetric shared-secret derivation handles user permissions securely at build-time.
- **firmware/external/**: Submodules containing third-party cryptographic primitives (`ascon`, `micro-ecc`).

1.2 Documentation & API Reference

We have heavily documented the internal C source code and Python utilities using a concise, Linux-kernel documentation style to explain the *how* and *why* behind the critical hardware security mechanisms.

1.2.1 Generating the Doxygen Website

To view the complete API reference, function call graphs, and structural definitions, you can generate the local HTML documentation site:

1. Ensure you have **Doxygen** installed on your machine.

1. From the root of the repository, run:

```
doxygen Doxyfile
```

1. Open `docs/html/index.html` in your preferred web browser.

1.2.2 Exporting documentation to PDF

The `Doxyfile` is also configured to support high-quality PDF generation. If you have a LaTeX distribution (like `texlive`) installed:

1. Run `doxygen Doxyfile` as shown above.

1. Navigate to the newly created `latex` directory:

```
cd docs/latex
```

1. Build the PDF:

```
make
```

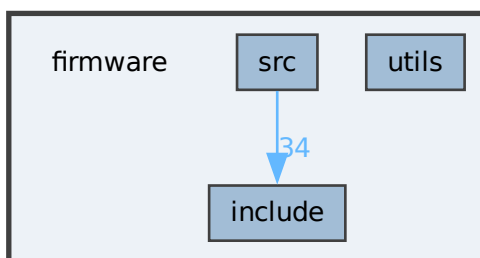
1. Open the generated `refman.pdf` document.

Chapter 2

Directory Documentation

2.1 firmware Directory Reference

Directory dependency graph for firmware:



Directories

- directory [include](#)
- directory [src](#)
- directory [utils](#)

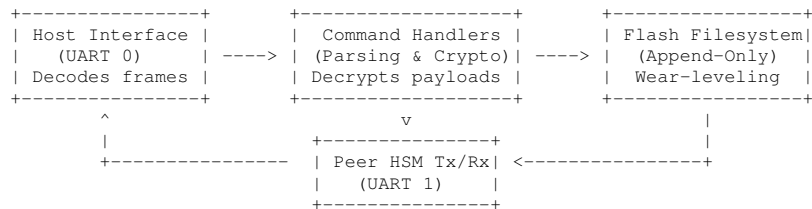
2.1.1 Detailed Description

2.1.2 SHIFT Firmware

An advanced, cryptographically verifiable append-only filesystem and Host Security Module (HSM) designed for the ECTF 2026 competition.

2.1.3 Architecture Overview

The firmware implements a strict boundary between Host requests and internal secure operations. All data written to the HSM is protected via state-of-the-art AEAD cryptography (ASCON-128a), ensuring confidentiality and tamper-evident integrity.



2.1.4 Source Layout

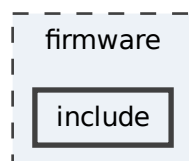
- `include/` & `src/`: Core implementation.
- `utils/`: Contains [derive_secrets.py](#), the core script for provisioning identities and permissions during the build process.

2.1.5 Building & the Doxygen Setup

```
cd SHIFT/
doxygen Doxyfile
# Open docs/html/index.html in your browser
```

2.2 firmware/include Directory Reference

Directory dependency graph for include:



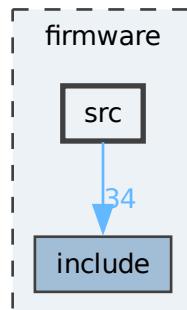
Files

- file [dl_config.h](#)
Device Library Configuration for MSPM0.
- file [filesystem.h](#)
Secure Flash Filesystem Layout and Data Structures.

- file [flash.h](#)
Low-level Flash Memory Controller Driver for MSPM0.
- file [handler.h](#)
Command Handlers for Host and Peer interactions.
- file [parser.h](#)
Command Parsing and Validation layer.
- file [randombytes.h](#)
Hardware-backed Random Number Generation.
- file [status.h](#)
Global Status Codes and Operations.
- file [stubs.h](#)
Secure Memory and Timing-Safe Operations.
- file [uart.h](#)
UART Ring-Buffer Hardware Abstraction.
- file [uECC.h](#)
Micro-ECC Library for ECDH and ECDSA.

2.3 firmware/src Directory Reference

Directory dependency graph for src:



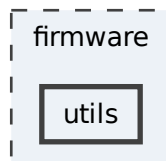
Files

- file [dl_config.c](#)
Device Library Configuration Source File.
- file [filesystem.c](#)
Flash Filesystem Implementation and Storage Tables.
- file [flash.c](#)
Flash Memory Controller Implementation.
- file [handler.c](#)
Implementation of Host and Peer Command Handlers.

- file [hsm.c](#)
Main entry point and core dispatch loop for the SHIFT HSM.
- file [parser.c](#)
Request stream parsing and initial validation layer.
- file [randombytes.c](#)
Implementation of TRNG-backed random number generation.
- file [startup_msp0l222x_ticlang.c](#)
Startup and Vector Table for TI Clang.
- file [stubs.c](#)
Implementation of Secure Memory and Side-Channel Protection routines.
- file [uart.c](#)
Non-blocking UART driver with Ring Buffer management.
- file [uECC.c](#)
Implementation of the Micro-ECC library.

2.4 firmware/utls Directory Reference

Directory dependency graph for utls:



Files

- file [derive_secrets.py](#)
Code generator for HSM cryptographic identities and access control.

2.4.1 Detailed Description

2.4.2 Firmware Utilities

This directory contains the operational scripts used during the SHIFT firmware build and provisioning processes.

2.4.3 `derive_secrets.py`

This is the core key-generation and attribution script for the HSM access control system. Rather than storing static plaintext keys where they might be vulnerable to flash memory extraction, this script embeds *asymmetric key material* (compiled into `secrets.c` and `secrets.h`).

2.4.3.1 How It Works

The script implements a unified, atomic code generator that reads a serialized Python dictionary of cryptographic materials (`global.secrets`) and outputs a tailored set of C key-derivation routines based on the permissions given to the current device.

Inputs:

1. **`global.secrets` file:** Generated by the top-level deployment scripts. Contains ECC keypairs and shared secrets for all known groups and devices.
1. **`hsm_pin`:** A 6-character user PIN required to authorize the device.
1. **`permissions`:** A formatted string dictating exactly what the firmware is allowed to do for each group (R = Read, W = Write, C = Receive).

Example Invocation:

```
python3 derive_secrets.py ../global.secrets 123456 "1234=R--:4321=RW-"
```

For full details on the generated cryptographic flows, use Doxygen to view the in-code documentation for the `Select*Key` and `Gen*Key` APIs.

Chapter 3

Namespace Documentation

3.1 `derive_secrets` Namespace Reference

Data Structures

- class [Permission](#)
- class [PermissionList](#)

Functions

- [key_as_array](#) (int groupid, dict[str, dict[str, dict[str, bytes]]] [secrets](#), str perm, str sel)
- [interrogate_as_array](#) (dict[str, dict[str, dict[str, bytes]]] [secrets](#))
- [generateCaseList](#) (str permission, [PermissionList](#) permissionlist)
- [generateSecrets](#) (str outfilename, [PermissionList](#) permissionlist, dict[str, dict[str, dict[str, bytes]]] [secrets](#))
- `argparse.Namespace` [arg_parse](#) ()

Variables

- tuple [VALID_PERMS](#) = ("Write", "Read", "Send", "Recv")
- str [DISCLAIMER](#)
- str [DEFINITIONS](#)
- Callable [HEADER_CONTENT](#)
- tuple [GEN_WRITEKEY](#)
- tuple [GEN_READKEY](#)
- tuple [GEN_SENDKEY](#)
- tuple [GEN_RECVKEY](#)
- tuple [GEN_INTERROGATEKEY](#)
- Callable [GEN_REPLYKEY](#)
- Callable [GEN_STUB](#)
- Callable [GEN_CASE_SMT](#)
- Callable [GEN_CASE_STUB](#)
- Callable [CASELIST](#)
- Callable [RECV_GROUPS](#)
- `argparse.Namespace` [args](#) = [arg_parse](#)()
- [secrets](#) = `pickle.load(args.secrets)`
- [permissions](#) = [PermissionList.deserialize](#)(args.permissions)
- [_](#)

3.1.1 Function Documentation

3.1.1.1 `arg_parse()`

```
argparse.Namespace derive_secrets.arg_parse ()
```

@brief Defines and parses command line arguments for secret derivation.

@return argparse.Namespace: The parsed arguments.

Definition at line 723 of file [derive_secrets.py](#).

3.1.1.2 `generateCaseList()`

```
derive_secrets.generateCaseList (  
    str permission,  
    PermissionList permissionlist)
```

@brief Generates a C switch-case block for selecting the correct key derivation routine based on group ID and the requested operation.

@param permission (str): The permission type (Read, Write, etc.).

@param permissionlist (PermissionList): The list of permissions.

@return str: The generated C code.

Definition at line 657 of file [derive_secrets.py](#).

References [CASELIST](#), and [GEN_CASE_SMT](#).

Referenced by [generateSecrets\(\)](#).

Here is the caller graph for this function:



3.1.1.3 generateSecrets()

```
derive_secrets.generateSecrets (
    str outfilename,
    PermissionList permissionlist,
    dict[str, dict[str, dict[str, bytes]]] secrets)
```

@brief Compiles all derived key routines and lookup tables into a single C source file.

@param outfilename (str): The output file path.
@param permissionlist (PermissionList): The list of permissions.
@param secrets (dict): The master secrets.

Definition at line 683 of file [derive_secrets.py](#).

References [GEN_INTERROGATEKEY](#), [GEN_READKEY](#), [GEN_RECVKEY](#), [GEN_REPLYKEY](#), [GEN_SENDKEY](#), [GEN_STUB](#), [GEN_WRITEKEY](#), [generateCaseList\(\)](#), and [RECV_GROUPS](#).

Here is the call graph for this function:



3.1.1.4 interrogate_as_array()

```
derive_secrets.interrogate_as_array (
    dict[str, dict[str, dict[str, bytes]]] secrets)
```

Definition at line 56 of file [derive_secrets.py](#).

3.1.1.5 key_as_array()

```
derive_secrets.key_as_array (
    int groupid,
    dict[str, dict[str, dict[str, bytes]]] secrets,
    str perm,
    str sel)
```

@brief Format a specific secret as a C-style array initializer list.

@param groupid (int): The group ID to retrieve.
@param secrets (dict): The secrets dictionary.
@param sel (str): The selector ('internal' or 'external').

@return str: The formatted C array.

Definition at line 41 of file [derive_secrets.py](#).

3.1.2 Variable Documentation

3.1.2.1 `_`

`derive_secrets._` [protected]

Initial value:

```
00001 = headerfile.write(
00002     DISCLAIMER
00003     + "\n\n"
00004     + HEADER_CONTENT(args.hsm_pin, len([g for g in permissions if g.receive]))
00005 )
```

Definition at line 754 of file [derive_secrets.py](#).

3.1.2.2 `args`

`argparse.Namespace` `derive_secrets.args` = [arg_parse\(\)](#)

Definition at line 745 of file [derive_secrets.py](#).

3.1.2.3 `CASELIST`

`Callable` `derive_secrets.CASELIST`

Initial value:

```
00001 = lambda permstr, cases: (
00002     f
00003 )
```

Definition at line 572 of file [derive_secrets.py](#).

Referenced by [generateCaseList\(\)](#).

3.1.2.4 `DEFINITIONS`

`str` `derive_secrets.DEFINITIONS`

Definition at line 73 of file [derive_secrets.py](#).

3.1.2.5 `DISCLAIMER`

`str` `derive_secrets.DISCLAIMER`

Initial value:

```
00001 = """\
00002 // -----
00003 //                                     DISCLAIMER
00004 // This file is generated by a utility script. Do not edit this file directly,
00005 // as it will be overwritten by the script. Instead, view the script to
00006 // understand how this file is generated and make changes to the script if
00007 // necessary.
00008 // -----
00009 """
```

Definition at line 63 of file [derive_secrets.py](#).

3.1.2.6 GEN_CASE_SMT

Callable `derive_secrets.GEN_CASE_SMT`

Initial value:

```
00001 = lambda groupid, permstr: (  
00002     f  
00003 )
```

Definition at line 558 of file [derive_secrets.py](#).

Referenced by [generateCaseList\(\)](#).

3.1.2.7 GEN_CASE_STUB

Callable `derive_secrets.GEN_CASE_STUB`

Initial value:

```
00001 = lambda groupid: (  
00002     f  
00003 )
```

Definition at line 565 of file [derive_secrets.py](#).

3.1.2.8 GEN_INTERROGATEKEY

tuple `derive_secrets.GEN_INTERROGATEKEY`

Initial value:

```
00001 = (  
00002     lambda secrets: (  
00003         f  
00004     )  
00005 )
```

Definition at line 478 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.9 GEN_READKEY

tuple `derive_secrets.GEN_READKEY`

Initial value:

```
00001 = (  
00002     lambda groupid, secrets: (  
00003         f  
00004     )  
00005 )
```

Definition at line 275 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.10 GEN_RECVKEY

tuple derive_secrets.GEN_RECVKEY

Initial value:

```
00001 = (  
00002     lambda groupid, secrets: (  
00003         f  
00004     )  
00005 )
```

Definition at line 416 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.11 GEN_REPLYKEY

Callable derive_secrets.GEN_REPLYKEY

Initial value:

```
00001 = lambda secrets: (  
00002     f  
00003 )
```

Definition at line 515 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.12 GEN_SENDKEY

tuple derive_secrets.GEN_SENDKEY

Initial value:

```
00001 = (  
00002     lambda groupid, secrets: (  
00003         f  
00004     )  
00005 )
```

Definition at line 336 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.13 GEN_STUB

Callable derive_secrets.GEN_STUB

Initial value:

```
00001 = lambda: (  
00002  
00003 )
```

Definition at line 549 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.14 GEN_WRITEKEY

tuple derive_secrets.GEN_WRITEKEY

Initial value:

```
00001 = (  
00002     lambda groupid, secrets: (  
00003         f  
00004     )  
00005 )
```

Definition at line 196 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.15 HEADER_CONTENT

Callable derive_secrets.HEADER_CONTENT

Initial value:

```
00001 = lambda pin, n_groups: (  
00002     f  
00003 )
```

Definition at line 157 of file [derive_secrets.py](#).

3.1.2.16 permissions

derive_secrets.permissions = [PermissionList.deserialize](#)(args.permissions)

Definition at line 748 of file [derive_secrets.py](#).

3.1.2.17 RECV_GROUPS

Callable derive_secrets.RECV_GROUPS

Initial value:

```
00001 = lambda groups: (  
00002     f  
00003 )
```

Definition at line 585 of file [derive_secrets.py](#).

Referenced by [generateSecrets\(\)](#).

3.1.2.18 secrets

derive_secrets.secrets = [pickle.load](#)(args.secrets)

Definition at line 747 of file [derive_secrets.py](#).

3.1.2.19 VALID_PERMS

tuple derive_secrets.VALID_PERMS = ("Write", "Read", "Send", "Recv")

Definition at line 60 of file [derive_secrets.py](#).

Chapter 4

Data Structure Documentation

4.1 EccPoint Struct Reference

Data Fields

- `uECC_word_t x` [[uECC_WORDS](#)]
- `uECC_word_t y` [[uECC_WORDS](#)]

4.1.1 Detailed Description

Definition at line [399](#) of file [uECC.c](#).

4.1.2 Field Documentation

4.1.2.1 x

```
uECC_word_t EccPoint::x[uECC\_WORDS]
```

Definition at line [401](#) of file [uECC.c](#).

Referenced by [EccPoint_isZero\(\)](#), [uECC_decompress\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign_with_k\(\)](#), and [uECC_verify\(\)](#).

4.1.2.2 y

```
uECC_word_t EccPoint::y[uECC\_WORDS]
```

Definition at line [402](#) of file [uECC.c](#).

Referenced by [EccPoint_isZero\(\)](#), [uECC_decompress\(\)](#), [uECC_shared_secret\(\)](#), and [uECC_verify\(\)](#).

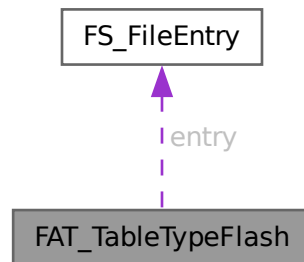
The documentation for this struct was generated from the following file:

- `firmware/src/uECC.c`

4.2 FAT_TableTypeFlash Union Reference

```
#include <filesystem.h>
```

Collaboration diagram for FAT_TableTypeFlash:



Data Fields

- `uint8_t asBytes` [DL_FLASHCTL_SECTOR_SIZE]
- `FS_FileEntry entry` [NUM_SLOTS]

4.2.1 Detailed Description

Definition at line 106 of file `filesystem.h`.

4.2.2 Field Documentation

4.2.2.1 asBytes

```
uint8_t FAT_TableTypeFlash::asBytes[DL_FLASHCTL_SECTOR_SIZE]
```

Definition at line 108 of file `filesystem.h`.

4.2.2.2 entry

```
FS_FileEntry FAT_TableTypeFlash::entry[NUM_SLOTS]
```

Definition at line 109 of file `filesystem.h`.

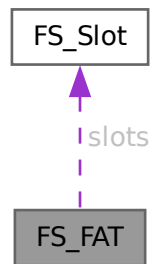
The documentation for this union was generated from the following file:

- `firmware/include/filesystem.h`

4.3 FS_FAT Struct Reference

```
#include <filesystem.h>
```

Collaboration diagram for FS_FAT:



Data Fields

- uint32_t [numEntries](#)
- [FS_Slot slots](#) [[NUM_SLOTS](#)]

4.3.1 Detailed Description

Definition at line [88](#) of file [filesystem.h](#).

4.3.2 Field Documentation

4.3.2.1 numEntries

```
uint32_t FS_FAT::numEntries
```

Definition at line [90](#) of file [filesystem.h](#).

4.3.2.2 slots

```
FS\_Slot FS_FAT::slots[NUM\_SLOTS]
```

Definition at line [91](#) of file [filesystem.h](#).

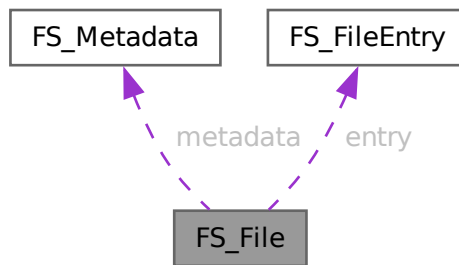
The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.4 FS_File Struct Reference

```
#include <filesystem.h>
```

Collaboration diagram for FS_File:



Data Fields

- [FS_Metadata metadata](#)
- [FS_FileEntry entry](#)
- [FS_FileData content](#)

4.4.1 Detailed Description

Definition at line 72 of file [filesystem.h](#).

4.4.2 Field Documentation

4.4.2.1 content

```
FS_FileData FS_File::content
```

Definition at line 76 of file [filesystem.h](#).

4.4.2.2 entry

```
FS_FileEntry FS_File::entry
```

Definition at line 75 of file [filesystem.h](#).

4.4.2.3 metadata

`FS_Metadata` `FS_File::metadata`

Definition at line 74 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.5 FS_FiledataFlash Struct Reference

```
#include <filesystem.h>
```

Data Fields

- `uint8_t` [data](#) [[NUM_SLOTS](#)][8][[DL_FLASHCTL_SECTOR_SIZE](#)]

4.5.1 Detailed Description

Definition at line 128 of file [filesystem.h](#).

4.5.2 Field Documentation

4.5.2.1 data

```
uint8_t FS_FiledataFlash::data[NUM\_SLOTS][8][DL\_FLASHCTL\_SECTOR\_SIZE]
```

Definition at line 130 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.6 FS_FileEntry Struct Reference

Allocation entry for a file within the FAT table.

```
#include <filesystem.h>
```

Data Fields

- `uint8_t` [uuid](#) [16]
- `uint16_t` [filesize](#)
- `uint16_t` [_pad](#)
- `uint32_t` [fileaddr](#)

4.6.1 Detailed Description

Allocation entry for a file within the FAT table.

Definition at line 64 of file [filesystem.h](#).

4.6.2 Field Documentation

4.6.2.1 `_pad`

```
uint16_t FS_FileEntry::_pad
```

Definition at line 68 of file [filesystem.h](#).

4.6.2.2 `fileaddr`

```
uint32_t FS_FileEntry::fileaddr
```

Definition at line 69 of file [filesystem.h](#).

4.6.2.3 `filesize`

```
uint16_t FS_FileEntry::filesize
```

Definition at line 67 of file [filesystem.h](#).

4.6.2.4 `uuid`

```
uint8_t FS_FileEntry::uuid[16]
```

Definition at line 66 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.7 FS_Metadata Struct Reference

Encrypted metadata for a stored file.

```
#include <filesystem.h>
```


Data Fields

- uint16_t [groupid](#)
- uint8_t [key](#) [ECC_PUB_KEYSIZE]
- uint8_t [nonce](#) [NONCE_SIZE]
- uint8_t [fileid](#) [ID_SIZE]
- uint8_t [tag](#) [ASCON_TAG_SIZE]
- uint8_t [filename](#) [32]

4.7.1 Detailed Description

Encrypted metadata for a stored file.

Contains ownership and cryptographic verification data. Validated against the [FS_Preamble](#) and firmware secrets upon load.

Definition at line 51 of file [filesystem.h](#).

4.7.2 Field Documentation

4.7.2.1 fileid

```
uint8_t FS_Metadata::fileid[ID_SIZE]
```

Definition at line 56 of file [filesystem.h](#).

4.7.2.2 filename

```
uint8_t FS_Metadata::filename[32]
```

Definition at line 58 of file [filesystem.h](#).

4.7.2.3 groupid

```
uint16_t FS_Metadata::groupid
```

Definition at line 53 of file [filesystem.h](#).

4.7.2.4 key

```
uint8_t FS_Metadata::key[ECC_PUB_KEYSIZE]
```

Definition at line 54 of file [filesystem.h](#).

4.7.2.5 nonce

```
uint8_t FS_Metadata::nonce[NONCE_SIZE]
```

Definition at line 55 of file [filesystem.h](#).

4.7.2.6 tag

```
uint8_t FS_Metadata::tag[ASCON_TAG_SIZE]
```

Definition at line 57 of file [filesystem.h](#).

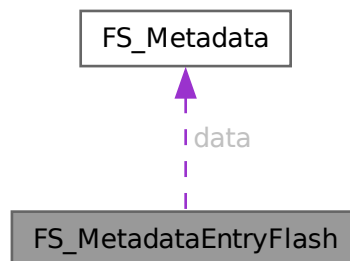
The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.8 FS_MetadataEntryFlash Union Reference

```
#include <filesystem.h>
```

Collaboration diagram for FS_MetadataEntryFlash:



Data Fields

- uint8_t [asBytes](#) [DL_FLASHCTL_SECTOR_SIZE]
- [FS_Metadata](#) [data](#)
- struct {
 - uint8_t [_padding](#) [512]
 - uint8_t [id](#) [16]
 - uint32_t [magic](#)
- } [status](#)

4.8.1 Detailed Description

Definition at line 114 of file [filesystem.h](#).

4.8.2 Field Documentation

4.8.2.1 _padding

```
uint8_t FS_MetadataEntryFlash::_padding[512]
```

Definition at line 120 of file [filesystem.h](#).

4.8.2.2 asBytes

```
uint8_t FS_MetadataEntryFlash::asBytes[DL_FLASHCTL_SECTOR_SIZE]
```

Definition at line 116 of file [filesystem.h](#).

Referenced by [StoreFile\(\)](#).

4.8.2.3 data

```
FS_Metadata FS_MetadataEntryFlash::data
```

Definition at line 117 of file [filesystem.h](#).

Referenced by [StoreFile\(\)](#).

4.8.2.4 id

```
uint8_t FS_MetadataEntryFlash::id[16]
```

Definition at line 121 of file [filesystem.h](#).

Referenced by [StoreFile\(\)](#).

4.8.2.5 magic

```
uint32_t FS_MetadataEntryFlash::magic
```

Definition at line 122 of file [filesystem.h](#).

Referenced by [StoreFile\(\)](#).

4.8.2.6 [struct]

```
struct { ... } FS_MetadataEntryFlash::status
```

Referenced by [StoreFile\(\)](#).

The documentation for this union was generated from the following file:

- firmware/include/[filesystem.h](#)

4.9 FS_Preamble Struct Reference

Cryptographic preamble for external data transmission.

```
#include <filesystem.h>
```

Data Fields

- `uint8_t` [key](#) [ECC_PUB_KEYSIZE]
- `uint8_t` [tag](#) [ASCON_TAG_SIZE]
- `uint8_t` [nonce](#) [NONCE_SIZE]
- `uint8_t` [mesgid](#) [ID_SIZE]

4.9.1 Detailed Description

Cryptographic preamble for external data transmission.

Precedes filesystem payloads when sent over UART. Contains the public key, Ascon tag, nonce, and message ID for authentication and decryption.

Definition at line 31 of file [filesystem.h](#).

4.9.2 Field Documentation

4.9.2.1 key

```
uint8_t FS_Preamble::key[ECC_PUB_KEYSIZE]
```

Definition at line 33 of file [filesystem.h](#).

4.9.2.2 mesgid

```
uint8_t FS_Preamble::mesgid[ID_SIZE]
```

Definition at line 36 of file [filesystem.h](#).

4.9.2.3 nonce

```
uint8_t FS_Preamble::nonce[NONCE_SIZE]
```

Definition at line 35 of file [filesystem.h](#).

4.9.2.4 tag

```
uint8_t FS_Preamble::tag[ASCON_TAG_SIZE]
```

Definition at line 34 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.10 FS_Slot Struct Reference

```
#include <filesystem.h>
```

Data Fields

- uint8_t [slot](#)
- uint16_t [groupid](#)
- uint8_t [filename](#) [32]

4.10.1 Detailed Description

Definition at line 81 of file [filesystem.h](#).

4.10.2 Field Documentation

4.10.2.1 filename

```
uint8_t FS_Slot::filename[32]
```

Definition at line 85 of file [filesystem.h](#).

4.10.2.2 groupid

```
uint16_t FS_Slot::groupid
```

Definition at line 84 of file [filesystem.h](#).

4.10.2.3 slot

```
uint8_t FS_Slot::slot
```

Definition at line 83 of file [filesystem.h](#).

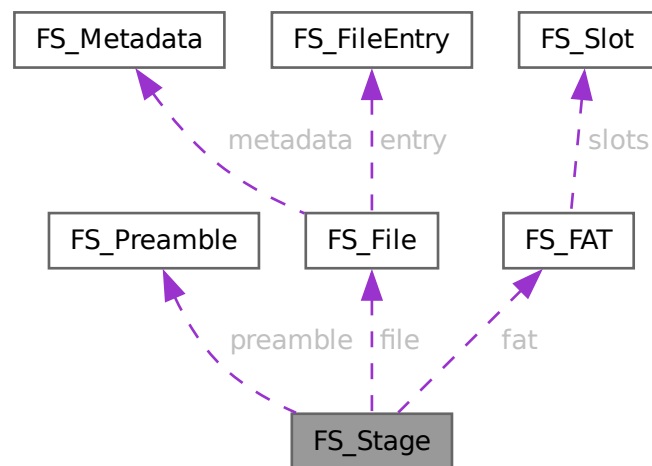
The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.11 FS_Stage Struct Reference

```
#include <filesystem.h>
```

Collaboration diagram for FS_Stage:



Data Fields

- [FS_Preamble](#) preamble
- union {
 - [FS_File](#) file
 - [FS_FAT](#) fat
 } as

4.11.1 Detailed Description

Definition at line 94 of file [filesystem.h](#).

4.11.2 Field Documentation

4.11.2.1 [union]

```
union { ... } FS_Stage::as
```

4.11.2.2 fat

```
FS_FAT FS_Stage::fat
```

Definition at line 100 of file [filesystem.h](#).

4.11.2.3 file

```
FS_File FS_Stage::file
```

Definition at line 99 of file [filesystem.h](#).

4.11.2.4 preamble

```
FS_Preamble FS_Stage::preamble
```

Definition at line 96 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- [firmware/include/filesystem.h](#)

4.12 FS_SystemStatusFlash Struct Reference

Internal configuration timeout state stored in flash.

```
#include <filesystem.h>
```

Data Fields

- `uint64_t` [timeout](#)

4.12.1 Detailed Description

Internal configuration timeout state stored in flash.

Definition at line 138 of file [filesystem.h](#).

4.12.2 Field Documentation

4.12.2.1 timeout

```
uint64_t FS_SystemStatusFlash::timeout
```

Definition at line 140 of file [filesystem.h](#).

The documentation for this struct was generated from the following file:

- firmware/include/[filesystem.h](#)

4.13 `derive_secrets.Permission` Class Reference

Public Member Functions

- [deserialize](#) (cls, str perms)
- [serialize](#) (self)

Static Public Attributes

- int [group_id](#) = 0
- bool [read](#) = False
- bool [write](#) = False
- bool [receive](#) = False

4.13.1 Detailed Description

@brief Represents permissions for a specific group.

@param group_id (int): The 16-bit group identifier.

@param read (bool): Permission to read files in this group.

@param write (bool): Permission to write files in this group.

@param receive (bool): Permission to receive files from peers for this group.

Definition at line 596 of file [derive_secrets.py](#).

4.13.2 Member Function Documentation

4.13.2.1 `deserialize()`

```
derive_secrets.Permission.deserialize (  
    cls,  
    str perms)
```

Definition at line 612 of file [derive_secrets.py](#).

4.13.2.2 serialize()

```
derive_secrets.Permission.serialize (  
    self)
```

Definition at line 623 of file [derive_secrets.py](#).

4.13.3 Field Documentation

4.13.3.1 group_id

```
int derive_secrets.Permission.group_id = 0 [static]
```

Definition at line 606 of file [derive_secrets.py](#).

4.13.3.2 read

```
bool derive_secrets.Permission.read = False [static]
```

Definition at line 607 of file [derive_secrets.py](#).

4.13.3.3 receive

```
bool derive_secrets.Permission.receive = False [static]
```

Definition at line 609 of file [derive_secrets.py](#).

4.13.3.4 write

```
bool derive_secrets.Permission.write = False [static]
```

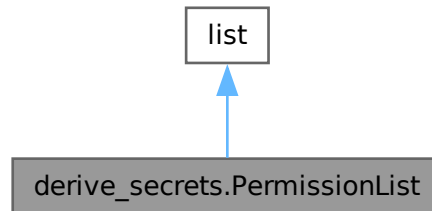
Definition at line 608 of file [derive_secrets.py](#).

The documentation for this class was generated from the following file:

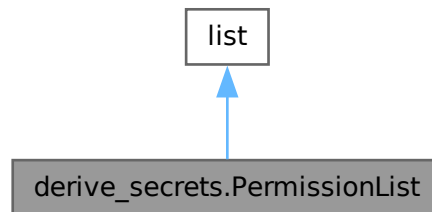
- [firmware/utils/derive_secrets.py](#)

4.14 `derive_secrets.PermissionList` Class Reference

Inheritance diagram for `derive_secrets.PermissionList`:



Collaboration diagram for `derive_secrets.PermissionList`:



Public Member Functions

- `__init__` (self, *[Permission](#) args)
- `deserialize` (cls, str perms)
- `serialize` (self)

4.14.1 Detailed Description

@brief A collection of `Permission` objects with serialization support.

Definition at line 631 of file [derive_secrets.py](#).

4.14.2 Constructor & Destructor Documentation

4.14.2.1 __init__()

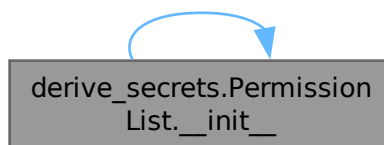
```
derive_secrets.PermissionList.__init__ (  
    self,  
    *Permission args)
```

Definition at line 636 of file [derive_secrets.py](#).

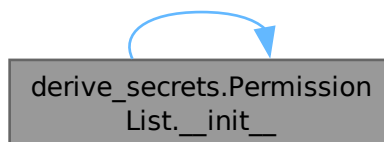
References [__init__\(\)](#).

Referenced by [__init__\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.14.3 Member Function Documentation

4.14.3.1 deserialize()

```
derive_secrets.PermissionList.deserialize (  
    cls,  
    str perms)
```

Definition at line 642 of file [derive_secrets.py](#).

4.14.3.2 `serialize()`

```
derive_secrets.PermissionList.serialize (  
    self)
```

Definition at line 650 of file [derive_secrets.py](#).

The documentation for this class was generated from the following file:

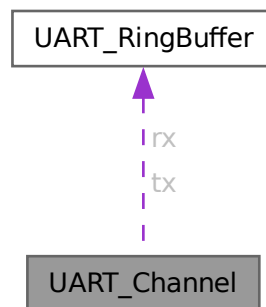
- [firmware/utils/derive_secrets.py](#)

4.15 `UART_Channel` Struct Reference

A complete bidirectional UART channel state.

```
#include <uart.h>
```

Collaboration diagram for `UART_Channel`:



Data Fields

- [UART_RingBuffer rx](#)
- [UART_RingBuffer tx](#)

4.15.1 Detailed Description

A complete bidirectional UART channel state.

Definition at line 37 of file [uart.h](#).

4.15.2 Field Documentation

4.15.2.1 rx

`UART_RingBuffer` `UART_Channel::rx`

Definition at line 39 of file `uart.h`.

Referenced by `IRQ_Handler()`, `UART_ClearChannel()`, and `UART_RecvBytes()`.

4.15.2.2 tx

`UART_RingBuffer` `UART_Channel::tx`

Definition at line 40 of file `uart.h`.

Referenced by `UART_ClearChannel()`, and `UART_SendBytes()`.

The documentation for this struct was generated from the following file:

- `firmware/include/uart.h`

4.16 UART_RingBuffer Struct Reference

Circular ring buffer control structure.

```
#include <uart.h>
```

Data Fields

- `uint8_t` `data` [`UART_BUFFER_SIZE`]
- `uint8_t` `head`
- `uint8_t` `tail`
- `uint16_t` `count`
- `uint16_t` `progress`
- `bool` `dirty`

4.16.1 Detailed Description

Circular ring buffer control structure.

Definition at line 24 of file `uart.h`.

4.16.2 Field Documentation

4.16.2.1 count

```
uint16_t UART_RingBuffer::count
```

Number of bytes currently enqueued.

Definition at line 29 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), [UART_ClearBuffer\(\)](#), [UART_PopByte\(\)](#), [UART_PushByte\(\)](#), [UART_RecvUntil\(\)](#), and [UART_TransmitAll\(\)](#).

4.16.2.2 data

```
uint8_t UART_RingBuffer::data[UART_BUFFER_SIZE]
```

The physical byte array buffer.

Definition at line 26 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), [UART_ClearBuffer\(\)](#), [UART_PopByte\(\)](#), [UART_PushByte\(\)](#), and [UART_TransmitAll\(\)](#).

4.16.2.3 dirty

```
bool UART_RingBuffer::dirty
```

Flag indicating unread new data.

Definition at line 31 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), [UART_ClearBuffer\(\)](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), and [UART_SendBytes\(\)](#).

4.16.2.4 head

```
uint8_t UART_RingBuffer::head
```

Insertion index.

Definition at line 27 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), [UART_ClearBuffer\(\)](#), and [UART_PushByte\(\)](#).

4.16.2.5 progress

```
uint16_t UART_RingBuffer::progress
```

Bytes processed during chunked ops.

Definition at line 30 of file [uart.h](#).

Referenced by [UART_ClearBuffer\(\)](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UART_SendBytes\(\)](#), and [UART_TransmitAll\(\)](#).

4.16.2.6 tail

```
uint8_t UART_RingBuffer::tail
```

Removal index.

Definition at line 28 of file [uart.h](#).

Referenced by [UART_ClearBuffer\(\)](#), [UART_PopByte\(\)](#), and [UART_TransmitAll\(\)](#).

The documentation for this struct was generated from the following file:

- [firmware/include/uart.h](#)

4.17 uECC_HashContext Struct Reference

Interface for passing arbitrary hash functions to uECC.

```
#include <uECC.h>
```

Data Fields

- void(* [init_hash](#))(struct [uECC_HashContext](#) *context)
Initialize the hash state.
- void(* [update_hash](#))(struct [uECC_HashContext](#) *context, const uint8_t *message, unsigned message_size)
Feed data into the hash.
- void(* [finish_hash](#))(struct [uECC_HashContext](#) *context, uint8_t *hash_result)
Extract the final digest.
- unsigned [block_size](#)
- unsigned [result_size](#)
- uint8_t * [tmp](#)

4.17.1 Detailed Description

Interface for passing arbitrary hash functions to uECC.

Definition at line 141 of file [uECC.h](#).

4.17.2 Field Documentation

4.17.2.1 block_size

```
unsigned uECC_HashContext::block_size
```

Hash function internal block size in bytes.

Definition at line 150 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#), and [HMAC_init\(\)](#).

4.17.2.2 finish_hash

```
void(* uECC_HashContext::finish_hash) (struct uECC_HashContext *context, uint8_t *hash_result)
```

Extract the final digest.

Definition at line 149 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#).

4.17.2.3 init_hash

```
void(* uECC_HashContext::init_hash) (struct uECC_HashContext *context)
```

Initialize the hash state.

Definition at line 144 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#), and [HMAC_init\(\)](#).

4.17.2.4 result_size

```
unsigned uECC_HashContext::result_size
```

Hash function final digest size in bytes.

Definition at line 151 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#), [HMAC_init\(\)](#), [uECC_sign_deterministic\(\)](#), and [update_V\(\)](#).

4.17.2.5 tmp

```
uint8_t* uECC_HashContext::tmp
```

Working buffer of at least (2 * result_size + block_size) bytes.

Definition at line 152 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#), [HMAC_init\(\)](#), and [uECC_sign_deterministic\(\)](#).

4.17.2.6 update_hash

```
void(* uECC_HashContext::update_hash) (struct uECC_HashContext *context, const uint8_t *message, unsigned message_size)
```

Feed data into the hash.

Definition at line 146 of file [uECC.h](#).

Referenced by [HMAC_finish\(\)](#), [HMAC_init\(\)](#), and [HMAC_update\(\)](#).

The documentation for this struct was generated from the following file:

- [firmware/include/uECC.h](#)

Chapter 5

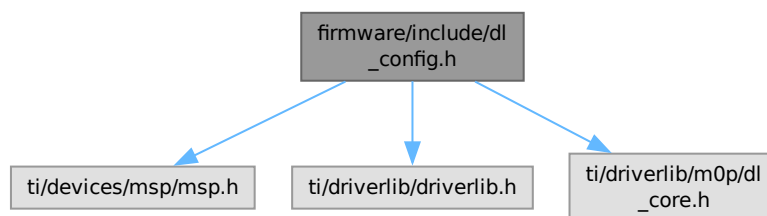
File Documentation

5.1 firmware/include/dl_config.h File Reference

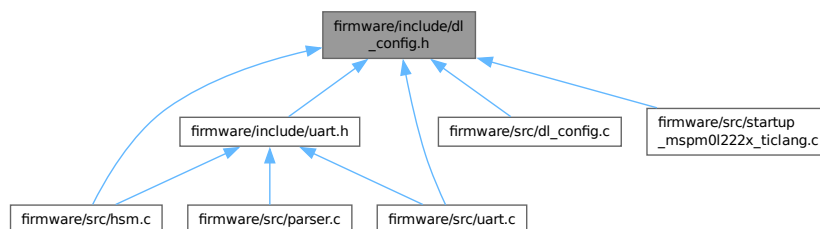
Device Library Configuration for MSPM0.

```
#include <ti/devices/msp/msp.h>
#include <ti/driverlib/driverlib.h>
#include <ti/driverlib/m0p/dl_core.h>
```

Include dependency graph for dl_config.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define [CONFIG_MSPM0L222X](#)
- #define [CONFIG_MSPM0L2228](#)
- #define [SYSCONFIG_WEAK __attribute__\(\(weak\)\)](#)
- #define [POWER_STARTUP_DELAY](#) (16)
Delay cycles required for power domain stabilization during startup.
- #define [CPUCLK_FREQ](#) 32000000
Central CPU clock frequency (32MHz).
- #define [LEDS_PORT](#) (GPIOB)
- #define [LEDS_STATUS_LED_PIN](#) (DL_GPIO_PIN_14)
- #define [LEDS_STATUS_LED_IOMUX](#) (IOMUX_PINCM35)
- #define [UART_0_INST](#) UART0
- #define [UART_0_INST_FREQUENCY](#) 32000000
- #define [UART_0_INST_IRQHandler](#) UART0_IRQHandler
- #define [UART_0_INST_INT_IRQn](#) UART0_INT_IRQn
- #define [GPIO_UART_0_RX_PORT](#) GPIOA
- #define [GPIO_UART_0_TX_PORT](#) GPIOA
- #define [GPIO_UART_0_RX_PIN](#) DL_GPIO_PIN_11
- #define [GPIO_UART_0_TX_PIN](#) DL_GPIO_PIN_10
- #define [GPIO_UART_0_IOMUX_RX](#) (IOMUX_PINCM26)
- #define [GPIO_UART_0_IOMUX_TX](#) (IOMUX_PINCM25)
- #define [GPIO_UART_0_IOMUX_RX_FUNC](#) IOMUX_PINCM26_PF_UART0_RX
- #define [GPIO_UART_0_IOMUX_TX_FUNC](#) IOMUX_PINCM25_PF_UART0_TX
- #define [UART_0_BAUD_RATE](#) (115200)
- #define [UART_0_IBRD_32_MHZ_115200_BAUD](#) (17)
- #define [UART_0_FBRD_32_MHZ_115200_BAUD](#) (23)
- #define [UART_1_INST](#) UART1
- #define [UART_1_INST_FREQUENCY](#) 32000000
- #define [UART_1_INST_IRQHandler](#) UART1_IRQHandler
- #define [UART_1_INST_INT_IRQn](#) UART1_INT_IRQn
- #define [GPIO_UART_1_RX_PORT](#) GPIOA
- #define [GPIO_UART_1_TX_PORT](#) GPIOA
- #define [GPIO_UART_1_RX_PIN](#) DL_GPIO_PIN_9
- #define [GPIO_UART_1_TX_PIN](#) DL_GPIO_PIN_8
- #define [GPIO_UART_1_IOMUX_RX](#) (IOMUX_PINCM20)
- #define [GPIO_UART_1_IOMUX_TX](#) (IOMUX_PINCM19)
- #define [GPIO_UART_1_IOMUX_RX_FUNC](#) IOMUX_PINCM20_PF_UART1_RX
- #define [GPIO_UART_1_IOMUX_TX_FUNC](#) IOMUX_PINCM19_PF_UART1_TX
- #define [UART_1_BAUD_RATE](#) (115200)
- #define [UART_1_IBRD_32_MHZ_115200_BAUD](#) (17)
- #define [UART_1_FBRD_32_MHZ_115200_BAUD](#) (23)

Functions

- void [SYSCFG_DL_init](#) (void)
Performs all required MSP DriverLib initialization.
- void [SYSCFG_DL_initPower](#) (void)
Enables power and resets peripherals.
- void [SYSCFG_DL_GPIO_init](#) (void)
Initializes GPIO muxing and pin directions.
- void [SYSCFG_DL_SYCTL_init](#) (void)
Configures System Control (SYCTL) settings.

- void [SYSCFG_DL_UART_0_init](#) (void)
Initializes UART0 for Host Communication (115200 baud).
- void [SYSCFG_DL_UART_1_init](#) (void)
Initializes UART1 for Inter-HSM Communication (115200 baud).
- void [SYSCFG_DL_TRNG_init](#) (void)
Initializes the hardware True Random Number Generator.

5.1.1 Detailed Description

Device Library Configuration for MSPM0.

Author

Sumedh Girish

This header contains the hardware abstraction layer configuration for the SHIFT firmware. It defines the pinout, clock frequencies, and peripheral instances for the TI MSPM0L2228 microcontroller.

Definition in file [dl_config.h](#).

5.1.2 Macro Definition Documentation

5.1.2.1 CONFIG_MSPM0L2228

```
#define CONFIG_MSPM0L2228
```

Definition at line 15 of file [dl_config.h](#).

5.1.2.2 CONFIG_MSPM0L222X

```
#define CONFIG_MSPM0L222X
```

Definition at line 14 of file [dl_config.h](#).

5.1.2.3 CPUCLK_FREQ

```
#define CPUCLK_FREQ 32000000
```

Central CPU clock frequency (32MHz).

Definition at line 38 of file [dl_config.h](#).

5.1.2.4 GPIO_UART_0_IOMUX_RX

```
#define GPIO_UART_0_IOMUX_RX (IOMUX_PINCM26)
```

Definition at line 56 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.5 GPIO_UART_0_IOMUX_RX_FUNC

```
#define GPIO_UART_0_IOMUX_RX_FUNC IOMUX_PINCM26_PF_UART0_RX
```

Definition at line 58 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.6 GPIO_UART_0_IOMUX_TX

```
#define GPIO_UART_0_IOMUX_TX (IOMUX_PINCM25)
```

Definition at line 57 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.7 GPIO_UART_0_IOMUX_TX_FUNC

```
#define GPIO_UART_0_IOMUX_TX_FUNC IOMUX_PINCM25_PF_UART0_TX
```

Definition at line 59 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.8 GPIO_UART_0_RX_PIN

```
#define GPIO_UART_0_RX_PIN DL_GPIO_PIN_11
```

Definition at line 54 of file [dl_config.h](#).

5.1.2.9 GPIO_UART_0_RX_PORT

```
#define GPIO_UART_0_RX_PORT GPIOA
```

Definition at line 52 of file [dl_config.h](#).

5.1.2.10 GPIO_UART_0_TX_PIN

```
#define GPIO_UART_0_TX_PIN DL_GPIO_PIN_10
```

Definition at line 55 of file [dl_config.h](#).

5.1.2.11 GPIO_UART_0_TX_PORT

```
#define GPIO_UART_0_TX_PORT GPIOA
```

Definition at line 53 of file [dl_config.h](#).

5.1.2.12 GPIO_UART_1_IOMUX_RX

```
#define GPIO_UART_1_IOMUX_RX (IOMUX_PINCM20)
```

Definition at line 73 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.13 GPIO_UART_1_IOMUX_RX_FUNC

```
#define GPIO_UART_1_IOMUX_RX_FUNC IOMUX_PINCM20_PF_UART1_RX
```

Definition at line 75 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.14 GPIO_UART_1_IOMUX_TX

```
#define GPIO_UART_1_IOMUX_TX (IOMUX_PINCM19)
```

Definition at line 74 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.15 GPIO_UART_1_IOMUX_TX_FUNC

```
#define GPIO_UART_1_IOMUX_TX_FUNC IOMUX_PINCM19_PF_UART1_TX
```

Definition at line 76 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.16 GPIO_UART_1_RX_PIN

```
#define GPIO_UART_1_RX_PIN DL_GPIO_PIN_9
```

Definition at line 71 of file [dl_config.h](#).

5.1.2.17 GPIO_UART_1_RX_PORT

```
#define GPIO_UART_1_RX_PORT GPIOA
```

Definition at line 69 of file [dl_config.h](#).

5.1.2.18 GPIO_UART_1_TX_PIN

```
#define GPIO_UART_1_TX_PIN DL_GPIO_PIN_8
```

Definition at line 72 of file [dl_config.h](#).

5.1.2.19 GPIO_UART_1_TX_PORT

```
#define GPIO_UART_1_TX_PORT GPIOA
```

Definition at line 70 of file [dl_config.h](#).

5.1.2.20 LEDS_PORT

```
#define LEDS_PORT (GPIOB)
```

Definition at line 41 of file [dl_config.h](#).

Referenced by [BlinkLED\(\)](#), and [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.21 LEDS_STATUS_LED_IOMUX

```
#define LEDS_STATUS_LED_IOMUX (IOMUX_PINCM35)
```

Definition at line 45 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.22 LEDS_STATUS_LED_PIN

```
#define LEDS_STATUS_LED_PIN (DL_GPIO_PIN_14)
```

Definition at line 44 of file [dl_config.h](#).

Referenced by [BlinkLED\(\)](#), and [SYSCFG_DL_GPIO_init\(\)](#).

5.1.2.23 POWER_STARTUP_DELAY

```
#define POWER_STARTUP_DELAY (16)
```

Delay cycles required for power domain stabilization during startup.

Definition at line 35 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_initPower\(\)](#).

5.1.2.24 SYSCONFIG_WEAK

```
#define SYSCONFIG_WEAK __attribute__((weak))
```

Definition at line 17 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_GPIO_init\(\)](#), [SYSCFG_DL_init\(\)](#), [SYSCFG_DL_initPower\(\)](#), [SYSCFG_DL_SYSCTL_init\(\)](#), [SYSCFG_DL_TRNG_init\(\)](#), [SYSCFG_DL_UART_0_init\(\)](#), and [SYSCFG_DL_UART_1_init\(\)](#).

5.1.2.25 UART_0_BAUD_RATE

```
#define UART_0_BAUD_RATE (115200)
```

Definition at line 60 of file [dl_config.h](#).

5.1.2.26 UART_0_FBRD_32_MHZ_115200_BAUD

```
#define UART_0_FBRD_32_MHZ_115200_BAUD (23)
```

Definition at line 62 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_0_init\(\)](#).

5.1.2.27 UART_0_IBRD_32_MHZ_115200_BAUD

```
#define UART_0_IBRD_32_MHZ_115200_BAUD (17)
```

Definition at line 61 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_0_init\(\)](#).

5.1.2.28 UART_0_INST

```
#define UART_0_INST UART0
```

Definition at line 48 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_initPower\(\)](#), [SYSCFG_DL_UART_0_init\(\)](#), and [UART0_IRQHandler\(\)](#).

5.1.2.29 UART_0_INST_FREQUENCY

```
#define UART_0_INST_FREQUENCY 32000000
```

Definition at line 49 of file [dl_config.h](#).

5.1.2.30 UART_0_INST_INT_IRQN

```
#define UART_0_INST_INT_IRQN UART0_INT_IRQn
```

Definition at line 51 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_0_init\(\)](#).

5.1.2.31 UART_0_INST_IRQHandler

```
#define UART_0_INST_IRQHandler UART0_IRQHandler
```

Definition at line 50 of file [dl_config.h](#).

5.1.2.32 UART_1_BAUD_RATE

```
#define UART_1_BAUD_RATE (115200)
```

Definition at line 77 of file [dl_config.h](#).

5.1.2.33 UART_1_FBRD_32_MHZ_115200_BAUD

```
#define UART_1_FBRD_32_MHZ_115200_BAUD (23)
```

Definition at line 79 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_1_init\(\)](#).

5.1.2.34 UART_1_IBRD_32_MHZ_115200_BAUD

```
#define UART_1_IBRD_32_MHZ_115200_BAUD (17)
```

Definition at line 78 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_1_init\(\)](#).

5.1.2.35 UART_1_INST

```
#define UART_1_INST UART1
```

Definition at line 65 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_initPower\(\)](#), [SYSCFG_DL_UART_1_init\(\)](#), and [UART1_IRQHandler\(\)](#).

5.1.2.36 UART_1_INST_FREQUENCY

```
#define UART_1_INST_FREQUENCY 32000000
```

Definition at line 66 of file [dl_config.h](#).

5.1.2.37 UART_1_INST_INT_IRQN

```
#define UART_1_INST_INT_IRQN UART1_INT_IRQn
```

Definition at line 68 of file [dl_config.h](#).

Referenced by [SYSCFG_DL_UART_1_init\(\)](#).

5.1.2.38 UART_1_INST_IRQHandler

```
#define UART_1_INST_IRQHandler UART1_IRQHandler
```

Definition at line 67 of file [dl_config.h](#).

5.1.3 Function Documentation

5.1.3.1 SYSCFG_DL_GPIO_init()

```
void SYSCFG_DL_GPIO_init (
    void )
```

Initializes GPIO muxing and pin directions.

Assigns pins to their respective peripheral functions (UART RX/TX) and configures output pins like the Status LED.

Initializes GPIO muxing and pin directions.

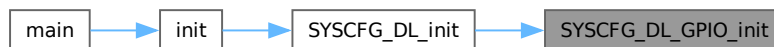
Configures the secondary functions for UART pins and initializes the Status LED pin as a digital output.

Definition at line 67 of file [dl_config.c](#).

References [GPIO_UART_0_IOMUX_RX](#), [GPIO_UART_0_IOMUX_RX_FUNC](#), [GPIO_UART_0_IOMUX_TX](#), [GPIO_UART_0_IOMUX_TX_FUNC](#), [GPIO_UART_1_IOMUX_RX](#), [GPIO_UART_1_IOMUX_RX_FUNC](#), [GPIO_UART_1_IOMUX_TX](#), [GPIO_UART_1_IOMUX_TX_FUNC](#), [LEDS_PORT](#), [LEDS_STATUS_LED_IOMUX](#), [LEDS_STATUS_LED_PIN](#), and [SYSCONFIG_WEAK](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.1.3.2 SYSCFG_DL_init()

```
void SYSCFG_DL_init (
    void )
```

Performs all required MSP DriverLib initialization.

Orchestrates the full hardware startup sequence: power domains, system clocks, GPIO muxing, TRNG seed initialization, and dual-channel UART configuration. This function must be called as the first action in [main\(\)](#).

Performs all required MSP DriverLib initialization.

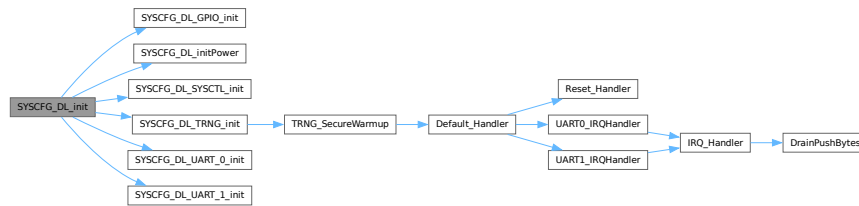
This function orchestrates the initialization sequence for all required hardware modules. It must be called before any application logic.

Definition at line 23 of file [dl_config.c](#).

References [SYSCFG_DL_GPIO_init\(\)](#), [SYSCFG_DL_initPower\(\)](#), [SYSCFG_DL_SYSCCTL_init\(\)](#), [SYSCFG_DL_TRNG_init\(\)](#), [SYSCFG_DL_UART_0_init\(\)](#), [SYSCFG_DL_UART_1_init\(\)](#), and [SYSCONFIG_WEAK](#).

Referenced by [init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.1.3.3 SYSCFG_DL_initPower()

```
void SYSCFG_DL_initPower (
    void )
```

Enables power and resets peripherals.

Configures power domains for GPIOA, GPIOB, UART0, UART1, and TRNG. Includes a mandatory startup delay for supply stabilization.

Enables power and resets peripherals.

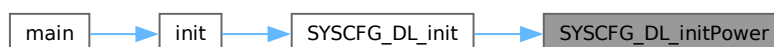
Resets the state of GPIO and UART peripherals and then enables their power domains. Includes a mandatory delay for stabilization.

Definition at line 41 of file [dl_config.c](#).

References [POWER_STARTUP_DELAY](#), [SYSCONFIG_WEAK](#), [UART_0_INST](#), and [UART_1_INST](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.1.3.4 SYSCFG_DL_SYSTL_init()

```
void SYSCFG_DL_SYSTL_init (
    void )
```

Configures System Control (SYSTL) settings.

Implements high-strictness brownout monitoring (BOR3) and system clock tree configuration (32MHz base frequency).

Configures System Control (SYSTL) settings.

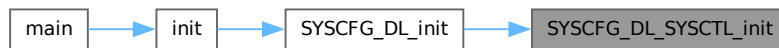
Configures the system for maximum brownout protection by setting the BOR threshold to Level 3 (~2.7V) and activating the monitoring hardware. Also configures the system oscillator frequency to its base (32 MHz).

Definition at line 91 of file [dl_config.c](#).

References [SYSCONFIG_WEAK](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.1.3.5 SYSCFG_DL_TRNG_init()

```
void SYSCFG_DL_TRNG_init (
    void )
```

Initializes the hardware True Random Number Generator.

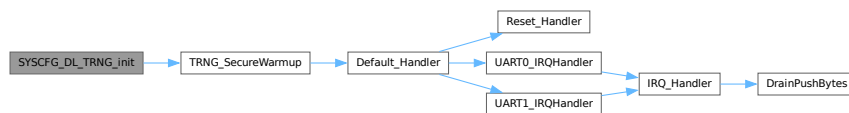
Initializes the hardware True Random Number Generator.

Definition at line 227 of file [dl_config.c](#).

References [SYSCONFIG_WEAK](#), and [TRNG_SecureWarmup\(\)](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.1.3.6 SYSCFG_DL_UART_0_init()

```
void SYSCFG_DL_UART_0_init (
    void )
```

Initializes UART0 for Host Communication (115200 baud).

Initializes UART0 for Host Communication (115200 baud).

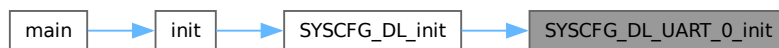
Configures the baud rate to 115200 based on the 32 MHz BUSCLK.

Definition at line 125 of file [dl_config.c](#).

References [gUART_0ClockConfig](#), [gUART_0Config](#), [SYSCONFIG_WEAK](#), [UART_0_FBRD_32_MHZ_115200_BAUD](#), [UART_0_IBRD_32_MHZ_115200_BAUD](#), [UART_0_INST](#), and [UART_0_INST_INT_IRQN](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.1.3.7 SYSCFG_DL_UART_1_init()

```
void SYSCFG_DL_UART_1_init (
    void )
```

Initializes UART1 for Inter-HSM Communication (115200 baud).

Initializes UART1 for Inter-HSM Communication (115200 baud).

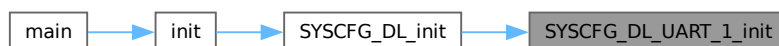
Configures the baud rate to 115200 based on the 32 MHz BUSCLK.

Definition at line 173 of file [dl_config.c](#).

References [gUART_1ClockConfig](#), [gUART_1Config](#), [SYSCONFIG_WEAK](#), [UART_1_FBRD_32_MHZ_115200_BAUD](#), [UART_1_IBRD_32_MHZ_115200_BAUD](#), [UART_1_INST](#), and [UART_1_INST_INT_IRQN](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.2 dl_config.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 #ifndef ti_msp_dl_config_h
00012 #define ti_msp_dl_config_h
00013
00014 #define CONFIG_MSPM0L222X
00015 #define CONFIG_MSPM0L2228
00016
00017 #define SYSCFG_WEAK __attribute__((weak))
00018
00019 #include <ti/devices/msp/msp.h>
00020 #include <ti/driverlib/driverlib.h>
00021 #include <ti/driverlib/m0p/dl_core.h>
00022
00030 void SYSCFG_DL_init(void);
00031
00032 /* clang-format off */
00033
00035 #define POWER_STARTUP_DELAY (16)
00036
00038 #define CPUCLK_FREQ 32000000
00039
00040 /* Port definition for Pin Group LEDS */
00041 #define LEDS_PORT (GPIOB)
00042
00043 /* Defines for STATUS_LED: GPIOB.14 with pinCMx 35 on package pin 2 */
00044 #define LEDS_STATUS_LED_PIN (DL_GPIO_PIN_14)
00045 #define LEDS_STATUS_LED_IOMUX (IOMUX_PINCM35)
00046
00047 /* Defines for UART_0 (Host Interface) */
00048 #define UART_0_INST UART0
00049 #define UART_0_INST_FREQUENCY 32000000
00050 #define UART_0_INST_IRQHandler UART0_IRQHandler
00051 #define UART_0_INST_INT_IRQn UART0_INT_IRQn
00052 #define GPIO_UART_0_RX_PORT GPIOA
00053 #define GPIO_UART_0_TX_PORT GPIOA
00054 #define GPIO_UART_0_RX_PIN DL_GPIO_PIN_11
00055 #define GPIO_UART_0_TX_PIN DL_GPIO_PIN_10
00056 #define GPIO_UART_0_IOMUX_RX (IOMUX_PINCM26)
00057 #define GPIO_UART_0_IOMUX_TX (IOMUX_PINCM25)
00058 #define GPIO_UART_0_IOMUX_RX_FUNC IOMUX_PINCM26_PF_UART0_RX
00059 #define GPIO_UART_0_IOMUX_TX_FUNC IOMUX_PINCM25_PF_UART0_TX
00060 #define UART_0_BAUD_RATE (115200)
00061 #define UART_0_IBRD_32_MHZ_115200_BAUD (17)
00062 #define UART_0_FBRD_32_MHZ_115200_BAUD (23)
00063
00064 /* Defines for UART_1 (Peer/Engineering Interface) */
00065 #define UART_1_INST UART1
00066 #define UART_1_INST_FREQUENCY 32000000
00067 #define UART_1_INST_IRQHandler UART1_IRQHandler
00068 #define UART_1_INST_INT_IRQn UART1_INT_IRQn
00069 #define GPIO_UART_1_RX_PORT GPIOA
00070 #define GPIO_UART_1_TX_PORT GPIOA
00071 #define GPIO_UART_1_RX_PIN DL_GPIO_PIN_9
00072 #define GPIO_UART_1_TX_PIN DL_GPIO_PIN_8
00073 #define GPIO_UART_1_IOMUX_RX (IOMUX_PINCM20)
00074 #define GPIO_UART_1_IOMUX_TX (IOMUX_PINCM19)
00075 #define GPIO_UART_1_IOMUX_RX_FUNC IOMUX_PINCM20_PF_UART1_RX
00076 #define GPIO_UART_1_IOMUX_TX_FUNC IOMUX_PINCM19_PF_UART1_TX
00077 #define UART_1_BAUD_RATE (115200)
00078 #define UART_1_IBRD_32_MHZ_115200_BAUD (17)
00079 #define UART_1_FBRD_32_MHZ_115200_BAUD (23)
00080
00081 /* clang-format on */
00082
00089 void SYSCFG_DL_initPower(void);
00090
00097 void SYSCFG_DL_GPIO_init(void);
00098
00105 void SYSCFG_DL_SYSTCL_init(void);
00106
00110 void SYSCFG_DL_UART_0_init(void);
00111
00115 void SYSCFG_DL_UART_1_init(void);
00116
00120 void SYSCFG_DL_TRNG_init(void);
00121
00122 #endif /* ti_msp_dl_config_h */

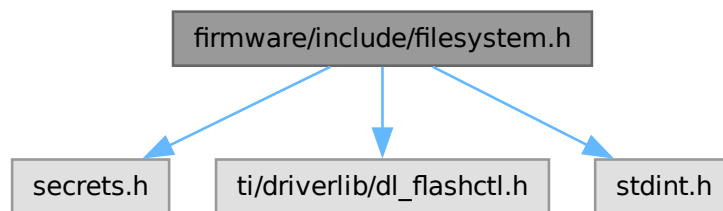
```

5.3 firmware/include/filesystem.h File Reference

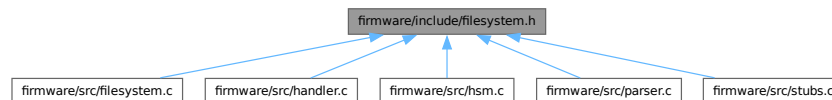
Secure Flash Filesystem Layout and Data Structures.

```
#include "secrets.h"
#include "ti/driverlib/dl_flashctl.h"
#include <stdint.h>
```

Include dependency graph for filesystem.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [FS_Preamble](#)
Cryptographic preamble for external data transmission.
- struct [FS_Metadata](#)
Encrypted metadata for a stored file.
- struct [FS_FileEntry](#)
Allocation entry for a file within the FAT table.
- struct [FS_File](#)
- struct [FS_Slot](#)
- struct [FS_FAT](#)
- struct [FS_Stage](#)
- union [FAT_TableTypeFlash](#)
- union [FS_MetadataEntryFlash](#)
- struct [FS_FiledataFlash](#)
- struct [FS_SystemStatusFlash](#)
Internal configuration timeout state stored in flash.

Macros

- `#define RAMFUNC \_\_attribute\_\_\(\(section\(".TI.ramfunc"\)\)\) \_\_attribute\_\_\(\(noinline\)\)`
- `#define MAX\_FILE\_SIZE 8192`
Maximum allowed file size in bytes (8 KB).
- `#define NUM\_SLOTS 8`

Typedefs

- `typedef uint8_t FS\_FileData[MAX\_FILE\_SIZE]`
Raw file data buffer.

Functions

- `void LoadFile (uint8_t slot, StatusCode *status)`
Loads a file from flash into the staging RAM area.
- `void StoreFile (uint8_t slot, StatusCode *status)`
Stores a file from the staging RAM area into flash memory.

Variables

- `volatile FS\_Stage stage`
Staging area in SRAM for cryptographic ops before syncing to Flash.
- `volatile const FAT\_TableTypeFlash FAT\_Table`
- `volatile const FS\_MetadataEntryFlash Metadata\_Table [NUM\_SLOTS]`
- `volatile const FS\_FiledataFlash Filedata\_Table`
- `volatile const FS\_SystemStatusFlash SystemStatus`

5.3.1 Detailed Description

Secure Flash Filesystem Layout and Data Structures.

Author

Sumedh Girish

Defines the static memory layout for the cryptographically verified, append-only filesystem. Includes definitions for FAT tables, encrypted metadata, and data blocks stored in flash memory.

Definition in file [filesystem.h](#).

5.3.2 Macro Definition Documentation

5.3.2.1 [MAX_FILE_SIZE](#)

```
#define MAX\_FILE\_SIZE 8192
```

Maximum allowed file size in bytes (8 KB).

Definition at line 40 of file [filesystem.h](#).

Referenced by [__attribute__\(\)](#), [ReadHandler\(\)](#), [StoreFile\(\)](#), [ValidateBodySize\(\)](#), [WriteHandler\(\)](#), and [WriteParser\(\)](#).

5.3.2.2 NUM_SLOTS

```
#define NUM_SLOTS 8
```

Definition at line 79 of file [filesystem.h](#).

Referenced by [__attribute__\(\)](#), [ListHandler\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), [ReplyHandler\(\)](#), [SendParser\(\)](#), [StoreFile\(\)](#), and [WriteParser\(\)](#).

5.3.2.3 RAMFUNC

```
#define RAMFUNC __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
```

Definition at line 22 of file [filesystem.h](#).

5.3.3 Typedef Documentation

5.3.3.1 FS_FileData

```
typedef uint8_t FS_FileData[MAX_FILE_SIZE]
```

Raw file data buffer.

Definition at line 43 of file [filesystem.h](#).

5.3.4 Function Documentation

5.3.4.1 LoadFile()

```
void LoadFile (
    uint8_t slot,
    StatusCode * status)
```

Loads a file from flash into the staging RAM area.

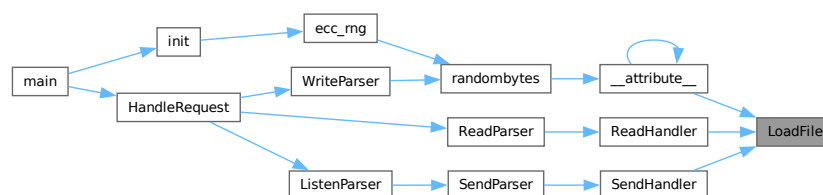
Verifies the file's cryptographic tag. If verification fails, the staging area is cleared and the status is updated.

Parameters

<i>slot</i>	Index of the file slot (0 to NUM_SLOTS-1).
<i>status</i>	Pointer to global status code.

Referenced by [__attribute__\(\)](#), [ReadHandler\(\)](#), and [SendHandler\(\)](#).

Here is the caller graph for this function:



5.3.4.2 StoreFile()

```
void StoreFile (
    uint8_t slot,
    StatusCode * status)
```

Stores a file from the staging RAM area into flash memory.

Parameters

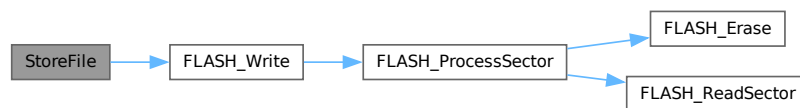
<i>slot</i>	Index of the file slot (0 to NUM_SLOTS-1).
<i>status</i>	Pointer to global status code.

Definition at line 62 of file [filesystem.c](#).

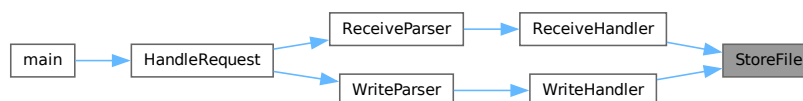
References [FS_MetadataEntryFlash::asBytes](#), [FS_MetadataEntryFlash::data](#), [FAT_Table](#), [Filedata_Table](#), [FLASH_Write\(\)](#), [FLASH_ERASEERROR](#), [FLASHWRITEERROR](#), [FS_MetadataEntryFlash::id](#), [INVALIDSLOT](#), [FS_MetadataEntryFlash::magic](#), [MAX_FILE_SIZE](#), [Metadata_Table](#), [NUM_SLOTS](#), [stage](#), and [FS_MetadataEntryFlash::status](#).

Referenced by [ReceiveHandler\(\)](#), and [WriteHandler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.5 Variable Documentation

5.3.5.1 FAT_Table

```
volatile const FAT_TableTypeFlash FAT_Table [extern]
```

Referenced by [__attribute__\(\)](#), [ListHandler\(\)](#), [ReplyHandler\(\)](#), and [StoreFile\(\)](#).

5.3.5.2 Filedata_Table

```
volatile const FS_FiledataFlash Filedata_Table [extern]
```

Referenced by [__attribute__\(\)](#), and [StoreFile\(\)](#).

5.3.5.3 Metadata_Table

```
volatile const FS_MetadataEntryFlash Metadata_Table[NUM_SLOTS] [extern]
```

Referenced by [__attribute__\(\)](#), [ListHandler\(\)](#), [ReplyHandler\(\)](#), and [StoreFile\(\)](#).

5.3.5.4 stage

```
volatile FS_Stage stage [extern]
```

Staging area in SRAM for cryptographic ops before syncing to Flash.

Definition at line 18 of file [filesystem.c](#).

Referenced by [__attribute__\(\)](#), [init\(\)](#), [InterrogateHandler\(\)](#), [InterrogateParser\(\)](#), [ListHandler\(\)](#), [main\(\)](#), [ReadHandler\(\)](#), [ReceiveHandler\(\)](#), [ReceiveParser\(\)](#), [ReplyHandler\(\)](#), [ReplyParser\(\)](#), [SendHandler\(\)](#), [SendParser\(\)](#), [SendResponse\(\)](#), [StoreFile\(\)](#), [WriteHandler\(\)](#), and [WriteParser\(\)](#).

5.3.5.5 SystemStatus

```
volatile const FS_SystemStatusFlash SystemStatus [extern]
```

Referenced by [__attribute__\(\)](#), [checkpin\(\)](#), and [main\(\)](#).

5.4 filesystem.h

[Go to the documentation of this file.](#)

```
00001
00010
00011 #ifndef __FILESYSTEM_H__
00012 #define __FILESYSTEM_H__
00013
00014 #include "secrets.h"
00015 #include "ti/driverlib/dl_flashctl.h"
00016 #include <stdint.h>
00017
00018 #ifndef PACKED
00019 #define PACKED __attribute__((packed))
00020 #endif
00021
00022 #define RAMFUNC
00023     __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
00024
00031 typedef struct
00032 {
00033     uint8_t key[ECC_PUB_KEYSIZE];
00034     uint8_t tag[ASCON_TAG_SIZE];
00035     uint8_t nonce[NONCE_SIZE];
00036     uint8_t msgid[ID_SIZE];
00037 } PACKED FS_Preamble;
00038
00040 #define MAX_FILE_SIZE 8192
```

```

00041
00043 typedef uint8_t FS_FileData[MAX_FILE_SIZE];
00044
00051 typedef struct
00052 {
00053     uint16_t groupid;
00054     uint8_t key[ECC_PUB_KEYSIZE];
00055     uint8_t nonce[NONCE_SIZE];
00056     uint8_t fileid[ID_SIZE];
00057     uint8_t tag[ASCON_TAG_SIZE];
00058     uint8_t filename[32];
00059 } PACKED FS_Metadate;
00060
00064 typedef struct
00065 {
00066     uint8_t uuid[16];
00067     uint16_t filesize;
00068     uint16_t _pad;
00069     uint32_t fileaddr;
00070 } PACKED FS_FileEntry;
00071
00072 typedef struct
00073 {
00074     FS_Metadate metadata;
00075     FS_FileEntry entry;
00076     FS_FileData content;
00077 } PACKED FS_File;
00078
00079 #define NUM_SLOTS 8
00080
00081 typedef struct
00082 {
00083     uint8_t slot;
00084     uint16_t groupid;
00085     uint8_t filename[32];
00086 } PACKED FS_Slot;
00087
00088 typedef struct
00089 {
00090     uint32_t numEntries;
00091     FS_Slot slots[NUM_SLOTS];
00092 } PACKED FS_FAT;
00093
00094 typedef struct
00095 {
00096     FS_Preamble preamble;
00097     union
00098     {
00099         FS_File file;
00100         FS_FAT fat;
00101     } as;
00102 } PACKED FS_Stage;
00103
00104 extern volatile FS_Stage stage;
00105
00106 typedef union
00107 {
00108     uint8_t asBytes[DL_FLASHCTL_SECTOR_SIZE];
00109     FS_FileEntry entry[NUM_SLOTS];
00110 } FAT_TableTypeFlash;
00111
00112 extern volatile const FAT_TableTypeFlash FAT_Table;
00113
00114 typedef union
00115 {
00116     uint8_t asBytes[DL_FLASHCTL_SECTOR_SIZE];
00117     FS_Metadate data;
00118     struct
00119     {
00120         uint8_t _padding[512];
00121         uint8_t id[16];
00122         uint32_t magic;
00123     } status;
00124 } FS_MetadateEntryFlash;
00125
00126 extern volatile const FS_MetadateEntryFlash Metadata_Table[NUM_SLOTS];
00127
00128 typedef struct
00129 {
00130     uint8_t data[NUM_SLOTS][8][DL_FLASHCTL_SECTOR_SIZE];
00131 } FS_FiledataFlash;
00132
00133 extern volatile const FS_FiledataFlash Filedata_Table;
00134
00138 typedef struct
00139 {
00140     uint64_t timeout;

```

```

00141 } FS_SystemStatusFlash;
00142
00143 extern volatile const FS_SystemStatusFlash SystemStatus;
00144
00154 void LoadFile(uint8_t slot, StatusCode *status);
00155
00162 void StoreFile(uint8_t slot, StatusCode *status);
00163
00164 #endif

```

5.5 firmware/include/flash.h File Reference

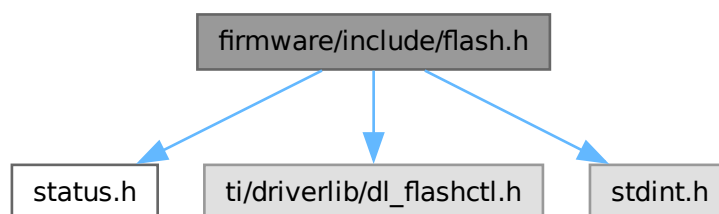
Low-level Flash Memory Controller Driver for MSPM0.

```

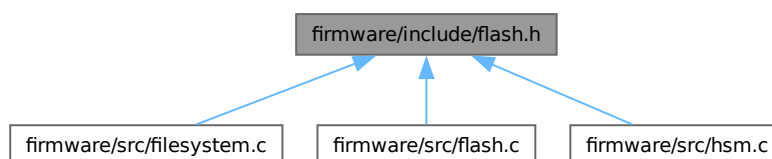
#include "status.h"
#include "ti/driverlib/dl_flashctl.h"
#include <stdint.h>

```

Include dependency graph for flash.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define `FLASH_PAGE_SIZE` `DL_FLASHCTL_SECTOR_SIZE`
Size of a single physical flash page/sector on the MSPM0.
- #define `RAMFUNC __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))`
Macro to place functions in the .TI.ramfunc section.

Functions

- **RAMFUNC** void **FLASH_Erase** (uint32_t address, **StatusCode** *status)
Erases a physical sector of flash memory.
- **RAMFUNC** void **FLASH_Write** (uint32_t address, uint8_t *buffer, uint32_t size, **StatusCode** *status)
Writes data into a physical flash sector.

5.5.1 Detailed Description

Low-level Flash Memory Controller Driver for MSPM0.

Author

Sumedh Girish

This module provides a thread-safe (via polling) abstraction for flash programming on the TI MSPM0L2228. It implements critical safety features including:

1. mandatory SRAM execution for erase/write routines (RAMFUNC).
2. Automatic sector-level unprotection/re-protection.
3. Hardware-software synchronization to prevent bus contention hangs.

Definition in file [flash.h](#).

5.5.2 Macro Definition Documentation

5.5.2.1 FLASH_PAGE_SIZE

```
#define FLASH_PAGE_SIZE DL_FLASHCTL_SECTOR_SIZE
```

Size of a single physical flash page/sector on the MSPM0.

Definition at line 24 of file [flash.h](#).

5.5.2.2 RAMFUNC

```
#define RAMFUNC __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
```

Macro to place functions in the .TI.ramfunc section.

On the MSPM0, the flash controller cannot fetch instructions from the flash bank it is currently modifying. Functions tagged with RAMFUNC are relocated to SRAM by the linker/startup code to allow the CPU to continue execution during flash write cycles.

Definition at line 34 of file [flash.h](#).

5.5.3 Function Documentation

5.5.3.1 FLASH_Erase()

```
RAMFUNC void FLASH_Erase (
    uint32_t address,
    StatusCode * status)
```

Erases a physical sector of flash memory.

Temporarily disables flash protection for the target sector, issues the erase command to the controller, and re-engages protection. Safe to call only when executing from SRAM.

Parameters

<i>address</i>	Base address of the 1024-byte sector to erase.
<i>status</i>	Global status variable to update on failure.

Note

This function blocks until the erase cycle completes.

References [RAMFUNC](#).

Referenced by [__attribute__\(\)](#), [FLASH_ProcessSector\(\)](#), and [main\(\)](#).

Here is the caller graph for this function:



5.5.3.2 FLASH_Write()

```
RAMFUNC void FLASH_Write (
    uint32_t address,
    uint8_t * buffer,
    uint32_t size,
    StatusCode * status)
```

Writes data into a physical flash sector.

Writes a buffer into flash memory in 64-bit words. Handles flash unlocking, execution of the write sequence, and locking. Automatically pads unaligned writes to 64-bit boundaries with 0xFF.

Parameters

<i>address</i>	Base address of the flash memory to write to.
----------------	---

<i>buffer</i>	Pointer to the source data in SRAM.
<i>size</i>	Number of bytes to write.
<i>status</i>	Global status variable to update on failure/misalignment.

Note

Must execute from SRAM.

Definition at line 104 of file [flash.c](#).

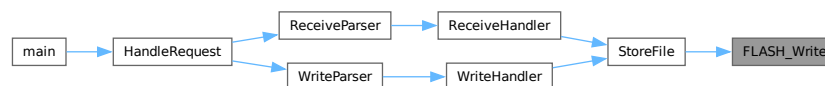
References [FLASH_ProcessSector\(\)](#), [FLASH_ERASEERROR](#), [FLASH_WRITEERROR](#), and [RAMFUNC](#).

Referenced by [StoreFile\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.6 flash.h

[Go to the documentation of this file.](#)

```

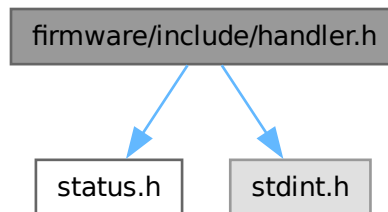
00001
00013
00014 #ifndef __FLASH_H__
00015 #define __FLASH_H__
00016
00017 #include "status.h"
00018 #include "ti/driverlib/dl_flashctl.h"
00019 #include <stdint.h>
00020
00024 #define FLASH_PAGE_SIZE DL_FLASHCTL_SECTOR_SIZE
00025
00034 #define RAMFUNC
00035     __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
00036
00049 RAMFUNC void FLASH_Erase(uint32_t address, StatusCode *status);
00050
00065 RAMFUNC void FLASH_Write(uint32_t address, uint8_t *buffer, uint32_t size,
00066                           StatusCode *status);
00067
00068 #endif /* __FLASH_H__ */
  
```

5.7 firmware/include/handler.h File Reference

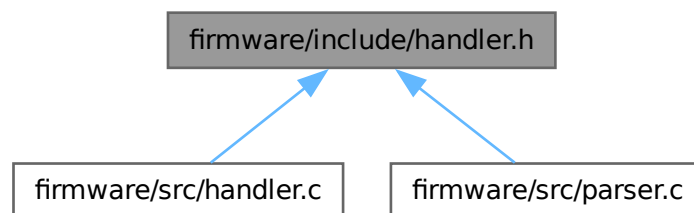
Command Handlers for Host and Peer interactions.

```
#include "status.h"
#include <stdint.h>
```

Include dependency graph for handler.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [ListHandler](#) ([StatusCode](#) *status)
Handles the LIST command.
- void [ReadHandler](#) (uint8_t slot, [StatusCode](#) *status)
Handles the READ command for a specific slot.
- void [WriteHandler](#) (uint8_t slot, [StatusCode](#) *status)
Handles the WRITE command for a specific slot.
- void [InterrogateHandler](#) ([StatusCode](#) *status)
Handles the INTERROGATE command.
- void [ReplyHandler](#) ([StatusCode](#) *status)
Handles the REPLY command from a peer.
- void [SendHandler](#) (uint8_t slot, [StatusCode](#) *status)
Begins the SEND command logic (HSM to HSM transfer).
- void [ReceiveHandler](#) (uint8_t slot, [StatusCode](#) *status)
Begins the RECEIVE command logic (HSM to HSM transfer).

5.7.1 Detailed Description

Command Handlers for Host and Peer interactions.

Author

Sumedh Girish

Contains the business logic for all implemented protocol commands. These functions parse decrypted command structures and invoke the necessary filesystem or crypto routines.

Definition in file [handler.h](#).

5.7.2 Function Documentation

5.7.2.1 InterrogateHandler()

```
void InterrogateHandler (  
    StatusCode * status)
```

Handles the INTERROGATE command.

Handles the INTERROGATE command.

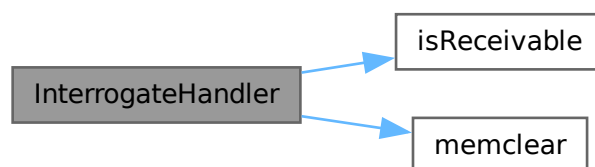
Decrypts a filesystem allocation table (FAT) sent by a peer to determine which files the peer holds that this HSM is authorized to RECEIVE. Filters the FAT array in-place.

Definition at line [147](#) of file [handler.c](#).

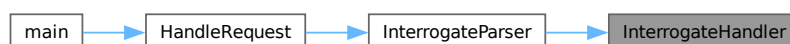
References [DECRYPTIONERROR](#), [isReceivable\(\)](#), [memset\(\)](#), [OPINTERROGATE](#), and [stage](#).

Referenced by [InterrogateParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.2 ListHandler()

```
void ListHandler (  
    StatusCode * status)
```

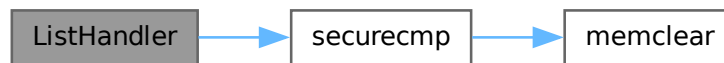
Handles the LIST command.

Definition at line 19 of file [handler.c](#).

References [FAT_Table](#), [Metadata_Table](#), [NUM_SLOTS](#), [OPLIST](#), [securecmp\(\)](#), and [stage](#).

Referenced by [ListParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.3 ReadHandler()

```
void ReadHandler (  
    uint8\_t slot,  
    StatusCode * status)
```

Handles the READ command for a specific slot.

Handles the READ command for a specific slot.

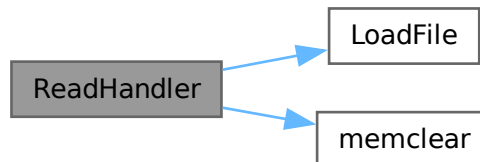
1. Loads the target slot into the volatile [stage](#).
2. Derives the unique read key for the file's owner group.
3. In-place decrypts the file payload using ASCON-128a.

Definition at line 51 of file [handler.c](#).

References [DECRYPTIONERROR](#), [LoadFile\(\)](#), [MAX_FILE_SIZE](#), [memset\(\)](#), [OPREAD](#), and [stage](#).

Referenced by [ReadParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.4 ReceiveHandler()

```
void ReceiveHandler (  
    uint8_t slot,  
    StatusCode * status)
```

Begins the RECEIVE command logic (HSM to HSM transfer).

Begins the RECEIVE command logic (HSM to HSM transfer).

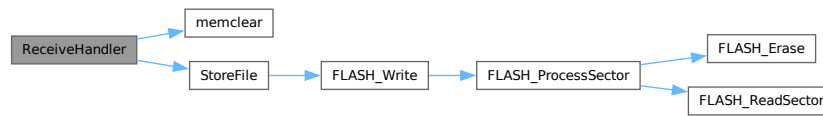
Decrypts the metadata of an incoming file transfer using the inter-HSM RECEIVE key. On success, commits the newly received file to flash.

Definition at line 294 of file [handler.c](#).

References [DECRYPTIONERROR](#), [memset\(\)](#), [OPRECEIVE](#), [stage](#), and [StoreFile\(\)](#).

Referenced by [ReceiveParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.5 ReplyHandler()

```
void ReplyHandler (
    StatusCode * status)
```

Handles the REPLY command from a peer.

Handles the REPLY command from a peer.

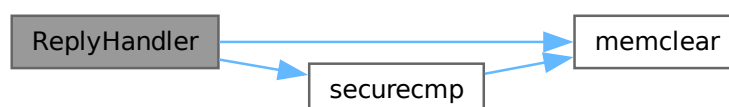
Constructs a FAT table containing all valid files currently held by this HSM. Encrypts the table with the secure Interrogate key before transmission.

Definition at line 195 of file [handler.c](#).

References [ENCRYPTIONERROR](#), [FAT_Table](#), [memclear\(\)](#), [Metadata_Table](#), [NUM_SLOTS](#), [OPREPLY](#), [securecmp\(\)](#), and [stage](#).

Referenced by [ReplyParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.6 SendHandler()

```
void SendHandler (  
    uint8_t slot,  
    StatusCode * status)
```

Begins the SEND command logic (HSM to HSM transfer).

Begins the SEND command logic (HSM to HSM transfer).

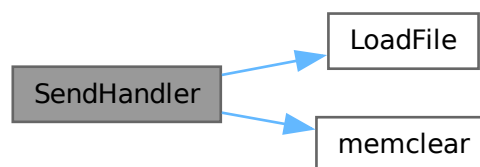
Prepares a file sequence for inter-HSM transfer. Loads the specific slot, re-encrypts the metadata (including the file tag and internal keys) using the inter-HSM SEND key for secure wire transport.

Definition at line 251 of file [handler.c](#).

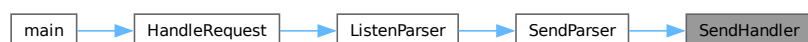
References [ENCRYPTIONERROR](#), [LoadFile\(\)](#), [memset\(\)](#), [OPSEND](#), and [stage](#).

Referenced by [SendParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.7 WriteHandler()

```
void WriteHandler (
    uint8_t slot,
    StatusCode * status)
```

Handles the WRITE command for a specific slot.

Handles the WRITE command for a specific slot.

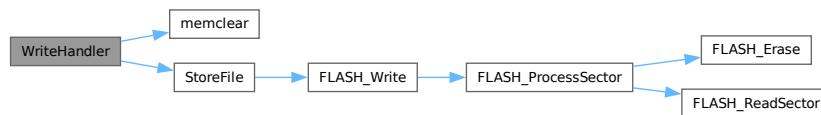
1. Derives the unique write key for the targeted group ID.
2. In-place encrypts the staged plaintext using ASCON-128a.
3. Triggers a commit to persistent flash storage.

Definition at line 95 of file [handler.c](#).

References [ENCRYPTIONERROR](#), [MAX_FILE_SIZE](#), [memset\(\)](#), [OPWRITE](#), [stage](#), and [StoreFile\(\)](#).

Referenced by [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.8 handler.h

[Go to the documentation of this file.](#)

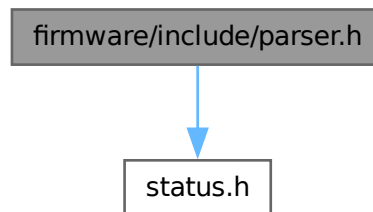
```
00001
00010
00011 #ifndef __HANDLER_H__
00012 #define __HANDLER_H__
00013
00014 #include "status.h"
00015 #include <stdint.h>
00016
00018 void ListHandler(StatusCode *status);
00019
00021 void ReadHandler(uint8_t slot, StatusCode *status);
00022
00024 void WriteHandler(uint8_t slot, StatusCode *status);
00025
00027 void InterrogateHandler(StatusCode *status);
00028
00030 void ReplyHandler(StatusCode *status);
00031
00033 void SendHandler(uint8_t slot, StatusCode *status);
00034
00036 void ReceiveHandler(uint8_t slot, StatusCode *status);
00037
00038 #endif
```

5.9 firmware/include/parser.h File Reference

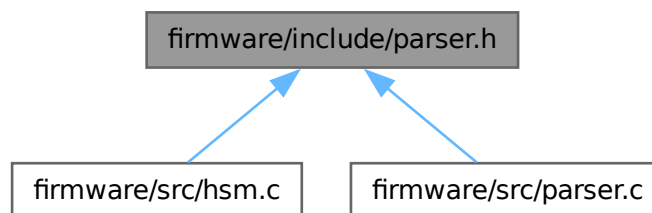
Command Parsing and Validation layer.

```
#include "status.h"
```

Include dependency graph for parser.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [ListParser](#) ([StatusCode](#) *status)
Parses and validates a LIST command frame.
- void [ReadParser](#) ([StatusCode](#) *status)
Parses and validates a READ command frame.
- void [WriteParser](#) ([StatusCode](#) *status)
Parses and validates a WRITE command frame.
- void [InterrogateParser](#) ([StatusCode](#) *status)
Parses and validates an INTERROGATE command frame.
- void [ReceiveParser](#) ([StatusCode](#) *status)
Parses and validates a RECEIVE command frame.
- void [ListenParser](#) ([StatusCode](#) *status)
Parses and validates a LISTEN command frame.

5.9.1 Detailed Description

Command Parsing and Validation layer.

Author

Sumedh Girish

Implements the protocol decoding logic. Reads raw frames from the UART host interface, validates their syntax and payload sizes, and forwards them to the appropriately handlers.

Definition in file [parser.h](#).

5.9.2 Function Documentation

5.9.2.1 InterrogateParser()

```
void InterrogateParser (
    StatusCode * status)
```

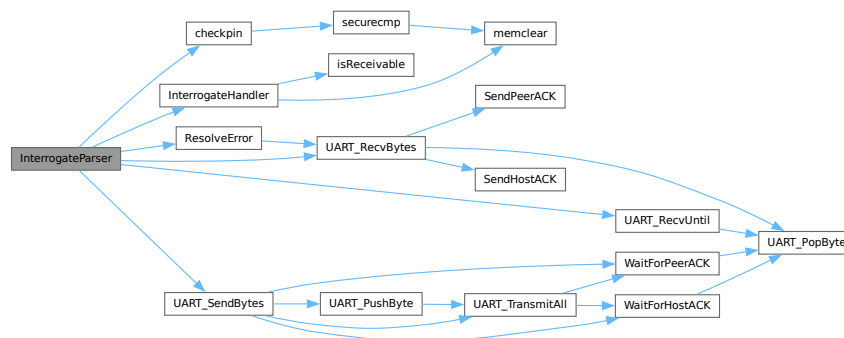
Parses and validates an INTERROGATE command frame.

Definition at line 150 of file [parser.c](#).

References [checkpin\(\)](#), [InterrogateHandler\(\)](#), [INVALIDBODYSIZE](#), [OP_ERROR](#), [OP_INTERROGATE](#), [OPINTERROGATE](#), [peer_magic_byte](#), [PERMISSIONERROR](#), [ResolveError\(\)](#), [stage](#), [UART_HOST](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UART_SendBytes\(\)](#), and [UNKNOWNOP](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.2 ListenParser()

```
void ListenParser (
    StatusCode * status)
```

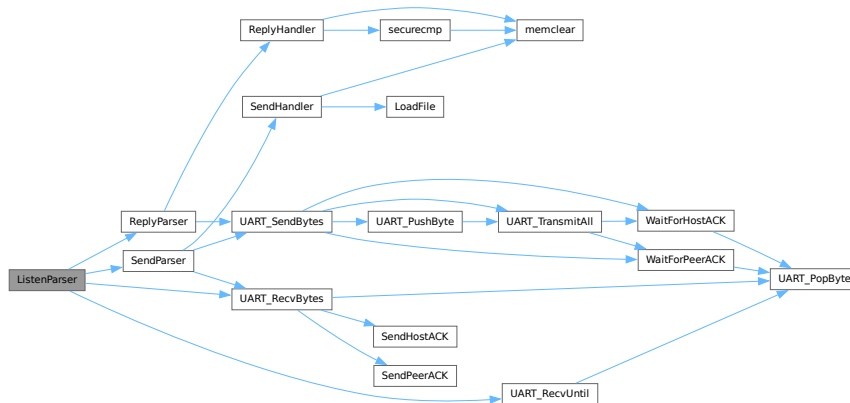
Parses and validates a LISTEN command frame.

Definition at line 364 of file [parser.c](#).

References [INVALIDBODYSIZE](#), [OP_INTERROGATE](#), [OP_RECEIVE](#), [OPREPLY](#), [OPSEND](#), [peer_magic_byte](#), [ReplyParser\(\)](#), [SendParser\(\)](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), and [UNKNOWNOP](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.3 ListParser()

```
void ListParser (
    StatusCode * status)
```

Parses and validates a LIST command frame.

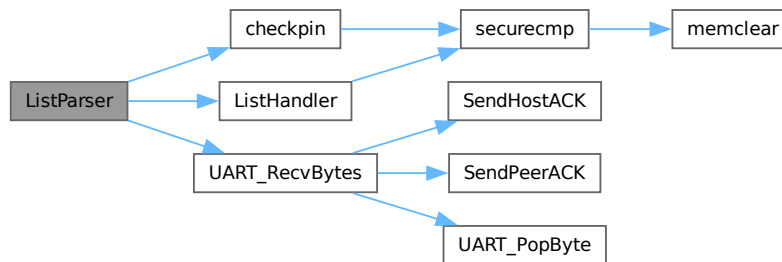
Parses and validates a LIST command frame.

Definition at line 27 of file [parser.c](#).

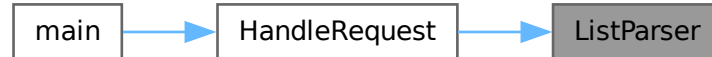
References [checkpin\(\)](#), [ListHandler\(\)](#), [PERMISSIONERROR](#), [UART_HOST](#), and [UART_RecvBytes\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.4 ReadParser()

```
void ReadParser (
    StatusCode * status)
```

Parses and validates a READ command frame.

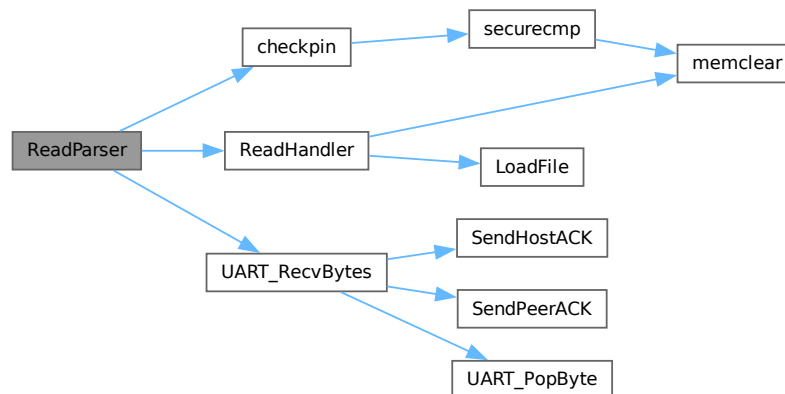
Parses and validates a READ command frame.

Definition at line 44 of file [parser.c](#).

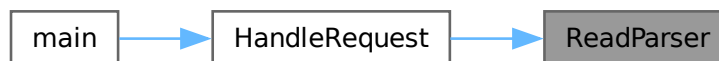
References [checkpin\(\)](#), [INVALIDSLOT](#), [NUM_SLOTS](#), [PERMISSIONERROR](#), [ReadHandler\(\)](#), [UART_HOST](#), and [UART_RecvBytes\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.5 ReceiveParser()

```
void ReceiveParser (
    StatusCode * status)
```

Parses and validates a RECEIVE command frame.

Parses and validates a RECEIVE command frame.

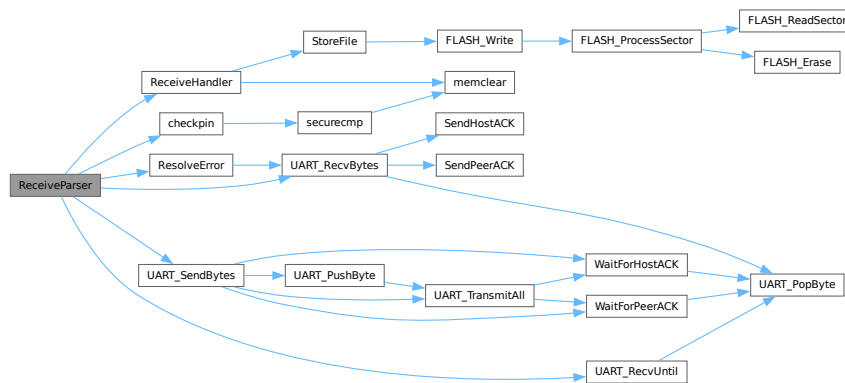
Reads metadata and the encrypted file chunk from the peer link, syncing it into the `stage` buffer before requesting handler decryption and storage.

Definition at line 219 of file `parser.c`.

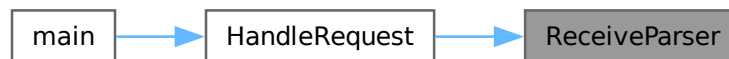
References `checkpin()`, `INVALIDBODYSIZE`, `INVALIDSLOT`, `NUM_SLOTS`, `OP_ERROR`, `OP_RECEIVE`, `OPRECEIVE`, `peer_magic_byte`, `PERMISSIONERROR`, `ReceiveHandler()`, `ResolveError()`, `stage`, `UART_HOST`, `UART_PEER`, `UART_RecvBytes()`, `UART_RecvUntil()`, `UART_SendBytes()`, and `UNKNOWNOP`.

Referenced by `HandleRequest()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.2.6 WriteParser()

```
void WriteParser (
    StatusCode * status)
```

Parses and validates a WRITE command frame.

Parses and validates a WRITE command frame.

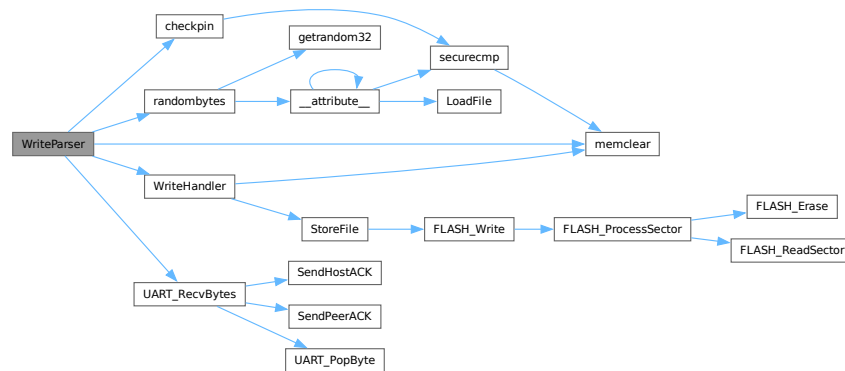
Consumes the PIN, target slot, group assignment, filename, and payload. If verification fails natively, the UART is drained to maintain synchronization.

Definition at line 73 of file [parser.c](#).

References [checkpin\(\)](#), [INVALIDBODYSIZE](#), [INVALIDSLOT](#), [MAX_FILE_SIZE](#), [memclear\(\)](#), [NUM_SLOTS](#), [PERMISSIONERROR](#), [randombytes\(\)](#), [stage](#), [UART_HOST](#), [UART_RecvBytes\(\)](#), and [WriteHandler\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.10 parser.h

[Go to the documentation of this file.](#)

```

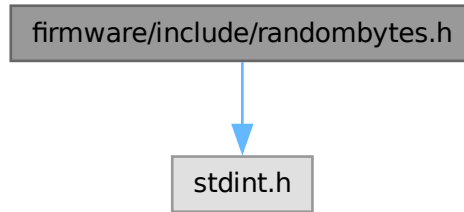
00001
00010
00011 #ifndef __PARSER_H__
00012 #define __PARSER_H__
00013
00014 #include "status.h"
00015
00017 void ListParser(StatusCode *status);
00018
00020 void ReadParser(StatusCode *status);
00021
00023 void WriteParser(StatusCode *status);
00024
00026 void InterrogateParser(StatusCode *status);
00027
00029 void ReceiveParser(StatusCode *status);
00030
00032 void ListenParser(StatusCode *status);
00033
00034 #endif
  
```

5.11 firmware/include/randombytes.h File Reference

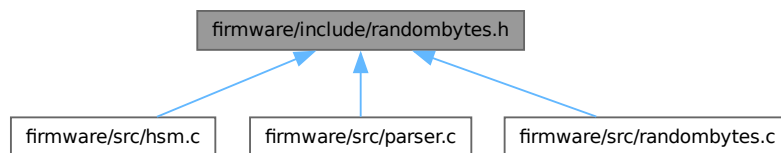
Hardware-backed Random Number Generation.

```
#include <stdint.h>
```

Include dependency graph for randombytes.h:



This graph shows which files directly or indirectly include this file:



Functions

- `uint32_t` [getrandom32](#) (void)
Retrieves a 32-bit random word from the hardware TRNG.
- `void` [randombytes](#) (uint8_t *buf, uint64_t len)
Fills a buffer with random bytes.

5.11.1 Detailed Description

Hardware-backed Random Number Generation.

Author

Sumedh Girish

Provides an interface for the MSPM0 True Random Number Generator (TRNG) peripheral. Used for generating nonces, session keys, and randomized filesystem metadata.

Definition in file [randombytes.h](#).

5.11.2 Function Documentation

5.11.2.1 getRandom32()

```
uint32_t getRandom32 (  
    void )
```

Retrieves a 32-bit random word from the hardware TRNG.

Blocks indefinitely until the TRNG capture register is ready with new entropy. The TRNG must be properly initialized in the system configuration.

Returns

uint32_t A fresh random 32-bit unsigned integer.

Retrieves a 32-bit random word from the hardware TRNG.

Blocks execution until the TRNG capture register is ready.

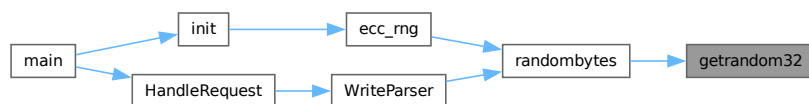
Returns

uint32_t A true random 32-bit word.

Definition at line 21 of file [randombytes.c](#).

Referenced by [randombytes\(\)](#).

Here is the caller graph for this function:



5.11.2.2 randombytes()

```
void randombytes (  
    uint8_t * buf,  
    uint64_t len)
```

Fills a buffer with random bytes.

Populates the destination buffer using entropy from the 32-bit hardware TRNG. Handles fractional words and byte-level alignment internally to ensure the entire buffer is filled.

Parameters

<i>buf</i>	Destination buffer to fill with random data.
<i>len</i>	Total number of bytes to generate (up to $2^{64}-1$).

Fills a buffer with random bytes.

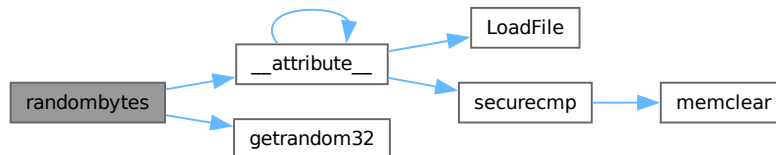
Utilizes [getrandom32\(\)](#) internally and handles byte alignment.

Definition at line 34 of file [randombytes.c](#).

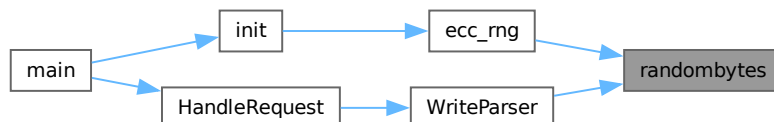
References [__attribute__\(\)](#), and [getrandom32\(\)](#).

Referenced by [ecc_rng\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.12 randombytes.h

[Go to the documentation of this file.](#)

```

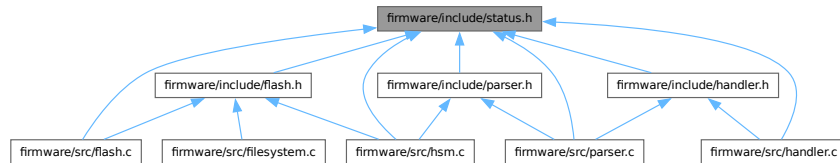
00001
00010
00011 #ifndef __RANDOMBYTES_H__
00012 #define __RANDOMBYTES_H__
00013
00014 #include <stdint.h>
00015
00024 uint32_t getrandom32(void);
00025
00036 void randombytes(uint8_t *buf, uint64_t len);
00037
00038 #endif /* __RANDOMBYTES_H__ */

```


5.13 firmware/include/status.h File Reference

Global Status Codes and Operations.

This graph shows which files directly or indirectly include this file:



Enumerations

- enum `StatusCode` {
`OPLIST` , `OPWRITE` , `OPREAD` , `OPSEND` ,
`OPRECEIVE` , `OPINTERROGATE` , `OPREPLY` , `OPLISTEN` ,
`UNKNOWNOP` , `INVALIDBODYSIZE` , `INVALIDSLOT` , `PERMISSIONERROR` ,
`KEYGENERERROR` , `SLOTEEMPTY` , `FLASHERASEERROR` , `FLASHWRITEERROR` ,
`DECRYPTIONERROR` , `ENCRYPTIONERROR` , `PEERERROR` }

Represents the active operation or a terminal error state.

5.13.1 Detailed Description

Global Status Codes and Operations.

Author

Sumedh Girish

Defines the canonical `StatusCode` enumeration used throughout the firmware to track the current operation type and report fatal exceptions.

Definition in file `status.h`.

5.13.2 Enumeration Type Documentation

5.13.2.1 StatusCode

enum `StatusCode`

Represents the active operation or a terminal error state.

Enumerator

OPLIST	Execution of a LIST operation.
OPWRITE	Execution of a WRITE operation.
OPREAD	Execution of a READ operation.
OPSEND	Execution of a peer SEND.
OPRECEIVE	Execution of a peer RECEIVE.
OPINTERROGATE	Execution of a peer INTERROGATE.
OPREPLY	Execution of a peer REPLY.
OPLISTEN	Waiting for peer provisioning.
UNKNOWNOP	Unrecognized command opcode received.
INVALIDBODYSIZE	Payload size does not match expected bounds.
INVALIDSLOT	Requested slot ID is out of range.
PERMISSIONERROR	Access denied due to key mismatch.
KEYGENERERROR	Failure during cryptographic key generation.
SLOTEEMPTY	Target slot contains no data.
FLASHERASEERROR	Hard fault during flash sector erase.
FLASHWRITEERROR	Hard fault or misalignment during flash write.
DECRYPTIONERROR	Payload authentication/decryption failed.
ENCRYPTIONERROR	Payload encryption failed.
PEERERROR	Peer HSM responded with an error or timeout.

Definition at line 16 of file [status.h](#).

5.14 status.h

[Go to the documentation of this file.](#)

```

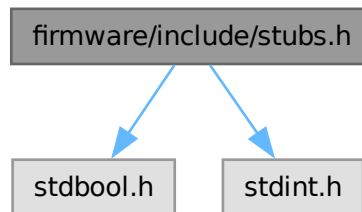
00001
00009
00010 #ifndef __STATUS_H__
00011 #define __STATUS_H__
00012
00016 typedef enum
00017 {
00018     // --- Operations ---
00019     OPLIST,
00020     OPWRITE,
00021     OPREAD,
00022     OPSEND,
00023     OPRECEIVE,
00024     OPINTERROGATE,
00025     OPREPLY,
00026     OPLISTEN,
00027
00028     // --- Exceptions ---
00029     UNKNOWNOP,
00030     INVALIDBODYSIZE,
00031     INVALIDSLOT,
00032     PERMISSIONERROR,
00033     KEYGENERERROR,
00034     SLOTEEMPTY,
00035     FLASHERASEERROR,
00036     FLASHWRITEERROR,
00037     DECRYPTIONERROR,
00038     ENCRYPTIONERROR,
00039     PEERERROR,
00040 } StatusCode;
00041
00042 #endif

```

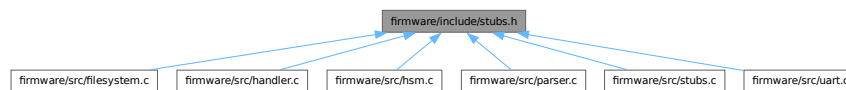
5.15 firmware/include/stubs.h File Reference

Secure Memory and Timing-Safe Operations.

```
#include <stdbool.h>
#include <stdint.h>
Include dependency graph for stubs.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- bool [securecmp](#) (const uint8_t *a, const uint8_t *b, uint32_t size)
Secure memory comparison using domain-separated hashing.
- bool [checkpin](#) (uint8_t inputPin[6])
Securely validates a user-provided PIN against the stored system PIN.
- void [memclear](#) (void *v, uint32_t n, uint8_t value)
Securely clears memory context using volatile-safe operations.

5.15.1 Detailed Description

Secure Memory and Timing-Safe Operations.

Author

Sumedh Girish

Provides utility functions for secure memory management and timing-safe comparisons to protect against side-channel attacks. These routines are critical for preventing information leakage during cryptographic operations.

Definition in file [stubs.h](#).

5.15.2 Function Documentation

5.15.2.1 `checkpin()`

```
bool checkpin (
    uint8_t inputPin[6])
```

Securely validates a user-provided PIN against the stored system PIN.

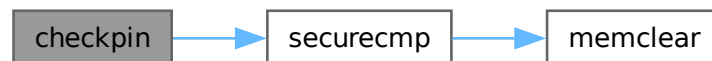
Executes both a standard memory comparison and a secure, constant-time hash-based comparison ([securecmp\(\)](#)). Any discrepancy immediately corrupts the timeout token in flash memory to trip the internal trap system, permanently mitigating brute force attempts.

Definition at line 53 of file [stubs.c](#).

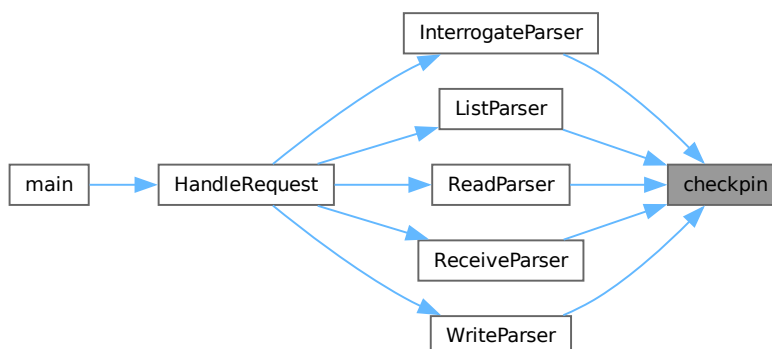
References [securecmp\(\)](#), and [SystemStatus](#).

Referenced by [InterrogateParser\(\)](#), [ListParser\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.15.2.2 memclear()

```
void memclear (
    void * v,
    uint32_t n,
    uint8_t value)
```

Securely clears memory context using volatile-safe operations.

Implements a robust and high-performance memory clearing strategy:

1. Byte-by-byte head cleanup until word alignment is reached.
2. High-speed 32-bit word-level clearing for the aligned core.
3. Byte-by-byte tail cleanup for any remaining bytes.

This function uses 'volatile' pointers to ensure that the compiler does NOT optimize away the memory write (which often happens with standard memset when the buffer is about to be freed or go out of scope), which is essential for ensuring sensitive keys are actually wiped from SRAM.

Parameters

<i>v</i>	Pointer to the memory region to clear.
<i>n</i>	Length of the memory region in bytes.
<i>value</i>	Byte value to fill (typically 0x00).

Securely clears memory context using volatile-safe operations.

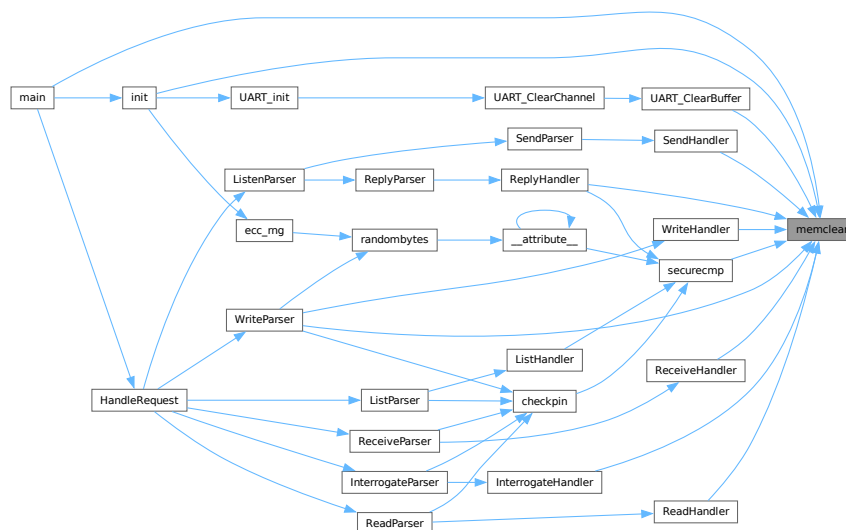
Standard memset() may be optimized away by compilers if the buffer is not read again before it goes out of scope. [memclear\(\)](#) uses volatile pointers and a three-stage clear (alignment, word-clear, tail-clear) to ensure that the memory is actually modified in SRAM, protecting against data remanence.

Definition at line 81 of file [stubs.c](#).

References [RAMFUNC](#).

Referenced by [init\(\)](#), [InterrogateHandler\(\)](#), [main\(\)](#), [ReadHandler\(\)](#), [ReceiveHandler\(\)](#), [ReplyHandler\(\)](#), [securecmp\(\)](#), [SendHandler\(\)](#), [UART_ClearBuffer\(\)](#), [WriteHandler\(\)](#), and [WriteParser\(\)](#).

Here is the caller graph for this function:



5.15.2.3 `securecmp()`

```
bool securecmp (  
    const uint8_t * a,  
    const uint8_t * b,  
    uint32_t size)
```

Secure memory comparison using domain-separated hashing.

Instead of a direct byte-by-byte comparison (which is susceptible to timing attacks), this function hashes both inputs using Ascon-XOF and compares the resulting fixed-length digests. This ensures that the execution time is independent of how many prefix bytes match, providing strong protection against timing-based side-channel analysis.

Parameters

<i>a</i>	First memory block to compare.
<i>b</i>	Second memory block to compare.
<i>size</i>	Number of bytes to compare from each block.

Returns

bool true if the digests (and thus the original data) are identical.

Secure memory comparison using domain-separated hashing.

Instead of comparing bytes directly (which is vulnerable to timing attacks), this function hashes both inputs into a temporary buffer and then performs a bitwise accumulated XOR comparison on the hashes.

Definition at line 27 of file [stubs.c](#).

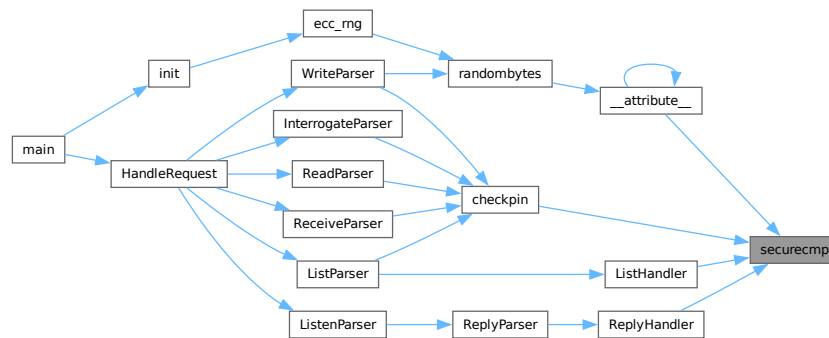
References [memclear\(\)](#).

Referenced by [__attribute__\(\)](#), [checkpin\(\)](#), [ListHandler\(\)](#), and [ReplyHandler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.16 stubs.h

[Go to the documentation of this file.](#)

```

00001
00010
00011 #ifndef __STUBS_H__
00012 #define __STUBS_H__
00013
00014 #include <stdbool.h>
00015 #include <stdint.h>
00016
00031 bool securecmp(const uint8_t *a, const uint8_t *b, uint32_t size);
00032
00033 bool checkpin(uint8_t inputPin[6]);
00034
00052 void memclear(void *v, uint32_t n, uint8_t value);
00053
00054 #endif /* __STUBS_H__ */

```

5.17 firmware/include/uart.h File Reference

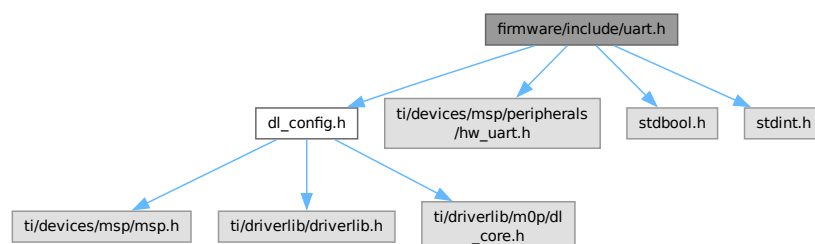
UART Ring-Buffer Hardware Abstraction.

```

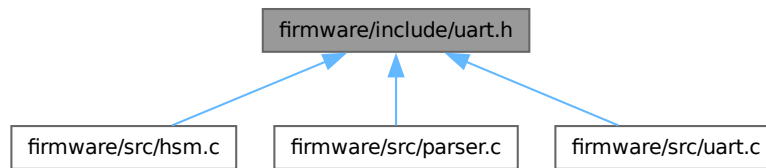
#include "dl_config.h"
#include "ti/devices/msp/peripherals/hw_uart.h"
#include <stdbool.h>
#include <stdint.h>

```

Include dependency graph for uart.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [UART_RingBuffer](#)
Circular ring buffer control structure.
- struct [UART_Channel](#)
A complete bidirectional UART channel state.

Macros

- `#define` [UART_BUFFER_SIZE](#) 256
Dimension of the RX/TX circular ring buffers.
- `#define` [MAX_DRAIN_SIZE](#) 4
Maximum bytes to drain per ISR execution.
- `#define` [UART_HOST](#) [UART_0_INST](#)
- `#define` [UART_PEER](#) [UART_1_INST](#)

Functions

- void [UART0_IRQHandler](#) (void)
- void [UART1_IRQHandler](#) (void)
- void [UART_init](#) (void)
Initializes the software buffers for all UART channels.
- void [UART_RecvUntil](#) (UART_Regs *channel, uint8_t chr)
Blocks until a specific character is received.
- void [UART_RecvBytes](#) (UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
Synchronously receives a fixed number of bytes.
- void [UART_SendBytes](#) (UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
Synchronously transmits a fixed number of bytes.

Variables

- volatile [UART_Channel](#) [host](#)
- volatile [UART_Channel](#) [peer](#)

5.17.1 Detailed Description

UART Ring-Buffer Hardware Abstraction.

Author

Sumedh Girish

Provides a non-blocking, interrupt-driven UART driver using dual ring buffers (RX/TX) for host and peer communications.

Definition in file [uart.h](#).

5.17.2 Macro Definition Documentation

5.17.2.1 MAX_DRAIN_SIZE

```
#define MAX_DRAIN_SIZE 4
```

Maximum bytes to drain per ISR execution.

Definition at line 47 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), and [IRQ_Handler\(\)](#).

5.17.2.2 UART_BUFFER_SIZE

```
#define UART_BUFFER_SIZE 256
```

Dimension of the RX/TX circular ring buffers.

Definition at line 19 of file [uart.h](#).

Referenced by [DrainPushBytes\(\)](#), [UART_ClearBuffer\(\)](#), [UART_PopByte\(\)](#), [UART_PushByte\(\)](#), [UART_RecvBytes\(\)](#), and [UART_TransmitAll\(\)](#).

5.17.2.3 UART_HOST

```
#define UART_HOST UART_0_INST
```

Definition at line 52 of file [uart.h](#).

Referenced by [HandleRequest\(\)](#), [InterrogateParser\(\)](#), [ListParser\(\)](#), [main\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), [SendError\(\)](#), [SendHeader\(\)](#), [SendHostACK\(\)](#), [SendResponse\(\)](#), [UART_PopByte\(\)](#), [UART_PushByte\(\)](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UART_SendBytes\(\)](#), [UART_TransmitAll\(\)](#), [WaitForHostACK\(\)](#), and [WriteParser\(\)](#).

5.17.2.4 UART_PEER

```
#define UART_PEER UART_1_INST
```

Definition at line 53 of file [uart.h](#).

Referenced by [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ReceiveParser\(\)](#), [ReplyParser\(\)](#), [ResolveError\(\)](#), [SendParser\(\)](#), [SendPeerACK\(\)](#), and [WaitForPeerACK\(\)](#).

5.17.3 Function Documentation

5.17.3.1 UART0_IRQHandler()

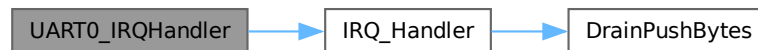
```
void UART0_IRQHandler (
    void ) [extern]
```

Definition at line 261 of file [uart.c](#).

References [host](#), [IRQ_Handler\(\)](#), and [UART_0_INST](#).

Referenced by [Default_Handler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.3.2 UART1_IRQHandler()

```
void UART1_IRQHandler (
    void ) [extern]
```

Definition at line 266 of file [uart.c](#).

References [IRQ_Handler\(\)](#), [peer](#), and [UART_1_INST](#).

Referenced by [Default_Handler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.3.3 UART_init()

```
void UART_init (
    void )
```

Initializes the software buffers for all UART channels.

Definition at line 234 of file `uart.c`.

References [host](#), [peer](#), and [UART_ClearChannel\(\)](#).

Referenced by [init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.3.4 UART_RecvBytes()

```
void UART_RecvBytes (
    UART_Regs * channel,
    uint8_t * out,
    uint32_t length,
    bool eof)
```

Synchronously receives a fixed number of bytes.

Parameters

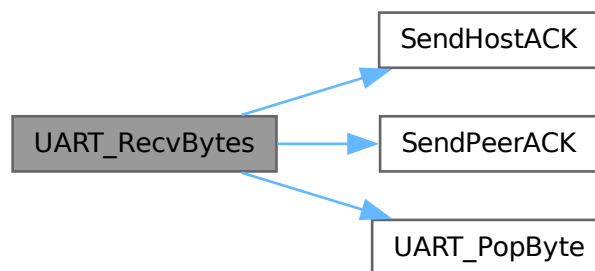
<i>channel</i>	The UART controller register base.
<i>out</i>	Destination buffer for the received bytes.
<i>length</i>	Exact number of bytes to wait for.
<i>eof</i>	If true, expects an EOF marker after the payload.

Definition at line 144 of file [uart.c](#).

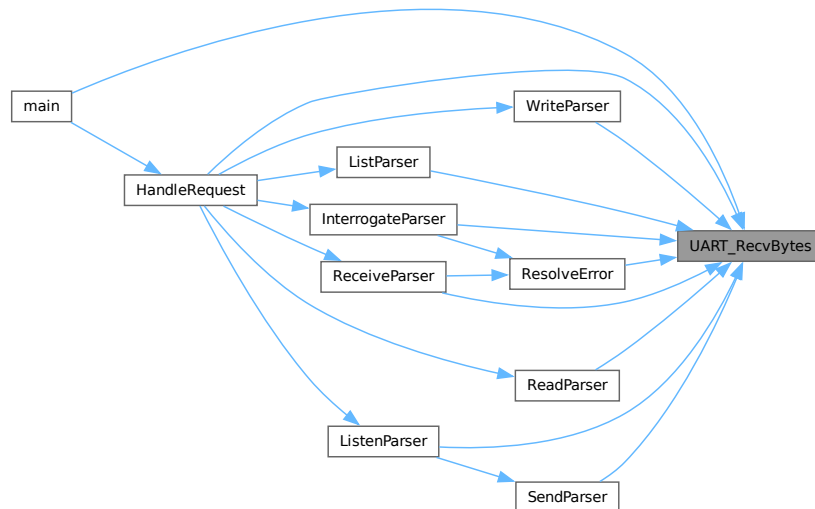
References [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_Channel::rx](#), [SendHostACK\(\)](#), [SendPeerACK\(\)](#), [UART_BUFFER_SIZE](#), [UART_HOST](#), and [UART_PopByte\(\)](#).

Referenced by [HandleRequest\(\)](#), [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ListParser\(\)](#), [main\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), [ResolveError\(\)](#), [SendParser\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.3.5 UART_RecvUntil()

```
void UART_RecvUntil (
    UART_Regs * channel,
    uint8_t chr)
```

Blocks until a specific character is received.

Parameters

<i>channel</i>	The UART controller register base (e.g. UART_HOST).
<i>chr</i>	The character to block for.

Definition at line 132 of file [uart.c](#).

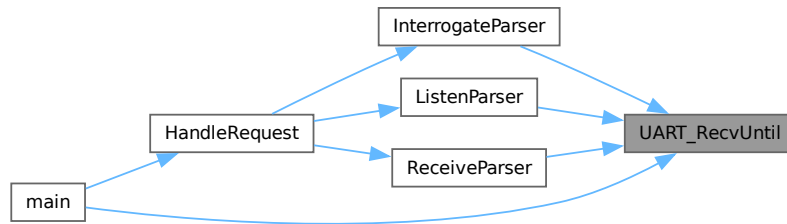
References [UART_RingBuffer::count](#), [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_HOST](#), and [UART_PopByte\(\)](#).

Referenced by [InterrogateParser\(\)](#), [ListenParser\(\)](#), [main\(\)](#), and [ReceiveParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.3.6 UART_SendBytes()

```

void UART_SendBytes (
    UART_Regs * channel,
    uint8_t * out,
    uint32_t length,
    bool eof)
  
```

Synchronously transmits a fixed number of bytes.

Parameters

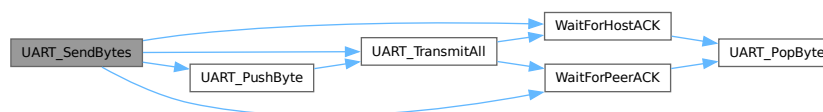
<i>channel</i>	The UART controller register base.
<i>out</i>	Source buffer to transmit.
<i>length</i>	Exact number of bytes to send.
<i>eof</i>	If true, appends an EOF marker after transmission.

Definition at line 178 of file [uart.c](#).

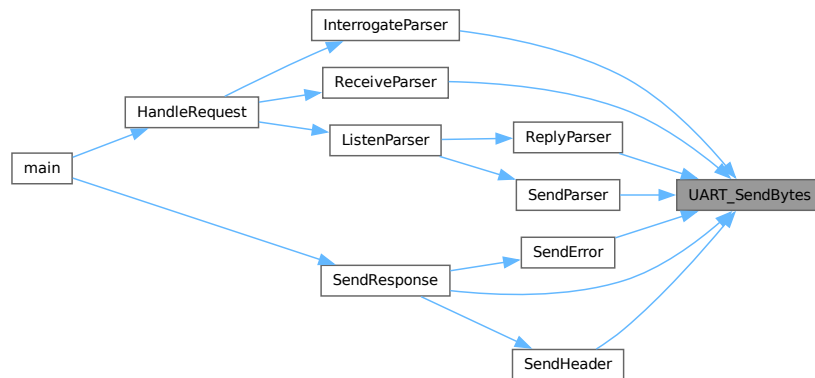
References [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_Channel::tx](#), [UART_HOST](#), [UART_PushByte\(\)](#), [UART_TransmitAll\(\)](#), [WaitForHostACK\(\)](#), and [WaitForPeerACK\(\)](#).

Referenced by [InterrogateParser\(\)](#), [ReceiveParser\(\)](#), [ReplyParser\(\)](#), [SendError\(\)](#), [SendHeader\(\)](#), [SendParser\(\)](#), and [SendResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.4 Variable Documentation

5.17.4.1 host

```
volatile UART_Channel host [extern]
```

Definition at line 18 of file `uart.c`.

Referenced by `UART0_IRQHandler()`, `UART_init()`, `UART_PopByte()`, `UART_PushByte()`, `UART_RcvBytes()`, `UART_RcvUntil()`, `UART_SendBytes()`, and `UART_TransmitAll()`.

5.17.4.2 peer

```
volatile UART_Channel peer [extern]
```

Definition at line 19 of file `uart.c`.

Referenced by `UART1_IRQHandler()`, `UART_init()`, `UART_PopByte()`, `UART_PushByte()`, `UART_RcvBytes()`, `UART_RcvUntil()`, `UART_SendBytes()`, and `UART_TransmitAll()`.

5.18 uart.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #ifndef __UART_H__
00011 #define __UART_H__
00012
00013 #include "dl_config.h"
00014 #include "ti/devices/msp/peripherals/hw_uart.h"
00015 #include <stdbool.h>
00016 #include <stdint.h>
00017
00019 #define UART_BUFFER_SIZE 256
00020

```

```

00024 typedef struct
00025 {
00026     uint8_t data[UART_BUFFER_SIZE];
00027     uint8_t head;
00028     uint8_t tail;
00029     uint16_t count;
00030     uint16_t progress;
00031     bool dirty;
00032 } UART_RingBuffer;
00033
00037 typedef struct
00038 {
00039     UART_RingBuffer rx;
00040     UART_RingBuffer tx;
00041 } UART_Channel;
00042
00043 extern volatile UART_Channel host;
00044 extern volatile UART_Channel peer;
00045
00047 #define MAX_DRAIN_SIZE 4
00048
00049 extern void UART0_IRQHandler(void);
00050 extern void UART1_IRQHandler(void);
00051
00052 #define UART_HOST UART_0_INST
00053 #define UART_PEER UART_1_INST
00054
00058 void UART_init(void);
00059
00066 void UART_RecvUntil(UART_Regs *channel, uint8_t chr);
00067
00076 void UART_RecvBytes(UART_Regs *channel, uint8_t *out, uint32_t length,
00077                     bool eof);
00078
00087 void UART_SendBytes(UART_Regs *channel, uint8_t *out, uint32_t length,
00088                     bool eof);
00089
00090 #endif

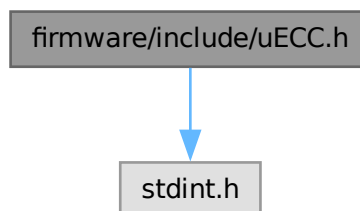
```

5.19 firmware/include/uECC.h File Reference

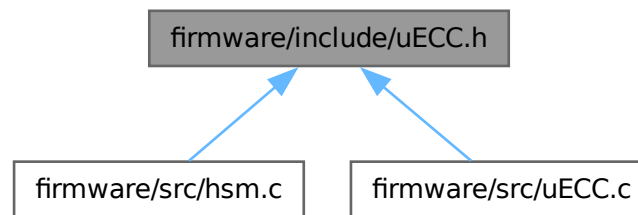
Micro-ECC Library for ECDH and ECDSA.

```
#include <stdint.h>
```

Include dependency graph for uECC.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [uECC_HashContext](#)
Interface for passing arbitrary hash functions to uECC.

Macros

- `#define uECC_arch_other 0`
Other architecture selection value.
- `#define uECC_x86 1`
x86 architecture selection value.
- `#define uECC_x86_64 2`
x86_64 architecture selection value.
- `#define uECC_arm 3`
ARM architecture selection value.
- `#define uECC_arm_thumb 4`
ARM Thumb architecture selection value.
- `#define uECC_avr 5`
AVR architecture selection value.
- `#define uECC_arm_thumb2 6`
ARM Thumb-2 architecture selection value.
- `#define uECC_asm_none 0`
Use standard C99 implementation only.
- `#define uECC_asm_small 1`
Use GCC inline assembly optimized for minimum size.
- `#define uECC_asm_fast 2`
Use GCC inline assembly optimized for maximum speed.
- `#define uECC_ASM uECC_asm_fast`
Selected assembly optimization level.
- `#define uECC_secp160r1 1`
SECP160R1 Curve ID.
- `#define uECC_secp192r1 2`
SECP192R1 Curve ID.
- `#define uECC_secp256r1 3`

- *SECP256R1 (NIST P-256) Curve ID.*
- `#define uECC_secp256k1 4`
SECP256K1 Curve ID.
- `#define uECC_secp224r1 5`
SECP224R1 Curve ID.
- `#define uECC_CURVE uECC_secp256r1`
Selected elliptic curve for the project.
- `#define uECC_SQUARE_FUNC 1`
Enable optimized scalar squaring (8% faster, more code).
- `#define uECC_CONCAT1(a, b)`
- `#define uECC_CONCAT(a, b)`
- `#define uECC_size_1 20`
- `#define uECC_size_2 24`
- `#define uECC_size_3 32`
- `#define uECC_size_4 32`
- `#define uECC_size_5 28`
- `#define uECC_BYTES uECC_CONCAT(uECC_size_, uECC_CURVE)`
Number of bytes required for a single coordinate/scalar on the selected curve.

Typedefs

- `typedef int(* uECC_RNG_Function) (uint8_t *dest, unsigned size)`
RNG function type.
- `typedef struct uECC_HashContext uECC_HashContext`
Interface for passing arbitrary hash functions to uECC.

Functions

- `void uECC_set_rng (uECC_RNG_Function rng_function)`
Sets the function that will be used to generate random bytes.
- `int uECC_make_key (uint8_t public_key[uECC_BYTES *2], uint8_t private_key[uECC_BYTES])`
Create a public/private key pair.
- `int uECC_shared_secret (const uint8_t public_key[uECC_BYTES *2], const uint8_t private_key[uECC_BYTES], uint8_t secret[uECC_BYTES])`
Compute a shared secret given your secret key and someone else's public key (ECDH).
- `int uECC_sign (const uint8_t private_key[uECC_BYTES], const uint8_t message_hash[uECC_BYTES], uint8_t signature[uECC_BYTES *2])`
Generate an ECDSA signature for a given hash value.
- `int uECC_sign_deterministic (const uint8_t private_key[uECC_BYTES], const uint8_t message_hash[uECC_BYTES], uECC_HashContext *hash_context, uint8_t signature[uECC_BYTES *2])`
Generate an ECDSA signature using a deterministic algorithm (RFC 6979).
- `int uECC_verify (const uint8_t public_key[uECC_BYTES *2], const uint8_t hash[uECC_BYTES], const uint8_t signature[uECC_BYTES *2])`
Verify an ECDSA signature against a public key and hash.
- `void uECC_compress (const uint8_t public_key[uECC_BYTES *2], uint8_t compressed[uECC_BYTES+1])`
Compress a public key (Y coordinate is reduced to parity bit).
- `void uECC_decompress (const uint8_t compressed[uECC_BYTES+1], uint8_t public_key[uECC_BYTES *2])`
Decompress a compressed public key.
- `int uECC_valid_public_key (const uint8_t public_key[uECC_BYTES *2])`
Check if a public key is a valid point on the curve.

- int [uECC_compute_public_key](#) (const uint8_t private_key[uECC_BYTES], uint8_t public_key[uECC_BYTES *2])
Compute the corresponding public key for a given private key.
- int [uECC_bytes](#) (void)
Returns the value of uECC_BYTES for the current build.
- int [uECC_curve](#) (void)
Returns the value of uECC_CURVE for the current build.

5.19.1 Detailed Description

Micro-ECC Library for ECDH and ECDSA.

Author

Kenneth MacKay (refined by Sumedh Girish)

This version of Micro-ECC is used for all asymmetric cryptographic operations in the SHIFT firmware, specifically targeting NIST P-256 for secure key exchange and message signing.

Definition in file [uECC.h](#).

5.19.2 Macro Definition Documentation

5.19.2.1 uECC_arch_other

```
#define uECC_arch_other 0
```

Other architecture selection value.

Definition at line 19 of file [uECC.h](#).

5.19.2.2 uECC_arm

```
#define uECC_arm 3
```

ARM architecture selection value.

Definition at line 25 of file [uECC.h](#).

5.19.2.3 uECC_arm_thumb

```
#define uECC_arm_thumb 4
```

ARM Thumb architecture selection value.

Definition at line 27 of file [uECC.h](#).

5.19.2.4 uECC_arm_thumb2

```
#define uECC_arm_thumb2 6
```

ARM Thumb-2 architecture selection value.

Definition at line 31 of file [uECC.h](#).

5.19.2.5 uECC_ASM

```
#define uECC_ASM uECC_asm_fast
```

Selected assembly optimization level.

Definition at line 42 of file [uECC.h](#).

5.19.2.6 uECC_asm_fast

```
#define uECC_asm_fast 2
```

Use GCC inline assembly optimized for maximum speed.

Definition at line 38 of file [uECC.h](#).

5.19.2.7 uECC_asm_none

```
#define uECC_asm_none 0
```

Use standard C99 implementation only.

Definition at line 34 of file [uECC.h](#).

5.19.2.8 uECC_asm_small

```
#define uECC_asm_small 1
```

Use GCC inline assembly optimized for minimum size.

Definition at line 36 of file [uECC.h](#).

5.19.2.9 uECC_avr

```
#define uECC_avr 5
```

AVR architecture selection value.

Definition at line 29 of file [uECC.h](#).

5.19.2.10 uECC_BYTES

```
#define uECC_BYTES uECC_CONCAT(uECC_size_, uECC_CURVE)
```

Number of bytes required for a single coordinate/scalar on the selected curve.

Definition at line 77 of file [uECC.h](#).

Referenced by [uECC_bytes\(\)](#), [uECC_compress\(\)](#), [uECC_compute_public_key\(\)](#), [uECC_decompress\(\)](#), [uECC_make_key\(\)](#), [uECC_set_rng\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign\(\)](#), [uECC_sign_deterministic\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_valid_public_key\(\)](#), and [uECC_verify\(\)](#).

5.19.2.11 uECC_CONCAT

```
#define uECC_CONCAT(  
    a,  
    b)
```

Value:

[uECC_CONCAT1](#)(a, b)

Definition at line 67 of file [uECC.h](#).

5.19.2.12 uECC_CONCAT1

```
#define uECC_CONCAT1(  
    a,  
    b)
```

Value:

a##b

Definition at line 66 of file [uECC.h](#).

5.19.2.13 uECC_CURVE

```
#define uECC_CURVE uECC_secp256r1
```

Selected elliptic curve for the project.

Definition at line 58 of file [uECC.h](#).

Referenced by [uECC_curve\(\)](#).

5.19.2.14 uECC_secp160r1

```
#define uECC_secp160r1 1
```

SECP160R1 Curve ID.

Definition at line 46 of file [uECC.h](#).

5.19.2.15 uECC_secp192r1

```
#define uECC_secp192r1 2
```

SECP192R1 Curve ID.

Definition at line 48 of file [uECC.h](#).

5.19.2.16 uECC_secp224r1

```
#define uECC_secp224r1 5
```

SECP224R1 Curve ID.

Definition at line 54 of file [uECC.h](#).

5.19.2.17 uECC_secp256k1

```
#define uECC_secp256k1 4
```

SECP256K1 Curve ID.

Definition at line 52 of file [uECC.h](#).

5.19.2.18 uECC_secp256r1

```
#define uECC_secp256r1 3
```

SECP256R1 (NIST P-256) Curve ID.

Definition at line 50 of file [uECC.h](#).

5.19.2.19 uECC_size_1

```
#define uECC_size_1 20
```

Byte size for secp160r1.

Definition at line 69 of file [uECC.h](#).

5.19.2.20 uECC_size_2

```
#define uECC_size_2 24
```

Byte size for secp192r1.

Definition at line 70 of file [uECC.h](#).

5.19.2.21 uECC_size_3

```
#define uECC_size_3 32
```

Byte size for secp256r1 (P-256).

Definition at line 71 of file [uECC.h](#).

5.19.2.22 uECC_size_4

```
#define uECC_size_4 32
```

Byte size for secp256k1.

Definition at line 72 of file [uECC.h](#).

5.19.2.23 uECC_size_5

```
#define uECC_size_5 28
```

Byte size for secp224r1.

Definition at line 73 of file [uECC.h](#).

5.19.2.24 uECC_SQUARE_FUNC

```
#define uECC_SQUARE_FUNC 1
```

Enable optimized scalar squaring (8% faster, more code).

Definition at line 63 of file [uECC.h](#).

5.19.2.25 uECC_x86

```
#define uECC_x86 1
```

x86 architecture selection value.

Definition at line 21 of file [uECC.h](#).

5.19.2.26 uECC_x86_64

```
#define uECC_x86_64 2
```

x86_64 architecture selection value.

Definition at line 23 of file [uECC.h](#).

5.19.3 Typedef Documentation

5.19.3.1 uECC_HashContext

```
typedef struct uECC_HashContext uECC_HashContext
```

Interface for passing arbitrary hash functions to uECC.

5.19.3.2 uECC_RNG_Function

```
typedef int(* uECC_RNG_Function) (uint8_t *dest, unsigned size)
```

RNG function type.

The RNG function should fill 'size' random bytes into 'dest'. It should return 1 on success, or 0 on failure.

Definition at line 85 of file [uECC.h](#).

5.19.4 Function Documentation

5.19.4.1 uECC_bytes()

```
int uECC_bytes (
    void )
```

Returns the value of uECC_BYTES for the current build.

Returns

int Current uECC_BYTES value.

Definition at line 2541 of file [uECC.c](#).

References [uECC_BYTES](#).

5.19.4.2 uECC_compress()

```
void uECC_compress (
    const uint8_t public_key[uECC_BYTES *2],
    uint8_t compressed[uECC_BYTES+1])
```

Compress a public key (Y coordinate is reduced to parity bit).

Parameters

in	<i>public_key</i>	The public key to compress.
out	<i>compressed</i>	Will be filled with the compressed key (uECC_BYTES + 1 bytes).

Definition at line 2448 of file [uECC.c](#).

References [uECC_BYTES](#).

5.19.4.3 uECC_compute_public_key()

```
int uECC_compute_public_key (
    const uint8_t private_key[uECC_BYTES],
    uint8_t public_key[uECC_BYTES *2])
```

Compute the corresponding public key for a given private key.

Parameters

in	<i>private_key</i>	The private key.
out	<i>public_key</i>	Will be filled with the corresponding public key.

Returns

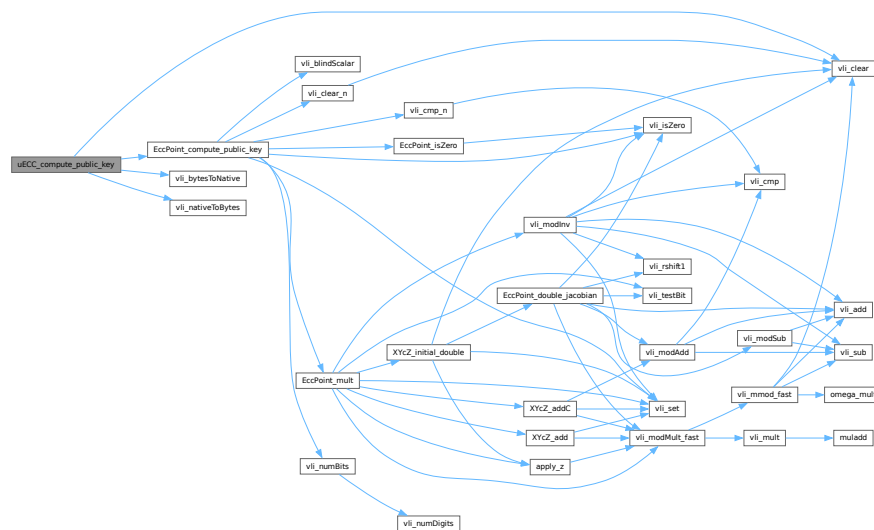
int 1 if the key was computed successfully, 0 otherwise.

Definition at line 2522 of file uECC.c.

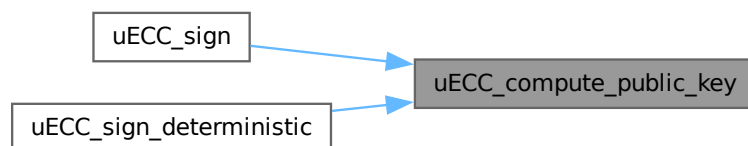
References [EccPoint_compute_public_key\(\)](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), and [vli_nativeToBytes\(\)](#).

Referenced by `uECC_sign()`, and `uECC_sign_deterministic()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.19.4.4 uECC_curve()

```
int uECC_curve (
    void )
```

Returns the value of uECC_CURVE for the current build.

Returns

int Current uECC_CURVE value.

Definition at line 2546 of file [uECC.c](#).

References [uECC_CURVE](#).

5.19.4.5 uECC_decompress()

```
void uECC_decompress (
    const uint8_t compressed[uECC_BYTES+1],
    uint8_t public_key[uECC_BYTES *2])
```

Decompress a compressed public key.

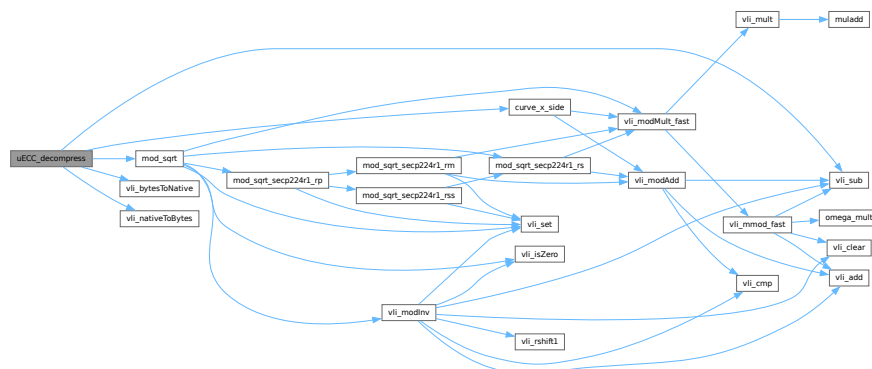
Parameters

in	<i>compressed</i>	The compressed public key.
out	<i>public_key</i>	Will be filled with the decompressed key (2 * uECC_BYTES bytes).

Definition at line 2477 of file [uECC.c](#).

References [curve_p](#), [curve_x_side\(\)](#), [mod_sqrt\(\)](#), [uECC_BYTES](#), [vli_bytesToNative\(\)](#), [vli_nativeToBytes\(\)](#), [vli_sub\(\)](#), [EccPoint::x](#), and [EccPoint::y](#).

Here is the call graph for this function:



5.19.4.6 uECC_make_key()

```
int uECC_make_key (
    uint8_t public_key[uECC_BYTES *2],
    uint8_t private_key[uECC_BYTES])
```

Create a public/private key pair.

Parameters

out	<i>public_key</i>	Will be filled with the public key (2* <code>uECC_BYTES</code> bytes).
out	<i>private_key</i>	Will be filled with the private key (<code>uECC_BYTES</code> bytes).

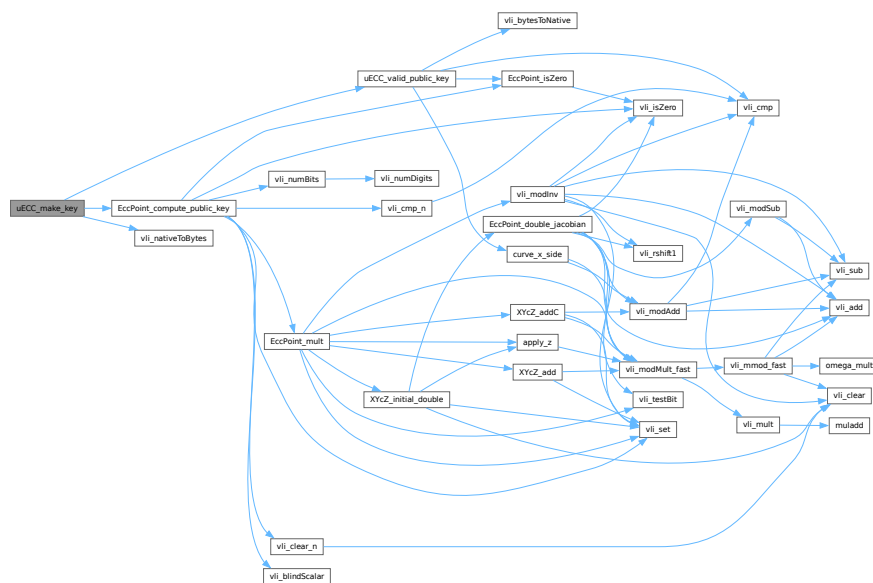
Returns

int 1 if the key pair was generated successfully, 0 otherwise.

Definition at line 2424 of file uECC.c.

References [EccPoint_compute_public_key\(\)](#), [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_valid_public_key\(\)](#), [uECC_WORDS](#), and [vli_nativeToBytes\(\)](#).

Here is the call graph for this function:



5.19.4.7 uECC_set_rng()

```
void uECC_set_rng (
    uECC_RNG_Function rng_function)
```

Sets the function that will be used to generate random bytes.

On embedded platforms, this MUST be called before any key generation or signing if true randomness is required.

Parameters

<i>rng_function</i>	The function that will be used to generate random bytes.
---------------------	--

References [uECC_BYTES](#).

Referenced by [init\(\)](#).

Here is the caller graph for this function:



5.19.4.8 uECC_shared_secret()

```

int uECC_shared_secret (
    const uint8_t public_key[uECC_BYTES *2],
    const uint8_t private_key[uECC_BYTES],
    uint8_t secret[uECC_BYTES])
  
```

Compute a shared secret given your secret key and someone else's public key (ECDH).

Note: It is recommended that you hash the result before using it for symmetric encryption.

Parameters

in	<i>public_key</i>	The public key of the remote party.
in	<i>private_key</i>	Your private key.
out	<i>secret</i>	Will be filled with the shared secret value (uECC_BYTES bytes).

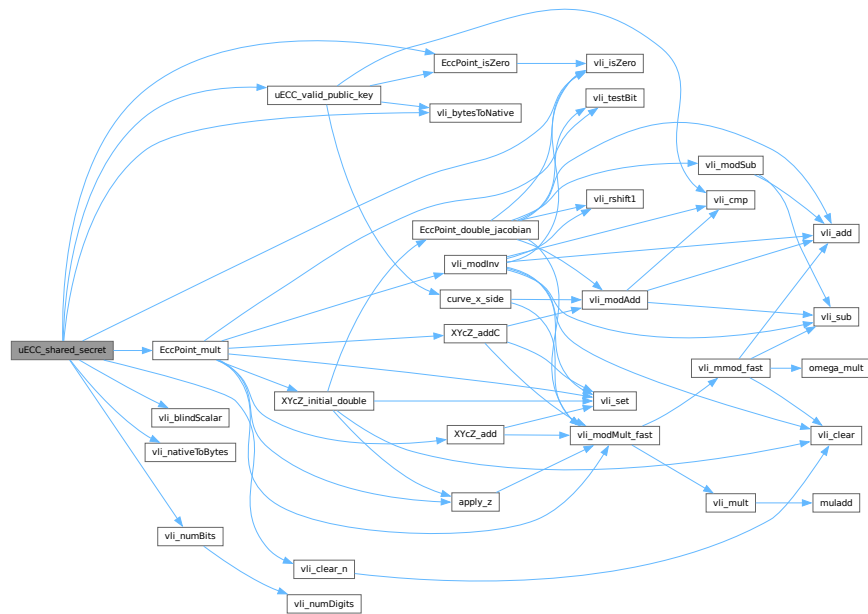
Returns

int 1 if the shared secret was generated successfully, 0 otherwise.

Definition at line [2923](#) of file [uECC.c](#).

References [EccPoint_isZero\(\)](#), [EccPoint_mult\(\)](#), [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_valid_public_key\(\)](#), [uECC_WORDS](#), [vli_blindScalar\(\)](#), [vli_bytesToNative\(\)](#), [vli_clear_n\(\)](#), [vli_isZero\(\)](#), [vli_nativeToBytes\(\)](#), [vli_numBits\(\)](#), [EccPoint::x](#), and [EccPoint::y](#).

Here is the call graph for this function:



5.19.4.9 uECC_sign()

```

int uECC_sign (
    const uint8_t private_key[uECC_BYTES],
    const uint8_t message_hash[uECC_BYTES],
    uint8_t signature[uECC_BYTES * 2])

```

Generate an ECDSA signature for a given hash value.

Parameters

in	<i>private_key</i>	Your private key.
in	<i>message_hash</i>	The hash of the message to sign (already computed).
out	<i>signature</i>	Will be filled with the signature (2*uECC_BYTES bytes).

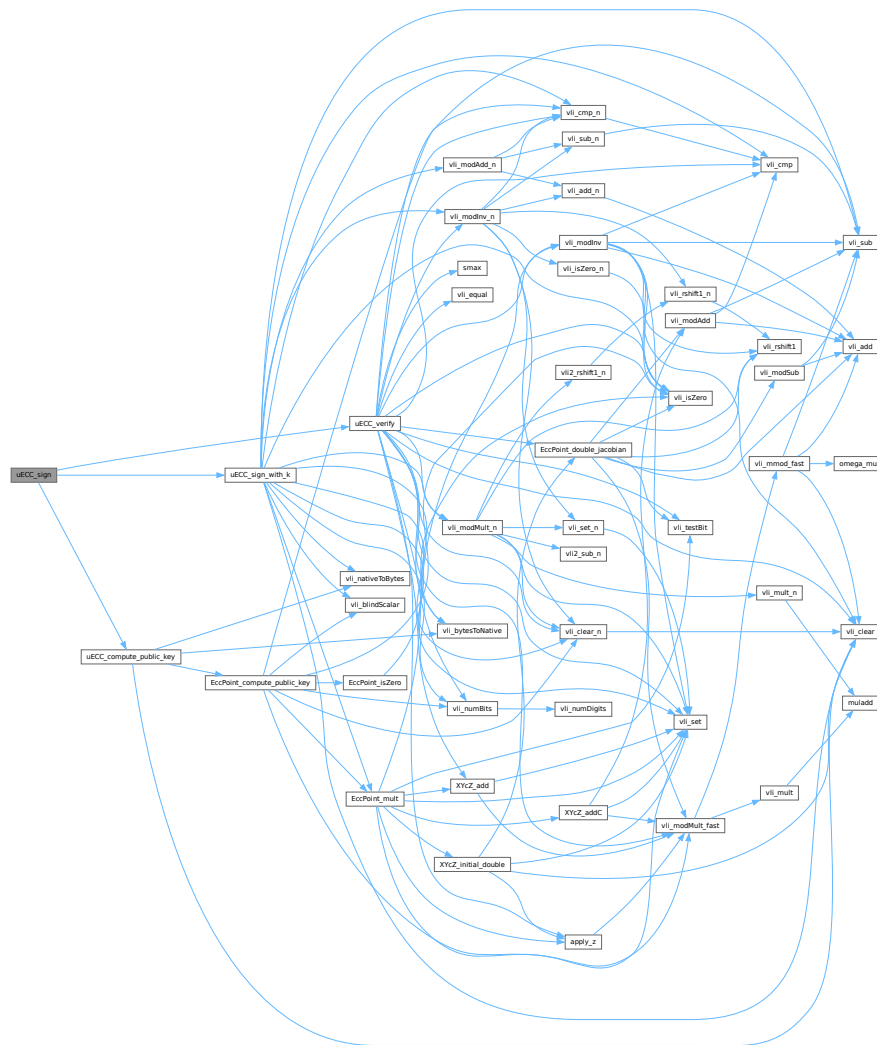
Returns

int 1 if the signature generated successfully, 0 otherwise.

Definition at line 3054 of file [uECC.c](#).

References [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_compute_public_key\(\)](#), [uECC_N_WORDS](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), and [uECC_WORDS](#).

Here is the call graph for this function:



5.19.4.10 uECC_sign_deterministic()

```
int uECC_sign_deterministic (
    const uint8_t private_key[uECC_BYTES],
    const uint8_t message_hash[uECC_BYTES],
    uECC_HashContext * hash_context,
    uint8_t signature[uECC_BYTES *2])
```

Generate an ECDSA signature using a deterministic algorithm (RFC 6979).

Parameters

in	<i>private_key</i>	Your private key.
in	<i>message_hash</i>	The hash of the message to sign.
in	<i>hash_context</i>	A hash context for HMAC-DRBG operations.

out	<i>signature</i>	Will be filled with the signature value.
-----	------------------	--

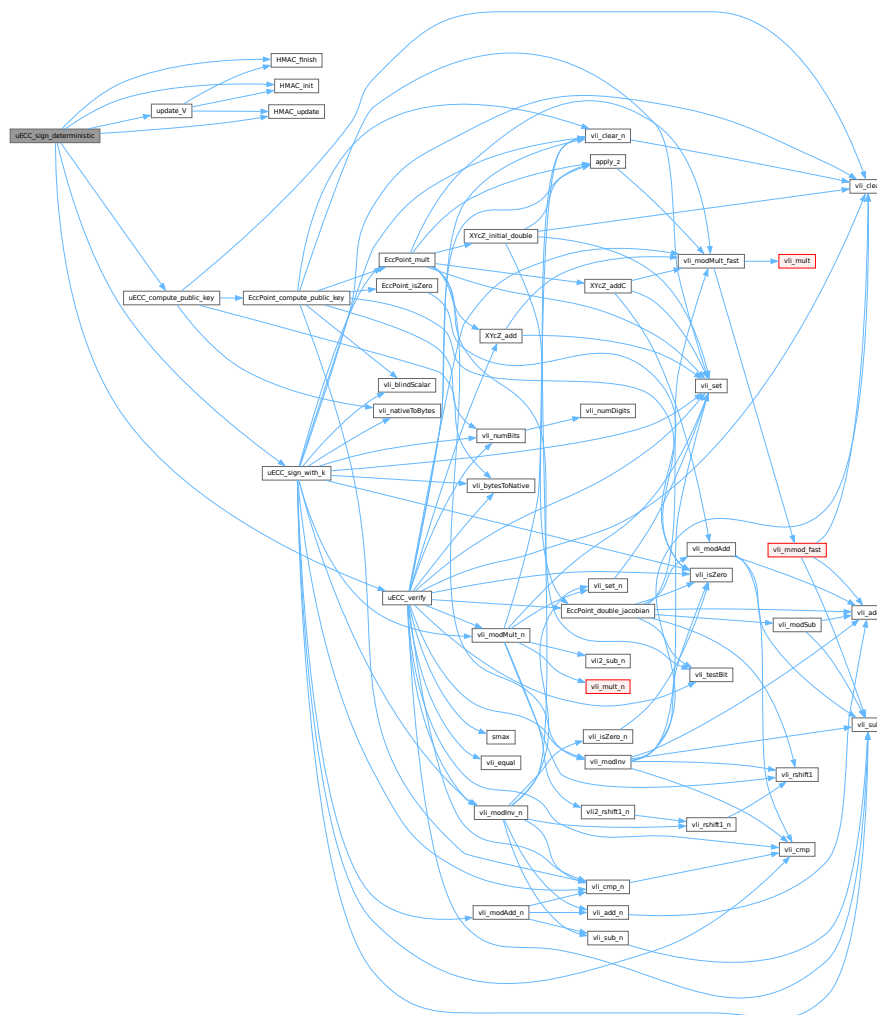
Returns

int 1 if the signature generated successfully, 0 otherwise.

Definition at line 3141 of file uECC.c.

References `HMAC_finish()`, `HMAC_init()`, `HMAC_update()`, `MAX_TRIES`, `uECC_HashContext::result_size`, `uECC_HashContext::tmp`, `uECC_BYTES`, `uECC_compute_public_key()`, `uECC_N_WORDS`, `uECC_sign_with_k()`, `uECC_verify()`, `uECC_WORDS`, and `update_V()`.

Here is the call graph for this function:



5.19.4.11 uECC_valid_public_key()

```
int uECC_valid_public_key (
    const uint8_t public_key[uECC_BYTES *2])
```

Check if a public key is a valid point on the curve.

Includes checks for the point at infinity and whether the point is on the curve.

Parameters

in	<i>public_key</i>	The public key to check.
----	-------------------	--------------------------

Returns

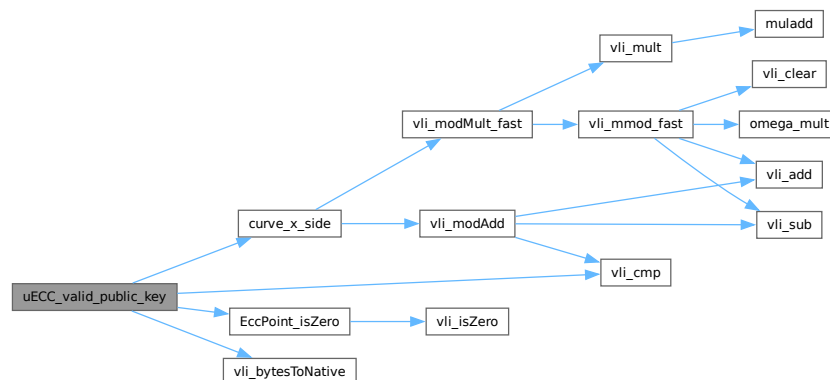
int 1 if the public key is valid, 0 otherwise.

Definition at line 2494 of file [uECC.c](#).

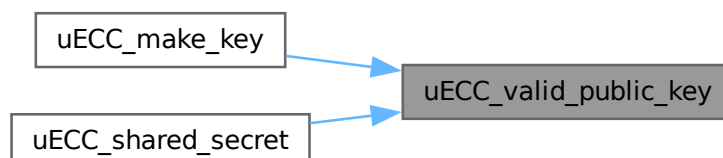
References [curve_p](#), [curve_x_side\(\)](#), [EccPoint_isZero\(\)](#), [uECC_BYTES](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_cmp\(\)](#), and [vli_modSquare_fast](#).

Referenced by [uECC_make_key\(\)](#), and [uECC_shared_secret\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.19.4.12 uECC_verify()

```
int uECC_verify (
    const uint8_t public_key[uECC_BYTES *2],
    const uint8_t hash[uECC_BYTES],
    const uint8_t signature[uECC_BYTES *2])
```

Verify an ECDSA signature against a public key and hash.

Parameters

in	<i>public_key</i>	The signer's public key.
in	<i>hash</i>	The hash of the signed data.
in	<i>signature</i>	The signature value.

Returns

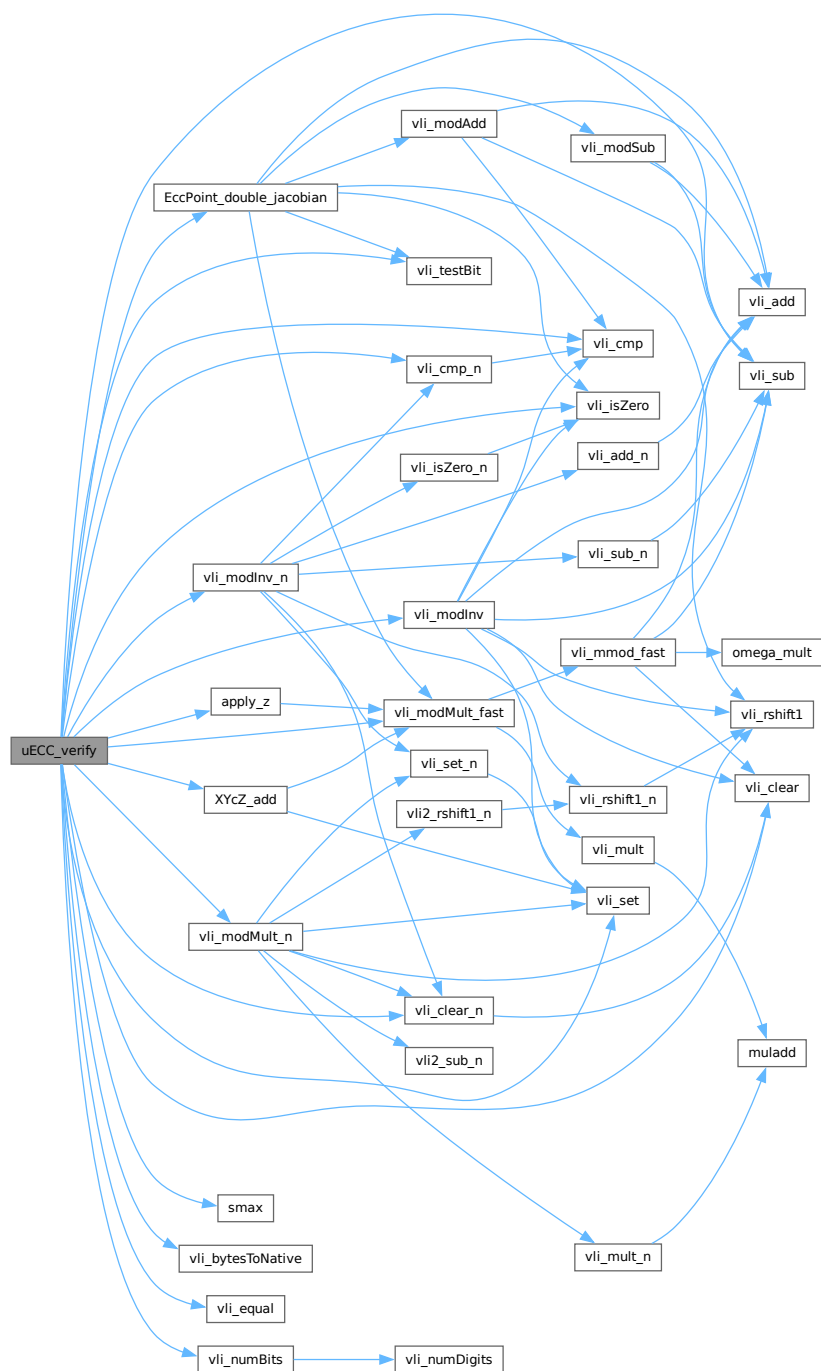
int 1 if the signature is valid, 0 if it is invalid or an error occurred.

Definition at line [3224](#) of file [uECC.c](#).

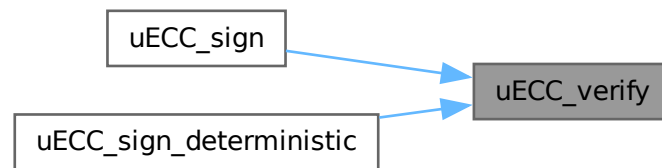
References [apply_z\(\)](#), [curve_G](#), [curve_n](#), [curve_p](#), [EccPoint_double_jacobian\(\)](#), [smax\(\)](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), [vli_clear_n\(\)](#), [vli_cmp\(\)](#), [vli_cmp_n\(\)](#), [vli_equal\(\)](#), [vli_isZero\(\)](#), [vli_modInv\(\)](#), [vli_modInv_n\(\)](#), [vli_modMult_fast\(\)](#), [vli_modMult_n\(\)](#), [vli_modSub_fast](#), [vli_numBits\(\)](#), [vli_set\(\)](#), [vli_sub\(\)](#), [vli_testBit\(\)](#), [EccPoint::x](#), [XYcZ_add\(\)](#), and [EccPoint::y](#).

Referenced by [uECC_sign\(\)](#), and [uECC_sign_deterministic\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.20 uECC.h

[Go to the documentation of this file.](#)

```

00001 /* Copyright 2014, Kenneth MacKay. Licensed under the BSD 2-clause license. */
00002
00012
00013 #ifndef _MICRO_ECC_H_
00014 #define _MICRO_ECC_H_
00015
00016 #include <stdint.h>
00017
00019 #define uECC_arch_other 0
00021 #define uECC_x86 1
00023 #define uECC_x86_64 2
00025 #define uECC_arm 3
00027 #define uECC_arm_thumb 4
00029 #define uECC_avr 5
00031 #define uECC_arm_thumb2 6
00032
00034 #define uECC_asm_none 0
00036 #define uECC_asm_small 1
00038 #define uECC_asm_fast 2
00039
00040 #ifndef uECC_ASM
00042 #define uECC_ASM uECC_asm_fast
00043 #endif
00044
00046 #define uECC_secp160r1 1
00048 #define uECC_secp192r1 2
00050 #define uECC_secp256r1 3
00052 #define uECC_secp256k1 4
00054 #define uECC_secp224r1 5
00055
00056 #ifndef uECC_CURVE
00058 #define uECC_CURVE uECC_secp256r1
00059 #endif
00060
00061 #ifndef uECC_SQUARE_FUNC
00063 #define uECC_SQUARE_FUNC 1
00064 #endif
00065
00066 #define uECC_CONCAT1(a, b) a##b
00067 #define uECC_CONCAT(a, b) uECC_CONCAT1(a, b)
00068
00069 #define uECC_size_1 20
00070 #define uECC_size_2 24
00071 #define uECC_size_3 32
00072 #define uECC_size_4 32
00073 #define uECC_size_5 28
00074
00077 #define uECC_BYTES uECC_CONCAT(uECC_size_, uECC_CURVE)
00078
00085 typedef int (*uECC_RNG_Function)(uint8_t *dest, unsigned size);
00086
00095 void uECC_set_rng(uECC_RNG_Function rng_function);
00096
00106 int uECC_make_key(uint8_t public_key[uECC_BYTES * 2],
00107                  uint8_t private_key[uECC_BYTES]);
  
```

```

00108
00122 int uECC_shared_secret(const uint8_t public_key[uECC_BYTES * 2],
00123                        const uint8_t private_key[uECC_BYTES],
00124                        uint8_t secret[uECC_BYTES]);
00125
00134 int uECC_sign(const uint8_t private_key[uECC_BYTES],
00135              const uint8_t message_hash[uECC_BYTES],
00136              uint8_t signature[uECC_BYTES * 2]);
00137
00141 typedef struct uECC_HashContext
00142 {
00144     void (*init_hash)(struct uECC_HashContext *context);
00146     void (*update_hash)(struct uECC_HashContext *context,
00147                       const uint8_t *message, unsigned message_size);
00149     void (*finish_hash)(struct uECC_HashContext *context, uint8_t *hash_result);
00150     unsigned block_size;
00151     unsigned result_size;
00152     uint8_t *tmp;
00154 } uECC_HashContext;
00155
00166 int uECC_sign_deterministic(const uint8_t private_key[uECC_BYTES],
00167                            const uint8_t message_hash[uECC_BYTES],
00168                            uECC_HashContext *hash_context,
00169                            uint8_t signature[uECC_BYTES * 2]);
00170
00180 int uECC_verify(const uint8_t public_key[uECC_BYTES * 2],
00181               const uint8_t hash[uECC_BYTES],
00182               const uint8_t signature[uECC_BYTES * 2]);
00183
00191 void uECC_compress(const uint8_t public_key[uECC_BYTES * 2],
00192                   uint8_t compressed[uECC_BYTES + 1]);
00193
00201 void uECC_decompress(const uint8_t compressed[uECC_BYTES + 1],
00202                    uint8_t public_key[uECC_BYTES * 2]);
00203
00213 int uECC_valid_public_key(const uint8_t public_key[uECC_BYTES * 2]);
00214
00222 int uECC_compute_public_key(const uint8_t private_key[uECC_BYTES],
00223                             uint8_t public_key[uECC_BYTES * 2]);
00224
00229 int uECC_bytes(void);
00230
00235 int uECC_curve(void);
00236
00237 #endif /* _MICRO_ECC_H_ */

```

5.21 firmware/src/dl_config.c File Reference

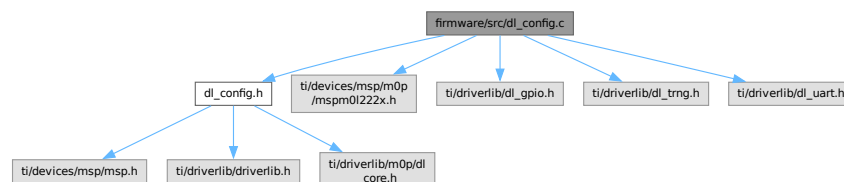
Device Library Configuration Source File.

```

#include "dl_config.h"
#include "ti/devices/msp/m0p/mspm01222x.h"
#include "ti/driverlib/dl_gpio.h"
#include "ti/driverlib/dl_trng.h"
#include "ti/driverlib/dl_uart.h"

```

Include dependency graph for dl_config.c:



Functions

- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_init](#) (void)

Main Entrypoint for DriverLib Configuration.

- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_initPower](#) (void)
Enable power lines and perform resets for hardware ports.
- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_GPIO_init](#) (void)
Initialize GPIO peripherals and IOMUX.
- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_SYSCTL_init](#) (void)
Initialize System Control (SYSCTL).
- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_UART_0_init](#) (void)
Initialize UART 0 (The Host Interface).
- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_UART_1_init](#) (void)
Initialize UART 1 (The Engineering/Peer Interface).
- static void [TRNG_SecureWarmup](#) (void)
Performs a secure warm-up and health check of the TRNG hardware.
- [SYSCONFIG_WEAK](#) void [SYSCFG_DL_TRNG_init](#) (void)
Initialize True Random Number Generator (TRNG).

Variables

- static const DL_UART_ClockConfig [gUART_0ClockConfig](#)
Clock configuration for UART 0.
- static const DL_UART_Config [gUART_0Config](#)
Protocol configuration for UART 0 (8N1).
- static const DL_UART_ClockConfig [gUART_1ClockConfig](#)
Clock configuration for UART 1.
- static const DL_UART_Config [gUART_1Config](#)
Protocol configuration for UART 1 (8N1).

5.21.1 Detailed Description

Device Library Configuration Source File.

Author

Sumedh Girish

Implements the hardware abstraction layer (HAL) initialization for the MSPM0L2228. This includes clock tree setup, power domain enabling, and peripheral configuration for UART, GPIO, and the TRNG.

Definition in file [dl_config.c](#).

5.21.2 Function Documentation

5.21.2.1 SYSCFG_DL_GPIO_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_GPIO_init (
    void )
```

Initialize GPIO peripherals and IOMUX.

Initializes GPIO muxing and pin directions.

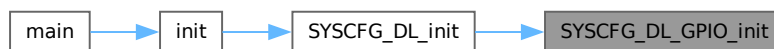
Configures the secondary functions for UART pins and initializes the Status LED pin as a digital output.

Definition at line 67 of file [dl_config.c](#).

References [GPIO_UART_0_IOMUX_RX](#), [GPIO_UART_0_IOMUX_RX_FUNC](#), [GPIO_UART_0_IOMUX_TX](#), [GPIO_UART_0_IOMUX_TX_FUNC](#), [GPIO_UART_1_IOMUX_RX](#), [GPIO_UART_1_IOMUX_RX_FUNC](#), [GPIO_UART_1_IOMUX_TX](#), [GPIO_UART_1_IOMUX_TX_FUNC](#), [LEDS_PORT](#), [LEDS_STATUS_LED_IOMUX](#), [LEDS_STATUS_LED_PIN](#), and [SYSCONFIG_WEAK](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.21.2.2 SYSCFG_DL_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_init (
    void )
```

Main Entrypoint for DriverLib Configuration.

Performs all required MSP DriverLib initialization.

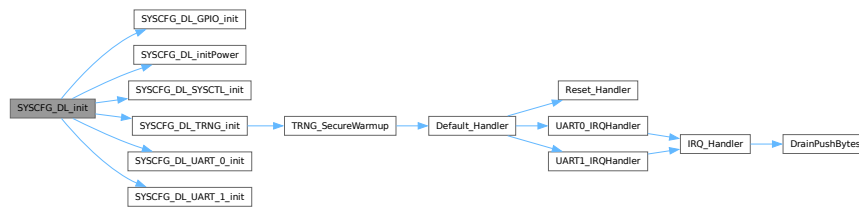
This function orchestrates the initialization sequence for all required hardware modules. It must be called before any application logic.

Definition at line 23 of file [dl_config.c](#).

References [SYSCFG_DL_GPIO_init\(\)](#), [SYSCFG_DL_initPower\(\)](#), [SYSCFG_DL_SYSCCTL_init\(\)](#), [SYSCFG_DL_TRNG_init\(\)](#), [SYSCFG_DL_UART_0_init\(\)](#), [SYSCFG_DL_UART_1_init\(\)](#), and [SYSCONFIG_WEAK](#).

Referenced by [init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.21.2.3 SYSCFG_DL_initPower()

```

SYSCONFIG_WEAK void SYSCFG_DL_initPower (
    void )

```

Enable power lines and perform resets for hardware ports.

Enables power and resets peripherals.

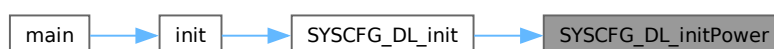
Resets the state of GPIO and UART peripherals and then enables their power domains. Includes a mandatory delay for stabilization.

Definition at line 41 of file [dl_config.c](#).

References [POWER_STARTUP_DELAY](#), [SYSCONFIG_WEAK](#), [UART_0_INST](#), and [UART_1_INST](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.21.2.4 SYSCFG_DL_SYCTL_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_SYCTL_init (
    void )
```

Initialize System Control (SYCTL).

Configures System Control (SYCTL) settings.

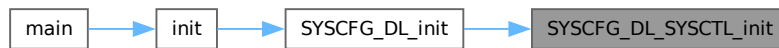
Configures the system for maximum brownout protection by setting the BOR threshold to Level 3 (~2.7V) and activating the monitoring hardware. Also configures the system oscillator frequency to its base (32 MHz).

Definition at line 91 of file [dl_config.c](#).

References [SYSCONFIG_WEAK](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.21.2.5 SYSCFG_DL_TRNG_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_TRNG_init (
    void )
```

Initialize True Random Number Generator (TRNG).

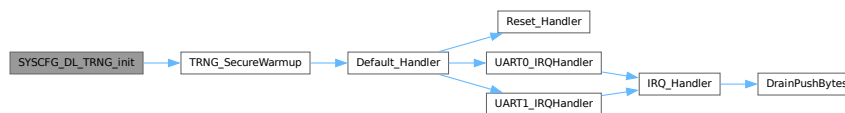
Initializes the hardware True Random Number Generator.

Definition at line 227 of file [dl_config.c](#).

References [SYSCONFIG_WEAK](#), and [TRNG_SecureWarmup\(\)](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.21.2.6 SYSCFG_DL_UART_0_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_UART_0_init (  
    void )
```

Initialize UART 0 (The Host Interface).

Initializes UART0 for Host Communication (115200 baud).

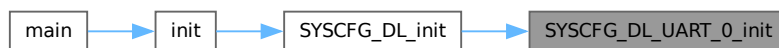
Configures the baud rate to 115200 based on the 32 MHz BUSCLK.

Definition at line 125 of file [dl_config.c](#).

References [gUART_0ClockConfig](#), [gUART_0Config](#), [SYSCONFIG_WEAK](#), [UART_0_FBRD_32_MHZ_115200_BAUD](#), [UART_0_IBRD_32_MHZ_115200_BAUD](#), [UART_0_INST](#), and [UART_0_INST_INT_IRQN](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.21.2.7 SYSCFG_DL_UART_1_init()

```
SYSCONFIG\_WEAK void SYSCFG_DL_UART_1_init (  
    void )
```

Initialize UART 1 (The Engineering/Peer Interface).

Initializes UART1 for Inter-HSM Communication (115200 baud).

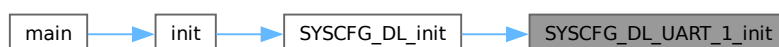
Configures the baud rate to 115200 based on the 32 MHz BUSCLK.

Definition at line 173 of file [dl_config.c](#).

References [gUART_1ClockConfig](#), [gUART_1Config](#), [SYSCONFIG_WEAK](#), [UART_1_FBRD_32_MHZ_115200_BAUD](#), [UART_1_IBRD_32_MHZ_115200_BAUD](#), [UART_1_INST](#), and [UART_1_INST_INT_IRQN](#).

Referenced by [SYSCFG_DL_init\(\)](#).

Here is the caller graph for this function:



5.21.2.8 TRNG_SecureWarmup()

```
void TRNG_SecureWarmup (
    void ) [static]
```

Performs a secure warm-up and health check of the TRNG hardware.

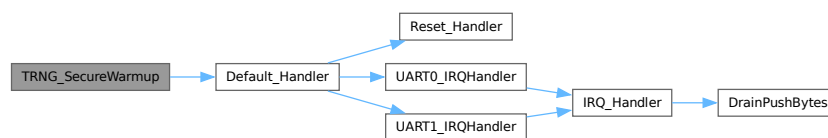
1. Discards the first 128 words to flush initial oscillator bias.
2. Verifies that the hardware health tests (repetition/adaptive) have passed.
3. Halts the system via Default_Handler if a failure is detected.

Definition at line 209 of file [dl_config.c](#).

References [Default_Handler\(\)](#).

Referenced by [SYSCFG_DL_TRNG_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.21.3 Variable Documentation

5.21.3.1 gUART_0ClockConfig

```
const DL_UART_ClockConfig gUART_0ClockConfig [static]
```

Initial value:

```
= {
    .clockSel = DL_UART_CLOCK_BUSCLK,
    .divideRatio = DL_UART_CLOCK_DIVIDE_RATIO_1}
```

Clock configuration for UART 0.

Definition at line 107 of file [dl_config.c](#).

Referenced by [SYSCFG_DL_UART_0_init\(\)](#).

5.21.3.2 gUART_0Config

```
const DL_UART_Config gUART_0Config [static]
```

Initial value:

```
                                = {  
.mode = DL_UART_MODE_NORMAL,  
.direction = DL_UART_DIRECTION_TX_RX,  
.flowControl = DL_UART_FLOW_CONTROL_NONE,  
.parity = DL_UART_PARITY_NONE,  
.wordLength = DL_UART_WORD_LENGTH_8_BITS,  
.stopBits = DL_UART_STOP_BITS_ONE}
```

Protocol configuration for UART 0 (8N1).

Definition at line 112 of file [dl_config.c](#).

Referenced by [SYSCFG_DL_UART_0_init\(\)](#).

5.21.3.3 gUART_1ClockConfig

```
const DL_UART_ClockConfig gUART_1ClockConfig [static]
```

Initial value:

```
                                = {  
.clockSel = DL_UART_CLOCK_BUSCLK,  
.divideRatio = DL_UART_CLOCK_DIVIDE_RATIO_1}
```

Clock configuration for UART 1.

Definition at line 155 of file [dl_config.c](#).

Referenced by [SYSCFG_DL_UART_1_init\(\)](#).

5.21.3.4 gUART_1Config

```
const DL_UART_Config gUART_1Config [static]
```

Initial value:

```
                                = {  
.mode = DL_UART_MODE_NORMAL,  
.direction = DL_UART_DIRECTION_TX_RX,  
.flowControl = DL_UART_FLOW_CONTROL_NONE,  
.parity = DL_UART_PARITY_NONE,  
.wordLength = DL_UART_WORD_LENGTH_8_BITS,  
.stopBits = DL_UART_STOP_BITS_ONE}
```

Protocol configuration for UART 1 (8N1).

Definition at line 160 of file [dl_config.c](#).

Referenced by [SYSCFG_DL_UART_1_init\(\)](#).

5.22 dl_config.c

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "dl_config.h"
00012 #include "ti/devices/msp/m0p/mspm01222x.h"
00013 #include "ti/driverlib/dl_gpio.h"
00014 #include "ti/driverlib/dl_trng.h"
00015 #include "ti/driverlib/dl_uart.h"
00016
00023 SYSCONFIG_WEAK void SYSCFG_DL_init(void)
00024 {
00025     SYSCFG_DL_initPower();
00026     SYSCFG_DL_GPIO_init();
00027
00028     // Module-Specific Initializations
00029     SYSCFG_DL_TRNG_init();
00030     SYSCFG_DL_SYSTCL_init();
00031     SYSCFG_DL_UART_0_init();
00032     SYSCFG_DL_UART_1_init();
00033 }
00034
00041 SYSCONFIG_WEAK void SYSCFG_DL_initPower(void)
00042 {
00043     DL_GPIO_reset(GPIOA);
00044     DL_GPIO_reset(GPIOB);
00045
00046     DL_UART_reset(UART_0_INST);
00047     DL_UART_reset(UART_1_INST);
00048
00049     DL_GPIO_enablePower(GPIOA);
00050     DL_GPIO_enablePower(GPIOB);
00051
00052     DL_UART_enablePower(UART_0_INST);
00053     DL_UART_enablePower(UART_1_INST);
00054
00055     DL_TRNG_reset(TRNG);
00056     DL_TRNG_enablePower(TRNG);
00057
00058     delay_cycles(POWER_STARTUP_DELAY);
00059 }
00060
00067 SYSCONFIG_WEAK void SYSCFG_DL_GPIO_init(void)
00068 {
00069     DL_GPIO_initPeripheralOutputFunction(GPIO_UART_0_IOMUX_TX,
00070                                         GPIO_UART_0_IOMUX_TX_FUNC);
00071     DL_GPIO_initPeripheralInputFunction(GPIO_UART_0_IOMUX_RX,
00072                                        GPIO_UART_0_IOMUX_RX_FUNC);
00073     DL_GPIO_initPeripheralOutputFunction(GPIO_UART_1_IOMUX_TX,
00074                                        GPIO_UART_1_IOMUX_TX_FUNC);
00075     DL_GPIO_initPeripheralInputFunction(GPIO_UART_1_IOMUX_RX,
00076                                        GPIO_UART_1_IOMUX_RX_FUNC);
00077
00078     DL_GPIO_initDigitalOutput(LEDS_STATUS_LED_IOMUX);
00079
00080     DL_GPIO_setPins(LEDS_PORT, LEDS_STATUS_LED_PIN);
00081     DL_GPIO_enableOutput(LEDS_PORT, LEDS_STATUS_LED_PIN);
00082 }
00083
00091 SYSCONFIG_WEAK void SYSCFG_DL_SYSTCL_init(void)
00092 {
00093     // Configure for maximum strictness (BOR3 ~2.7V)
00094     DL_SYSTCL_setBORTThreshold(DL_SYSTCL_BOR_THRESHOLD_LEVEL_3);
00095     DL_SYSTCL_activateBORTThreshold();
00096
00097     DL_SYSTCL_setSYSOSCFreq(DL_SYSTCL_SYSOSC_FREQ_BASE);
00098     DL_SYSTCL_setMCLKDivider(DL_SYSTCL_MCLK_DIVIDER_DISABLE);
00099
00100     // Partition SRAM: lower 28KB = RW (data/stack), upper 4KB = RX (ramfuncs)
00101     // Address sourced from linker symbol __sram_boundary = ORIGIN(SRAM_CODE)
00102     extern uint32_t __sram_boundary;
00103     DL_SYSTCL_setSRAMBoundaryAddress((uint32_t) &__sram_boundary);
00104 }
00105
00107 static const DL_UART_ClockConfig gUART_0ClockConfig = {
00108     .clockSel = DL_UART_CLOCK_BUSCLK,
00109     .divideRatio = DL_UART_CLOCK_DIVIDE_RATIO_1;
00110
00112 static const DL_UART_Config gUART_0Config = {
00113     .mode = DL_UART_MODE_NORMAL,
00114     .direction = DL_UART_DIRECTION_TX_RX,
00115     .flowControl = DL_UART_FLOW_CONTROL_NONE,
00116     .parity = DL_UART_PARITY_NONE,
00117     .wordLength = DL_UART_WORD_LENGTH_8_BITS,

```

```

00118     .stopBits = DL_UART_STOP_BITS_ONE};
00119
00125 SYSCONFIG_WEAK void SYSCFG_DL_UART_0_init(void)
00126 {
00127     DL_UART_setClockConfig(UART_0_INST,
00128                           (DL_UART_ClockConfig *) &gUART_0ClockConfig);
00129
00130     DL_UART_init(UART_0_INST, (DL_UART_Config *) &gUART_0Config);
00131     /*
00132      * Configure baud rate by setting oversampling and baud rate divisors.
00133      * Target baud rate: 115200
00134      * Actual baud rate: 115211.52
00135      */
00136     DL_UART_setOversampling(UART_0_INST, DL_UART_OVERSAMPLING_RATE_16X);
00137     DL_UART_setBaudRateDivisor(UART_0_INST, UART_0_IBRD_32_MHZ_115200_BAUD,
00138                               UART_0_FBRD_32_MHZ_115200_BAUD);
00139
00140     DL_UART_enableFIFOs(UART_0_INST);
00141     DL_UART_setRXFIFOThreshold(UART_0_INST, DL_UART_RX_FIFO_LEVEL_ONE_ENTRY);
00142
00143     DL_UART_setRXInterruptTimeout(UART_0_INST, 10);
00144
00145     DL_UART_enableInterrupt(
00146         UART_0_INST, DL_UART_INTERRUPT_RX | DL_UART_INTERRUPT_RX_TIMEOUT_ERROR |
00147                     DL_UART_INTERRUPT_OVERRUN_ERROR);
00148
00149     DL_UART_enable(UART_0_INST);
00150
00151     NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
00152 }
00153
00155 static const DL_UART_ClockConfig gUART_1ClockConfig = {
00156     .clockSel = DL_UART_CLOCK_BUSCLK,
00157     .divideRatio = DL_UART_CLOCK_DIVIDE_RATIO_1};
00158
00160 static const DL_UART_Config gUART_1Config = {
00161     .mode = DL_UART_MODE_NORMAL,
00162     .direction = DL_UART_DIRECTION_TX_RX,
00163     .flowControl = DL_UART_FLOW_CONTROL_NONE,
00164     .parity = DL_UART_PARITY_NONE,
00165     .wordLength = DL_UART_WORD_LENGTH_8_BITS,
00166     .stopBits = DL_UART_STOP_BITS_ONE};
00167
00173 SYSCONFIG_WEAK void SYSCFG_DL_UART_1_init(void)
00174 {
00175     DL_UART_setClockConfig(UART_1_INST,
00176                           (DL_UART_ClockConfig *) &gUART_1ClockConfig);
00177
00178     DL_UART_init(UART_1_INST, (DL_UART_Config *) &gUART_1Config);
00179     /*
00180      * Configure baud rate by setting oversampling and baud rate divisors.
00181      * Target baud rate: 115200
00182      * Actual baud rate: 115211.52
00183      */
00184     DL_UART_setOversampling(UART_1_INST, DL_UART_OVERSAMPLING_RATE_16X);
00185     DL_UART_setBaudRateDivisor(UART_1_INST, UART_1_IBRD_32_MHZ_115200_BAUD,
00186                               UART_1_FBRD_32_MHZ_115200_BAUD);
00187
00188     DL_UART_enableFIFOs(UART_1_INST);
00189     DL_UART_setRXFIFOThreshold(UART_1_INST, DL_UART_RX_FIFO_LEVEL_ONE_ENTRY);
00190
00191     DL_UART_setRXInterruptTimeout(UART_1_INST, 10);
00192
00193     DL_UART_enableInterrupt(
00194         UART_1_INST, DL_UART_INTERRUPT_RX | DL_UART_INTERRUPT_RX_TIMEOUT_ERROR |
00195                     DL_UART_INTERRUPT_OVERRUN_ERROR);
00196
00197     DL_UART_enable(UART_1_INST);
00198
00199     NVIC_EnableIRQ(UART_1_INST_INT_IRQN);
00200 }
00201
00209 static void TRNG_SecureWarmup(void)
00210 {
00211     for (int i = 0; i < 128; i++)
00212     {
00213         while (!DL_TRNG_isCaptureReady(TRNG))
00214             ;
00215         (void) DL_TRNG_getCapture(TRNG);
00216     }
00217
00218     if (DL_TRNG_isHealthTestFail(TRNG))
00219     {
00220         Default_Handler();
00221     }
00222 }
00223

```

```

00227 SYSCONFIG_WEAK void SYSCFG_DL_TRNG_init(void)
00228 {
00229     DL_TRNG_setDecimationRate(TRNG, DL_TRNG_DECIMATION_RATE_8);
00230     DL_TRNG_sendCommand(TRNG, DL_TRNG_CMD_NORM_FUNC);
00231
00232     TRNG_SecureWarmup();
00233 }

```

5.23 firmware/src/filesystem.c File Reference

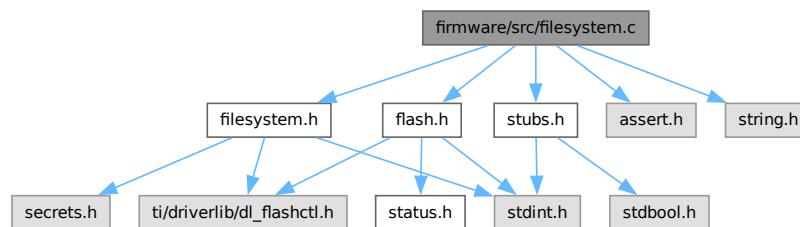
Flash Filesystem Implementation and Storage Tables.

```

#include "filesystem.h"
#include "flash.h"
#include "stubs.h"
#include <assert.h>
#include <string.h>

```

Include dependency graph for filesystem.c:



Functions

- `__attribute__((section(".fat"))) const volatile`
File Allocation Table, mapped to fixed flash section.
- void `StoreFile` (uint8_t slot, `StatusCode` *status)
Stores a file from the staging RAM area into flash memory.

Variables

- volatile `FS_Stage` stage
Staging area in SRAM for cryptographic ops before syncing to Flash.

5.23.1 Detailed Description

Flash Filesystem Implementation and Storage Tables.

Author

Sumedh Girish

Implements the core flash storage tables linked directly into designated memory segments via the linker script. Provides load/store abstractions that interact safely with the cryptographic staging area.

Definition in file `filesystem.c`.

5.23.2 Function Documentation

5.23.2.1 __attribute__()

```
__attribute__ (
    (section(".fat")) ) const volatile
```

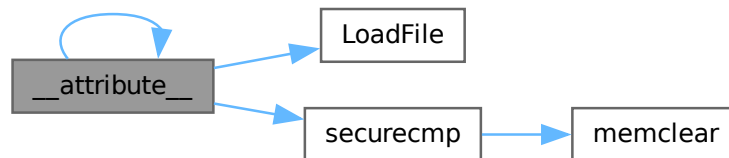
File Allocation Table, mapped to fixed flash section.

Definition at line 21 of file [filesystem.c](#).

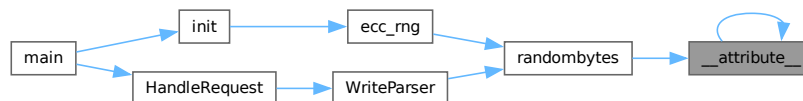
References [__attribute__\(\)](#), [FAT_Table](#), [Filedata_Table](#), [INVALIDSLOT](#), [LoadFile\(\)](#), [MAX_FILE_SIZE](#), [Metadata_Table](#), [NUM_SLOTS](#), [securecmp\(\)](#), [stage](#), and [SystemStatus](#).

Referenced by [__attribute__\(\)](#), and [randombytes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.23.2.2 StoreFile()

```
void StoreFile (
    uint8_t slot,
    StatusCode * status)
```

Stores a file from the staging RAM area into flash memory.

Parameters

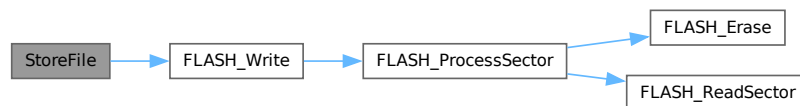
<i>slot</i>	Index of the file slot (0 to NUM_SLOTS-1).
<i>status</i>	Pointer to global status code.

Definition at line 62 of file [filesystem.c](#).

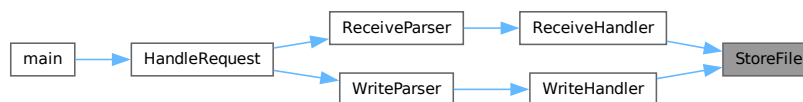
References [FS_MetadataEntryFlash::asBytes](#), [FS_MetadataEntryFlash::data](#), [FAT_Table](#), [Filedata_Table](#), [FLASH_Write\(\)](#), [FLASH_ERASEERROR](#), [FLASHWRITEERROR](#), [FS_MetadataEntryFlash::id](#), [INVALIDSLOT](#), [FS_MetadataEntryFlash::magic](#), [MAX_FILE_SIZE](#), [Metadata_Table](#), [NUM_SLOTS](#), [stage](#), and [FS_MetadataEntryFlash::status](#).

Referenced by [ReceiveHandler\(\)](#), and [WriteHandler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.23.3 Variable Documentation

5.23.3.1 stage

```
volatile FS_Stage stage
```

Staging area in SRAM for cryptographic ops before syncing to Flash.

Definition at line 18 of file [filesystem.c](#).

Referenced by [__attribute__\(\)](#), [init\(\)](#), [InterrogateHandler\(\)](#), [InterrogateParser\(\)](#), [ListHandler\(\)](#), [main\(\)](#), [ReadHandler\(\)](#), [ReceiveHandler\(\)](#), [ReceiveParser\(\)](#), [ReplyHandler\(\)](#), [ReplyParser\(\)](#), [SendHandler\(\)](#), [SendParser\(\)](#), [SendResponse\(\)](#), [StoreFile\(\)](#), [WriteHandler\(\)](#), and [WriteParser\(\)](#).

5.24 filesystem.c

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "filesystem.h"
00012 #include "flash.h"
00013 #include "stubs.h"
00014 #include <assert.h>
00015 #include <string.h>
00016
00018 volatile FS_Stage stage;
00019
00021 __attribute__((section(".fat"))) // Link into the location of the FAT in flash
00022 volatile const FAT_TableTypeFlash FAT_Table;
00023
00025 __attribute__((section(".metadata"))) // Link into metadata in flash
00026 volatile const FS_MetadataEntryFlash Metadata_Table[NUM_SLOTS];
00027
00029 __attribute__((section(".filedata"))) // Link into filedata in flash
00030 volatile const FS_FiledataFlash Filedata_Table;
00031
00033 __attribute__((section(".systemconf"))) // Link into filedata in flash
00034 volatile const FS_SystemStatusFlash SystemStatus;
00035
00036 void LoadFile(uint8_t slot, StatusCode *status)
00037 {
00038     if (slot >= NUM_SLOTS)
00039     {
00040         *status = INVALIDSLOT;
00041         return;
00042     }
00043
00044     if (Metadata_Table[slot].status.magic != 0xdeadf00d ||
00045         securecmp((uint8_t *) Metadata_Table[slot].status.id,
00046                 (uint8_t *) FAT_Table.entry[slot].uuid, 16) == false)
00047     {
00048         *status = INVALIDSLOT;
00049         return;
00050     }
00051
00052     memcpy((uint8_t *) &stage.as.file.entry, (uint8_t *) &FAT_Table.entry[slot],
00053           sizeof(FS_FileEntry));
00054
00055     memcpy((uint8_t *) &stage.as.file.metadata,
00056           (uint8_t *) &Metadata_Table[slot].data, sizeof(FS_Metadata));
00057
00058     memcpy((uint8_t *) &stage.as.file.content,
00059           (uint8_t *) Filedata_Table.data[slot], MAX_FILE_SIZE);
00060 }
00061
00062 void StoreFile(uint8_t slot, StatusCode *status)
00063 {
00064     if (slot >= NUM_SLOTS)
00065     {
00066         *status = INVALIDSLOT;
00067         return;
00068     }
00069
00070     FLASH_Write((uint32_t) Filedata_Table.data[slot],
00071                (uint8_t *) stage.as.file.content, MAX_FILE_SIZE, status);
00072
00073     FS_MetadataEntryFlash metadata_entry;
00074
00075     metadata_entry.data = stage.as.file.metadata;
00076     memcpy(metadata_entry.status.id, (uint8_t *) &stage.as.file.entry.uuid, 16);
00077     metadata_entry.status.magic = 0xdeadf00d;
00078
00079     FLASH_Write((uint32_t) &Metadata_Table[slot].asBytes,
00080                metadata_entry.asBytes, sizeof(FS_MetadataEntryFlash), status);
00081
00082     FLASH_Write((uint32_t) &FAT_Table.entry[slot],
00083                (uint8_t *) &stage.as.file.entry, sizeof(FS_FileEntry), status);
00084     if (*status == FLASHWRITEERROR || *status == FLASHERASEERROR)
00085         return;
00086 }

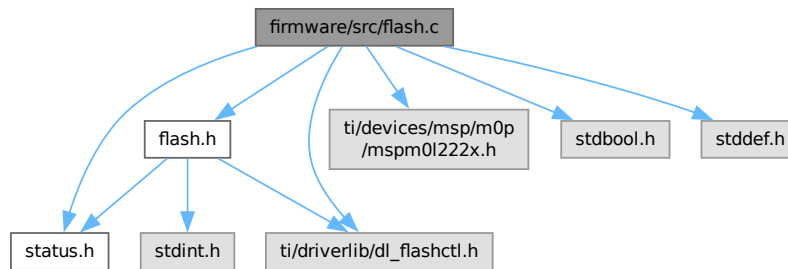
```

5.25 firmware/src/flash.c File Reference

Flash Memory Controller Implementation.

```
#include "flash.h"
#include "status.h"
#include "ti/devices/msp/m0p/mspm01222x.h"
#include "ti/driverlib/dl_flashctl.h"
#include <stdbool.h>
#include <stddef.h>
```

Include dependency graph for flash.c:



Functions

- `__attribute__((aligned(8)))`
- static `RAMFUNC` void `FLASH_ReadSector` (uint32_t address, uint32_t *buffer)
Reads a full flash sector into a 64-bit aligned SRAM buffer.
- static `RAMFUNC` void `FLASH_ProcessSector` (uint32_t sector_base, uint32_t offset, uint32_t chunk, const uint8_t *src_ptr, `StatusCode` *status, bool fullChunk)
Executes a read-modify-write cycle for a single flash sector.
- `RAMFUNC` void `FLASH_Write` (uint32_t address, uint8_t *buffer, uint32_t size, `StatusCode` *status)
Writes data into a physical flash sector.

5.25.1 Detailed Description

Flash Memory Controller Implementation.

Author

Sumedh Girish

Provides safe routines for flash sector erase and program operations. Operations are strictly constrained to SRAM execution spaces via `RAMFUNC`.

Definition in file `flash.c`.

5.25.2 Function Documentation

5.25.2.1 `__attribute__()`

```
__attribute__ (
    (aligned(8)) )
```

Definition at line 17 of file [flash.c](#).

References [FLASH_Erase\(\)](#), [FLASH_ERASEERROR](#), and [RAMFUNC](#).

Here is the call graph for this function:



5.25.2.2 `FLASH_ProcessSector()`

```
RAMFUNC void FLASH_ProcessSector (
    uint32_t sector_base,
    uint32_t offset,
    uint32_t chunk,
    const uint8_t * src_ptr,
    StatusCode * status,
    bool fullChunk) [static]
```

Executes a read-modify-write cycle for a single flash sector.

If the data chunk doesn't cleanly overwrite the entire sector, the current sector is loaded into SRAM, patched, erased, and reprogrammed.

Parameters

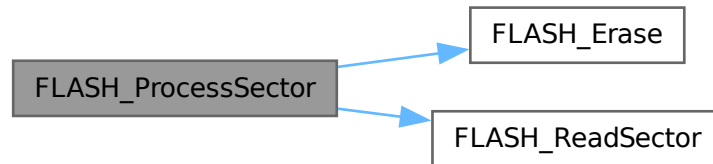
<i>sector_base</i>	Aligned flash sector base address.
<i>offset</i>	Byte-offset within the sector to apply the new data.
<i>chunk</i>	Size of the new data in bytes.
<i>src_ptr</i>	Pointer to the new data in SRAM.
<i>status</i>	Global status variable.
<i>fullChunk</i>	True if the chunk size precisely matches the sector size.

Definition at line 68 of file [flash.c](#).

References [FLASH_Erase\(\)](#), [FLASH_ReadSector\(\)](#), [FLASH_ERASEERROR](#), [FLASH_WRITEERROR](#), and [RAMFUNC](#).

Referenced by [FLASH_Write\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.25.2.3 FLASH_ReadSector()

```

RAMFUNC void FLASH_ReadSector (
    uint32_t address,
    uint32_t * buffer) [static]
  
```

Reads a full flash sector into a 64-bit aligned SRAM buffer.

Essential for the read-modify-write cycle. Executes from SRAM to avoid bus contention during subsequent flash erase commands.

Parameters

<i>address</i>	Base address of the sector to read.
<i>buffer</i>	Pointer to the 64-bit aligned SRAM buffer.

Definition at line 46 of file [flash.c](#).

References [RAMFUNC](#).

Referenced by [FLASH_ProcessSector\(\)](#).

Here is the caller graph for this function:



5.25.2.4 FLASH_Write()

```
RAMFUNC void FLASH_Write (
    uint32_t address,
    uint8_t * buffer,
    uint32_t size,
    StatusCode * status)
```

Writes data into a physical flash sector.

Writes a buffer into flash memory in 64-bit words. Handles flash unlocking, execution of the write sequence, and locking. Automatically pads unaligned writes to 64-bit boundaries with 0xFF.

Parameters

<i>address</i>	Base address of the flash memory to write to.
<i>buffer</i>	Pointer to the source data in SRAM.
<i>size</i>	Number of bytes to write.
<i>status</i>	Global status variable to update on failure/misalignment.

Note

Must execute from SRAM.

Definition at line 104 of file [flash.c](#).

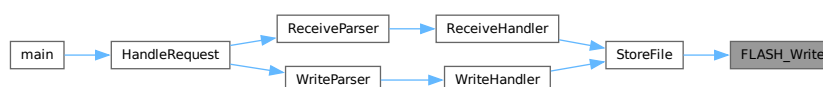
References [FLASH_ProcessSector\(\)](#), [FLASH_ERASEERROR](#), [FLASH_WRITEERROR](#), and [RAMFUNC](#).

Referenced by [StoreFile\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.26 flash.c

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "flash.h"
00011 #include "status.h"
00012 #include "ti/devices/msp/m0p/mspm01222x.h"
00013 #include "ti/driverlib/dl_flashctl.h"
00014 #include <stdbool.h>
00015 #include <stdint.h>
00016
00017 __attribute__((aligned(8))) // Make this a valid 64 bit address
00018 static uint32_t sector_buffer[DL_FLASHCTL_SECTOR_SIZE / 4U];
00019
00020 RAMFUNC void FLASH_Erase(uint32_t address, StatusCode *status)
00021 {
00022     DL_FlashCTL_executeClearStatus(FLASHCTL);
00023     DL_FlashCTL_unprotectSector(FLASHCTL, address,
00024                               DL_FLASHCTL_REGION_SELECT_MAIN);
00025     if (DL_FlashCTL_eraseMemoryFromRAM(FLASHCTL, address,
00026                                       DL_FLASHCTL_COMMAND_SIZE_SECTOR) !=
00027         DL_FLASHCTL_COMMAND_STATUS_PASSED)
00028     {
00029         *status = FLASHERASEERROR;
00030         return;
00031     }
00032
00033     if (DL_FlashCTL_waitForCmdDone(FLASHCTL) == false)
00034         *status = FLASHERASEERROR;
00035 }
00036
00046 static RAMFUNC void FLASH_ReadSector(uint32_t address, uint32_t *buffer)
00047 {
00048     const volatile uint32_t *src = (const uint32_t *) address;
00049     for (uint32_t i = 0U; i < DL_FLASHCTL_SECTOR_SIZE / 4U; i++)
00050     {
00051         buffer[i] = src[i];
00052     }
00053 }
00054
00068 static RAMFUNC void FLASH_ProcessSector(uint32_t sector_base, uint32_t offset,
00069                                         uint32_t chunk, const uint8_t *src_ptr,
00070                                         StatusCode *status, bool fullChunk)
00071 {
00072     if (!fullChunk)
00073         FLASH_ReadSector(sector_base, sector_buffer);
00074
00075     uint8_t *dst = (uint8_t *) sector_buffer;
00076     for (uint32_t i = 0; i < chunk; i++)
00077     {
00078         dst[i + offset] = src_ptr[i];
00079     }
00080
00081     FLASH_Erase(sector_base, status);
00082     if (*status == FLASHERASEERROR)
00083         return;
00084
00085     DL_FlashCTL_executeClearStatus(FLASHCTL);
00086     DL_FlashCTL_unprotectSector(FLASHCTL, sector_base,
00087                               DL_FLASHCTL_REGION_SELECT_MAIN);
00088     if (DL_FlashCTL_programMemoryBlockingFromRAM64WithECCGenerated(
00089         FLASHCTL, sector_base, sector_buffer, DL_FLASHCTL_SECTOR_SIZE / 4U,
00090         DL_FLASHCTL_REGION_SELECT_MAIN) !=
00091         DL_FLASHCTL_COMMAND_STATUS_PASSED)
00092     {
00093         *status = FLASHWRITEERROR;
00094         return;
00095     }
00096
00097     if (DL_FlashCTL_waitForCmdDone(FLASHCTL) == false)
00098     {
00099         *status = FLASHWRITEERROR;
00100         return;
00101     }
00102 }
00103
00104 RAMFUNC void FLASH_Write(uint32_t address, uint8_t *buffer, uint32_t size,
00105                          StatusCode *status)
00106 {
00107     uint32_t remaining = size;
00108     uint32_t current_addr = address;
00109     const uint8_t *src_ptr = buffer;
00110
00111     while (remaining > 0U)

```

```

00112     {
00113         uint32_t sector_base =
00114             current_addr & ~(uint32_t) (DL_FLASHCTL_SECTOR_SIZE - 1U);
00115         uint32_t offset = current_addr % DL_FLASHCTL_SECTOR_SIZE;
00116         uint32_t chunk = DL_FLASHCTL_SECTOR_SIZE - offset;
00117
00118         if (chunk > remaining)
00119             chunk = remaining;
00120
00121         FLASH_ProcessSector(sector_base, offset, chunk, src_ptr, status,
00122                             chunk == DL_FLASHCTL_SECTOR_SIZE);
00123         if (*status == FLASHWRITEERROR || *status == FLASHERASEERROR)
00124             return;
00125
00126         remaining -= chunk;
00127         current_addr += chunk;
00128         src_ptr += chunk;
00129     }
00130 }

```

5.27 firmware/src/handler.c File Reference

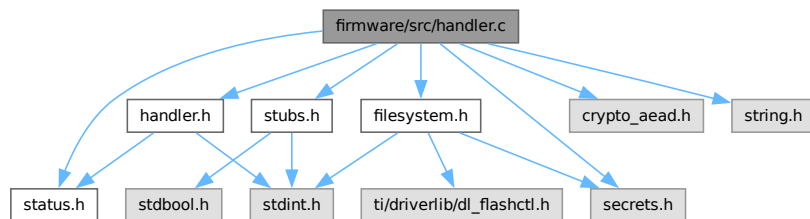
Implementation of Host and Peer Command Handlers.

```

#include "handler.h"
#include "crypto_aead.h"
#include "filesystem.h"
#include "secrets.h"
#include "status.h"
#include "stubs.h"
#include <string.h>

```

Include dependency graph for handler.c:



Functions

- void [ListHandler](#) ([StatusCode](#) *status)
Handles the LIST command.
- void [ReadHandler](#) (uint8_t slot, [StatusCode](#) *status)
Execution flow for a Host READ command.
- void [WriteHandler](#) (uint8_t slot, [StatusCode](#) *status)
Execution flow for a Host WRITE command.
- static bool [isReceivable](#) (uint16_t groupid)
- void [InterrogateHandler](#) ([StatusCode](#) *status)
Execution flow for a Peer INTERROGATE command.
- void [ReplyHandler](#) ([StatusCode](#) *status)
Execution flow for generating a Peer REPLY.
- void [SendHandler](#) (uint8_t slot, [StatusCode](#) *status)
Execution flow for a Peer SEND operation.
- void [ReceiveHandler](#) (uint8_t slot, [StatusCode](#) *status)
Execution flow for a Peer RECEIVE operation.

5.27.1 Detailed Description

Implementation of Host and Peer Command Handlers.

Author

Sumedh Girish

Implements the execution flows for all system operations. Handlers are responsible for orchestrating key derivation, AEAD encryption/decryption, and calling the appropriate filesystem routines.

Definition in file [handler.c](#).

5.27.2 Function Documentation

5.27.2.1 InterrogateHandler()

```
void InterrogateHandler (  
    StatusCode * status)
```

Execution flow for a Peer INTERROGATE command.

Handles the INTERROGATE command.

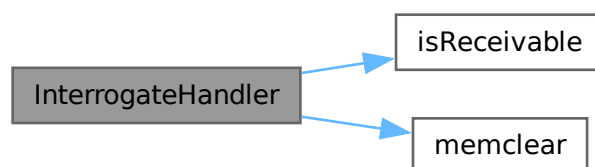
Decrypts a filesystem allocation table (FAT) sent by a peer to determine which files the peer holds that this HSM is authorized to RECEIVE. Filters the FAT array in-place.

Definition at line [147](#) of file [handler.c](#).

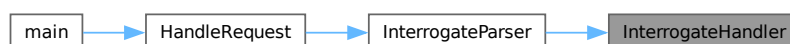
References [DECRYPTIONERROR](#), [isReceivable\(\)](#), [memset\(\)](#), [OPINTERROGATE](#), and [stage](#).

Referenced by [InterrogateParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



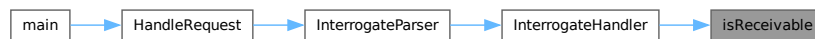
5.27.2.2 isReceivable()

```
bool isReceivable (  
    uint16_t groupid)  [inline], [static]
```

Definition at line 130 of file [handler.c](#).

Referenced by [InterrogateHandler\(\)](#).

Here is the caller graph for this function:



5.27.2.3 ListHandler()

```
void ListHandler (  
    StatusCode * status)
```

Handles the LIST command.

Definition at line 19 of file [handler.c](#).

References [FAT_Table](#), [Metadata_Table](#), [NUM_SLOTS](#), [OPLIST](#), [securecmp\(\)](#), and [stage](#).

Referenced by [ListParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.27.2.4 ReadHandler()

```
void ReadHandler (  
    uint8_t slot,  
    StatusCode * status)
```

Execution flow for a Host READ command.

Handles the READ command for a specific slot.

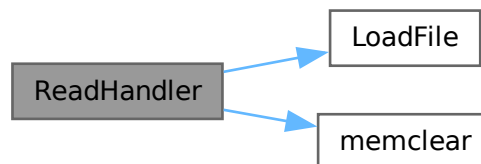
1. Loads the target slot into the volatile [stage](#).
2. Derives the unique read key for the file's owner group.
3. In-place decrypts the file payload using ASCON-128a.

Definition at line [51](#) of file [handler.c](#).

References [DECRYPTIONERROR](#), [LoadFile\(\)](#), [MAX_FILE_SIZE](#), [memset\(\)](#), [OPREAD](#), and [stage](#).

Referenced by [ReadParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.27.2.5 ReceiveHandler()

```
void ReceiveHandler (
    uint8_t slot,
    StatusCode * status)
```

Execution flow for a Peer RECEIVE operation.

Begins the RECEIVE command logic (HSM to HSM transfer).

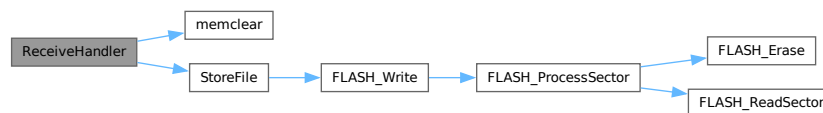
Decrypts the metadata of an incoming file transfer using the inter-HSM RECEIVE key. On success, commits the newly received file to flash.

Definition at line 294 of file [handler.c](#).

References [DECRYPTIONERROR](#), [memset\(\)](#), [OPRECEIVE](#), [stage](#), and [StoreFile\(\)](#).

Referenced by [ReceiveParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.27.2.6 ReplyHandler()

```
void ReplyHandler (
    StatusCode * status)
```

Execution flow for generating a Peer REPLY.

Handles the REPLY command from a peer.

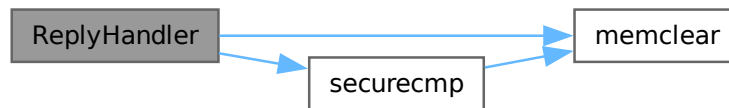
Constructs a FAT table containing all valid files currently held by this HSM. Encrypts the table with the secure Interrogate key before transmission.

Definition at line 195 of file [handler.c](#).

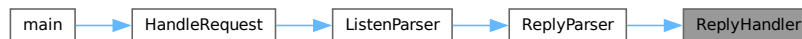
References [ENCRYPTIONERROR](#), [FAT_Table](#), [memclear\(\)](#), [Metadata_Table](#), [NUM_SLOTS](#), [OPREPLY](#), [securecmp\(\)](#), and [stage](#).

Referenced by [ReplyParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.27.2.7 SendHandler()

```

void SendHandler (
    uint8_t slot,
    StatusCode * status)
  
```

Execution flow for a Peer SEND operation.

Begins the SEND command logic (HSM to HSM transfer).

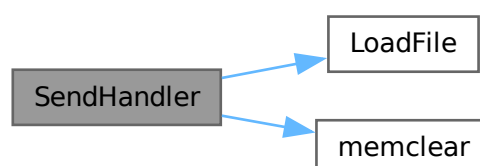
Prepares a file sequence for inter-HSM transfer. Loads the specific slot, re-encrypts the metadata (including the file tag and internal keys) using the inter-HSM SEND key for secure wire transport.

Definition at line 251 of file [handler.c](#).

References [ENCRYPTIONERROR](#), [LoadFile\(\)](#), [memclear\(\)](#), [OPSEND](#), and [stage](#).

Referenced by [SendParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.27.2.8 WriteHandler()

```
void WriteHandler (
    uint8_t slot,
    StatusCode * status)
```

Execution flow for a Host WRITE command.

Handles the WRITE command for a specific slot.

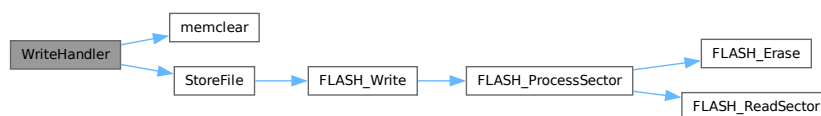
1. Derives the unique write key for the targeted group ID.
2. In-place encrypts the staged plaintext using ASCON-128a.
3. Triggers a commit to persistent flash storage.

Definition at line 95 of file [handler.c](#).

References [ENCRYPTIONERROR](#), [MAX_FILE_SIZE](#), [memset\(\)](#), [OPWRITE](#), [stage](#), and [StoreFile\(\)](#).

Referenced by [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.28 handler.c

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "handler.h"
00012 #include "crypto_aead.h"
00013 #include "filesystem.h"
00014 #include "secrets.h"
00015 #include "status.h"
00016 #include "stubs.h"
00017 #include <string.h>
00018
00019 void ListHandler(StatusCode *status)
00020 {
00021     if (*status != OPLIST)
00022         return;
00023
00024     uint32_t numEntries = 0;
00025     for (uint8_t i = 0; i < NUM_SLOTS; i++)
00026     {
00027         if (Metadata_Table[i].status.magic == 0xdeadf00d &&
00028             securecmp((uint8_t *) Metadata_Table[i].status.id,
00029                     (uint8_t *) FAT_Table.entry[i].uuid, 16) == true)
00030         {
00031             stage.as.fat.slots[numEntries].groupid =
00032                 Metadata_Table[i].data.groupid;
00033             stage.as.fat.slots[numEntries].slot = i;
00034             memcpy((uint8_t *) stage.as.fat.slots[numEntries].filename,
00035                 (uint8_t *) Metadata_Table[i].data.filename, 32);
00036
00037             numEntries++;
00038         }
00039     }
00040
00041     stage.as.fat.numEntries = numEntries;
00042 }
00043
00051 void ReadHandler(uint8_t slot, StatusCode *status)
00052 {
00053     if (*status != OPREAD)
00054         return;
00055
00056     LoadFile(slot, status);
00057     if (*status != OPREAD)
00058         return;
00059
00060     uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00061     *status = SelectReadKey(stage.as.file.metadata.groupid, derivedKey);
00062
00063     int dstat;
00064     if (*status == OPREAD)
00065     {
00066         dstat = ascon_aead_decrypt(
00067             (uint8_t *) &stage.as.file.content,
00068             (uint8_t *) &stage.as.file.metadata.tag,
00069             (uint8_t *) &stage.as.file.content, MAX_FILE_SIZE,
00070             (uint8_t *) stage.as.file.metadata.filename,
00071             32 + sizeof(FS_FileEntry), (uint8_t *) stage.as.file.metadata.nonce,
00072             derivedKey);
00073         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00074     }
00075     else
00076     {
00077         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00078         return;
00079     }
00080
00081     if (dstat != 0)
00082     {
00083         *status = DECRYPTIONERROR;
00084         return;
00085     }
00086 }
00087
00095 void WriteHandler(uint8_t slot, StatusCode *status)
00096 {
00097     if (*status != OPWRITE)
00098         return;
00099
00100     uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00101     *status = SelectWriteKey(stage.as.file.metadata.groupid, derivedKey);
00102
00103     int estat;
00104     if (*status == OPWRITE)

```

```

00105     {
00106         estat = ascon_aead_encrypt(
00107             (uint8_t *) &stage.as.file.metadata.tag,
00108             (uint8_t *) &stage.as.file.content,
00109             (uint8_t *) &stage.as.file.content, MAX_FILE_SIZE,
00110             (uint8_t *) stage.as.file.metadata.filename,
00111             32 + sizeof(FS_FileEntry), (uint8_t *) stage.as.file.metadata.nonce,
00112             derivedKey);
00113         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00114     }
00115     else
00116     {
00117         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00118         return;
00119     }
00120
00121     if (estat != 0)
00122     {
00123         *status = ENCRYPTIONERROR;
00124         return;
00125     }
00126
00127     StoreFile(slot, status);
00128 }
00129
00130 static inline bool isReceivable(uint16_t groupid)
00131 {
00132     for (uint8_t i = 0; i < N_RECV_GROUPS; ++i)
00133     {
00134         if (recv_groups[i] == groupid)
00135             return true;
00136     }
00137     return false;
00138 }
00139
00140 void InterrogateHandler(StatusCode *status)
00141 {
00142     if (*status != OPINTERROGATE)
00143         return;
00144
00145     uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00146     *status = SelectInterrogateKey(derivedKey);
00147
00148     int dstat;
00149     if (*status == OPINTERROGATE)
00150     {
00151         dstat = ascon_aead_decrypt(
00152             (uint8_t *) &stage.as.fat, (uint8_t *) &stage.preamble.tag,
00153             (uint8_t *) &stage.as.fat, sizeof(FS_FAT),
00154             (uint8_t *) &stage.preamble.nonce, NONCE_SIZE + ID_SIZE,
00155             (uint8_t *) stage.preamble.nonce, derivedKey);
00156         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00157     }
00158     else
00159     {
00160         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00161         return;
00162     }
00163
00164     if (dstat != 0)
00165     {
00166         *status = DECRYPTIONERROR;
00167         return;
00168     }
00169
00170     uint32_t recvFiles = 0;
00171     for (uint8_t i = 0; i < stage.as.fat.numEntries; ++i)
00172     {
00173         if (isReceivable(stage.as.fat.slots[i].groupid))
00174         {
00175             stage.as.fat.slots[recvFiles] = stage.as.fat.slots[i];
00176             recvFiles++;
00177         }
00178     }
00179     stage.as.fat.numEntries = recvFiles;
00180 }
00181
00182 void ReplyHandler(StatusCode *status)
00183 {
00184     if (*status != OPREPLY)
00185         return;
00186
00187     uint32_t numEntries = 0;
00188     for (uint8_t i = 0; i < NUM_SLOTS; i++)
00189     {
00190         if (Metadata_Table[i].status.magic == 0xdeadf00d &&
00191             securecmp((uint8_t *) Metadata_Table[i].status.id,

```

```

00205         (uint8_t *) FAT_Table.entry[i].uuid, 16) == true)
00206     {
00207         stage.as.fat.slots[numEntries].groupid =
00208             Metadata_Table[i].data.groupid;
00209         stage.as.fat.slots[numEntries].slot = i;
00210         memcpy((uint8_t *) stage.as.fat.slots[numEntries].filename,
00211             (uint8_t *) Metadata_Table[i].data.filename, 32);
00212
00213         numEntries++;
00214     }
00215 }
00216 stage.as.fat.numEntries = numEntries;
00217
00218 uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00219 *status = SelectReplyKey(derivedKey);
00220
00221 int estat;
00222 if (*status == OPREPLY)
00223 {
00224     estat = ascon_aead_encrypt(
00225         (uint8_t *) &stage.preamble.tag, (uint8_t *) &stage.as.fat,
00226         (uint8_t *) &stage.as.fat, sizeof(FS_FAT),
00227         (uint8_t *) &stage.preamble.nonce, NONCE_SIZE + ID_SIZE,
00228         (uint8_t *) stage.preamble.nonce, derivedKey);
00229     memclear(derivedKey, ASCON_KEY_SIZE, 0);
00230 }
00231 else
00232 {
00233     memclear(derivedKey, ASCON_KEY_SIZE, 0);
00234     return;
00235 }
00236
00237 if (estat != 0)
00238 {
00239     *status = ENCRYPTIONERROR;
00240     return;
00241 }
00242 }
00243
00251 void SendHandler(uint8_t slot, StatusCode *status)
00252 {
00253     if (*status != OPSEND)
00254         return;
00255
00256     LoadFile(slot, status);
00257     if (*status != OPSEND)
00258         return;
00259
00260     uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00261     *status = SelectSendKey(stage.as.file.metadata.groupid, derivedKey);
00262
00263     int estat;
00264     if (*status == OPSEND)
00265     {
00266         estat = ascon_aead_encrypt(
00267             (uint8_t *) &stage.preamble.tag,
00268             (uint8_t *) &stage.as.file.metadata.key,
00269             (uint8_t *) &stage.as.file.metadata.key,
00270             sizeof(FS_FileEntry) + sizeof(FS_Metadata) - sizeof(uint16_t),
00271             (uint8_t *) &stage.preamble.nonce,
00272             NONCE_SIZE + ID_SIZE + sizeof(uint16_t),
00273             (uint8_t *) stage.preamble.nonce, derivedKey);
00274         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00275     }
00276     else
00277     {
00278         memclear(derivedKey, ASCON_KEY_SIZE, 0);
00279         return;
00280     }
00281     if (estat != 0)
00282     {
00283         *status = ENCRYPTIONERROR;
00284         return;
00285     }
00286 }
00287
00294 void ReceiveHandler(uint8_t slot, StatusCode *status)
00295 {
00296     if (*status != OPRECEIVE)
00297         return;
00298     uint8_t derivedKey[ASCON_KEY_SIZE] = {0};
00299     *status = SelectRecvKey(stage.as.file.metadata.groupid, derivedKey);
00300     int dstat;
00301     if (*status == OPRECEIVE)
00302     {
00303         dstat = ascon_aead_decrypt(
00304             (uint8_t *) &stage.as.file.metadata.key,

```



```

00305         (uint8_t *) &stage.preamble.tag,
00306         (uint8_t *) &stage.as.file.metadata.key,
00307         sizeof(FS_FileEntry) + sizeof(FS_Metadata) - sizeof(uint16_t),
00308         (uint8_t *) &stage.preamble.nonce,
00309         NONCE_SIZE + ID_SIZE + sizeof(uint16_t),
00310         (uint8_t *) stage.preamble.nonce, derivedKey);
00311     memclear(derivedKey, ASCON_KEY_SIZE, 0);
00312 }
00313 else
00314 {
00315     memclear(derivedKey, ASCON_KEY_SIZE, 0);
00316     return;
00317 }
00318 if (dstat != 0)
00319 {
00320     *status = DECRYPTIONERROR;
00321     return;
00322 }
00323 StoreFile(slot, status);
00324 }

```

5.29 firmware/src/hsm.c File Reference

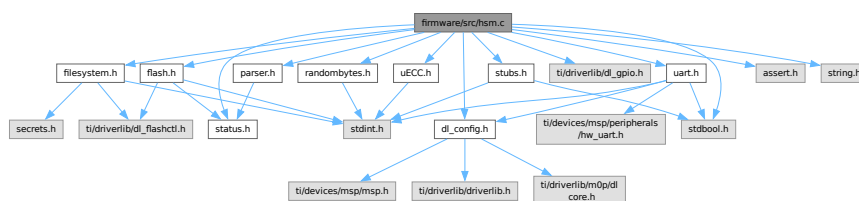
Main entry point and core dispatch loop for the SHIFT HSM.

```

#include "dl_config.h"
#include "filesystem.h"
#include "flash.h"
#include "parser.h"
#include "randombytes.h"
#include "status.h"
#include "stubs.h"
#include "ti/driverlib/dl_gpio.h"
#include "uECC.h"
#include "uart.h"
#include <assert.h>
#include <stdbool.h>
#include <string.h>

```

Include dependency graph for hsm.c:



Macros

- #define HSM_PIN_SIZE 6
- #define OP_LIST 'L'
- #define OP_READ 'R'
- #define OP_WRITE 'W'
- #define OP_INTERROGATE 'I'
- #define OP_RECEIVE 'C'
- #define OP_LISTEN 'N'
- #define LIST_METADATA_SIZE (HSM_PIN_SIZE)

- `#define READ_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t))`
- `#define WRITE_METADATA_SIZE`
- `#define INTERROGATE_METADATA_SIZE (HSM_PIN_SIZE)`
- `#define RECEIVE_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t) + sizeof(uint8_t))`
- `#define LISTEN_METADATA_SIZE 0`
- `#define OP_ERROR 'E'`

Functions

- static int `ecc_rng` (uint8_t *dest, unsigned size)
- static void `init` (void)
- static void `BlinkLED` (void)
- static void `ParseOpcode` (uint8_t opCode, `StatusCode` *status)
Parses an incoming raw byte into a recognized operational Status.
- static void `ValidateBodySize` (uint16_t bodyLen, `StatusCode` *status)
- static void `HandleRequest` (`StatusCode` *status, uint16_t bodyLen)
Top-level request dispatcher based on resolved Command State.
- static void `SendError` (const char *message)
- static void `SendHeader` (uint8_t opCode, uint16_t bodyLen)
- static void `SendResponse` (`StatusCode` *status)
Constructs and transmits the final output frame to the Host.
- int `main` (void)

Variables

- static const uint8_t `host_magic_byte` = '%'

5.29.1 Detailed Description

Main entry point and core dispatch loop for the SHIFT HSM.

Author

Sumedh Girish

This file contains the primary `main` loop that constantly listens for Host or Peer commands over UART. It handles opcode parsing, basic size validation, and dispatches to the appropriate parser, before finally formulating and transmitting a serialized response frame.

Definition in file `hsm.c`.

5.29.2 Macro Definition Documentation

5.29.2.1 HSM_PIN_SIZE

```
#define HSM_PIN_SIZE 6
```

Definition at line 26 of file `hsm.c`.

5.29.2.2 INTERROGATE_METADATA_SIZE

```
#define INTERROGATE_METADATA_SIZE (HSM_PIN_SIZE)
```

Definition at line 101 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.2.3 LIST_METADATA_SIZE

```
#define LIST_METADATA_SIZE (HSM_PIN_SIZE)
```

Definition at line 96 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.2.4 LISTEN_METADATA_SIZE

```
#define LISTEN_METADATA_SIZE 0
```

Definition at line 103 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.2.5 OP_ERROR

```
#define OP_ERROR 'E'
```

Definition at line 178 of file [hsm.c](#).

Referenced by [InterrogateParser\(\)](#), [ReceiveParser\(\)](#), [ReplyParser\(\)](#), [SendError\(\)](#), and [SendParser\(\)](#).

5.29.2.6 OP_INTERROGATE

```
#define OP_INTERROGATE 'I'
```

Definition at line 58 of file [hsm.c](#).

Referenced by [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ParseOpcode\(\)](#), [ReplyParser\(\)](#), and [SendResponse\(\)](#).

5.29.2.7 OP_LIST

```
#define OP_LIST 'L'
```

Definition at line 55 of file [hsm.c](#).

Referenced by [ParseOpcode\(\)](#), and [SendResponse\(\)](#).

5.29.2.8 OP_LISTEN

```
#define OP_LISTEN 'N'
```

Definition at line 60 of file [hsm.c](#).

Referenced by [ParseOpcode\(\)](#), and [SendResponse\(\)](#).

5.29.2.9 OP_READ

```
#define OP_READ 'R'
```

Definition at line 56 of file [hsm.c](#).

Referenced by [ParseOpcode\(\)](#), and [SendResponse\(\)](#).

5.29.2.10 OP_RECEIVE

```
#define OP_RECEIVE 'C'
```

Definition at line 59 of file [hsm.c](#).

Referenced by [ListenParser\(\)](#), [ParseOpcode\(\)](#), [ReceiveParser\(\)](#), [SendParser\(\)](#), and [SendResponse\(\)](#).

5.29.2.11 OP_WRITE

```
#define OP_WRITE 'W'
```

Definition at line 57 of file [hsm.c](#).

Referenced by [ParseOpcode\(\)](#), and [SendResponse\(\)](#).

5.29.2.12 READ_METADATA_SIZE

```
#define READ_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t))
```

Definition at line 97 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.2.13 RECEIVE_METADATA_SIZE

```
#define RECEIVE_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t) + sizeof(uint8_t))
```

Definition at line 102 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.2.14 WRITE_METADATA_SIZE

```
#define WRITE_METADATA_SIZE
```

Value:

```
(HSM_PIN_SIZE + sizeof(uint8_t) + sizeof(uint16_t) + 32 + 16 +  
 sizeof(uint16_t)) \
```

Definition at line 98 of file [hsm.c](#).

Referenced by [ValidateBodySize\(\)](#).

5.29.3 Function Documentation

5.29.3.1 BlinkLED()

```
void BlinkLED (  
    void ) [static]
```

Definition at line 46 of file [hsm.c](#).

References [LEDS_PORT](#), and [LEDS_STATUS_LED_PIN](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



5.29.3.2 ecc_rng()

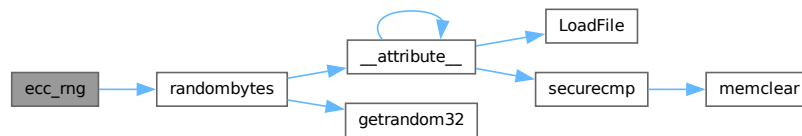
```
int ecc_rng (  
    uint8_t * dest,  
    unsigned size) [static]
```

Definition at line 28 of file [hsm.c](#).

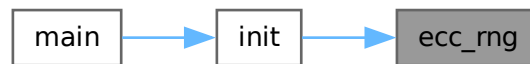
References [randombytes\(\)](#).

Referenced by [init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.3 HandleRequest()

```

void HandleRequest (
    StatusCode * status,
    uint16_t bodyLen) [inline], [static]
  
```

Top-level request dispatcher based on resolved Command State.

Routes execution to specific parsers which will handle subsequent UART payloads and trigger the actual Handler routines.

Parameters

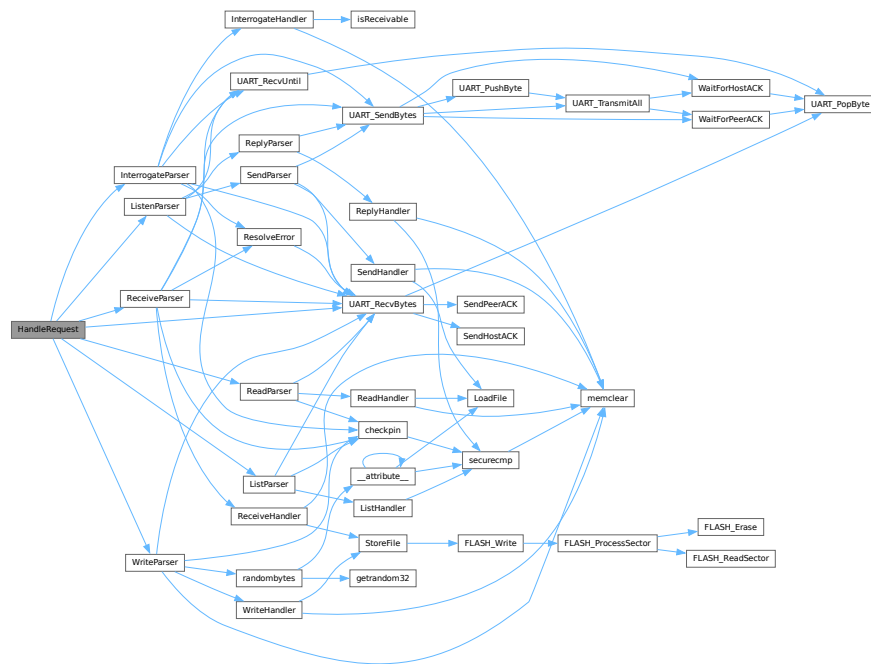
<i>status</i>	Current validated system state.
<i>bodyLen</i>	Expected length of the incoming payload body.

Definition at line 149 of file [hsm.c](#).

References [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ListParser\(\)](#), [OPINTERROGATE](#), [OPLIST](#), [OPLISTEN](#), [OPREAD](#), [OPRECEIVE](#), [OPWRITE](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), [UART_HOST](#), [UART_RecvBytes\(\)](#), and [WriteParser\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.4 init()

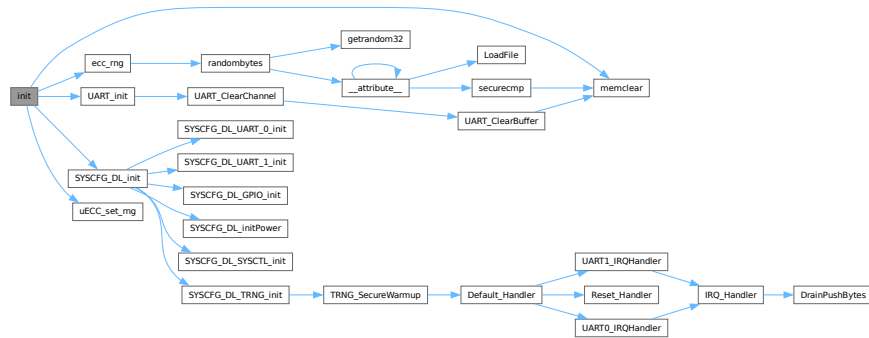
```
void init (
    void ) [static]
```

Definition at line 34 of file [hsm.c](#).

References [ecc_rng\(\)](#), [memclear\(\)](#), [stage](#), [SYSCFG_DL_init\(\)](#), [UART_init\(\)](#), and [uECC_set_rng\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.5 main()

```
int main (
    void )
```

Definition at line 291 of file [hsm.c](#).

References [BlinkLED\(\)](#), [FLASH_Erase\(\)](#), [HandleRequest\(\)](#), [host_magic_byte](#), [init\(\)](#), [memclear\(\)](#), [ParseOpcode\(\)](#), [SendResponse\(\)](#), [stage](#), [SystemStatus](#), [UART_HOST](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UNKNOWNOP](#), and [ValidateBodySize\(\)](#).


```
void ParseOpcode (
    uint8_t opCode,
    StatusCode * status) [inline], [static]
```

Parameters

<i>opCode</i>	The raw byte character from UART.
<i>status</i>	Pointer to the current system state.

Generated by Doxygen

References [OP_INTERROGATE](#), [OP_LIST](#), [OP_LISTEN](#), [OP_READ](#), [OP_RECEIVE](#), [OP_WRITE](#), [OPINTERROGATE](#), [OPLIST](#), [OPLISTEN](#), [OPREAD](#), [OPRECEIVE](#), [OPWRITE](#), and [UNKNOWNOP](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



5.29.3.7 SendError()

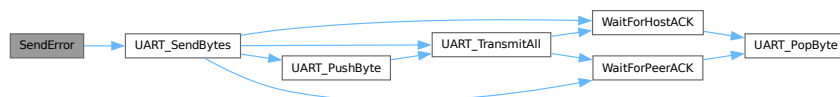
```
void SendError (
    const char * message) [inline], [static]
```

Definition at line 181 of file [hsm.c](#).

References [host_magic_byte](#), [OP_ERROR](#), [UART_HOST](#), and [UART_SendBytes\(\)](#).

Referenced by [SendResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.8 SendHeader()

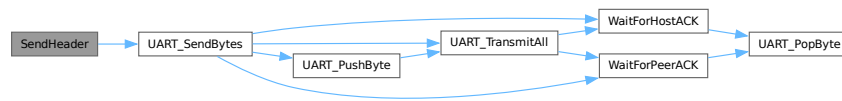
```
void SendHeader (
    uint8_t opCode,
    uint16_t bodyLen) [inline], [static]
```

Definition at line 191 of file [hsm.c](#).

References [host_magic_byte](#), [UART_HOST](#), and [UART_SendBytes\(\)](#).

Referenced by [SendResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.9 SendResponse()

```
void SendResponse (
    StatusCode * status) [inline], [static]
```

Constructs and transmits the final output frame to the Host.

Serializes data from the volatile [stage](#) into the UART TX buffer. It handles successful payload transmissions as well as sending human-readable error messages.

Parameters

<i>status</i>	Final state of the request (success or specific error code).
---------------	--

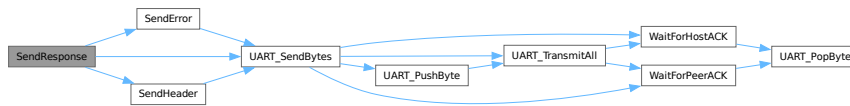
Definition at line 206 of file [hsm.c](#).

References [DECRYPTIONERROR](#), [ENCRYPTIONERROR](#), [FLASHERASEERROR](#), [FLASHWRITEERROR](#), [INVALIDBODYSIZE](#), [INVALIDSLOT](#), [KEYGENERROR](#), [OP_INTERROGATE](#), [OP_LIST](#), [OP_LISTEN](#), [OP_READ](#), [OP_RECEIVE](#), [OP_WRITE](#), [OPINTERROGATE](#), [OPLIST](#), [OPLISTEN](#), [OPREAD](#), [OPRECEIVE](#), [OPREPLY](#),

[OPSEND](#), [OPWRITE](#), [PEERERROR](#), [PERMISSIONERROR](#), [SendError\(\)](#), [SendHeader\(\)](#), [SLOTEMPTY](#), [stage](#), [UART_HOST](#), [UART_SendBytes\(\)](#), and [UNKNOWNOP](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.3.10 ValidateBodySize()

```

void ValidateBodySize (
    uint16_t bodyLen,
    StatusCode * status) [inline], [static]
  
```

Definition at line 105 of file [hsm.c](#).

References [INTERROGATE_METADATA_SIZE](#), [INVALIDBODYSIZE](#), [LIST_METADATA_SIZE](#), [LISTEN_METADATA_SIZE](#), [MAX_FILE_SIZE](#), [OPINTERROGATE](#), [OPLIST](#), [OPLISTEN](#), [OPREAD](#), [OPRECEIVE](#), [OPWRITE](#), [READ_METADATA_SIZE](#), [RECEIVE_METADATA_SIZE](#), and [WRITE_METADATA_SIZE](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



5.29.4 Variable Documentation

5.29.4.1 host_magic_byte

`const uint8_t host_magic_byte = '%' [static]`

Definition at line 179 of file `hsm.c`.

Referenced by `main()`, `SendError()`, and `SendHeader()`.

5.30 hsm.c

[Go to the documentation of this file.](#)

```

00001
00011
00012 #include "dl_config.h"
00013 #include "filesystem.h"
00014 #include "flash.h"
00015 #include "parser.h"
00016 #include "randombytes.h"
00017 #include "status.h"
00018 #include "stubs.h"
00019 #include "ti/driverlib/dl_gpio.h"
00020 #include "uECC.h"
00021 #include "uart.h"
00022 #include <assert.h>
00023 #include <stdbool.h>
00024 #include <string.h>
00025
00026 #define HSM_PIN_SIZE 6
00027
00028 static int ecc_rng(uint8_t *dest, unsigned size)
00029 {
00030     randombytes(dest, size);
00031     return 1;
00032 }
00033
00034 static void init(void)
00035 {
00036     __disable_irq();
00037
00038     UART_init();
00039     SYSCFG_DL_init();
00040     memclear((uint8_t *) &stage, sizeof(FS_Stage), 0);
00041     uECC_set_rng(&ecc_rng);
00042
00043     __enable_irq();
00044 }
00045
00046 static void BlinkLED(void)
00047 {
00048     while (true)
00049     {
00050         DL_GPIO_togglePins(LED_PORT, LED_STATUS_LED_PIN);
00051         delay_cycles(32000000 * 2); // 2 seconds
00052     }
00053 }
00054
00055 #define OP_LIST 'L'
00056 #define OP_READ 'R'
00057 #define OP_WRITE 'W'
00058 #define OP_INTERROGATE 'I'
00059 #define OP_RECEIVE 'C'
00060 #define OP_LISTEN 'N'
00061
00062 static inline void ParseOpcode(uint8_t opCode, StatusCode *status)
00063 {
00064     switch (opCode)
00065     {
00066     case OP_LIST:
00067         *status = OPLIST;
00068         break;
00069     case OP_READ:
00070         *status = OPREAD;
00071         break;
00072     }
00073 }

```

```

00078         case OP_WRITE:
00079             *status = OPWRITE;
00080             break;
00081         case OP_INTERROGATE:
00082             *status = OPINTERROGATE;
00083             break;
00084         case OP_RECEIVE:
00085             *status = OPRECEIVE;
00086             break;
00087         case OP_LISTEN:
00088             *status = OPLISTEN;
00089             break;
00090         default:
00091             *status = UNKNOWNOP;
00092             break;
00093     }
00094 }
00095
00096 #define LIST_METADATA_SIZE (HSM_PIN_SIZE)
00097 #define READ_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t))
00098 #define WRITE_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t) + sizeof(uint16_t) + 32 + 16 +
00099                                     sizeof(uint16_t))
00100 #define INTERROGATE_METADATA_SIZE (HSM_PIN_SIZE)
00101 #define RECEIVE_METADATA_SIZE (HSM_PIN_SIZE + sizeof(uint8_t) + sizeof(uint8_t))
00102 #define LISTEN_METADATA_SIZE 0
00103
00104 static inline void ValidateBodySize(uint16_t bodyLen, StatusCode *status)
00105 {
00106     bool validWriteSize = false;
00107     switch (*status)
00108     {
00109         case OPLIST:
00110             *status =
00111                 (bodyLen == LIST_METADATA_SIZE) ? (*status) : INVALIDBODYSIZE;
00112             break;
00113         case OPREAD:
00114             *status =
00115                 (bodyLen == READ_METADATA_SIZE) ? (*status) : INVALIDBODYSIZE;
00116             break;
00117         case OPWRITE:
00118             validWriteSize = (bodyLen >= WRITE_METADATA_SIZE) &&
00119                             ((bodyLen - WRITE_METADATA_SIZE) <= MAX_FILE_SIZE);
00120             *status = (validWriteSize) ? (*status) : INVALIDBODYSIZE;
00121             break;
00122         case OPINTERROGATE:
00123             *status = (bodyLen == INTERROGATE_METADATA_SIZE) ? (*status)
00124                                                         : INVALIDBODYSIZE;
00125             break;
00126         case OPRECEIVE:
00127             *status = (bodyLen == RECEIVE_METADATA_SIZE) ? (*status)
00128                                                         : INVALIDBODYSIZE;
00129             break;
00130         case OPLISTEN:
00131             *status =
00132                 (bodyLen == LISTEN_METADATA_SIZE) ? (*status) : INVALIDBODYSIZE;
00133             break;
00134         default:
00135             break;
00136     }
00137 }
00138
00139 static inline void HandleRequest(StatusCode *status, uint16_t bodyLen)
00140 {
00141     switch (*status)
00142     {
00143         case OPLIST:
00144             ListParser(status);
00145             break;
00146         case OPREAD:
00147             ReadParser(status);
00148             break;
00149         case OPWRITE:
00150             WriteParser(status);
00151             break;
00152         case OPINTERROGATE:
00153             InterrogateParser(status);
00154             break;
00155         case OPRECEIVE:
00156             ReceiveParser(status);
00157             break;
00158         case OPLISTEN:
00159             ListenParser(status);
00160             break;
00161         default:
00162             if (bodyLen > 0)
00163                 UART_RecvBytes(UART_HOST, NULL, bodyLen, true);
00164     }
00165 }

```

```

00174         break;
00175     }
00176 }
00177
00178 #define OP_ERROR 'E'
00179 static const uint8_t host_magic_byte = '%';
00180
00181 static inline void SendError(const char *message)
00182 {
00183     uint8_t responseCode = OP_ERROR;
00184     uint16_t responseLen = (uint16_t) strlen(message);
00185     UART_SendBytes(UART_HOST, (uint8_t *) &host_magic_byte, 1, false);
00186     UART_SendBytes(UART_HOST, (uint8_t *) &responseCode, 1, false);
00187     UART_SendBytes(UART_HOST, (uint8_t *) &responseLen, 2, true);
00188     UART_SendBytes(UART_HOST, (uint8_t *) message, responseLen, true);
00189 }
00190
00191 static inline void SendHeader(uint8_t opCode, uint16_t bodyLen)
00192 {
00193     UART_SendBytes(UART_HOST, (uint8_t *) &host_magic_byte, 1, false);
00194     UART_SendBytes(UART_HOST, (uint8_t *) &opCode, 1, false);
00195     UART_SendBytes(UART_HOST, (uint8_t *) &bodyLen, 2, true);
00196 }
00197
00206 static inline void SendResponse(StatusCode *status)
00207 {
00208     switch (*status)
00209     {
00210         case OPLIST:
00211             SendHeader(OP_LIST,
00212                 (uint16_t) (stage.as.fat.numEntries * sizeof(FS_Slot) +
00213                     sizeof(uint32_t)));
00214             UART_SendBytes(UART_HOST, (uint8_t *) &stage.as.fat.numEntries,
00215                 sizeof(uint32_t), false);
00216             UART_SendBytes(UART_HOST, (uint8_t *) stage.as.fat.slots,
00217                 (stage.as.fat.numEntries * sizeof(FS_Slot)), true);
00218             break;
00219         case OPINTERROGATE:
00220             SendHeader(OP_INTERROGATE,
00221                 (uint16_t) (stage.as.fat.numEntries * sizeof(FS_Slot) +
00222                     sizeof(uint32_t)));
00223             UART_SendBytes(UART_HOST, (uint8_t *) &stage.as.fat.numEntries,
00224                 sizeof(uint32_t), false);
00225             UART_SendBytes(UART_HOST, (uint8_t *) stage.as.fat.slots,
00226                 (stage.as.fat.numEntries * sizeof(FS_Slot)), true);
00227             break;
00228         case OPREAD:
00229             SendHeader(OP_READ, stage.as.file.entry.filesize + 32);
00230             UART_SendBytes(UART_HOST,
00231                 (uint8_t *) stage.as.file.metadata.filename, 32,
00232                 false);
00233             UART_SendBytes(UART_HOST, (uint8_t *) stage.as.file.content,
00234                 stage.as.file.entry.filesize, true);
00235             break;
00236         case OPWRITE:
00237             SendHeader(OP_WRITE, 0);
00238             break;
00239         case OPREPLY:
00240         case OPSEND:
00241         case OPLISTEN:
00242             SendHeader(OP_LISTEN, 0);
00243             break;
00244         case OPRECEIVE:
00245             SendHeader(OP_RECEIVE, 0);
00246             break;
00247         case FLASHWRITEERROR:
00248             SendError("ERROR: Damn, I couldn't commit that to memory.");
00249             break;
00250         case FLASHERASEERROR:
00251             SendError("ERROR: Damn, I can't forget that now that I know it.");
00252             break;
00253         case INVALIDSLOT:
00254             SendError("ERROR: That slot ain't flying.");
00255             break;
00256         case SLOTEEMPTY:
00257             SendError(
00258                 "ERROR: I can't read an empty slot. What did you expect?");
00259             break;
00260         case KEYGENERERROR:
00261             SendError("ERROR: Misplaced my keys! Oops.");
00262             break;
00263         case DECRYPTIONERROR:
00264             SendError(
00265                 "ERROR: Decryption failed. Maybe the data was corrupted?");
00266             break;
00267         case ENCRYPTIONERROR:
00268             SendError("ERROR: Encryption failed.");

```

```

00269         break;
00270     case PERMISSIONERROR:
00271         SendError(
00272             "ERROR: You dont have permission to do that. Sucks to be you.");
00273         break;
00274     case INVALIDBODYSIZE:
00275         SendError("ERROR: That message was too fat for comfort.");
00276         break;
00277     case PEERERROR:
00278         SendError("ERROR: Something went wrong with the peer. Maybe they "
00279             "sent something weird?");
00280         break;
00281     case UNKNOWNOP:
00282         SendError("ERROR: You somehow managed send something that I "
00283             "cant parse. Congrats!");
00284         break;
00285     default:
00286         SendError("ERROR: Request not sexy enough, hogli bidu");
00287         break;
00288     }
00289 }
00290
00291 int main(void)
00292 {
00293     init();
00294
00295     StatusCode status;
00296     uint8_t opCode;
00297
00298     while (true)
00299     {
00300         memclear((uint8_t *) &stage, sizeof(FS_Stage), 0);
00301         status = UNKNOWNOP;
00302         while (status == UNKNOWNOP)
00303         {
00304             UART_RecvUntil(UART_HOST, host_magic_byte);
00305             UART_RecvBytes(UART_HOST, &opCode, 1, false);
00306             ParseOpcode(opCode, &status);
00307         }
00308
00309         uint16_t bodyLen = 0;
00310         UART_RecvBytes(UART_HOST, (uint8_t *) &bodyLen, 2, true);
00311
00312         ValidateBodySize(bodyLen, &status);
00313
00314         HandleRequest(&status, bodyLen);
00315
00316         if (SystemStatus.timeout == 0)
00317         {
00318             delay_cycles((32000000 * 9) / 2);
00319             FLASH_Erase((uint32_t) &SystemStatus, &status);
00320         }
00321
00322         SendResponse(&status);
00323     }
00324
00325     BlinkLED();
00326     return 0; /* Unreachable */
00327 }

```

5.31 firmware/src/parser.c File Reference

Request stream parsing and initial validation layer.

```

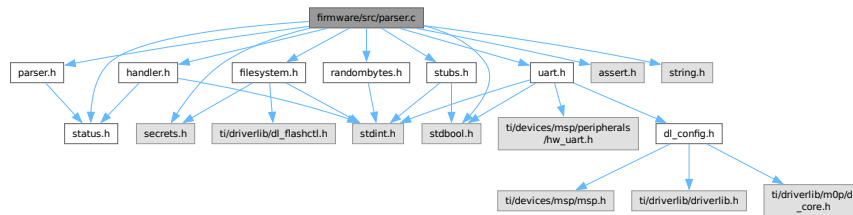
#include "parser.h"
#include "filesystem.h"
#include "handler.h"
#include "randombytes.h"
#include "secrets.h"
#include "status.h"
#include "stubs.h"
#include "uart.h"
#include <assert.h>
#include <stdbool.h>

```



```
#include <string.h>
```

Include dependency graph for parser.c:



Macros

- `#define OP_INTERROGATE 'I'`
- `#define OP_RECEIVE 'R'`
- `#define OP_ERROR 'E'`

Functions

- void `ListParser` (`StatusCode` *status)
Parses an inbound Host LIST command and verifies the user PIN.
- void `ReadParser` (`StatusCode` *status)
Parses an inbound Host READ command, fetching target slot and PIN.
- void `WriteParser` (`StatusCode` *status)
Parses an inbound Host WRITE command and manages volatile staging.
- static void `ResolveError` (`StatusCode` *status)
- void `InterrogateParser` (`StatusCode` *status)
Parses and validates an INTERROGATE command frame.
- void `ReceiveParser` (`StatusCode` *status)
Parses an inbound Peer RECEIVE command, intercepting secure file transfers.
- static void `ReplyParser` (`StatusCode` *status)
- static void `SendParser` (`StatusCode` *status)
- void `ListenParser` (`StatusCode` *status)
Parses and validates a LISTEN command frame.

Variables

- `const uint8_t peer_magic_byte = '^'`

5.31.1 Detailed Description

Request stream parsing and initial validation layer.

Author

Sumedh Girish

The parser layer reads incoming UART payloads, extracts parameters like user PINs, slot indices, and payload sizes, and performs the first layer of authorization (e.g., matching the User PIN) before handing off cleanly parsed structures to the core Handlers.

Definition in file [parser.c](#).

5.31.2 Macro Definition Documentation

5.31.2.1 OP_ERROR

```
#define OP_ERROR 'E'
```

Definition at line 127 of file [parser.c](#).

5.31.2.2 OP_INTERROGATE

```
#define OP_INTERROGATE 'I'
```

Definition at line 125 of file [parser.c](#).

5.31.2.3 OP_RECEIVE

```
#define OP_RECEIVE 'R'
```

Definition at line 126 of file [parser.c](#).

5.31.3 Function Documentation

5.31.3.1 InterrogateParser()

```
void InterrogateParser (
    StatusCode * status)
```

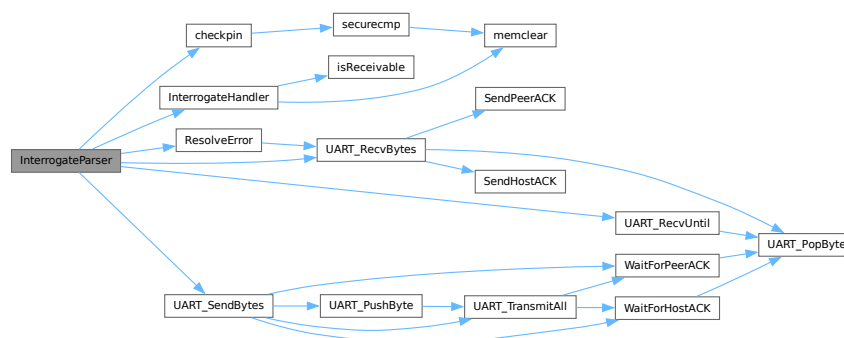
Parses and validates an INTERROGATE command frame.

Definition at line 150 of file [parser.c](#).

References [checkpin\(\)](#), [InterrogateHandler\(\)](#), [INVALIDBODYSIZE](#), [OP_ERROR](#), [OP_INTERROGATE](#), [OPINTERROGATE](#), [peer_magic_byte](#), [PERMISSIONERROR](#), [ResolveError\(\)](#), [stage](#), [UART_HOST](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UART_SendBytes\(\)](#), and [UNKNOWNNOP](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.2 ListenParser()

```
void ListenParser (
    StatusCode * status)
```

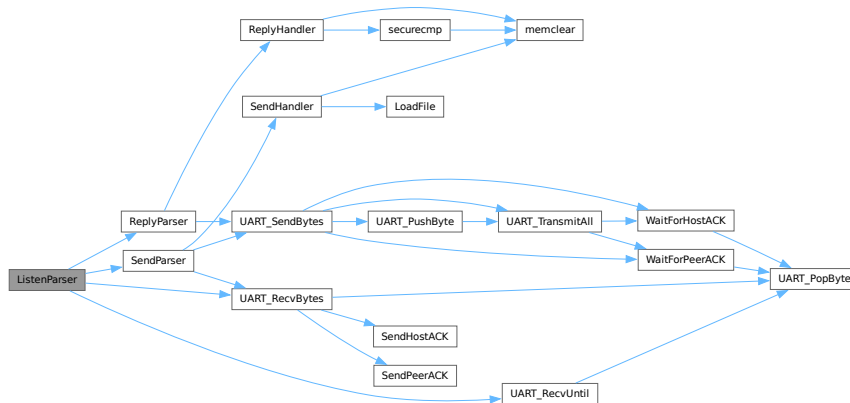
Parses and validates a LISTEN command frame.

Definition at line 364 of file [parser.c](#).

References [INVALIDBODYSIZE](#), [OP_INTERROGATE](#), [OP_RECEIVE](#), [OPREPLY](#), [OPSEND](#), [peer_magic_byte](#), [ReplyParser\(\)](#), [SendParser\(\)](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), and [UNKNOWNOP](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.3 ListParser()

```
void ListParser (
    StatusCode * status)
```

Parses an inbound Host LIST command and verifies the user PIN.

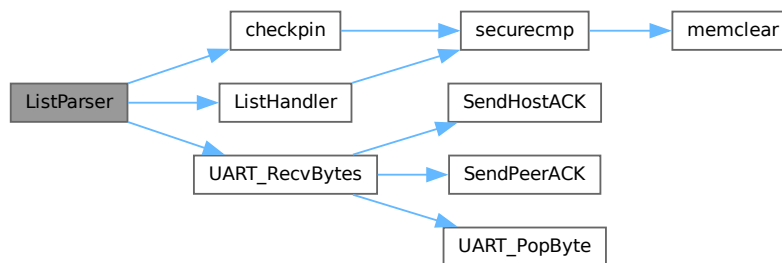
Parses and validates a LIST command frame.

Definition at line 27 of file [parser.c](#).

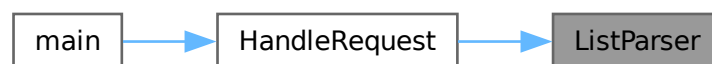
References [checkpin\(\)](#), [ListHandler\(\)](#), [PERMISSIONERROR](#), [UART_HOST](#), and [UART_RecvBytes\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.4 ReadParser()

```
void ReadParser (
    StatusCode * status)
```

Parses an inbound Host READ command, fetching target slot and PIN.

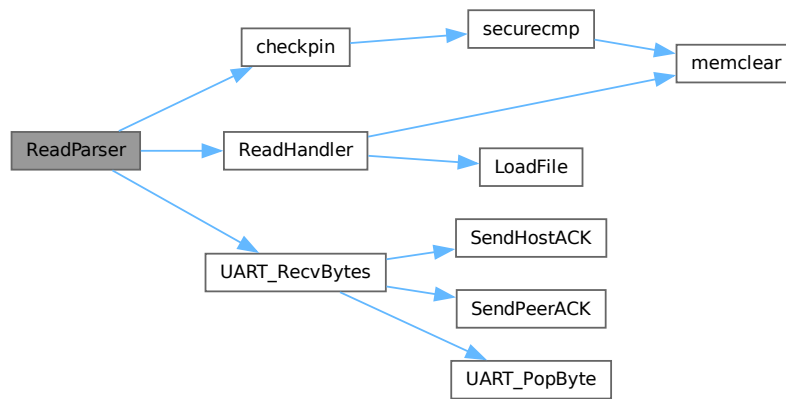
Parses and validates a READ command frame.

Definition at line 44 of file [parser.c](#).

References [checkpin\(\)](#), [INVALIDSLOT](#), [NUM_SLOTS](#), [PERMISSIONERROR](#), [ReadHandler\(\)](#), [UART_HOST](#), and [UART_RecvBytes\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.5 ReceiveParser()

```
void ReceiveParser (
    StatusCode * status)
```

Parses an inbound Peer RECEIVE command, intercepting secure file transfers.

Parses and validates a RECEIVE command frame.

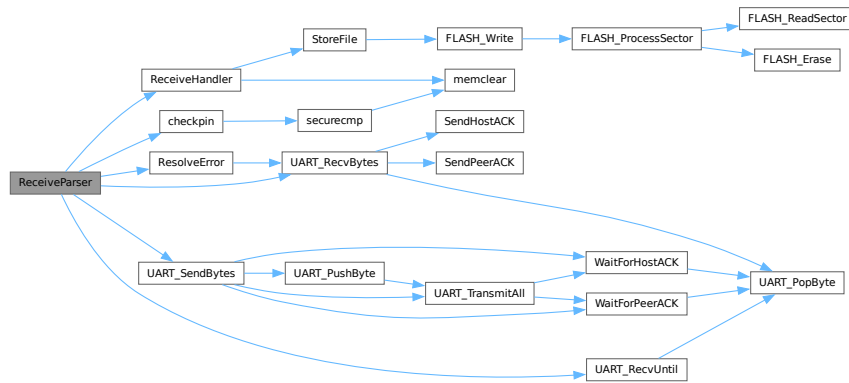
Reads metadata and the encrypted file chunk from the peer link, syncing it into the `stage` buffer before requesting handler decryption and storage.

Definition at line 219 of file [parser.c](#).

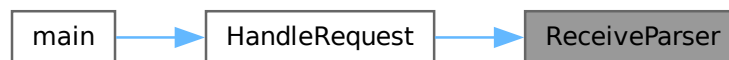
References [checkpin\(\)](#), [INVALIDBODYSIZE](#), [INVALIDSLOT](#), [NUM_SLOTS](#), [OP_ERROR](#), [OP_RECEIVE](#), [OPRECEIVE](#), [peer_magic_byte](#), [PERMISSIONERROR](#), [ReceiveHandler\(\)](#), [ResolveError\(\)](#), [stage](#), [UART_HOST](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [UART_SendBytes\(\)](#), and [UNKNOWNOP](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.6 ReplyParser()

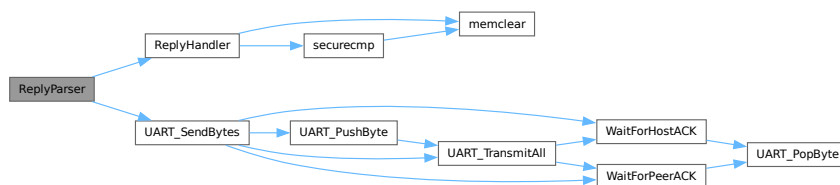
```
void ReplyParser (
    StatusCode * status) [static]
```

Definition at line 290 of file [parser.c](#).

References [OP_ERROR](#), [OP_INTERROGATE](#), [OPREPLY](#), [peer_magic_byte](#), [ReplyHandler\(\)](#), [stage](#), [UART_PEER](#), and [UART_SendBytes\(\)](#).

Referenced by [ListenParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.7 ResolveError()

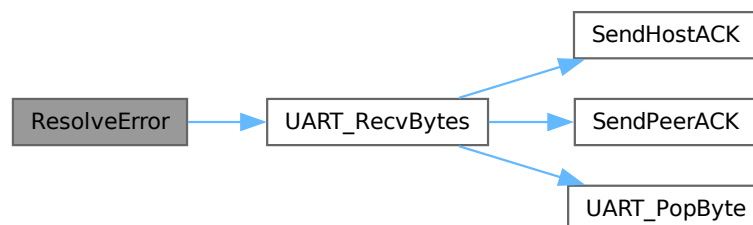
```
void ResolveError (  
    StatusCode * status) [inline], [static]
```

Definition at line 129 of file [parser.c](#).

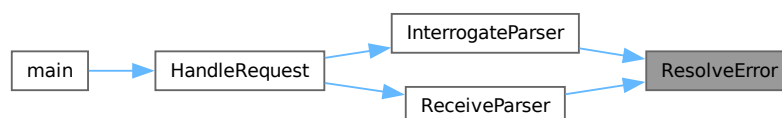
References [INVALIDBODYSIZE](#), [INVALIDSLOT](#), [PEERERROR](#), [PERMISSIONERROR](#), [UART_PEER](#), and [UART_RecvBytes\(\)](#).

Referenced by [InterrogateParser\(\)](#), and [ReceiveParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.8 SendParser()

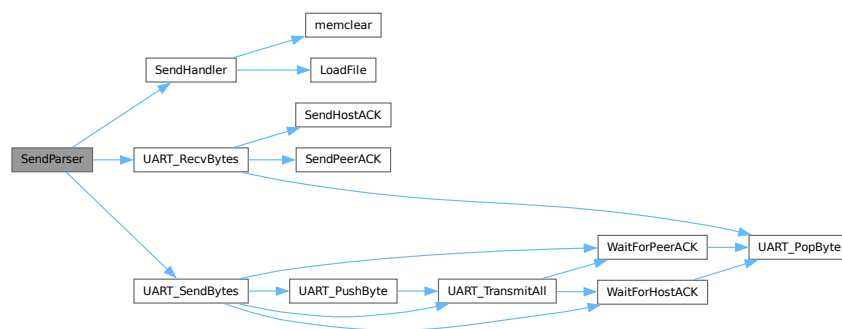
```
void SendParser (
    StatusCode * status) [static]
```

Definition at line 324 of file [parser.c](#).

References [INVALIDSLOT](#), [NUM_SLOTS](#), [OP_ERROR](#), [OP_RECEIVE](#), [OPSEND](#), [peer_magic_byte](#), [SendHandler\(\)](#), [stage](#), [UART_PEER](#), [UART_RecvBytes\(\)](#), and [UART_SendBytes\(\)](#).

Referenced by [ListenParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.3.9 WriteParser()

```
void WriteParser (
    StatusCode * status)
```

Parses an inbound Host WRITE command and manages volatile staging.

Parses and validates a WRITE command frame.

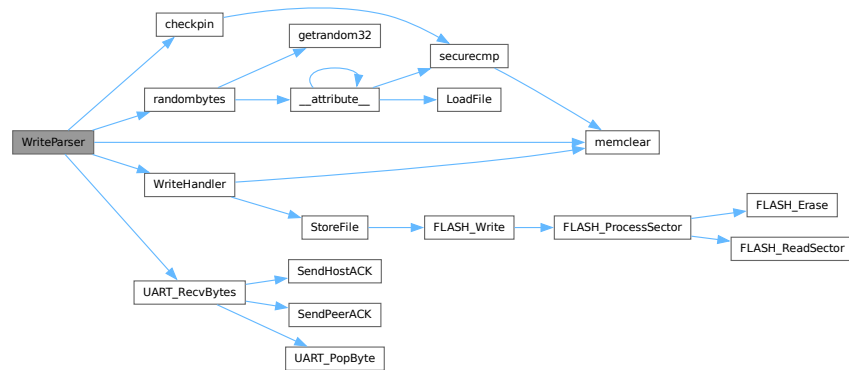
Consumes the PIN, target slot, group assignment, filename, and payload. If verification fails natively, the UART is drained to maintain synchronization.

Definition at line 73 of file [parser.c](#).

References [checkpin\(\)](#), [INVALIDBODYSIZE](#), [INVALIDSLOT](#), [MAX_FILE_SIZE](#), [memclear\(\)](#), [NUM_SLOTS](#), [PERMISSIONERROR](#), [randombytes\(\)](#), [stage](#), [UART_HOST](#), [UART_RecvBytes\(\)](#), and [WriteHandler\(\)](#).

Referenced by [HandleRequest\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.31.4 Variable Documentation

5.31.4.1 peer_magic_byte

```
const uint8_t peer_magic_byte = '^'
```

Definition at line 124 of file [parser.c](#).

Referenced by [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ReceiveParser\(\)](#), [ReplyParser\(\)](#), and [SendParser\(\)](#).

5.32 parser.c

[Go to the documentation of this file.](#)

```

00001
00011
00012 #include "parser.h"
00013 #include "filesystem.h"
00014 #include "handler.h"
00015 #include "randombytes.h"
00016 #include "secrets.h"
00017 #include "status.h"
00018 #include "stubs.h"
00019 #include "uart.h"
00020 #include <assert.h>
00021 #include <stdbool.h>
00022 #include <string.h>
00023
00027 void ListParser(StatusCode *status)
00028 {
00029     uint8_t pin[6] = {0};
00030     UART_RecvBytes(UART_HOST, pin, 6, true);
00031
00032     if (!checkpin(pin))
00033     {
00034         *status = PERMISSIONERROR;
00035         return;
00036     }
00037
00038     ListHandler(status);
00039 }
00040
00044 void ReadParser(StatusCode *status)
00045 {
00046     uint8_t pin[6] = {0};
00047     uint8_t readSlot;
00048
00049     UART_RecvBytes(UART_HOST, pin, 6, false);
00050     UART_RecvBytes(UART_HOST, &readSlot, 1, true);
00051
00052     if (!checkpin(pin))
00053     {
00054         *status = PERMISSIONERROR;
00055         return;
00056     }
00057
00058     if (readSlot >= NUM_SLOTS)
00059     {
00060         *status = INVALIDSLOT;
00061         return;
00062     }
00063
00064     ReadHandler(readSlot, status);
00065 }
00066
00073 void WriteParser(StatusCode *status)
00074 {
00075     uint8_t pin[6] = {0};
00076     uint8_t writeSlot;
00077     uint16_t groupid;
00078     uint8_t filename[32] = {0};
00079     uint16_t filesize = 0;
00080
00081     UART_RecvBytes(UART_HOST, pin, 6, false);
00082     UART_RecvBytes(UART_HOST, &writeSlot, 1, false);
00083     UART_RecvBytes(UART_HOST, (uint8_t *) &groupid, 2, false);
00084     UART_RecvBytes(UART_HOST, filename, 32, false);
00085     UART_RecvBytes(UART_HOST, (uint8_t *) stage.as.file.entry.uuid, 16, false);
00086     UART_RecvBytes(UART_HOST, (uint8_t *) &filesize, 2, false);
00087
00088     if (!checkpin(pin))
00089     {
00090         memclear((uint8_t *) &stage, sizeof(stage), 0);
00091         UART_RecvBytes(UART_HOST, NULL, filesize, true);
00092         *status = PERMISSIONERROR;
00093         return;
00094     }
00095
00096     if (writeSlot >= NUM_SLOTS)
00097     {
00098         memclear((uint8_t *) &stage, sizeof(stage), 0);
00099         UART_RecvBytes(UART_HOST, NULL, filesize, true);
00100         *status = INVALIDSLOT;
00101         return;
00102     }
00103

```

```

00104     if (filesize > MAX_FILE_SIZE)
00105     {
00106         memclear((uint8_t *) &stage, sizeof(stage), 0);
00107         UART_RecvBytes(UART_HOST, NULL, filesize, true);
00108         *status = INVALIDBODYSIZE;
00109         return;
00110     }
00111
00112     UART_RecvBytes(UART_HOST, (uint8_t *) stage.as.file.content, filesize,
00113                   true);
00114
00115     strncpy((char *) stage.as.file.metadata.filename, (const char *) filename,
00116            32);
00117     stage.as.file.metadata.groupid = groupid;
00118     stage.as.file.entry.filesize = filesize;
00119     randombytes((uint8_t *) stage.as.file.metadata.fileid, ID_SIZE);
00120
00121     WriteHandler(writeSlot, status);
00122 }
00123
00124 const uint8_t peer_magic_byte = '^';
00125 #define OP_INTERROGATE 'I'
00126 #define OP_RECEIVE 'R'
00127 #define OP_ERROR 'E'
00128
00129 static inline void ResolveError(StatusCode *status)
00130 {
00131     uint16_t errorCode;
00132     UART_RecvBytes(UART_PEER, (uint8_t *) &errorCode, 2, true);
00133     switch (errorCode)
00134     {
00135         case PERMISSIONERROR:
00136             *status = PERMISSIONERROR;
00137             break;
00138         case INVALIDSLOT:
00139             *status = INVALIDSLOT;
00140             break;
00141         case INVALIDBODYSIZE:
00142             *status = INVALIDBODYSIZE;
00143             break;
00144         default:
00145             *status = PEERERROR;
00146             break;
00147     }
00148 }
00149
00150 void InterrogateParser(StatusCode *status)
00151 {
00152     uint8_t pin[6] = {0};
00153     UART_RecvBytes(UART_HOST, pin, 6, true);
00154
00155     if (!checkpin(pin))
00156     {
00157         *status = PERMISSIONERROR;
00158         return;
00159     }
00160
00161     uint8_t opCode = OP_INTERROGATE;
00162     UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00163     UART_SendBytes(UART_PEER, &opCode, 1, false);
00164
00165     uint16_t bodyLen = 0;
00166     UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00167
00168     *status = UNKNOWNOP;
00169     while (*status == UNKNOWNOP)
00170     {
00171         UART_RecvUntil(UART_PEER, peer_magic_byte);
00172         UART_RecvBytes(UART_PEER, &opCode, 1, false);
00173         switch (opCode)
00174         {
00175             case OP_INTERROGATE:
00176                 *status = OPINTERROGATE;
00177                 break;
00178             case OP_ERROR:
00179                 ResolveError(status);
00180                 return;
00181             default:
00182                 *status = UNKNOWNOP;
00183                 break;
00184         }
00185     }
00186
00187     uint16_t bodySize;
00188     UART_RecvBytes(UART_PEER, (uint8_t *) &bodySize, 2, true);
00189
00190     *status = (bodySize <= sizeof(FS_FAT)) ? OPINTERROGATE : INVALIDBODYSIZE;

```

```

00191
00192     UART_RcvBytes(UART_PEER, (uint8_t *) &stage.preamble.nonce, NONCE_SIZE,
00193                   false);
00194     UART_RcvBytes(UART_PEER, (uint8_t *) &stage.preamble.mesgid, ID_SIZE,
00195                   false);
00196     UART_RcvBytes(UART_PEER, (uint8_t *) &stage.preamble.tag, ASCON_TAG_SIZE,
00197                   true);
00198
00199     switch (*status)
00200     {
00201     case OPINTERROGATE:
00202         UART_RcvBytes(UART_PEER, (uint8_t *) &stage.as.fat, bodySize,
00203                       true);
00204         break;
00205     default:
00206         UART_RcvBytes(UART_PEER, NULL, bodySize, true);
00207         break;
00208     }
00209
00210     InterrogateHandler(status);
00211 }
00212
00219 void ReceiveParser(StatusCode *status)
00220 {
00221     uint8_t pin[6] = {0};
00222     uint8_t readSlot;
00223     uint8_t writeSlot;
00224
00225     UART_RcvBytes(UART_HOST, pin, 6, false);
00226     UART_RcvBytes(UART_HOST, &readSlot, 1, false);
00227     UART_RcvBytes(UART_HOST, &writeSlot, 1, true);
00228
00229     if (!checkpin(pin))
00230     {
00231         *status = PERMISSIONERROR;
00232         return;
00233     }
00234
00235     if (readSlot >= NUM_SLOTS || writeSlot >= NUM_SLOTS)
00236     {
00237         *status = INVALIDSLOT;
00238         return;
00239     }
00240
00241     uint8_t opCode = OP_RECEIVE;
00242     UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00243     UART_SendBytes(UART_PEER, &opCode, 1, false);
00244
00245     uint16_t bodyLen = 1;
00246     UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00247
00248     UART_SendBytes(UART_PEER, &readSlot, 1, true);
00249
00250     *status = UNKNOWNOP;
00251     while (*status == UNKNOWNOP)
00252     {
00253         UART_RcvUntil(UART_PEER, peer_magic_byte);
00254         UART_RcvBytes(UART_PEER, &opCode, 1, false);
00255         switch (opCode)
00256         {
00257         case OP_RECEIVE:
00258             *status = OPRECEIVE;
00259             break;
00260         case OP_ERROR:
00261             ResolveError(status);
00262             return;
00263         default:
00264             *status = UNKNOWNOP;
00265             break;
00266         }
00267     }
00268
00269     uint16_t bodySize;
00270     UART_RcvBytes(UART_PEER, (uint8_t *) &bodySize, 2, true);
00271
00272     *status = (bodySize == sizeof(FS_File)) ? OPRECEIVE : INVALIDBODYSIZE;
00273
00274     UART_RcvBytes(UART_PEER, (uint8_t *) &stage.preamble, sizeof(FS_Preamble),
00275                   true);
00276
00277     switch (*status)
00278     {
00279     case OPRECEIVE:
00280         UART_RcvBytes(UART_PEER, (uint8_t *) &stage.as.file,
00281                       sizeof(FS_File), true);
00282         ReceiveHandler(writeSlot, status);
00283         break;

```

```

00284         default:
00285             UART_RecvBytes(UART_PEER, NULL, bodySize, true);
00286             break;
00287     }
00288 }
00289
00290 static void ReplyParser(StatusCode *status)
00291 {
00292     ReplyHandler(status);
00293
00294     uint8_t opCode;
00295     uint16_t bodyLen;
00296
00297     if (*status != OPREPLY)
00298     {
00299         opCode = OP_ERROR;
00300         bodyLen = (uint16_t) (*status);
00301         UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00302         UART_SendBytes(UART_PEER, &opCode, 1, false);
00303         UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00304         return;
00305     }
00306
00307     opCode = OP_INTERROGATE;
00308     UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00309     UART_SendBytes(UART_PEER, &opCode, 1, false);
00310
00311     bodyLen = sizeof(FS_FAT);
00312     UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00313
00314     UART_SendBytes(UART_PEER, (uint8_t *) &stage.preamble.nonce, NONCE_SIZE,
00315                   false);
00316     UART_SendBytes(UART_PEER, (uint8_t *) &stage.preamble.mesgid, ID_SIZE,
00317                   false);
00318     UART_SendBytes(UART_PEER, (uint8_t *) &stage.preamble.tag, ASCON_TAG_SIZE,
00319                   true);
00320
00321     UART_SendBytes(UART_PEER, (uint8_t *) &stage.as.fat, bodyLen, true);
00322 }
00323
00324 static void SendParser(StatusCode *status)
00325 {
00326     uint8_t readSlot;
00327     UART_RecvBytes(UART_PEER, &readSlot, 1, true);
00328
00329     if (readSlot >= NUM_SLOTS)
00330     {
00331         *status = INVALID_SLOT;
00332         return;
00333     }
00334
00335     SendHandler(readSlot, status);
00336
00337     uint8_t opCode;
00338     uint16_t bodyLen;
00339
00340     if (*status != OPSEND)
00341     {
00342         opCode = OP_ERROR;
00343         bodyLen = (uint16_t) (*status);
00344         UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00345         UART_SendBytes(UART_PEER, &opCode, 1, false);
00346         UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00347         return;
00348     }
00349
00350     opCode = OP_RECEIVE;
00351     UART_SendBytes(UART_PEER, (uint8_t *) &peer_magic_byte, 1, false);
00352     UART_SendBytes(UART_PEER, &opCode, 1, false);
00353
00354     bodyLen = sizeof(FS_File);
00355     UART_SendBytes(UART_PEER, (uint8_t *) &bodyLen, 2, true);
00356
00357     UART_SendBytes(UART_PEER, (uint8_t *) &stage.preamble, sizeof(FS_Preamble),
00358                   true);
00359
00360     UART_SendBytes(UART_PEER, (uint8_t *) &stage.as.file, sizeof(FS_File),
00361                   true);
00362 }
00363
00364 void ListenParser(StatusCode *status)
00365 {
00366     uint8_t opCode;
00367
00368     *status = UNKNOWNOP;
00369     while (*status == UNKNOWNOP)
00370     {

```

```

00371     UART_RecvUntil(UART_PEER, peer_magic_byte);
00372     UART_RecvBytes(UART_PEER, &opCode, 1, false);
00373     switch (opCode)
00374     {
00375         case OP_INTERROGATE:
00376             *status = OPREPLY;
00377             break;
00378         case OP_RECEIVE:
00379             *status = OPSEND;
00380             break;
00381         default:
00382             *status = UNKNOWNOP;
00383             break;
00384     }
00385 }
00386
00387 uint16_t bodySize;
00388 UART_RecvBytes(UART_PEER, (uint8_t *) &bodySize, 2, true);
00389
00390 switch (*status)
00391 {
00392     case OPREPLY:
00393         *status = (bodySize == 0) ? OPREPLY : INVALIDBODYSIZE;
00394         break;
00395     case OPSEND:
00396         *status = (bodySize == 1) ? OPSEND : INVALIDBODYSIZE;
00397         break;
00398     default:
00399         *status = UNKNOWNOP;
00400 }
00401
00402 switch (*status)
00403 {
00404     case OPREPLY:
00405         ReplyParser(status);
00406         break;
00407     case OPSEND:
00408         SendParser(status);
00409         break;
00410     default:
00411         return;
00412 }
00413 }

```

5.33 firmware/src/randombytes.c File Reference

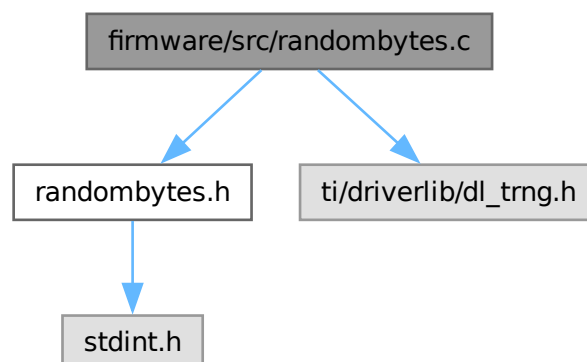
Implementation of TRNG-backed random number generation.

```

#include "randombytes.h"
#include "ti/driverlib/dl_trng.h"

```

Include dependency graph for randombytes.c:



Functions

- uint32_t [getrandom32](#) (void)
Retrieves a single 32-bit random word from the TRNG hardware.
- void [randombytes](#) (uint8_t *buf, uint64_t len)
Fills a buffer with true random bytes.

5.33.1 Detailed Description

Implementation of TRNG-backed random number generation.

Author

Sumedh Girish

Interfaces with the MSPM0 hardware True Random Number Generator (TRNG) to provide cryptographically secure random numbers for key generation and nonces.

Definition in file [randombytes.c](#).

5.33.2 Function Documentation

5.33.2.1 [getrandom32\(\)](#)

```
uint32_t getrandom32 (  
    void )
```

Retrieves a single 32-bit random word from the TRNG hardware.

Retrieves a 32-bit random word from the hardware TRNG.

Blocks execution until the TRNG capture register is ready.

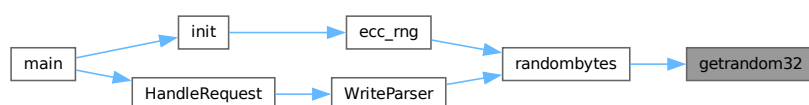
Returns

uint32_t A true random 32-bit word.

Definition at line 21 of file [randombytes.c](#).

Referenced by [randombytes\(\)](#).

Here is the caller graph for this function:



5.33.2.2 randombytes()

```
void randombytes (
    uint8_t * buf,
    uint64_t len)
```

Fills a buffer with true random bytes.

Fills a buffer with random bytes.

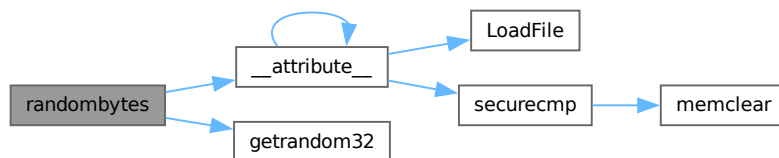
Utilizes [getrandom32\(\)](#) internally and handles byte alignment.

Definition at line 34 of file [randombytes.c](#).

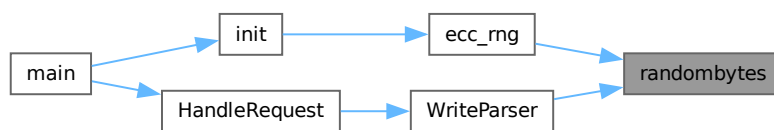
References [__attribute__\(\)](#), and [getrandom32\(\)](#).

Referenced by [ecc_rng\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.34 randombytes.c

[Go to the documentation of this file.](#)

```
00001
00010
00011 #include "randombytes.h"
00012 #include "ti/driverlib/dl_trng.h"
00013
00021 uint32_t getrandom32(void)
00022 {
00023     while (!DL_TRNG_isCaptureReady(TRNG))
00024         ;
00025
```



```

00026     return DL_TRNG_getCapture(TRNG);
00027 }
00028
00034 void randombytes(uint8_t *buf, uint64_t len)
00035 {
00036     uint8_t capturebuf[4] __attribute__((aligned(4)));
00037     uint32_t *capture = (uint32_t *) capturebuf;
00038     uint64_t genBytes = 0;
00039     while (genBytes < len)
00040     {
00041         if (genBytes % 4 == 0)
00042             *capture = getRandom32();
00043         buf[genBytes] = capturebuf[genBytes % 4];
00044         genBytes++;
00045     }
00046 }

```

5.35 firmware/src/startup_mspm0l222x_ticlang.c File Reference

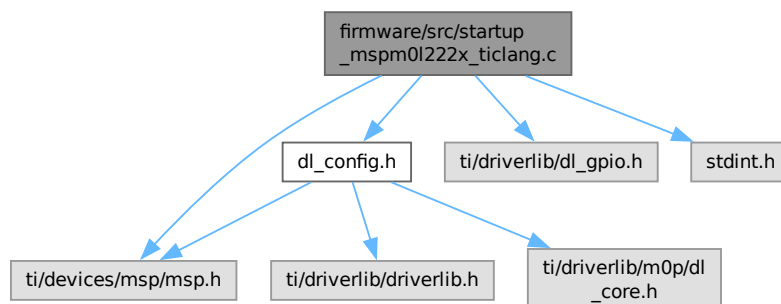
Startup and Vector Table for TI Clang.

```

#include "dl_config.h"
#include "ti/driverlib/dl_gpio.h"
#include <stdint.h>
#include <ti/devices/msp/msp.h>

```

Include dependency graph for startup_mspm0l222x_ticlang.c:



Functions

- `__NO_RETURN void __PROGRAM_START (void)`
TI Clang C runtime initialization entry point.
- `void Default_Handler (void)`
Interrupt vector table for the MSPM0L2228.
- `void Reset_Handler (void)`
Processor Reset Handler.

Variables

- `unsigned long __STACK_END`
Top of the stack, defined in the linker command file.

5.35.1 Detailed Description

Startup and Vector Table for TI Clang.

Author

Sumedh Girish

Defines the interrupt vector table and basic reset handling for the MSPM0L2228. This file is critical for ensuring the processor correctly identifies the initial stack pointer and the reset entry point.

Definition in file [startup_mspm0l222x_ticlang.c](#).

5.35.2 Function Documentation

5.35.2.1 __PROGRAM_START()

```
__NO_RETURN void __PROGRAM_START (
    void ) [extern]
```

TI Clang C runtime initialization entry point.

5.35.2.2 Default_Handler()

```
void Default_Handler (
    void )
```

Interrupt vector table for the MSPM0L2228.

Default interrupt handler.

Placed in the `.intvecs` section by the linker to reside at address 0x6000. Contains the initial SP and pointers to exception/interrupt handlers.

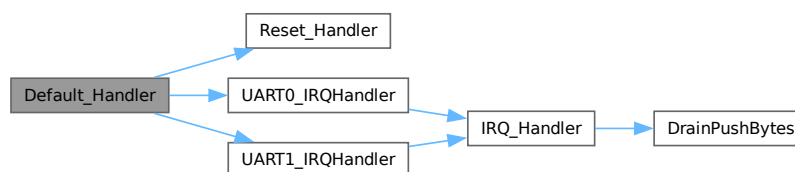
This is the code that gets called when the processor receives an unexpected interrupt. It turns off the Status LED and enters an infinite loop for debugger inspection.

Definition at line 23 of file [startup_mspm0l222x_ticlang.c](#).

References [__STACK_END](#), [Reset_Handler\(\)](#), [UART0_IRQHandler\(\)](#), and [UART1_IRQHandler\(\)](#).

Referenced by [TRNG_SecureWarmup\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.35.2.3 Reset_Handler()

```
void Reset_Handler (
    void )
```

Processor Reset Handler.

This is the code that gets called when the processor first starts execution following a reset event. It jumps to the TI Clang C initialization routine which sets up the stack and calls [main\(\)](#).

Definition at line 159 of file [startup_msp0l222x_ticlang.c](#).

Referenced by [Default_Handler\(\)](#).

Here is the caller graph for this function:



5.35.3 Variable Documentation

5.35.3.1 __STACK_END

```
unsigned long __STACK_END [extern]
```

Top of the stack, defined in the linker command file.

Referenced by [Default_Handler\(\)](#).

5.36 startup_msp0l222x_ticlang.c

[Go to the documentation of this file.](#)

```

00001
00010
00011 #include "dl_config.h"
00012 #include "ti/driverlib/dl_gpio.h"
00013 #include <stdint.h>
00014 #include <ti/devices/msp/msp.h>
00015
00017 extern unsigned long __STACK_END;
00018
00020 extern __NO_RETURN void __PROGRAM_START(void);
00021
00022 /* Forward declaration of the default fault handlers. */
00023 void Default_Handler(void) __attribute__((weak));
00024 extern void Reset_Handler(void) __attribute__((weak));
00025
00026 /* Processor Exceptions */
00027 extern void NMI_Handler(void) __attribute__((weak, alias("Default_Handler")));
00028 extern void HardFault_Handler(void)
00029     __attribute__((weak, alias("Default_Handler")));
00030 extern void SVC_Handler(void) __attribute__((weak, alias("Default_Handler")));
00031 extern void PendSV_Handler(void)
00032     __attribute__((weak, alias("Default_Handler")));
00033 extern void SysTick_Handler(void)
00034     __attribute__((weak, alias("Default_Handler")));
00035
00036 /* Device Specific Interrupt Handlers */
```

```

00037 extern void GROUP0_IRQHandler(void)
00038     __attribute__((weak, alias("Default_Handler")));
00039 extern void GROUP1_IRQHandler(void)
00040     __attribute__((weak, alias("Default_Handler")));
00041 extern void TIMG12_IRQHandler(void)
00042     __attribute__((weak, alias("Default_Handler")));
00043 extern void UART4_IRQHandler(void)
00044     __attribute__((weak, alias("Default_Handler")));
00045 extern void ADC0_IRQHandler(void)
00046     __attribute__((weak, alias("Default_Handler")));
00047 extern void SPI0_IRQHandler(void)
00048     __attribute__((weak, alias("Default_Handler")));
00049 extern void SPI1_IRQHandler(void)
00050     __attribute__((weak, alias("Default_Handler")));
00051 extern void UART2_IRQHandler(void)
00052     __attribute__((weak, alias("Default_Handler")));
00053 extern void UART3_IRQHandler(void)
00054     __attribute__((weak, alias("Default_Handler")));
00055 extern void UART0_IRQHandler(void)
00056     __attribute__((weak, alias("Default_Handler")));
00057 extern void UART1_IRQHandler(void)
00058     __attribute__((weak, alias("Default_Handler")));
00059 extern void TIMA0_IRQHandler(void)
00060     __attribute__((weak, alias("Default_Handler")));
00061 extern void TIMG8_IRQHandler(void)
00062     __attribute__((weak, alias("Default_Handler")));
00063 extern void TIMG0_IRQHandler(void)
00064     __attribute__((weak, alias("Default_Handler")));
00065 extern void TIMG4_IRQHandler(void)
00066     __attribute__((weak, alias("Default_Handler")));
00067 extern void TIMG5_IRQHandler(void)
00068     __attribute__((weak, alias("Default_Handler")));
00069 extern void I2C0_IRQHandler(void)
00070     __attribute__((weak, alias("Default_Handler")));
00071 extern void I2C1_IRQHandler(void)
00072     __attribute__((weak, alias("Default_Handler")));
00073 extern void I2C2_IRQHandler(void)
00074     __attribute__((weak, alias("Default_Handler")));
00075 extern void AESADV_IRQHandler(void)
00076     __attribute__((weak, alias("Default_Handler")));
00077 extern void LCD_IRQHandler(void)
00078     __attribute__((weak, alias("Default_Handler")));
00079 extern void LFSS_IRQHandler(void)
00080     __attribute__((weak, alias("Default_Handler")));
00081 extern void DMA_IRQHandler(void)
00082     __attribute__((weak, alias("Default_Handler")));
00083
00090 #if defined(__ARM_ARCH) && (__ARM_ARCH != 0)
00091 void (*const interruptVectors[]) (void) __attribute__((used))
00092     __attribute__((section(".intvecs"))) =
00093     #elif defined(__TI_ARM__)
00094     #pragma RETAIN(interruptVectors)
00095     #pragma DATA_SECTION(interruptVectors, ".intvecs")
00096     void (*const interruptVectors[]) (void) =
00097     #else
00098     #error "Compiler not supported"
00099 #endif
00100 {
00101     (void (*)(void))((uint32_t) &__STACK_END),
00102     /* The initial stack pointer */
00103     Reset_Handler,          /* The reset handler          */
00104     NMI_Handler,            /* The NMI handler            */
00105     HardFault_Handler,      /* The hard fault handler     */
00106     0,                      /* Reserved                    */
00107     0,                      /* Reserved                    */
00108     0,                      /* Reserved                    */
00109     0,                      /* Reserved                    */
00110     0,                      /* Reserved                    */
00111     0,                      /* Reserved                    */
00112     0,                      /* Reserved                    */
00113     SVC_Handler,           /* SVCcall handler            */
00114     0,                      /* Reserved                    */
00115     0,                      /* Reserved                    */
00116     PendSV_Handler,        /* The PendSV handler         */
00117     SysTick_Handler,       /* SysTick handler            */
00118     GROUP0_IRQHandler,     /* GROUP0 interrupt handler   */
00119     GROUP1_IRQHandler,     /* GROUP1 interrupt handler   */
00120     TIMG12_IRQHandler,     /* TIMG12 interrupt handler   */
00121     UART4_IRQHandler,      /* UART4 interrupt handler    */
00122     ADC0_IRQHandler,       /* ADC0 interrupt handler     */
00123     0,                      /* Reserved                    */
00124     0,                      /* Reserved                    */
00125     0,                      /* Reserved                    */
00126     0,                      /* Reserved                    */
00127     SPI0_IRQHandler,       /* SPI0 interrupt handler     */
00128     SPI1_IRQHandler,       /* SPI1 interrupt handler     */
00129     0,                      /* Reserved                    */

```

```

00130         0, /* Reserved */
00131         UART2_IRQHandler, /* UART2 interrupt handler */
00132         UART3_IRQHandler, /* UART3 interrupt handler */
00133         UART0_IRQHandler, /* UART0 interrupt handler */
00134         UART1_IRQHandler, /* UART1 interrupt handler */
00135         0, /* Reserved */
00136         TIMA0_IRQHandler, /* TIMA0 interrupt handler */
00137         0, /* Reserved */
00138         TIMG8_IRQHandler, /* TIMG8 interrupt handler */
00139         TIMG0_IRQHandler, /* TIMG0 interrupt handler */
00140         TIMG4_IRQHandler, /* TIMG4 interrupt handler */
00141         TIMG5_IRQHandler, /* TIMG5 interrupt handler */
00142         I2C0_IRQHandler, /* I2C0 interrupt handler */
00143         I2C1_IRQHandler, /* I2C1 interrupt handler */
00144         I2C2_IRQHandler, /* I2C2 interrupt handler */
00145         0, /* Reserved */
00146         AESADV_IRQHandler, /* AESADV interrupt handler */
00147         LCD_IRQHandler, /* LCD interrupt handler */
00148         LFSS_IRQHandler, /* LFSS interrupt handler */
00149         DMA_IRQHandler /* DMA interrupt handler */
00150     };
00151
00159 void Reset_Handler(void)
00160 {
00161     /* Disable SWD immediately on boot to prevent debugger attachment */
00162     DL_SYSCTL_disableSWD();
00163
00164     /* Jump to the ticlang C Initialization Routine. */
00165     __asm("    .global _c_int00\n"
00166          "    b     _c_int00");
00167 }
00168
00176 void Default_Handler(void)
00177 {
00178     DL_GPIO_clearPins(LED_PORT, LED_STATUS_LED_PIN);
00179     /* Enter an infinite loop. */
00180     while (1)
00181     {
00182     }
00183 }

```

5.37 firmware/src/stubs.c File Reference

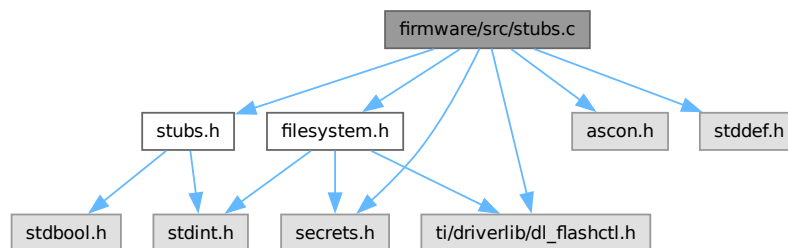
Implementation of Secure Memory and Side-Channel Protection routines.

```

#include "stubs.h"
#include "ascon.h"
#include "filesystem.h"
#include "secrets.h"
#include "ti/driverlib/dl_flashctl.h"
#include <stddef.h>

```

Include dependency graph for stubs.c:



Macros

- #define `RAMFUNC __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))`

Functions

- bool [securecmp](#) (const uint8_t *a, const uint8_t *b, uint32_t size)
Constant-time sensitive memory comparison using Ascon-XOF.
- bool [checkpin](#) (uint8_t inputPin[6])
Securely validates a user-provided PIN against the stored system PIN.
- [RAMFUNC](#) void [memclear](#) (void *v, uint32_t n, uint8_t value)
Volatile memory clearing/setting to prevent compiler optimization.

5.37.1 Detailed Description

Implementation of Secure Memory and Side-Channel Protection routines.

Author

Sumedh Girish

Provides specialized memory utilities designed to withstand side-channel attacks and ensure compiler-compliant volatile memory clearing.

Definition in file [stubs.c](#).

5.37.2 Macro Definition Documentation

5.37.2.1 RAMFUNC

```
#define RAMFUNC __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
```

Definition at line 17 of file [stubs.c](#).

Referenced by [__attribute__\(\)](#), [FLASH_Erase\(\)](#), [FLASH_ProcessSector\(\)](#), [FLASH_ReadSector\(\)](#), [FLASH_Write\(\)](#), and [memclear\(\)](#).

5.37.3 Function Documentation

5.37.3.1 checkpin()

```
bool checkpin (
    uint8_t inputPin[6])
```

Securely validates a user-provided PIN against the stored system PIN.

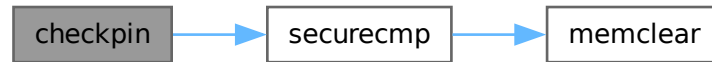
Executes both a standard memory comparison and a secure, constant-time hash-based comparison ([securecmp\(\)](#)). Any discrepancy immediately corrupts the timeout token in flash memory to trip the internal trap system, permanently mitigating brute force attempts.

Definition at line 53 of file [stubs.c](#).

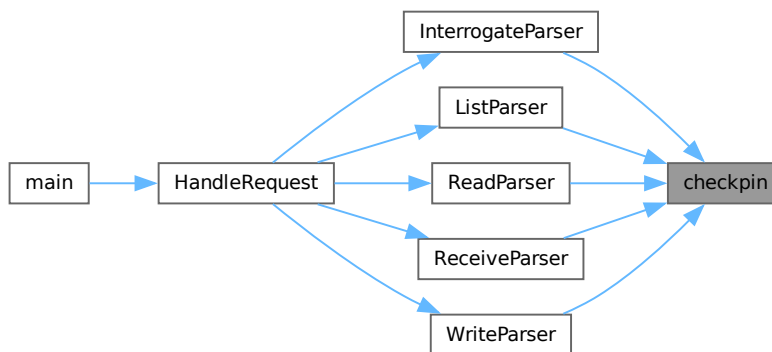
References [securecmp\(\)](#), and [SystemStatus](#).

Referenced by [InterrogateParser\(\)](#), [ListParser\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.37.3.2 memclear()

```

RAMFUNC void memclear (
    void * v,
    uint32_t n,
    uint8_t value)
  
```

Volatile memory clearing/setting to prevent compiler optimization.

Securely clears memory context using volatile-safe operations.

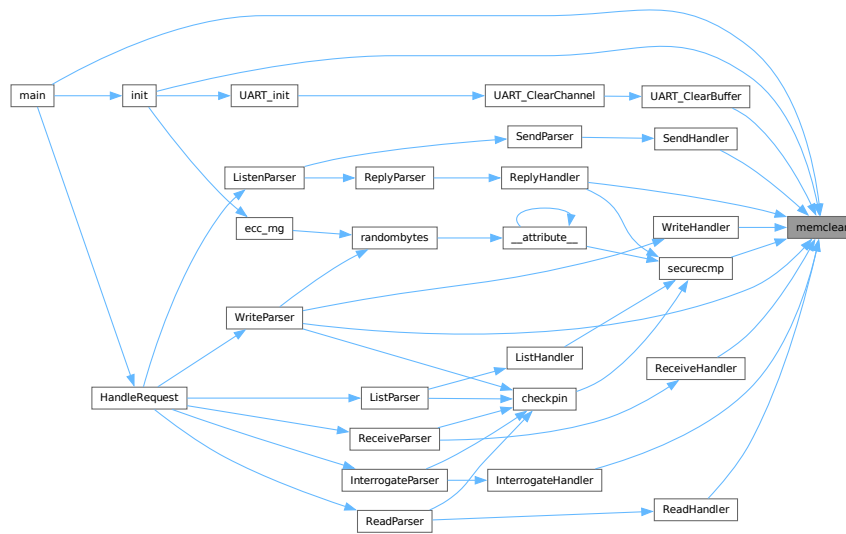
Standard `memset()` may be optimized away by compilers if the buffer is not read again before it goes out of scope. [memclear\(\)](#) uses volatile pointers and a three-stage clear (alignment, word-clear, tail-clear) to ensure that the memory is actually modified in SRAM, protecting against data remanence.

Definition at line 81 of file [stubs.c](#).

References [RAMFUNC](#).

Referenced by [init\(\)](#), [InterrogateHandler\(\)](#), [main\(\)](#), [ReadHandler\(\)](#), [ReceiveHandler\(\)](#), [ReplyHandler\(\)](#), [securecmp\(\)](#), [SendHandler\(\)](#), [UART_ClearBuffer\(\)](#), [WriteHandler\(\)](#), and [WriteParser\(\)](#).

Here is the caller graph for this function:



5.37.3.3 `securecmp()`

```

bool securecmp (
    const uint8_t * a,
    const uint8_t * b,
    uint32_t size)

```

Constant-time sensitive memory comparison using Ascon-XOF.

Secure memory comparison using domain-separated hashing.

Instead of comparing bytes directly (which is vulnerable to timing attacks), this function hashes both inputs into a temporary buffer and then performs a bitwise accumulated XOR comparison on the hashes.

Definition at line 27 of file [stubs.c](#).

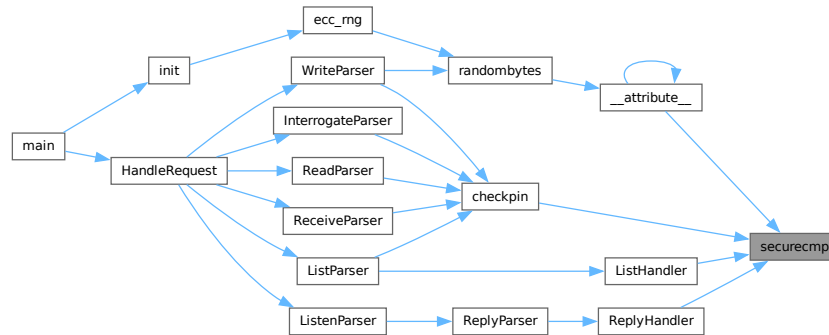
References [memclear\(\)](#).

Referenced by [__attribute__\(\)](#), [checkpin\(\)](#), [ListHandler\(\)](#), and [ReplyHandler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.38 stubs.c

[Go to the documentation of this file.](#)

```

00001
00009
00010 #include "stubs.h"
00011 #include "ascon.h"
00012 #include "filesystem.h"
00013 #include "secrets.h"
00014 #include "ti/driverlib/dl_flashctl.h"
00015 #include <stddef.h>
00016
00017 #define RAMFUNC
00018     __attribute__((section(".TI.ramfunc"))) __attribute__((noinline))
00019
00027 bool securecmp(const uint8_t *a, const uint8_t *b, uint32_t size)
00028 {
00029     uint8_t ahash[CRYPTO_ABYTES] = {0};
00030     uint8_t bhash[CRYPTO_ABYTES] = {0};
00031
00032     ascon_xof_masked(ahash, CRYPTO_ABYTES, a, size);
00033     ascon_xof_masked(bhash, CRYPTO_ABYTES, b, size);
00034
00035     uint8_t result = 0;
00036     for (uint32_t i = 0; i < CRYPTO_ABYTES; ++i)
00037         result |= ahash[i] ^ bhash[i];
00038
00039     memclear(ahash, CRYPTO_ABYTES, 0xFF);
00040     memclear(bhash, CRYPTO_ABYTES, 0xFF);
00041
00042     return result == 0;
00043 }
00044
00053 bool checkpin(uint8_t inputPin[6])
00054 {
00055     uint64_t storedPin = 0;
00056     memcpy(&storedPin, (const uint8_t *) HSM_PIN, 6);
00057
00058     uint64_t inputPinVal = 0;
00059     memcpy(&inputPinVal, inputPin, 6);
00060
00061     bool valid1 = storedPin == inputPinVal;
00062     bool valid2 =
00063         securecmp((const uint8_t *) HSM_PIN, (const uint8_t *) inputPin, 6);
00064
00065     uint64_t value = (0 - (uint64_t) (valid1 && valid2));
00066     DL_FlashCTL_programMemoryBlocking64WithECCGenerated(
00067         FLASHCTL, (uint32_t) &SystemStatus.timeout, (uint32_t *) &value, 2,
00068         DL_FLASHCTL_REGION_SELECT_MAIN);
00069
00070     return valid1 && valid2;
00071 }
00072
00081 RAMFUNC void memclear(void *v, uint32_t n, uint8_t value)
00082 {

```

```

00083     if (v == NULL || n == 0)
00084         return;
00085
00086     volatile uint8_t *p = (volatile uint8_t *) v;
00087
00088     // 1. Align pointer to 4-byte boundary (Byte-by-byte)
00089     while (n > 0 && ((uintptr_t) p & 3) != 0)
00090     {
00091         *p++ = value;
00092         n--;
00093     }
00094
00095     // 2. High-speed word clear (32-bit at a time)
00096     volatile uint32_t *pw;
00097     if (n >= 4)
00098     {
00099         uint32_t word_val = (uint32_t) value | ((uint32_t) value << 8) |
00100                             ((uint32_t) value << 16) | ((uint32_t) value << 24);
00101         pw = (volatile uint32_t *) (uintptr_t) p;
00102         while (n >= 4)
00103         {
00104             *pw++ = word_val;
00105             n -= 4;
00106         }
00107         p = (volatile uint8_t *) pw;
00108     }
00109
00110     // 3. Final Tail cleanup (Byte-by-byte)
00111     while (n > 0)
00112     {
00113         *p++ = value;
00114         n--;
00115     }
00116 }

```

5.39 firmware/src/uart.c File Reference

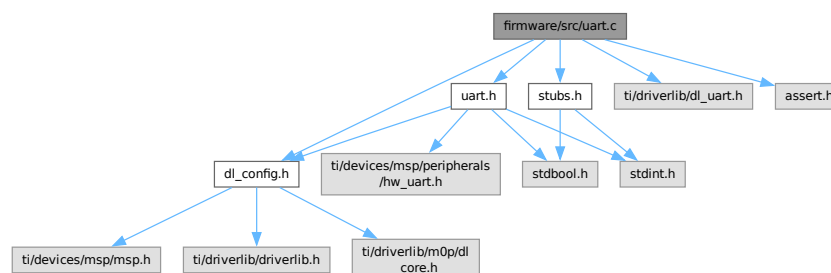
Non-blocking UART driver with Ring Buffer management.

```

#include "uart.h"
#include "dl_config.h"
#include "stubs.h"
#include "ti/driverlib/dl_uart.h"
#include <assert.h>

```

Include dependency graph for uart.c:



Functions

- static void [WaitForHostACK](#) (void)
- static void [WaitForPeerACK](#) (void)
- static void [UART_TransmitAll](#) (UART_Regs *channel)
Drains the internal ring buffer out to the physical TX FIFO.

- static void [UART_PushByte](#) (UART_Regs *source, uint8_t byte)
- static uint8_t [UART_PopByte](#) (UART_Regs *source)
- static void [SendHostACK](#) (void)
- static void [SendPeerACK](#) (void)
- void [UART_RecvUntil](#) (UART_Regs *channel, uint8_t chr)
Blocks until a specific character is received.
- void [UART_RecvBytes](#) (UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
Synchronously receives a fixed number of bytes.
- void [UART_SendBytes](#) (UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
Synchronously transmits a fixed number of bytes.
- static void [DrainPushBytes](#) (uint8_t tmp[[MAX_DRAIN_SIZE](#)], volatile [UART_RingBuffer](#) *buffer, uint32_t bytesRead)
- static void [UART_ClearBuffer](#) (volatile [UART_RingBuffer](#) *buffer)
- static void [UART_ClearChannel](#) (volatile [UART_Channel](#) *channel)
- void [UART_init](#) (void)
Initializes the software buffers for all UART channels.
- static void [IRQ_Handler](#) (UART_Regs *source, volatile [UART_Channel](#) *buffer)
Common UART Interrupt Service Routine for draining the hardware RX FIFO.
- void [UART0_IRQHandler](#) (void)
- void [UART1_IRQHandler](#) (void)

Variables

- volatile [UART_Channel](#) host = {0}
- volatile [UART_Channel](#) peer = {0}

5.39.1 Detailed Description

Non-blocking UART driver with Ring Buffer management.

Author

Sumedh Girish

Handles concurrent stream processing on two UART interfaces (Host and Peer). Incorporates software flow-control (ACK synchronization) to prevent buffer overflows during bulk cryptographic transfers.

Definition in file [uart.c](#).

5.39.2 Function Documentation

5.39.2.1 DrainPushBytes()

```
void DrainPushBytes (
    uint8_t tmp[MAX_DRAIN_SIZE],
    volatile UART_RingBuffer * buffer,
    uint32_t bytesRead) [inline], [static]
```

Definition at line 199 of file [uart.c](#).

References [UART_RingBuffer::count](#), [UART_RingBuffer::data](#), [UART_RingBuffer::dirty](#), [UART_RingBuffer::head](#), [MAX_DRAIN_SIZE](#), and [UART_BUFFER_SIZE](#).

Referenced by [IRQ_Handler\(\)](#).

Here is the caller graph for this function:



5.39.2.2 IRQ_Handler()

```
void IRQ_Handler (
    UART_Regs * source,
    volatile UART_Channel * buffer) [inline], [static]
```

Common UART Interrupt Service Routine for draining the hardware RX FIFO.

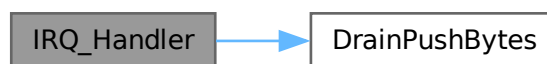
Automatically pulls hardware buffer contents into the volatile, asynchronous software ring buffers. Drops bytes silently if the software buffer is full.

Definition at line 246 of file [uart.c](#).

References [DrainPushBytes\(\)](#), [MAX_DRAIN_SIZE](#), and [UART_Channel::rx](#).

Referenced by [UART0_IRQHandler\(\)](#), and [UART1_IRQHandler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.3 SendHostACK()

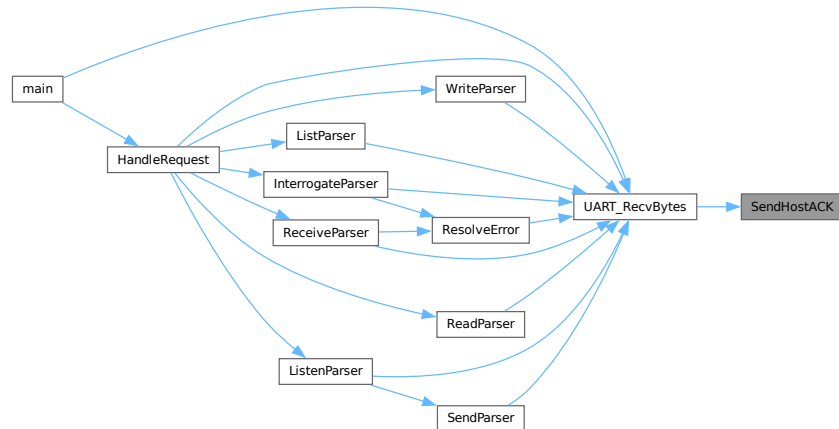
```
void SendHostACK (  
    void ) [inline], [static]
```

Definition at line 118 of file [uart.c](#).

References [UART_HOST](#).

Referenced by [UART_RecvBytes\(\)](#).

Here is the caller graph for this function:



5.39.2.4 SendPeerACK()

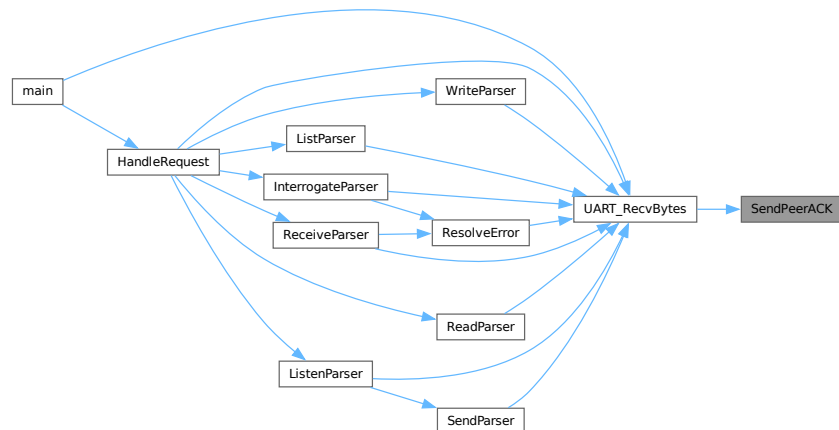
```
void SendPeerACK (  
    void ) [inline], [static]
```

Definition at line 125 of file [uart.c](#).

References [UART_PEER](#).

Referenced by [UART_RecvBytes\(\)](#).

Here is the caller graph for this function:



5.39.2.5 UART0_IRQHandler()

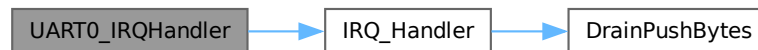
```
void UART0_IRQHandler (  
    void )
```

Definition at line 261 of file [uart.c](#).

References [host](#), [IRQ_Handler\(\)](#), and [UART_0_INST](#).

Referenced by [Default_Handler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.6 UART1_IRQHandler()

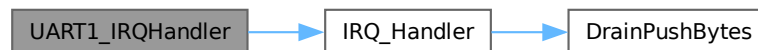
```
void UART1_IRQHandler (  
    void )
```

Definition at line 266 of file [uart.c](#).

References [IRQ_Handler\(\)](#), [peer](#), and [UART_1_INST](#).

Referenced by [Default_Handler\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.7 UART_ClearBuffer()

```
void UART_ClearBuffer (  
    volatile UART_RingBuffer * buffer)    [inline], [static]
```

Definition at line 219 of file [uart.c](#).

References [UART_RingBuffer::count](#), [UART_RingBuffer::data](#), [UART_RingBuffer::dirty](#), [UART_RingBuffer::head](#), [memset\(\)](#), [UART_RingBuffer::progress](#), [UART_RingBuffer::tail](#), and [UART_BUFFER_SIZE](#).

Referenced by [UART_ClearChannel\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.8 UART_ClearChannel()

```
void UART_ClearChannel (  
    volatile UART_Channel * channel)    [inline], [static]
```

Definition at line 228 of file [uart.c](#).

References [UART_Channel::rx](#), [UART_Channel::tx](#), and [UART_ClearBuffer\(\)](#).

Referenced by [UART_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.9 UART_init()

```
void UART_init (  
    void )
```

Initializes the software buffers for all UART channels.

Definition at line 234 of file `uart.c`.

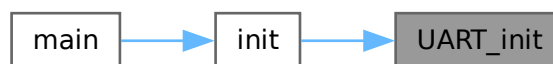
References [host](#), [peer](#), and [UART_ClearChannel\(\)](#).

Referenced by [init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.10 UART_PopByte()

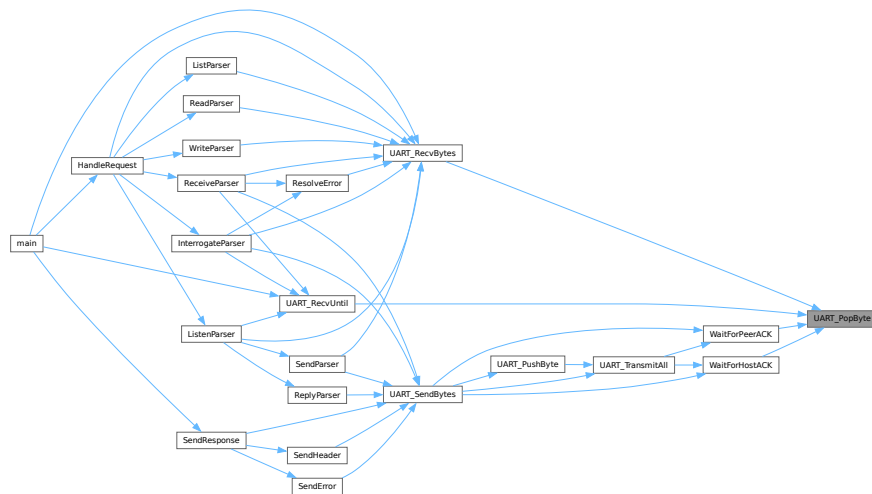
```
uint8_t UART_PopByte (
    UART_Regs * source) [inline], [static]
```

Definition at line 67 of file [uart.c](#).

References [UART_RingBuffer::count](#), [UART_RingBuffer::data](#), [host](#), [peer](#), [UART_RingBuffer::tail](#), [UART_BUFFER_SIZE](#), and [UART_HOST](#).

Referenced by [UART_RecvBytes\(\)](#), [UART_RecvUntil\(\)](#), [WaitForHostACK\(\)](#), and [WaitForPeerACK\(\)](#).

Here is the caller graph for this function:



5.39.2.11 UART_PushByte()

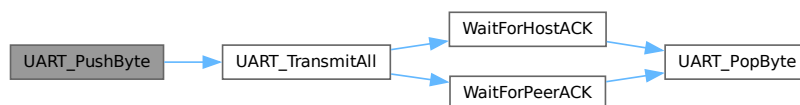
```
void UART_PushByte (
    UART_Regs * source,
    uint8_t byte) [inline], [static]
```

Definition at line 52 of file [uart.c](#).

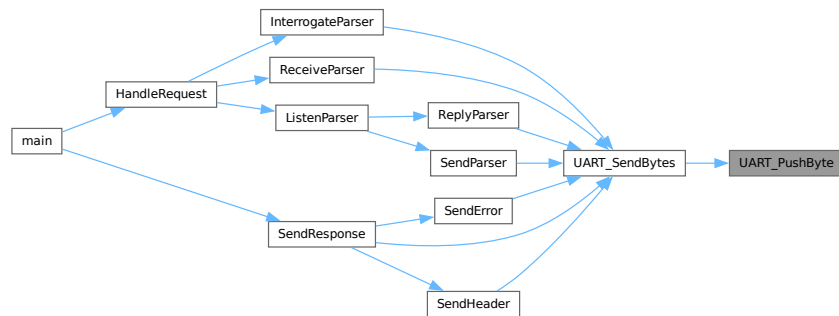
References [UART_RingBuffer::count](#), [UART_RingBuffer::data](#), [UART_RingBuffer::head](#), [host](#), [peer](#), [UART_BUFFER_SIZE](#), [UART_HOST](#), and [UART_TransmitAll\(\)](#).

Referenced by [UART_SendBytes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.12 UART_RcvBytes()

```

void UART_RcvBytes (
    UART_Regs * channel,
    uint8_t * out,
    uint32_t length,
    bool eof)

```

Synchronously receives a fixed number of bytes.

Parameters

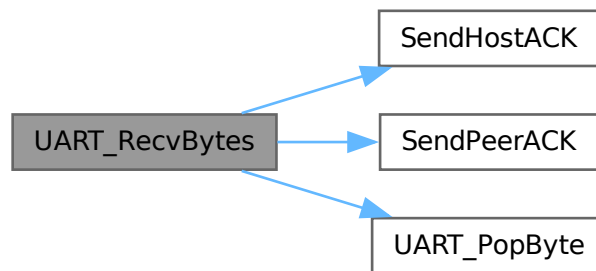
<i>channel</i>	The UART controller register base.
<i>out</i>	Destination buffer for the received bytes.
<i>length</i>	Exact number of bytes to wait for.
<i>eof</i>	If true, expects an EOF marker after the payload.

Definition at line 144 of file [uart.c](#).

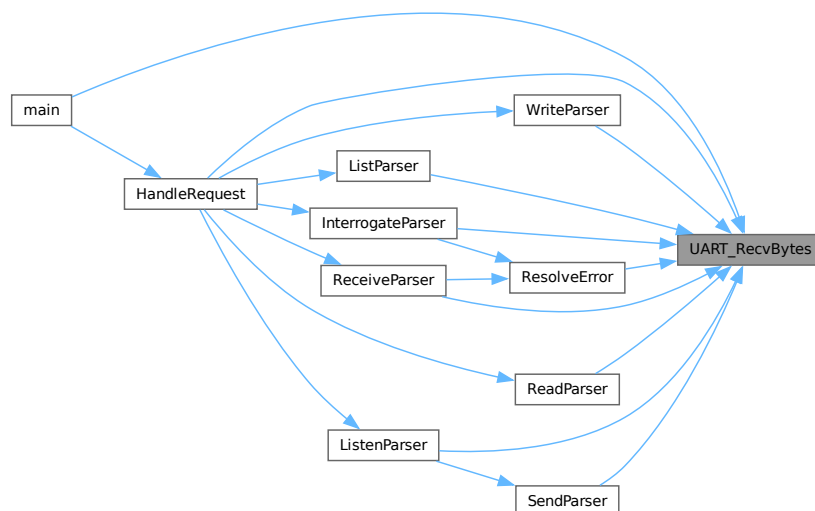
References [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_Channel::rx](#), [SendHostACK\(\)](#), [SendPeerACK\(\)](#), [UART_BUFFER_SIZE](#), [UART_HOST](#), and [UART_PopByte\(\)](#).

Referenced by [HandleRequest\(\)](#), [InterrogateParser\(\)](#), [ListenParser\(\)](#), [ListParser\(\)](#), [main\(\)](#), [ReadParser\(\)](#), [ReceiveParser\(\)](#), [ResolveError\(\)](#), [SendParser\(\)](#), and [WriteParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.13 UART_RecvUntil()

```

void UART_RecvUntil (
    UART_Regs * channel,
    uint8_t chr)
  
```

Blocks until a specific character is received.

Parameters

<i>channel</i>	The UART controller register base (e.g. <code>UART_HOST</code>).
----------------	---

<i>chr</i>	The character to block for.
------------	-----------------------------

Definition at line 132 of file [uart.c](#).

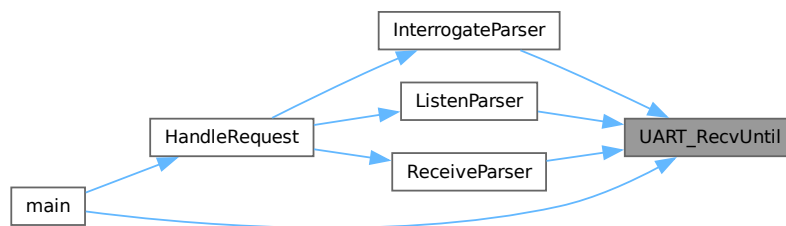
References [UART_RingBuffer::count](#), [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_HOST](#), and [UART_PopByte\(\)](#).

Referenced by [InterrogateParser\(\)](#), [ListenParser\(\)](#), [main\(\)](#), and [ReceiveParser\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.14 UART_SendBytes()

```

void UART_SendBytes (
    UART_Regs * channel,
    uint8_t * out,
    uint32_t length,
    bool eof)
  
```

Synchronously transmits a fixed number of bytes.

Parameters

<i>channel</i>	The UART controller register base.
<i>out</i>	Source buffer to transmit.

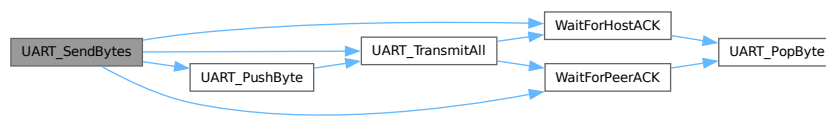
<i>length</i>	Exact number of bytes to send.
<i>eof</i>	If true, appends an EOF marker after transmission.

Definition at line 178 of file [uart.c](#).

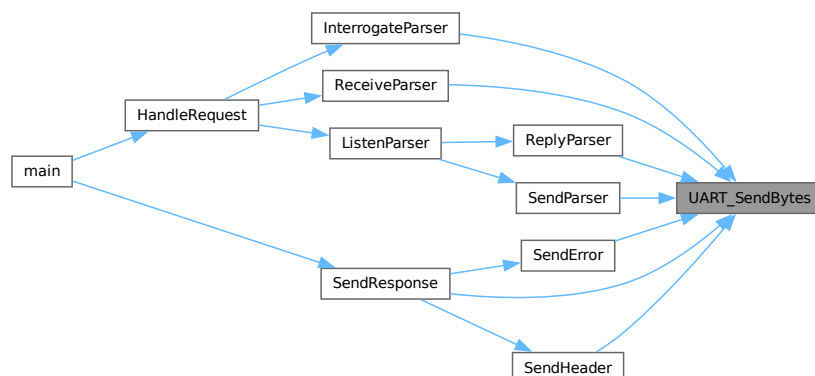
References [UART_RingBuffer::dirty](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_Channel::tx](#), [UART_HOST](#), [UART_PushByte\(\)](#), [UART_TransmitAll\(\)](#), [WaitForHostACK\(\)](#), and [WaitForPeerACK\(\)](#).

Referenced by [InterrogateParser\(\)](#), [ReceiveParser\(\)](#), [ReplyParser\(\)](#), [SendError\(\)](#), [SendHeader\(\)](#), [SendParser\(\)](#), and [SendResponse\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.15 UART_TransmitAll()

```
void UART_TransmitAll (
    UART_Regs * channel) [inline], [static]
```

Drains the internal ring buffer out to the physical TX FIFO.

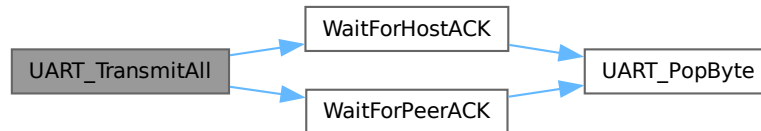
Enforces ACK-based synchronization chunks. If the transmitted count exceeds a chunk limit, it blocks waiting for a remote ACK to ensure the peer hasn't been overrun.

Definition at line 31 of file [uart.c](#).

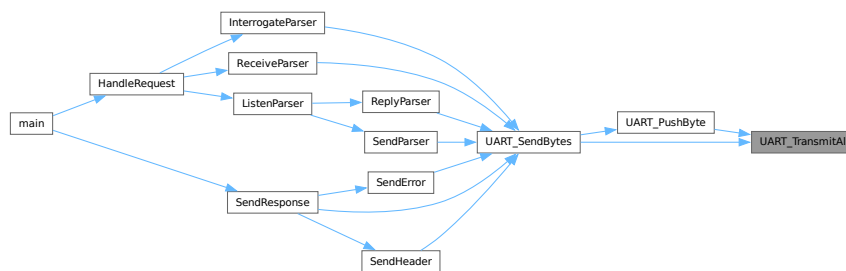
References [UART_RingBuffer::count](#), [UART_RingBuffer::data](#), [host](#), [peer](#), [UART_RingBuffer::progress](#), [UART_RingBuffer::tail](#), [UART_BUFFER_SIZE](#), [UART_HOST](#), [WaitForHostACK\(\)](#), and [WaitForPeerACK\(\)](#).

Referenced by [UART_PushByte\(\)](#), and [UART_SendBytes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.16 WaitForHostACK()

```
void WaitForHostACK (
    void ) [inline], [static]
```

Definition at line 82 of file [uart.c](#).

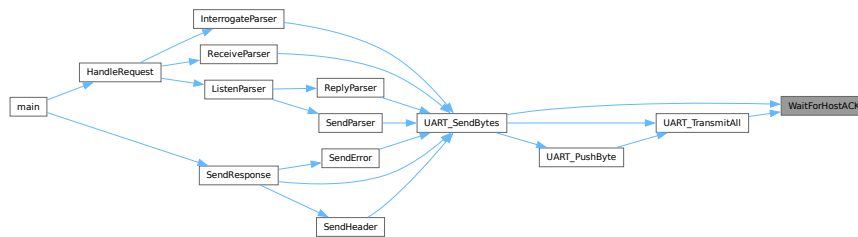
References [UART_HOST](#), and [UART_PopByte\(\)](#).

Referenced by [UART_SendBytes\(\)](#), and [UART_TransmitAll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.2.17 WaitForPeerACK()

```
void WaitForPeerACK (
    void ) [inline], [static]
```

Definition at line 100 of file [uart.c](#).

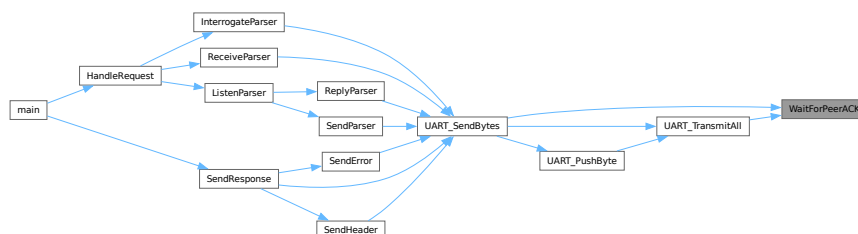
References [UART_PEER](#), and [UART_PopByte\(\)](#).

Referenced by [UART_SendBytes\(\)](#), and [UART_TransmitAll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.39.3 Variable Documentation

5.39.3.1 host

```
volatile UART_Channel host = {0}
```

Definition at line 18 of file `uart.c`.

Referenced by `UART0_IRQHandler()`, `UART_init()`, `UART_PopByte()`, `UART_PushByte()`, `UART_RecvBytes()`, `UART_RecvUntil()`, `UART_SendBytes()`, and `UART_TransmitAll()`.

5.39.3.2 peer

```
volatile UART_Channel peer = {0}
```

Definition at line 19 of file `uart.c`.

Referenced by `UART1_IRQHandler()`, `UART_init()`, `UART_PopByte()`, `UART_PushByte()`, `UART_RecvBytes()`, `UART_RecvUntil()`, `UART_SendBytes()`, and `UART_TransmitAll()`.

5.40 uart.c

[Go to the documentation of this file.](#)

```
00001
00011
00012 #include "uart.h"
00013 #include "dl_config.h"
00014 #include "stubs.h"
00015 #include "ti/driverlib/dl_uart.h"
00016 #include <assert.h>
00017
00018 volatile UART_Channel host = {0};
00019 volatile UART_Channel peer = {0};
00020
00021 static inline void WaitForHostACK(void);
00022 static inline void WaitForPeerACK(void);
00023
00031 static inline void UART_TransmitAll(UART_Regs *channel)
00032 {
00033     volatile UART_RingBuffer *buffer =
00034         (channel == UART_HOST) ? &host.tx : &peer.tx;
00035     void (*WaitForAck)(void) =
00036         (channel == UART_HOST) ? WaitForHostACK : WaitForPeerACK;
00037
00038     while (buffer->count > 0)
00039     {
00040         if (buffer->progress >= UART_BUFFER_SIZE)
00041         {
00042             WaitForAck();
00043             buffer->progress = 0;
00044         }
00045         DL_UART_transmitDataBlocking(channel, buffer->data[buffer->tail]);
00046         buffer->tail = (buffer->tail + 1) % UART_BUFFER_SIZE;
00047         buffer->count--;
00048         buffer->progress++;
00049     }
00050 }
00051
00052 static inline void UART_PushByte(UART_Regs *source, uint8_t byte)
00053 {
00054     volatile UART_RingBuffer *buffer =
00055         (source == UART_HOST) ? &host.tx : &peer.tx;
00056
00057     while (buffer->count >= UART_BUFFER_SIZE)
00058     {
00059         UART_TransmitAll(source);
00060     }
00061 }
```



```

00062     buffer->data[buffer->head] = byte;
00063     buffer->head = (buffer->head + 1) % UART_BUFFER_SIZE;
00064     buffer->count++;
00065 }
00066
00067 static inline uint8_t UART_PopByte(UART_Regs *source)
00068 {
00069     volatile UART_RingBuffer *buffer =
00070         (source == UART_HOST) ? &host.rx : &peer.rx;
00071
00072     while (buffer->count == 0)
00073         __WFI();
00074
00075     uint8_t byte = buffer->data[buffer->tail];
00076     buffer->tail = (buffer->tail + 1) % UART_BUFFER_SIZE;
00077     buffer->count--;
00078
00079     return byte;
00080 }
00081
00082 static inline void WaitForHostACK(void)
00083 {
00084     static const uint8_t ackbytes[4] = {'%', 'A', '\x00', '\x00'};
00085
00086     uint8_t idx = 0;
00087     uint8_t currentByte;
00088     while (idx < 4)
00089     {
00090         currentByte = UART_PopByte(UART_HOST);
00091         if (currentByte == ackbytes[idx])
00092             idx++;
00093         else if (currentByte == ackbytes[0])
00094             idx = 1;
00095         else
00096             idx = 0;
00097     }
00098 }
00099
00100 static inline void WaitForPeerACK(void)
00101 {
00102     static const uint8_t ackbytes[4] = {'^', 'A', '\x00', '\x00'};
00103
00104     uint8_t idx = 0;
00105     uint8_t currentByte;
00106     while (idx < 4)
00107     {
00108         currentByte = UART_PopByte(UART_PEER);
00109         if (currentByte == ackbytes[idx])
00110             idx++;
00111         else if (currentByte == ackbytes[0])
00112             idx = 1;
00113         else
00114             idx = 0;
00115     }
00116 }
00117
00118 static inline void SendHostACK(void)
00119 {
00120     const uint8_t ackbytes[4] = {'%', 'A', '\x00', '\x00'};
00121     for (size_t i = 0; i < 4; i++)
00122         DL_UART_transmitDataBlocking(UART_HOST, ackbytes[i]);
00123 }
00124
00125 static inline void SendPeerACK(void)
00126 {
00127     const uint8_t ackbytes[4] = {'^', 'A', '\x00', '\x00'};
00128     for (size_t i = 0; i < 4; i++)
00129         DL_UART_transmitDataBlocking(UART_PEER, ackbytes[i]);
00130 }
00131
00132 void UART_RecvUntil(UART_Regs *channel, uint8_t chr)
00133 {
00134     while (UART_PopByte(channel) != chr)
00135         ;
00136
00137     volatile UART_RingBuffer *buffer =
00138         (channel == UART_HOST) ? &host.rx : &peer.rx;
00139
00140     buffer->progress = 0;
00141     buffer->dirty = buffer->count > 0;
00142 }
00143
00144 void UART_RecvBytes(UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
00145 {
00146     volatile UART_Channel *source = (channel == UART_HOST) ? &host : &peer;
00147     void (*sendACK)(void) = (channel == UART_HOST) ? SendHostACK : SendPeerACK;
00148

```

```

00149     uint32_t bytesRead = 0;
00150     while (bytesRead < length)
00151     {
00152         if (source->rx.progress >= UART_BUFFER_SIZE && source->rx.dirty)
00153         {
00154             source->rx.progress = 0;
00155             source->rx.dirty = false;
00156             sendACK();
00157         }
00158
00159         if (out != NULL)
00160             out[bytesRead++] = UART_PopByte(channel);
00161         else
00162         {
00163             UART_PopByte(channel);
00164             bytesRead++;
00165         }
00166
00167         source->rx.progress++;
00168     }
00169
00170     if (eof)
00171     {
00172         source->rx.progress = 0;
00173         source->rx.dirty = false;
00174         sendACK();
00175     }
00176 }
00177
00178 void UART_SendBytes(UART_Regs *channel, uint8_t *out, uint32_t length, bool eof)
00179 {
00180     volatile UART_Channel *source = (channel == UART_HOST) ? &host : &peer;
00181     void (*waitACK)(void) =
00182         (channel == UART_HOST) ? WaitForHostACK : WaitForPeerACK;
00183
00184     uint32_t bytesSent = 0;
00185     while (bytesSent < length)
00186     {
00187         UART_PushByte(channel, out[bytesSent++]);
00188     }
00189
00190     UART_TransmitAll(channel);
00191     if (eof)
00192     {
00193         waitACK();
00194         source->tx.dirty = false;
00195         source->tx.progress = 0;
00196     }
00197 }
00198
00199 static inline void DrainPushBytes(uint8_t tmp[MAX_DRAIN_SIZE],
00200                                   volatile UART_RingBuffer *buffer,
00201                                   uint32_t bytesRead)
00202 {
00203     for (size_t i = 0; i < bytesRead; i++)
00204     {
00205         if (buffer->count < UART_BUFFER_SIZE)
00206         {
00207             buffer->data[buffer->head] = tmp[i];
00208             buffer->head = (buffer->head + 1) % UART_BUFFER_SIZE;
00209             buffer->count++;
00210             buffer->dirty = true;
00211         }
00212         else
00213         {
00214             break;
00215         }
00216     }
00217 }
00218
00219 static inline void UART_ClearBuffer(volatile UART_RingBuffer *buffer)
00220 {
00221     memclear((uint8_t *) &buffer->data, UART_BUFFER_SIZE, 0);
00222     buffer->count = 0;
00223     buffer->dirty = false;
00224     buffer->head = buffer->tail = 0;
00225     buffer->progress = 0;
00226 }
00227
00228 static inline void UART_ClearChannel(volatile UART_Channel *channel)
00229 {
00230     UART_ClearBuffer(&channel->rx);
00231     UART_ClearBuffer(&channel->tx);
00232 }
00233
00234 void UART_init(void)
00235 {

```

```

00236     UART_ClearChannel(&host);
00237     UART_ClearChannel(&peer);
00238 }
00239
00246 static inline void IRQ_Handler(UART_Regs *source, volatile UART_Channel *buffer)
00247 {
00248     uint8_t tmp[MAX_DRAIN_SIZE] = {0};
00249     switch (DL_UART_getPendingInterrupt(source))
00250     {
00251         case DL_UART_IIDX_RX:
00252         case DL_UART_IIDX_RX_TIMEOUT_ERROR:
00253             DrainPushBytes(tmp, &buffer->rx,
00254                             DL_UART_drainRXFIFO(source, tmp, MAX_DRAIN_SIZE));
00255             break;
00256         default:
00257             break;
00258     }
00259 }
00260
00261 void UART0_IRQHandler(void)
00262 {
00263     IRQ_Handler(UART_0_INST, &host);
00264 }
00265
00266 void UART1_IRQHandler(void)
00267 {
00268     IRQ_Handler(UART_1_INST, &peer);
00269 }

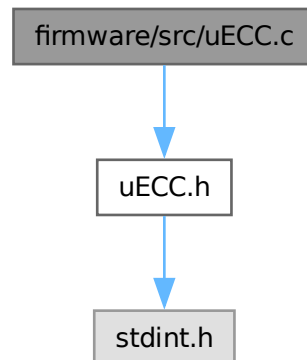
```

5.41 firmware/src/uECC.c File Reference

Implementation of the Micro-ECC library.

```
#include "uECC.h"
```

Include dependency graph for uECC.c:



Data Structures

- struct [EccPoint](#)

Macros

- `#define uECC_PLATFORM uECC_arch_other`
- `#define uECC_WORD_SIZE 1`
- `#define RESTRICT`
- `#define SUPPORTS_INT128 0`
- `#define MAX_TRIES 64`
- `#define uECC_WORDS uECC_CONCAT(uECC_WORDS_, uECC_CURVE)`
- `#define uECC_N_WORDS uECC_CONCAT(uECC_N_WORDS_, uECC_CURVE)`
- `#define muladd_exists 1`
- `#define vli_square(result, left, size)`
- `#define vli_modSub_fast(result, left, right)`
- `#define vli_modSquare_fast(result, left)`
- `#define EVEN(vli)`
- `#define muladd_exists 1`

Typedefs

- `typedef struct EccPoint EccPoint`

Functions

- static void `vli_clear` (uECC_word_t *vli)
- static uECC_word_t `vli_isZero` (const uECC_word_t *vli)
- static uECC_word_t `vli_testBit` (const uECC_word_t *vli, bitcount_t bit)
- static bitcount_t `vli_numBits` (const uECC_word_t *vli, wordcount_t max_words)
- static void `vli_set` (uECC_word_t *dest, const uECC_word_t *src)
- static cmpresult_t `vli_cmp` (const uECC_word_t *left, const uECC_word_t *right)
- static cmpresult_t `vli_equal` (const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_rshift1` (uECC_word_t *vli)
- static uECC_word_t `vli_add` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static uECC_word_t `vli_sub` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_mult` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_modAdd` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right, const uECC_word_t *mod)
- static void `vli_modInv` (uECC_word_t *result, const uECC_word_t *input, const uECC_word_t *mod)
- static void `vli_clear_n` (uECC_word_t *vli)
- static void `vli_set_n` (uECC_word_t *dest, const uECC_word_t *src)
- static cmpresult_t `vli_cmp_n` (const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_modMult_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_modAdd_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right, const uECC_word_t *mod)
- static void `vli_modInv_n` (uECC_word_t *result, const uECC_word_t *input, const uECC_word_t *mod)
- static void `vli_rshift1_n` (uECC_word_t *vli)
- static uECC_word_t `vli_add_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static uECC_word_t `vli_sub_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static void `vli_blindScalar` (uECC_word_t *result, const uECC_word_t *scalar)
- static int `EccPoint_compute_public_key` (EccPoint *result, uECC_word_t *private)
- static int `default_RNG` (uint8_t *dest, unsigned size)
- `__attribute__((used))`
- static wordcount_t `vli_numDigits` (const uECC_word_t *vli, wordcount_t max_words)
- static void `muladd` (uECC_word_t a, uECC_word_t b, uECC_word_t *r0, uECC_word_t *r1, uECC_word_t *r2)

- static void `vli_modSub` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right, const uECC_word_t *mod)
- static void `omega_mult` (uECC_word_t *RESTRICT result, const uECC_word_t *RESTRICT right)
- static void `vli_mmod_fast` (uECC_word_t *RESTRICT result, uECC_word_t *RESTRICT product)
- static void `vli_modMult_fast` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static cmpresult_t `EccPoint_isZero` (const EccPoint *point)
- static void `EccPoint_double_jacobian` (uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1, uECC_word_t *RESTRICT Z1)
- static void `apply_z` (uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1, const uECC_word_t *RESTRICT Z)
- static void `XYcZ_initial_double` (uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1, uECC_word_t *RESTRICT X2, uECC_word_t *RESTRICT Y2, const uECC_word_t *RESTRICT initial_Z)
- static void `XYcZ_add` (uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1, uECC_word_t *RESTRICT X2, uECC_word_t *RESTRICT Y2)
- static void `XYcZ_addC` (uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1, uECC_word_t *RESTRICT X2, uECC_word_t *RESTRICT Y2)
- static void `EccPoint_mult` (EccPoint *RESTRICT result, const EccPoint *RESTRICT point, const uECC_word_t *RESTRICT scalar, const uECC_word_t *RESTRICT initialZ, bitcount_t numBits)
- static void `mod_sqrt_secp224r1_rs` (uECC_word_t *d1, uECC_word_t *e1, uECC_word_t *f1, const uECC_word_t *d0, const uECC_word_t *e0, const uECC_word_t *f0)
- static void `mod_sqrt_secp224r1_rss` (uECC_word_t *d1, uECC_word_t *e1, uECC_word_t *f1, const uECC_word_t *d0, const uECC_word_t *e0, const uECC_word_t *f0, const bitcount_t j)
- static void `mod_sqrt_secp224r1_rm` (uECC_word_t *d2, uECC_word_t *e2, uECC_word_t *f2, const uECC_word_t *c, const uECC_word_t *d0, const uECC_word_t *e0, const uECC_word_t *d1, const uECC_word_t *e1)
- static void `mod_sqrt_secp224r1_rp` (uECC_word_t *d1, uECC_word_t *e1, uECC_word_t *f1, const uECC_word_t *c, const uECC_word_t *r)
- static void `mod_sqrt` (uECC_word_t *a)
- static void `vli_nativeToBytes` (uint8_t *bytes, const uint64_t *native)
- static void `vli_bytesToNative` (uint64_t *native, const uint8_t *bytes)
- int `uECC_make_key` (uint8_t public_key[uECC_BYTES *2], uint8_t private_key[uECC_BYTES])
Create a public/private key pair.
- void `uECC_compress` (const uint8_t public_key[uECC_BYTES *2], uint8_t compressed[uECC_BYTES+1])
Compress a public key (Y coordinate is reduced to parity bit).
- static void `curve_x_side` (uECC_word_t *RESTRICT result, const uECC_word_t *RESTRICT x)
- void `uECC_decompress` (const uint8_t compressed[uECC_BYTES+1], uint8_t public_key[uECC_BYTES *2])
Decompress a compressed public key.
- int `uECC_valid_public_key` (const uint8_t public_key[uECC_BYTES *2])
Check if a public key is a valid point on the curve.
- int `uECC_compute_public_key` (const uint8_t private_key[uECC_BYTES], uint8_t public_key[uECC_BYTES *2])
Compute the corresponding public key for a given private key.
- int `uECC_bytes` (void)
Returns the value of uECC_BYTES for the current build.
- int `uECC_curve` (void)
Returns the value of uECC_CURVE for the current build.
- static uECC_word_t `vli_isZero_n` (const uECC_word_t *vli)
- static void `vli_mult_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- static void `vli2_rshift1_n` (uECC_word_t *vli)
- static uECC_word_t `vli2_sub_n` (uECC_word_t *result, const uECC_word_t *left, const uECC_word_t *right)
- int `uECC_shared_secret` (const uint8_t public_key[uECC_BYTES *2], const uint8_t private_key[uECC_BYTES], uint8_t secret[uECC_BYTES])
Compute a shared secret given your secret key and someone else's public key (ECDH).

- static int `uECC_sign_with_k` (const uint8_t private_key[uECC_BYTES], const uint8_t message_hash[uECC_BYTES], uECC_word_t k[uECC_N_WORDS], uint8_t signature[uECC_BYTES * 2])
- int `uECC_sign` (const uint8_t private_key[uECC_BYTES], const uint8_t message_hash[uECC_BYTES], uint8_t signature[uECC_BYTES * 2])
Generate an ECDSA signature for a given hash value.
- static void `HMAC_init` (uECC_HashContext *hash_context, const uint8_t *K)
- static void `HMAC_update` (uECC_HashContext *hash_context, const uint8_t *message, unsigned message_size)
- static void `HMAC_finish` (uECC_HashContext *hash_context, const uint8_t *K, uint8_t *result)
- static void `update_V` (uECC_HashContext *hash_context, uint8_t *K, uint8_t *V)
- int `uECC_sign_deterministic` (const uint8_t private_key[uECC_BYTES], const uint8_t message_hash[uECC_BYTES], uECC_HashContext *hash_context, uint8_t signature[uECC_BYTES * 2])
Generate an ECDSA signature using a deterministic algorithm (RFC 6979).
- static bitcount_t `smax` (bitcount_t a, bitcount_t b)
- int `uECC_verify` (const uint8_t public_key[uECC_BYTES * 2], const uint8_t hash[uECC_BYTES], const uint8_t signature[uECC_BYTES * 2])
Verify an ECDSA signature against a public key and hash.
- void `uECC_mod_mult` (uint8_t result[32], const uint8_t left[32], const uint8_t right[32])

Variables

- static const uECC_word_t `curve_p` [uECC_WORDS]
- static const uECC_word_t `curve_b` [uECC_WORDS]
- static const `EccPoint curve_G` = `uECC_CONCAT`(Curve_G_, uECC_CURVE)
- static const uECC_word_t `curve_n` [uECC_N_WORDS]
- static `uECC_RNG_Function g_rng_function` = `&default_RNG`

5.41.1 Detailed Description

Implementation of the Micro-ECC library.

Author

Kenneth MacKay (refined by Sumedh Girish)

This file provides the core mathematical and cryptographic routines for ECDH and ECDSA, specifically optimized for the MSPM0 platform.

Definition in file [uECC.c](#).

5.41.2 Macro Definition Documentation

5.41.2.1 EVEN

```
#define EVEN(  
    vli)
```

Value:

```
(!(vli[0] & 1))
```

Definition at line 1857 of file [uECC.c](#).

Referenced by [vli_modInv\(\)](#), and [vli_modInv_n\(\)](#).

5.41.2.2 MAX_TRIES

```
#define MAX_TRIES 64
```

Definition at line 85 of file [uECC.c](#).

Referenced by [uECC_make_key\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign\(\)](#), [uECC_sign_deterministic\(\)](#), [uECC_sign_with_k\(\)](#), and [vli_blindScalar\(\)](#).

5.41.2.3 muladd_exists [1/2]

```
#define muladd_exists 1
```

Definition at line 771 of file [uECC.c](#).

5.41.2.4 muladd_exists [2/2]

```
#define muladd_exists 1
```

Definition at line 771 of file [uECC.c](#).

5.41.2.5 RESTRICT

```
#define RESTRICT
```

Definition at line 75 of file [uECC.c](#).

Referenced by [apply_z\(\)](#), [curve_x_side\(\)](#), [EccPoint_double_jacobian\(\)](#), [EccPoint_mult\(\)](#), [omega_mult\(\)](#), [vli_mmod_fast\(\)](#), [XYcZ_add\(\)](#), [XYcZ_addC\(\)](#), and [XYcZ_initial_double\(\)](#).

5.41.2.6 SUPPORTS_INT128

```
#define SUPPORTS_INT128 0
```

Definition at line 82 of file [uECC.c](#).

5.41.2.7 uECC_N_WORDS

```
#define uECC_N_WORDS uECC_CONCAT(uECC_N_WORDS_, uECC_CURVE)
```

Definition at line 397 of file [uECC.c](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign\(\)](#), [uECC_sign_deterministic\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), [vli2_rshift1_n\(\)](#), [vli2_sub_n\(\)](#), [vli_add_n\(\)](#), [vli_blindScalar\(\)](#), [vli_clear_n\(\)](#), [vli_cmp_n\(\)](#), [vli_isZero_n\(\)](#), [vli_modInv_n\(\)](#), [vli_modMult_n\(\)](#), [vli_mult_n\(\)](#), [vli_rshift1_n\(\)](#), [vli_set_n\(\)](#), and [vli_sub_n\(\)](#).

5.41.2.8 uECC_PLATFORM

```
#define uECC_PLATFORM uECC_arch_other
```

Definition at line 30 of file [uECC.c](#).

5.41.2.9 uECC_WORD_SIZE

```
#define uECC_WORD_SIZE 1
```

Definition at line 36 of file [uECC.c](#).

5.41.2.10 uECC_WORDS

```
#define uECC_WORDS uECC_CONCAT(uECC_WORDS_, uECC_CURVE)
```

Definition at line 396 of file [uECC.c](#).

Referenced by [apply_z\(\)](#), [curve_x_side\(\)](#), [EccPoint_compute_public_key\(\)](#), [EccPoint_double_jacobian\(\)](#), [EccPoint_mult\(\)](#), [mod_sqrt\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rp\(\)](#), [mod_sqrt_secp224r1_rs\(\)](#), [uECC_compute_public_key\(\)](#), [uECC_make_key\(\)](#), [uECC_mod_mult\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign\(\)](#), [uECC_sign_deterministic\(\)](#), [uECC_valid_public_key\(\)](#), [uECC_verify\(\)](#), [vli_add\(\)](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), [vli_cmp\(\)](#), [vli_equal\(\)](#), [vli_isZero\(\)](#), [vli_mmod_fast\(\)](#), [vli_modInv\(\)](#), [vli_modMult_fast\(\)](#), [vli_mult\(\)](#), [vli_nativeToBytes\(\)](#), [vli_rshift1\(\)](#), [vli_set\(\)](#), [vli_sub\(\)](#), [XYcZ_add\(\)](#), [XYcZ_addC\(\)](#), and [XYcZ_initial_double\(\)](#).

5.41.2.11 vli_modSquare_fast

```
#define vli_modSquare_fast(  
    result,  
    left)
```

Value:

```
vli_modMult_fast((result), (left), (left))
```

Definition at line 1852 of file [uECC.c](#).

Referenced by [apply_z\(\)](#), [curve_x_side\(\)](#), [EccPoint_double_jacobian\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rs\(\)](#), [uECC_valid_public_key\(\)](#), [XYcZ_add\(\)](#), and [XYcZ_addC\(\)](#).

5.41.2.12 vli_modSub_fast

```
#define vli_modSub_fast(  
    result,  
    left,  
    right)
```

Value:

```
vli_modSub((result), (left), (right), curve_p)
```

Definition at line 933 of file [uECC.c](#).

Referenced by [curve_x_side\(\)](#), [EccPoint_mult\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rp\(\)](#), [uECC_verify\(\)](#), [XYcZ_add\(\)](#), and [XYcZ_addC\(\)](#).

5.41.2.13 vli_square

```
#define vli_square(  
    result,  
    left,  
    size)
```

Value:

`vli_mult((result), (left), (left), (size))`

Definition at line 892 of file [uECC.c](#).

5.41.3 Typedef Documentation

5.41.3.1 EccPoint

```
typedef struct EccPoint EccPoint
```

5.41.4 Function Documentation

5.41.4.1 __attribute__((used))

```
__attribute__((  
    used))
```

Definition at line 541 of file [uECC.c](#).

References [g_rng_function](#).

5.41.4.2 apply_z()

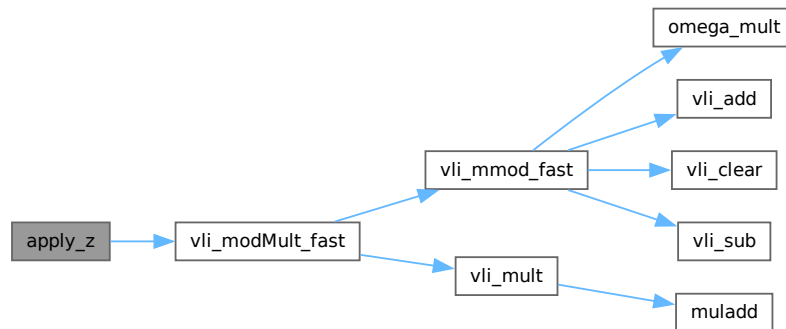
```
void apply_z (  
    uECC_word_t *RESTRICT X1,  
    uECC_word_t *RESTRICT Y1,  
    const uECC_word_t *RESTRICT Z) [static]
```

Definition at line 2060 of file [uECC.c](#).

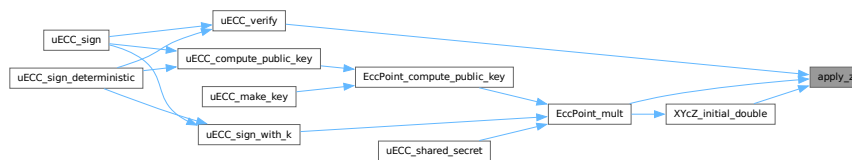
References [RESTRICT](#), [uECC_WORDS](#), [vli_modMult_fast\(\)](#), and [vli_modSquare_fast](#).

Referenced by [EccPoint_mult\(\)](#), [uECC_verify\(\)](#), and [XYcZ_initial_double\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.3 curve_x_side()

```

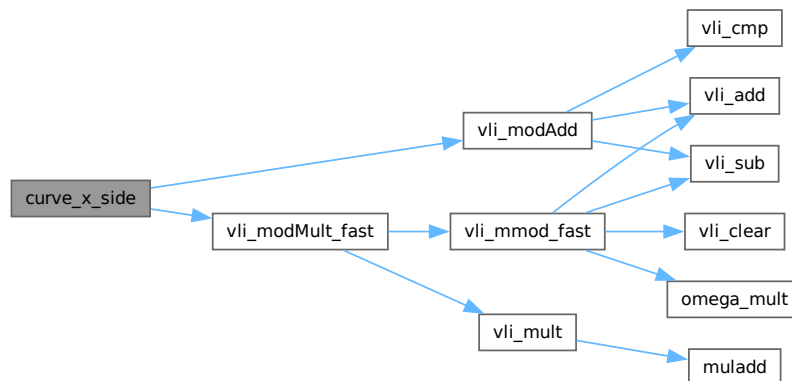
void curve_x_side (
    uECC_word_t *RESTRICT result,
    const uECC_word_t *RESTRICT x) [static]
  
```

Definition at line 2460 of file [uECC.c](#).

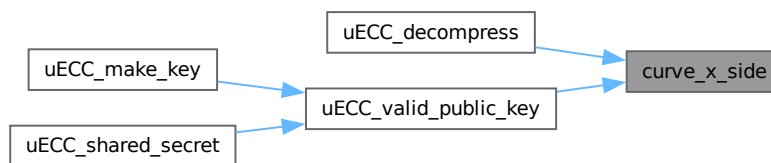
References [curve_b](#), [curve_p](#), [RESTRICT](#), [uECC_WORDS](#), [vli_modAdd\(\)](#), [vli_modMult_fast\(\)](#), [vli_modSquare_fast](#), and [vli_modSub_fast](#).

Referenced by [uECC_decompress\(\)](#), and [uECC_valid_public_key\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.4 default_RNG()

```
int default_RNG (
    uint8_t * dest,
    unsigned size) [static]
```

Definition at line 532 of file [uECC.c](#).

5.41.4.5 EccPoint_compute_public_key()

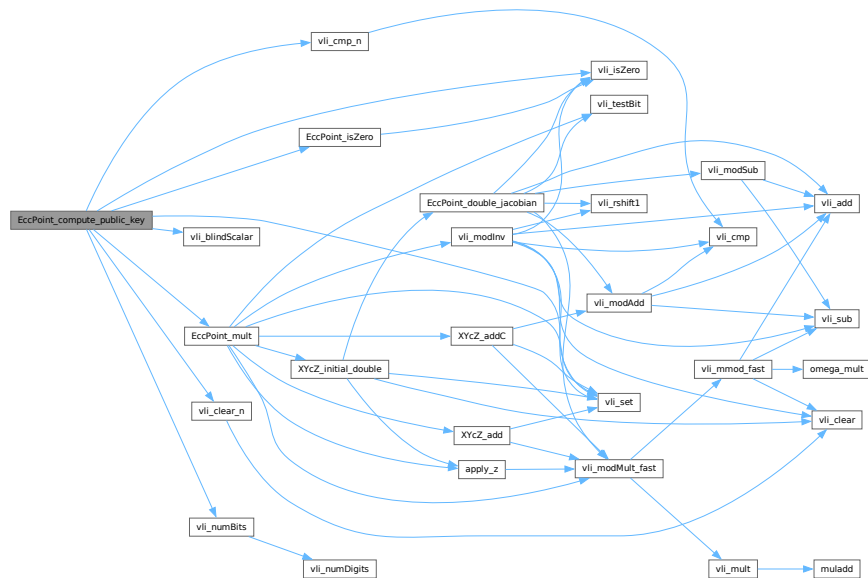
```
int EccPoint_compute_public_key (
    EccPoint * result,
    uECC_word_t * private) [static]
```

Definition at line 2891 of file [uECC.c](#).

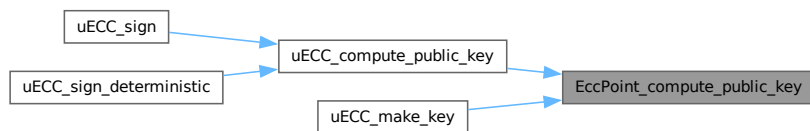
References [curve_G](#), [curve_n](#), [EccPoint_isZero\(\)](#), [EccPoint_mult\(\)](#), [g_rng_function](#), [uECC_N_WORDS](#), [uECC_WORDS](#), [vli_blindScalar\(\)](#), [vli_clear_n\(\)](#), [vli_cmp_n\(\)](#), [vli_isZero\(\)](#), [vli_numBits\(\)](#), and [vli_set\(\)](#).

Referenced by [uECC_compute_public_key\(\)](#), and [uECC_make_key\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.6 EccPoint_double_jacobian()

```

void EccPoint_double_jacobian (
    uECC_word_t *RESTRICT X1,
    uECC_word_t *RESTRICT Y1,
    uECC_word_t *RESTRICT Z1) [static]

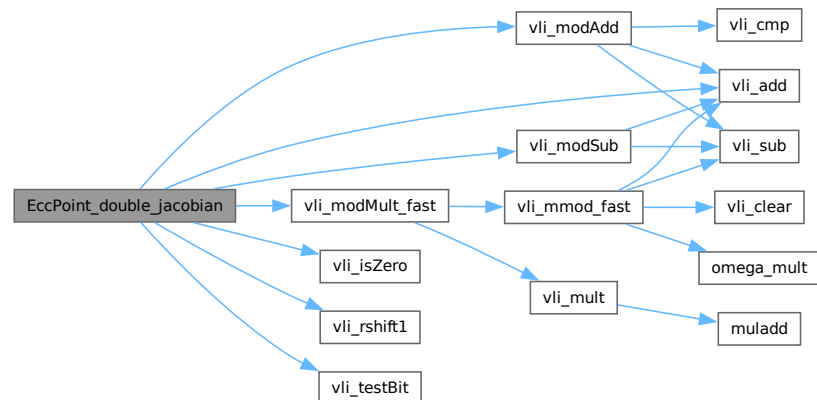
```

Definition at line 1966 of file [uECC.c](#).

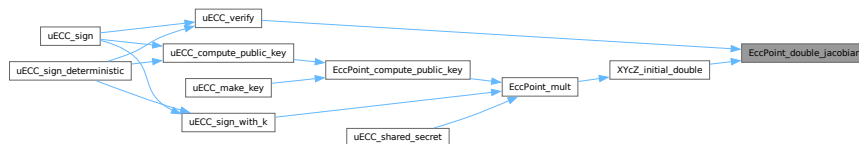
References [curve_p](#), [RESTRICT](#), [uECC_WORDS](#), [vli_add\(\)](#), [vli_isZero\(\)](#), [vli_modAdd\(\)](#), [vli_modMult_fast\(\)](#), [vli_modSquare_fast](#), [vli_modSub\(\)](#), [vli_rshift1\(\)](#), and [vli_testBit\(\)](#).

Referenced by [uECC_verify\(\)](#), and [XYZ_initial_double\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.7 EccPoint_isZero()

```

cmpresult_t EccPoint_isZero (
    const EccPoint * point) [static]

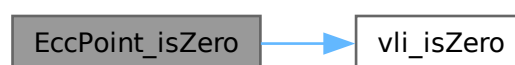
```

Definition at line 1955 of file [uECC.c](#).

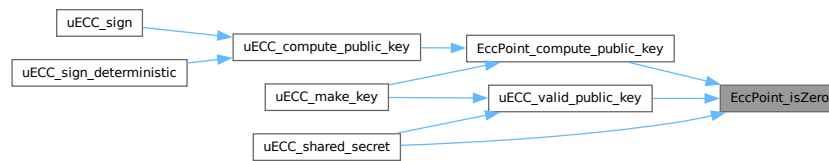
References [vli_isZero\(\)](#), [EccPoint::x](#), and [EccPoint::y](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), and [uECC_valid_public_key\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.8 EccPoint_mult()

```

void EccPoint_mult (
    EccPoint *RESTRICT result,
    const EccPoint *RESTRICT point,
    const uECC_word_t *RESTRICT scalar,
    const uECC_word_t *RESTRICT initialZ,
    bitcount_t numBits) [static]

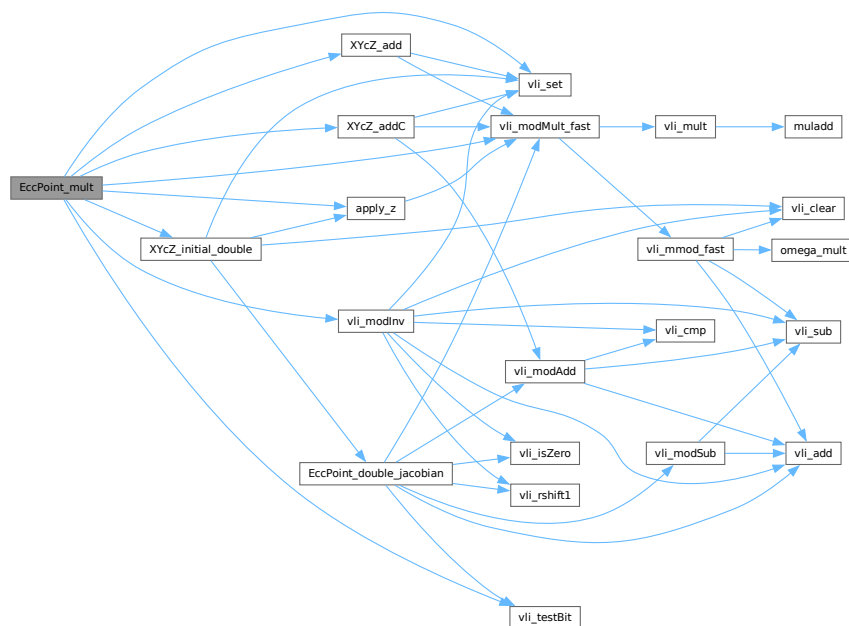
```

Definition at line 2163 of file [uECC.c](#).

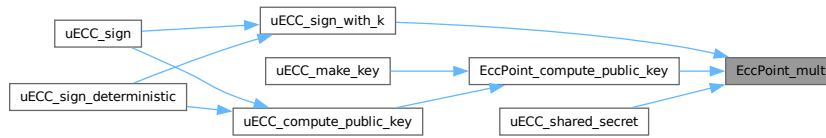
References [apply_z\(\)](#), [curve_p](#), [RESTRICT](#), [uECC_WORDS](#), [vli_modInv\(\)](#), [vli_modMult_fast\(\)](#), [vli_modSub_fast](#), [vli_set\(\)](#), [vli_testBit\(\)](#), [XYcZ_add\(\)](#), [XYcZ_addC\(\)](#), and [XYcZ_initial_double\(\)](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), and [uECC_sign_with_k\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.9 HMAC_finish()

```

void HMAC_finish (
    uECC_HashContext * hash_context,
    const uint8_t * K,
    uint8_t * result) [static]
  
```

Definition at line 3107 of file [uECC.c](#).

References [uECC_HashContext::block_size](#), [uECC_HashContext::finish_hash](#), [uECC_HashContext::init_hash](#), [uECC_HashContext::result_size](#), [uECC_HashContext::tmp](#), and [uECC_HashContext::update_hash](#).

Referenced by [uECC_sign_deterministic\(\)](#), and [update_V\(\)](#).

Here is the caller graph for this function:



5.41.4.10 HMAC_init()

```

void HMAC_init (
    uECC_HashContext * hash_context,
    const uint8_t * K) [static]
  
```

Definition at line 3088 of file [uECC.c](#).

References [uECC_HashContext::block_size](#), [uECC_HashContext::init_hash](#), [uECC_HashContext::result_size](#), [uECC_HashContext::tmp](#), and [uECC_HashContext::update_hash](#).

Referenced by [uECC_sign_deterministic\(\)](#), and [update_V\(\)](#).

Here is the caller graph for this function:



5.41.4.11 HMAC_update()

```
void HMAC_update (
    uECC_HashContext * hash_context,
    const uint8_t * message,
    unsigned message_size) [static]
```

Definition at line 3101 of file [uECC.c](#).

References [uECC_HashContext::update_hash](#).

Referenced by [uECC_sign_deterministic\(\)](#), and [update_V\(\)](#).

Here is the caller graph for this function:



5.41.4.12 mod_sqrt()

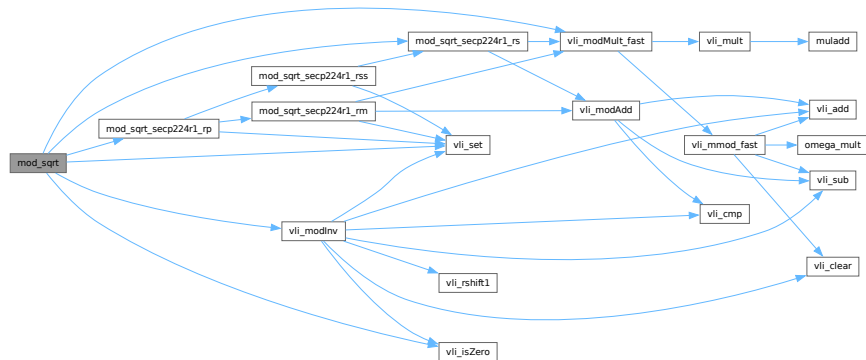
```
void mod_sqrt (
    uECC_word_t * a) [static]
```

Definition at line 2296 of file [uECC.c](#).

References [curve_p](#), [mod_sqrt_secp224r1_rp\(\)](#), [mod_sqrt_secp224r1_rs\(\)](#), [uECC_WORDS](#), [vli_isZero\(\)](#), [vli_modInv\(\)](#), [vli_modMult_fast\(\)](#), and [vli_set\(\)](#).

Referenced by [uECC_decompress\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.13 mod_sqrt_secp224r1_rm()

```

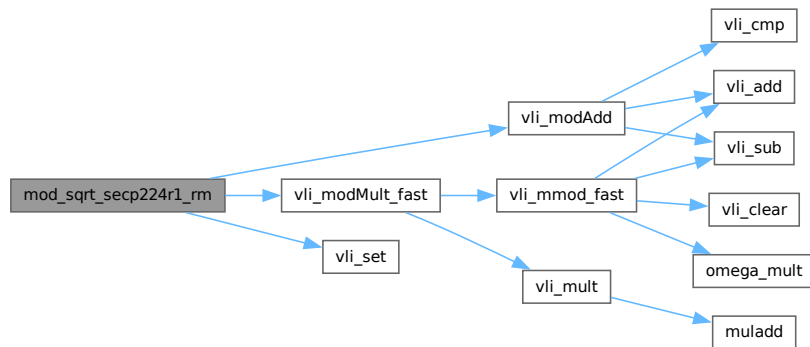
void mod_sqrt_secp224r1_rm (
    uECC_word_t * d2,
    uECC_word_t * e2,
    uECC_word_t * f2,
    const uECC_word_t * c,
    const uECC_word_t * d0,
    const uECC_word_t * e0,
    const uECC_word_t * d1,
    const uECC_word_t * e1) [static]
  
```

Definition at line 2245 of file [uECC.c](#).

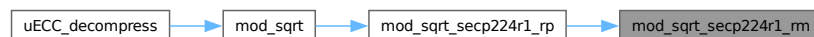
References [curve_p](#), [uECC_WORDS](#), [vli_modAdd\(\)](#), [vli_modMult_fast\(\)](#), [vli_modSquare_fast](#), [vli_modSub_fast](#), and [vli_set\(\)](#).

Referenced by [mod_sqrt_secp224r1_rp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.14 mod_sqrt_secp224r1_rp()

```

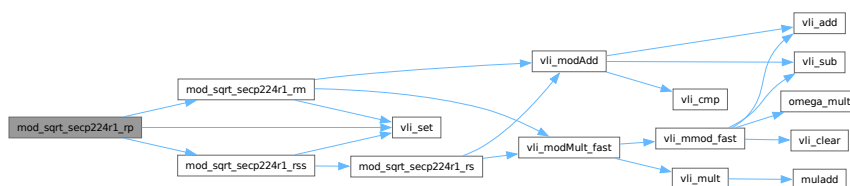
void mod_sqrt_secp224r1_rp (
    uECC_word_t * d1,
    uECC_word_t * e1,
    uECC_word_t * f1,
    const uECC_word_t * c,
    const uECC_word_t * r) [static]
  
```

Definition at line 2268 of file [uECC.c](#).

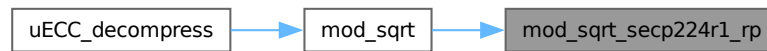
References [curve_p](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rss\(\)](#), [uECC_WORDS](#), [vli_modSub_fast](#), and [vli_set\(\)](#).

Referenced by [mod_sqrt\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.15 mod_sqrt_secp224r1_rs()

```

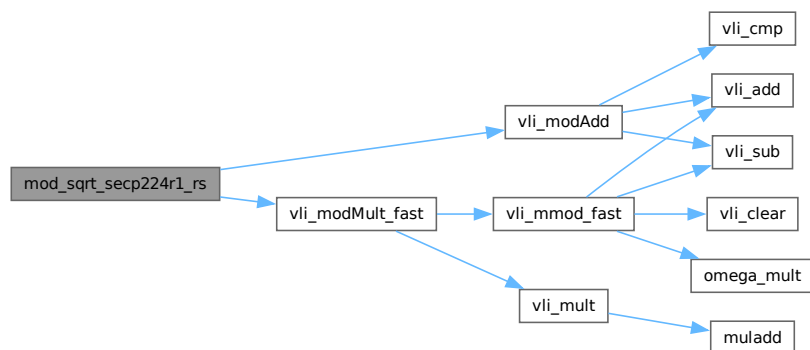
void mod_sqrt_secp224r1_rs (
    uECC_word_t * d1,
    uECC_word_t * e1,
    uECC_word_t * f1,
    const uECC_word_t * d0,
    const uECC_word_t * e0,
    const uECC_word_t * f0) [static]
  
```

Definition at line 2210 of file `uECC.c`.

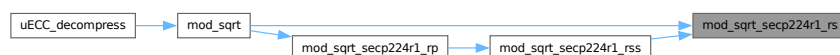
References `curve_p`, `uECC_WORDS`, `vli_modAdd()`, `vli_modMult_fast()`, and `vli_modSquare_fast`.

Referenced by `mod_sqrt()`, and `mod_sqrt_secp224r1_rss()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.16 mod_sqrt_secp224r1_rss()

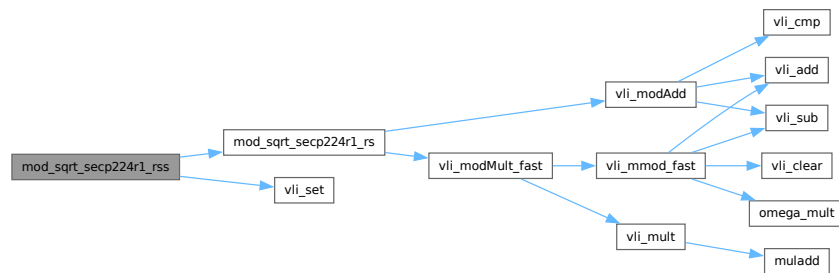
```
void mod_sqrt_secp224r1_rss (
    uECC_word_t * d1,
    uECC_word_t * e1,
    uECC_word_t * f1,
    const uECC_word_t * d0,
    const uECC_word_t * e0,
    const uECC_word_t * f0,
    const bitcount_t j) [static]
```

Definition at line 2227 of file [uECC.c](#).

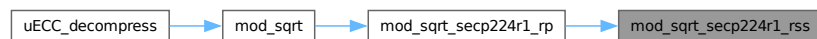
References [mod_sqrt_secp224r1_rs\(\)](#), and [vli_set\(\)](#).

Referenced by [mod_sqrt_secp224r1_rp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



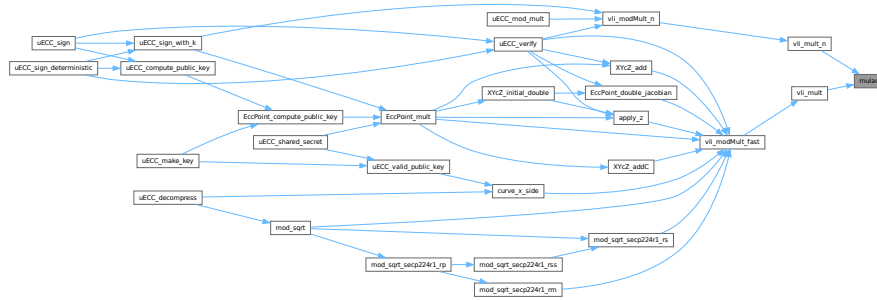
5.41.4.17 muladd()

```
void muladd (
    uECC_word_t a,
    uECC_word_t b,
    uECC_word_t * r0,
    uECC_word_t * r1,
    uECC_word_t * r2) [static]
```

Definition at line 733 of file [uECC.c](#).

Referenced by [vli_mult\(\)](#), and [vli_mult_n\(\)](#).

Here is the caller graph for this function:



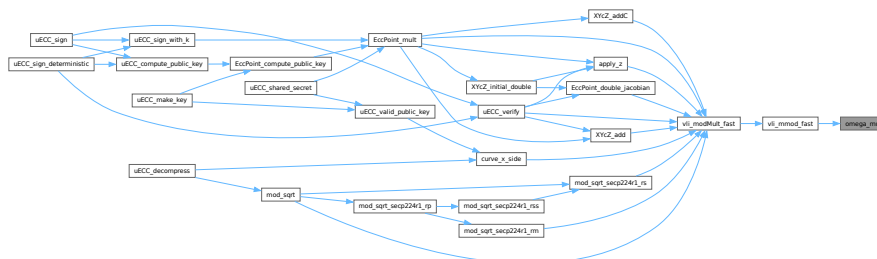
5.41.4.18 omega_mult()

```
void omega_mult (
    uECC_word_t *RESTRICT result,
    const uECC_word_t *RESTRICT right) [static]
```

References [RESTRICT](#).

Referenced by [vli_mmod_fast\(\)](#).

Here is the caller graph for this function:



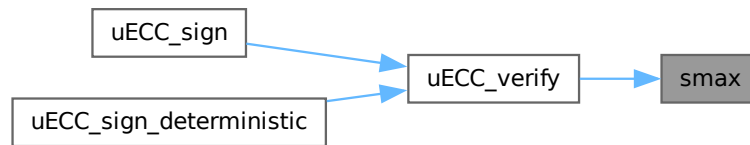
5.41.4.19 smax()

```
bitcount_t smax (
    bitcount_t a,
    bitcount_t b) [static]
```

Definition at line 3219 of file [uECC.c](#).

Referenced by [uECC_verify\(\)](#).

Here is the caller graph for this function:



5.41.4.20 uECC_bytes()

```
int uECC_bytes (
    void )
```

Returns the value of uECC_BYTES for the current build.

Returns

int Current uECC_BYTES value.

Definition at line [2541](#) of file [uECC.c](#).

References [uECC_BYTES](#).

5.41.4.21 uECC_compress()

```
void uECC_compress (
    const uint8_t public_key[uECC_BYTES *2],
    uint8_t compressed[uECC_BYTES+1])
```

Compress a public key (Y coordinate is reduced to parity bit).

Parameters

in	<i>public_key</i>	The public key to compress.
out	<i>compressed</i>	Will be filled with the compressed key (uECC_BYTES + 1 bytes).

Definition at line [2448](#) of file [uECC.c](#).

References [uECC_BYTES](#).

5.41.4.22 uECC_compute_public_key()

```
int uECC_compute_public_key (
    const uint8_t private_key[uECC_BYTES],
    uint8_t public_key[uECC_BYTES * 2])
```

Compute the corresponding public key for a given private key.

Parameters

in	<i>private_key</i>	The private key.
out	<i>public_key</i>	Will be filled with the corresponding public key.

Returns

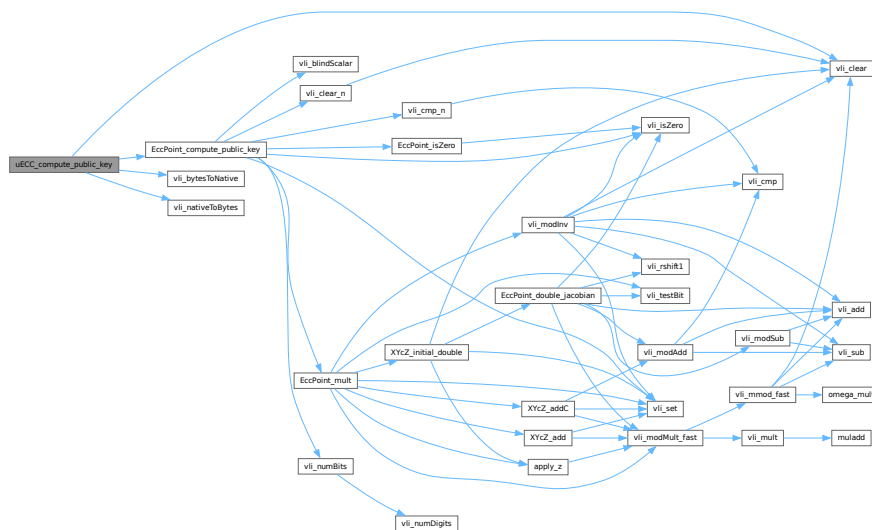
int 1 if the key was computed successfully, 0 otherwise.

Definition at line 2522 of file [uECC.c](#).

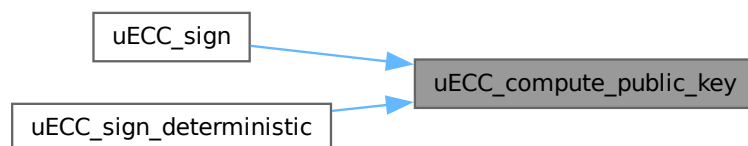
References [EccPoint_compute_public_key\(\)](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), and [vli_nativeToBytes\(\)](#).

Referenced by [uECC_sign\(\)](#), and [uECC_sign_deterministic\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.25 uECC_make_key()

```
int uECC_make_key (
    uint8_t public_key[uECC_BYTES *2],
    uint8_t private_key[uECC_BYTES])
```

Create a public/private key pair.

Parameters

out	<i>public_key</i>	Will be filled with the public key (2* <code>uECC_BYTES</code> bytes).
out	<i>private_key</i>	Will be filled with the private key (<code>uECC_BYTES</code> bytes).

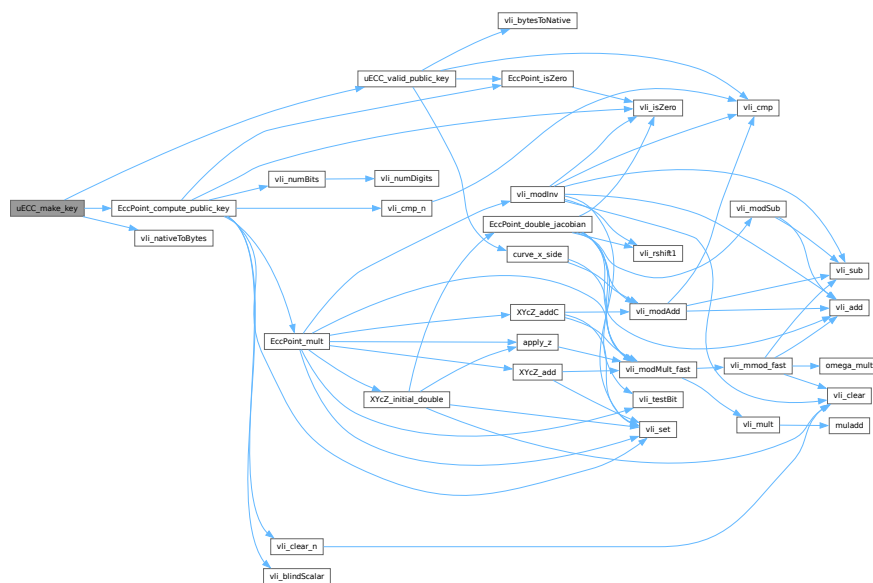
Returns

int 1 if the key pair was generated successfully, 0 otherwise.

Definition at line 2424 of file uECC.c.

References [EccPoint_compute_public_key\(\)](#), [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_valid_public_key\(\)](#), [uECC_WORDS](#), and [vli_nativeToBytes\(\)](#).

Here is the call graph for this function:



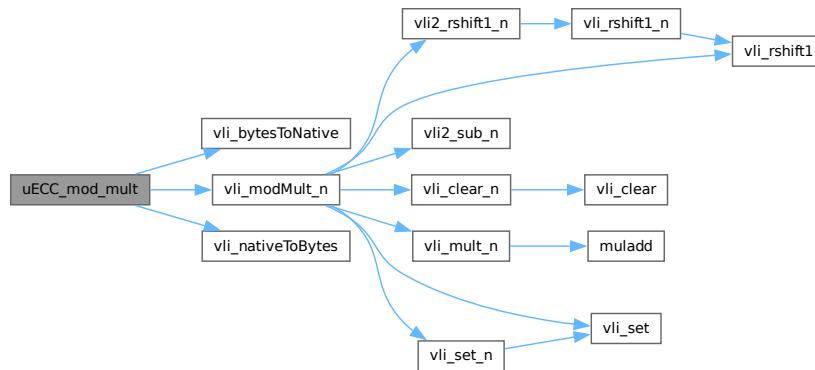
5.41.4.26 uECC_mod_mult()

```
void uECC_mod_mult (
    uint8_t result[32],
    const uint8_t left[32],
    const uint8_t right[32])
```

Definition at line 3327 of file [uECC.c](#).

References [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_modMult_n\(\)](#), and [vli_nativeToBytes\(\)](#).

Here is the call graph for this function:



5.41.4.27 uECC_shared_secret()

```
int uECC_shared_secret (
    const uint8_t public_key[uECC_BYTES *2],
    const uint8_t private_key[uECC_BYTES],
    uint8_t secret[uECC_BYTES])
```

Compute a shared secret given your secret key and someone else's public key (ECDH).

Note: It is recommended that you hash the result before using it for symmetric encryption.

Parameters

in	<i>public_key</i>	The public key of the remote party.
in	<i>private_key</i>	Your private key.
out	<i>secret</i>	Will be filled with the shared secret value (uECC_BYTES bytes).

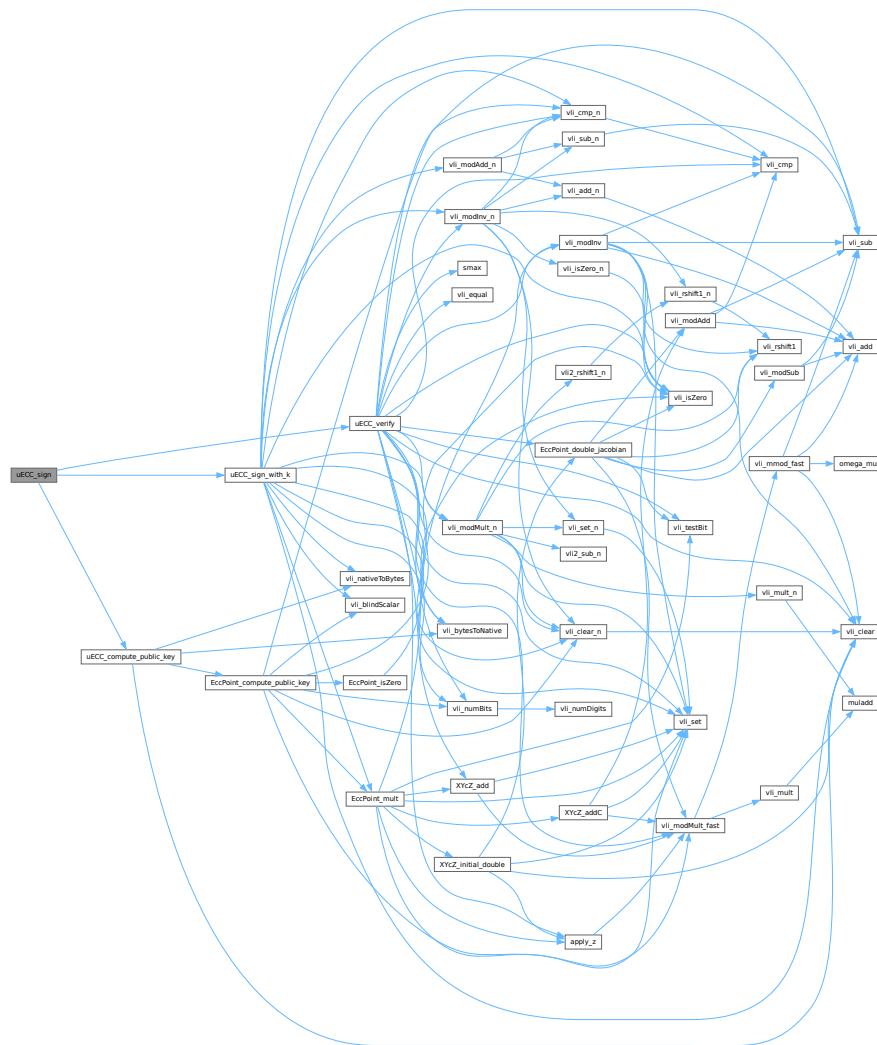
Returns

int 1 if the shared secret was generated successfully, 0 otherwise.

Definition at line 2923 of file [uECC.c](#).

References [EccPoint_isZero\(\)](#), [EccPoint_mult\(\)](#), [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_valid_public_key\(\)](#), [uECC_WORDS](#), [vli_blindScalar\(\)](#), [vli_bytesToNative\(\)](#), [vli_clear_n\(\)](#), [vli_isZero\(\)](#), [vli_nativeToBytes\(\)](#), [vli_numBits\(\)](#), [EccPoint::x](#), and [EccPoint::y](#).

Here is the call graph for this function:



5.41.4.29 uECC_sign_deterministic()

```
int uECC_sign_deterministic (
    const uint8_t private_key[uECC_BYTES],
    const uint8_t message_hash[uECC_BYTES],
    uECC_HashContext * hash_context,
    uint8_t signature[uECC_BYTES *2])
```

Generate an ECDSA signature using a deterministic algorithm (RFC 6979).

Parameters

in	<i>private_key</i>	Your private key.
in	<i>message_hash</i>	The hash of the message to sign.
in	<i>hash_context</i>	A hash context for HMAC-DRBG operations.

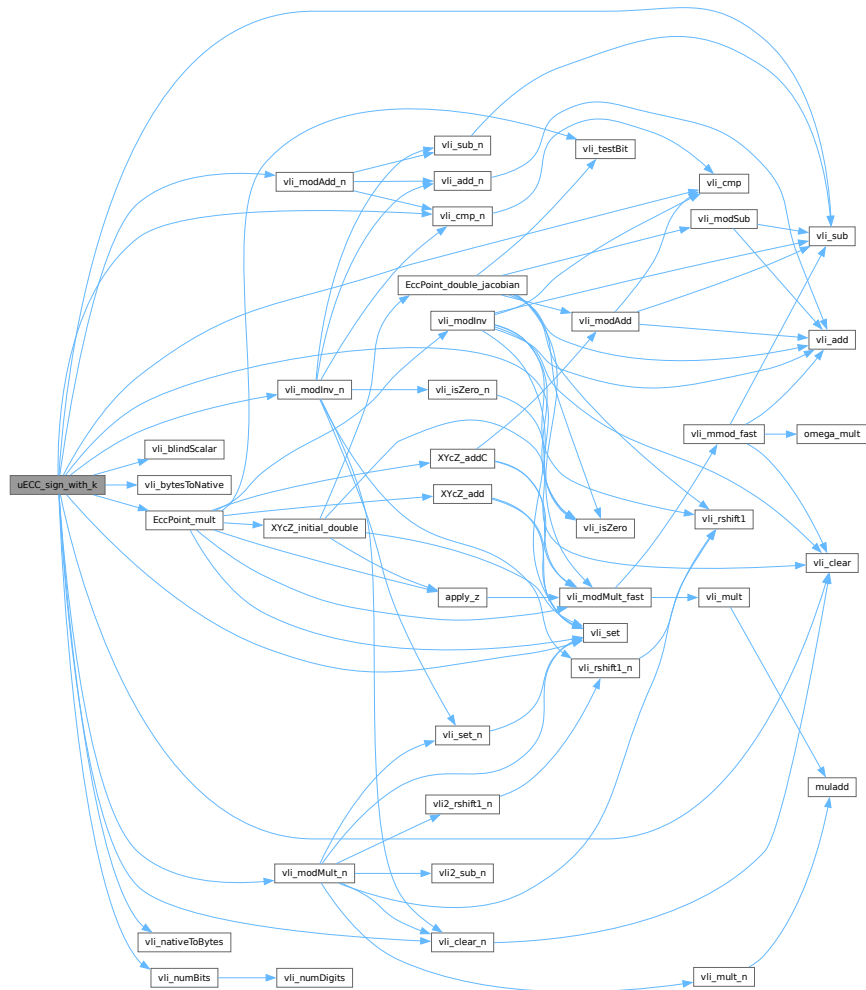

```
const uint8_t message_hash[uECC_BYTES],
uECC_word_t k[uECC_N_WORDS],
uint8_t signature[uECC_BYTES *2]) [static]
```

Definition at line 2971 of file [uECC.c](#).

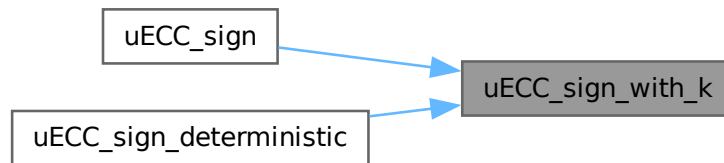
References [curve_G](#), [curve_n](#), [EccPoint_mult\(\)](#), [g_rng_function](#), [MAX_TRIES](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [vli_blindScalar\(\)](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), [vli_clear_n\(\)](#), [vli_cmp\(\)](#), [vli_cmp_n\(\)](#), [vli_isZero\(\)](#), [vli_modAdd_n\(\)](#), [vli_modInv_n\(\)](#), [vli_modMult_n\(\)](#), [vli_nativeToBytes\(\)](#), [vli_numBits\(\)](#), [vli_set\(\)](#), [vli_sub\(\)](#), and [EccPoint::x](#).

Referenced by [uECC_sign\(\)](#), and [uECC_sign_deterministic\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.31 uECC_valid_public_key()

```
int uECC_valid_public_key (
    const uint8_t public_key[uECC_BYTES *2])
```

Check if a public key is a valid point on the curve.

Includes checks for the point at infinity and whether the point is on the curve.

Parameters

in	<i>public_key</i>	The public key to check.
----	-------------------	--------------------------

Returns

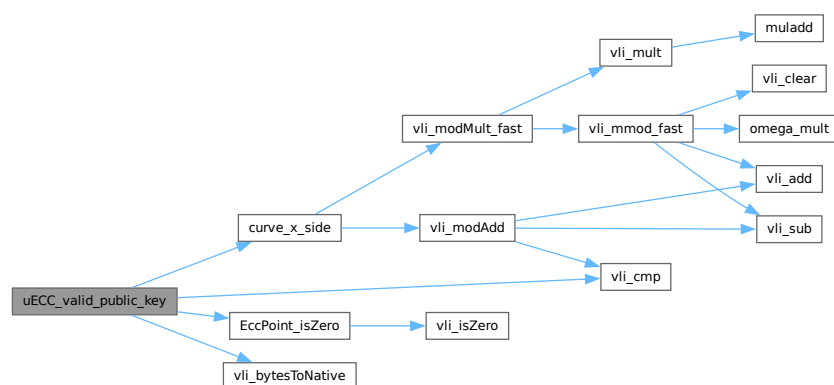
int 1 if the public key is valid, 0 otherwise.

Definition at line 2494 of file [uECC.c](#).

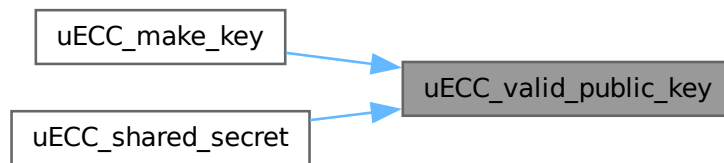
References [curve_p](#), [curve_x_side\(\)](#), [EccPoint_isZero\(\)](#), [uECC_BYTES](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_cmp\(\)](#), and [vli_modSquare_fast](#).

Referenced by [uECC_make_key\(\)](#), and [uECC_shared_secret\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.32 uECC_verify()

```
int uECC_verify (
    const uint8_t public_key[uECC_BYTES *2],
    const uint8_t hash[uECC_BYTES],
    const uint8_t signature[uECC_BYTES *2])
```

Verify an ECDSA signature against a public key and hash.

Parameters

in	<i>public_key</i>	The signer's public key.
in	<i>hash</i>	The hash of the signed data.
in	<i>signature</i>	The signature value.

Returns

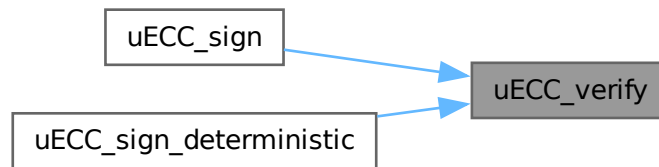
int 1 if the signature is valid, 0 if it is invalid or an error occurred.

Definition at line 3224 of file [uECC.c](#).

References [apply_z\(\)](#), [curve_G](#), [curve_n](#), [curve_p](#), [EccPoint_double_jacobian\(\)](#), [smax\(\)](#), [uECC_BYTES](#), [uECC_N_WORDS](#), [uECC_WORDS](#), [vli_bytesToNative\(\)](#), [vli_clear\(\)](#), [vli_clear_n\(\)](#), [vli_cmp\(\)](#), [vli_cmp_n\(\)](#), [vli_equal\(\)](#), [vli_isZero\(\)](#), [vli_modInv\(\)](#), [vli_modInv_n\(\)](#), [vli_modMult_fast\(\)](#), [vli_modMult_n\(\)](#), [vli_modSub_fast\(\)](#), [vli_numBits\(\)](#), [vli_set\(\)](#), [vli_sub\(\)](#), [vli_testBit\(\)](#), [EccPoint::x](#), [XYcZ_add\(\)](#), and [EccPoint::y](#).

Referenced by [uECC_sign\(\)](#), and [uECC_sign_deterministic\(\)](#).

Here is the caller graph for this function:



5.41.4.33 `update_V()`

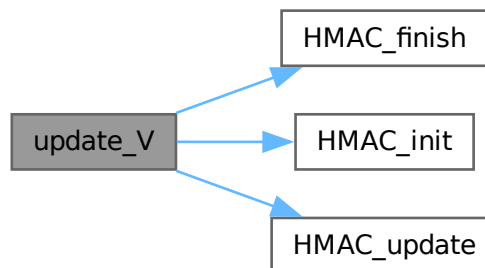
```
void update_V (  
    uECC_HashContext * hash_context,  
    uint8_t * K,  
    uint8_t * V) [static]
```

Definition at line 3126 of file `uECC.c`.

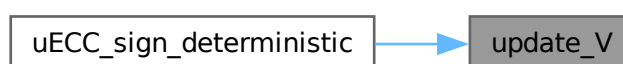
References `HMAC_finish()`, `HMAC_init()`, `HMAC_update()`, and `uECC_HashContext::result_size`.

Referenced by `uECC_sign_deterministic()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.34 vli2_rshift1_n()

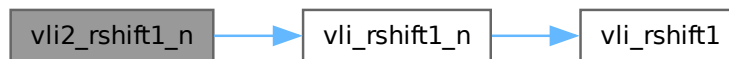
```
void vli2_rshift1_n (
    uECC_word_t * vli) [static]
```

Definition at line 2760 of file [uECC.c](#).

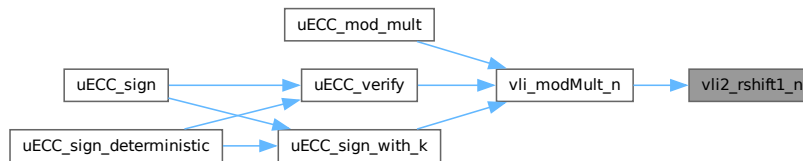
References [uECC_N_WORDS](#), and [vli_rshift1_n\(\)](#).

Referenced by [vli_modMult_n\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:

**5.41.4.35 vli2_sub_n()**

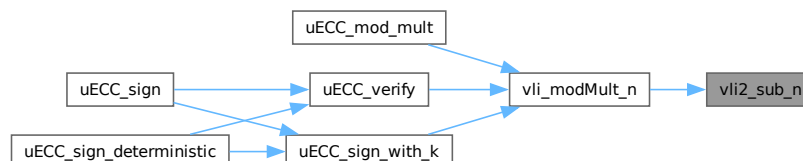
```
uECC_word_t vli2_sub_n (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
```

Definition at line 2767 of file [uECC.c](#).

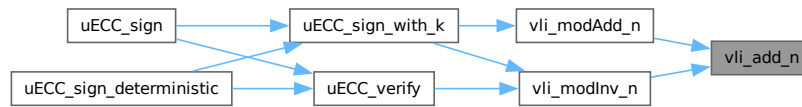
References [uECC_N_WORDS](#).

Referenced by [vli_modMult_n\(\)](#).

Here is the caller graph for this function:



Here is the caller graph for this function:



5.41.4.38 vli_blindScalar()

```

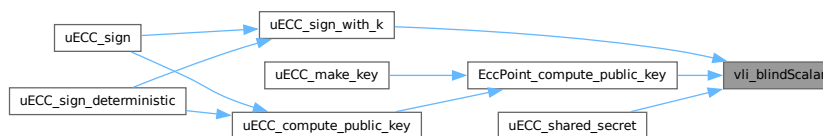
void vli_blindScalar (
    uECC_word_t * result,
    const uECC_word_t * scalar) [static]
  
```

Definition at line 2864 of file [uECC.c](#).

References [curve_n](#), [g_rng_function](#), [MAX_TRIES](#), and [uECC_N_WORDS](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), and [uECC_sign_with_k\(\)](#).

Here is the caller graph for this function:



5.41.4.39 vli_bytesToNative()

```

void vli_bytesToNative (
    uint64_t * native,
    const uint8_t * bytes) [static]
  
```

Definition at line 2409 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [uECC_compute_public_key\(\)](#), [uECC_decompress\(\)](#), [uECC_mod_mult\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_valid_public_key\(\)](#), and [uECC_verify\(\)](#).

5.41.4.41 vli_clear_n()

```
void vli_clear_n (
    uECC_word_t * vli) [static]
```

Definition at line 2554 of file [uECC.c](#).

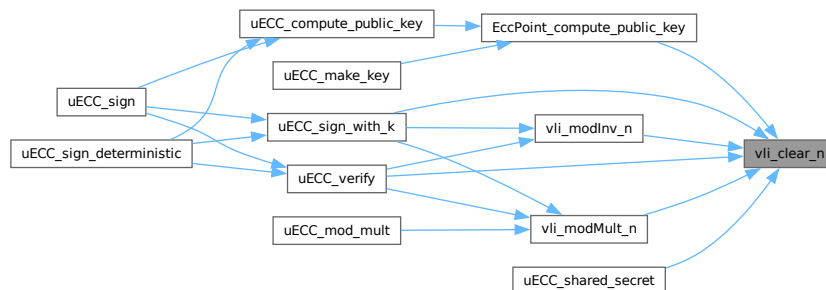
References [uECC_N_WORDS](#), and [vli_clear\(\)](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), [vli_modInv_n\(\)](#), and [vli_modMult_n\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.42 vli_cmp()

```
cmpresult_t vli_cmp (
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
```

Definition at line 645 of file [uECC.c](#).

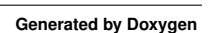
References [uECC_WORDS](#).

Referenced by [uECC_sign_with_k\(\)](#), [uECC_valid_public_key\(\)](#), [uECC_verify\(\)](#), [vli_cmp_n\(\)](#), [vli_modAdd\(\)](#), and [vli_modInv\(\)](#).

```
cmpresult_t vli_cmp_n (
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
```

References `uECC_N_WORDS`, and `vli_cmp()`.

Here is the call graph for this function:



5.41.4.44 vli_equal()

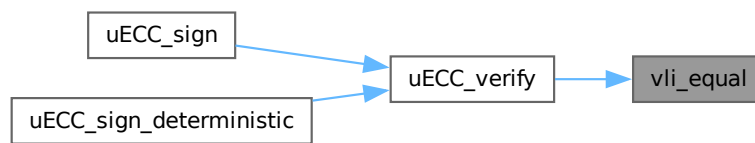
```
cmpresult_t vli_equal (
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
```

Definition at line 663 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [uECC_verify\(\)](#).

Here is the caller graph for this function:



5.41.4.45 vli_isZero()

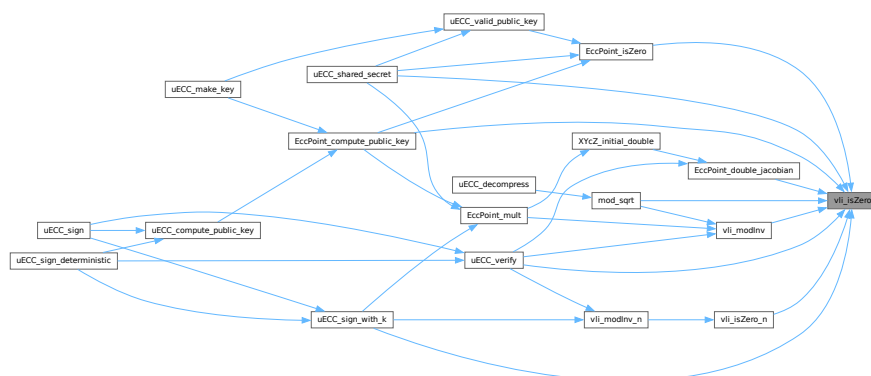
```
uECC_word_t vli_isZero (
    const uECC_word_t * vli) [static]
```

Definition at line 571 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [EccPoint_double_jacobian\(\)](#), [EccPoint_isZero\(\)](#), [mod_sqrt\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), [vli_isZero_n\(\)](#), and [vli_modInv\(\)](#).

Here is the caller graph for this function:



5.41.4.46 vli_isZero_n()

```
uECC_word_t vli_isZero_n (
    const uECC_word_t * vli)  [static]
```

Definition at line 2560 of file [uECC.c](#).

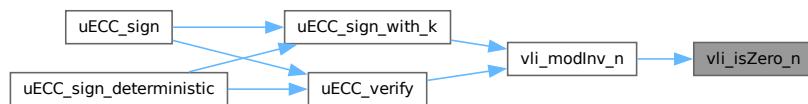
References [uECC_N_WORDS](#), and [vli_isZero\(\)](#).

Referenced by [vli_modInv_n\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.47 vli_mmod_fast()

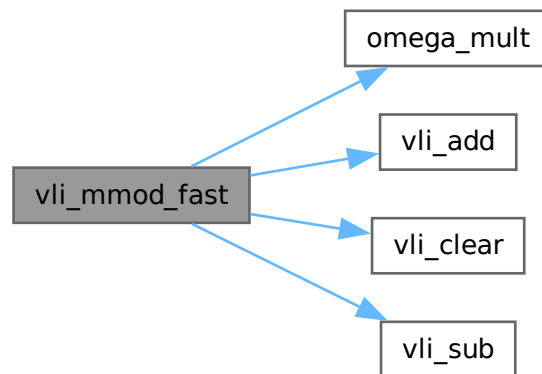
```
void vli_mmod_fast (
    uECC_word_t *RESTRICT result,
    uECC_word_t *RESTRICT product)  [static]
```

Definition at line 949 of file [uECC.c](#).

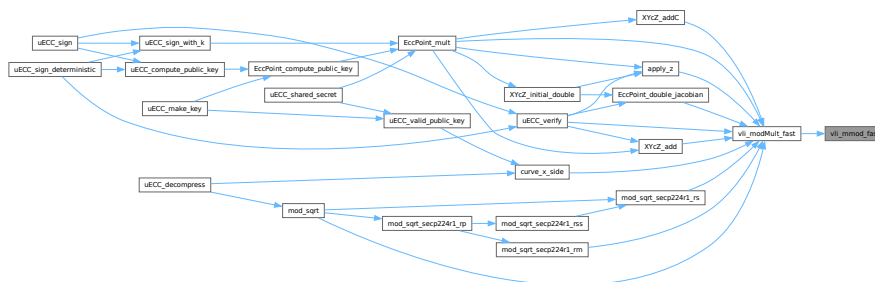
References [curve_p](#), [omega_mult\(\)](#), [RESTRICT](#), [uECC_WORDS](#), [vli_add\(\)](#), [vli_clear\(\)](#), and [vli_sub\(\)](#).

Referenced by [vli_modMult_fast\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.48 vli_modAdd()

```

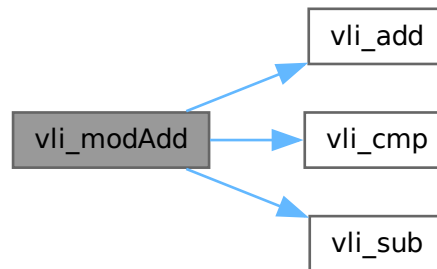
void vli_modAdd (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right,
    const uECC_word_t * mod) [static]
  
```

Definition at line 901 of file `uECC.c`.

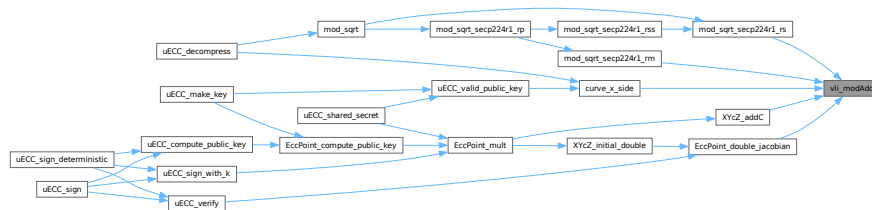
References `vli_add()`, `vli_cmp()`, and `vli_sub()`.

Referenced by `curve_x_side()`, `EccPoint_double_jacobian()`, `mod_sqrt_secp224r1_rm()`, `mod_sqrt_secp224r1_rs()`, and `XYcZ_addC()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.49 vli_modAdd_n()

```

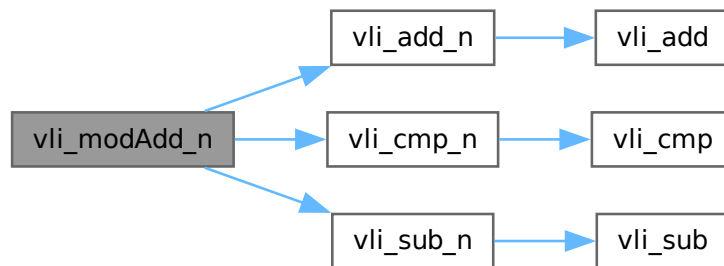
void vli_modAdd_n (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right,
    const uECC_word_t * mod) [static]
  
```

Definition at line 2660 of file `uECC.c`.

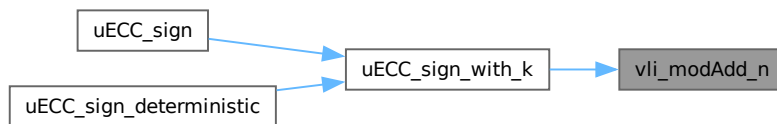
References `vli_add_n()`, `vli_cmp_n()`, and `vli_sub_n()`.

Referenced by `uECC_sign_with_k()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.50 vli_modInv()

```

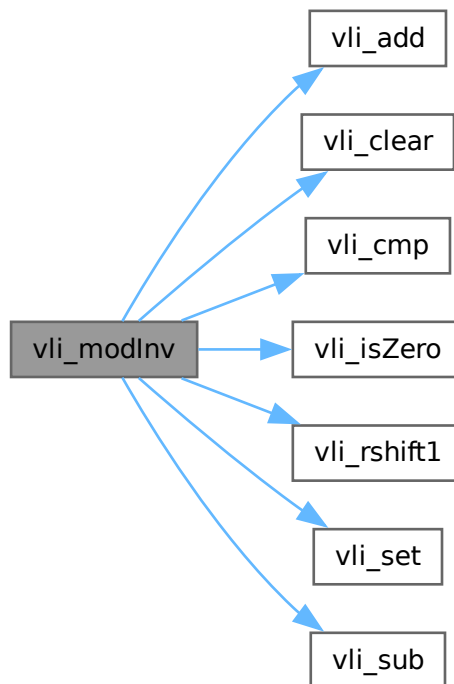
void vli_modInv (
    uECC_word_t * result,
    const uECC_word_t * input,
    const uECC_word_t * mod) [static]
  
```

Definition at line 1862 of file `uECC.c`.

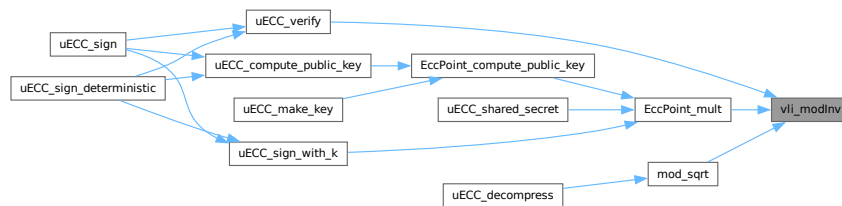
References [EVEN](#), [uECC_WORDS](#), [vli_add\(\)](#), [vli_clear\(\)](#), [vli_cmp\(\)](#), [vli_isZero\(\)](#), [vli_rshift1\(\)](#), [vli_set\(\)](#), and [vli_sub\(\)](#).

Referenced by [EccPoint_mult\(\)](#), [mod_sqrt\(\)](#), and [uECC_verify\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.51 vli_modInv_n()

```

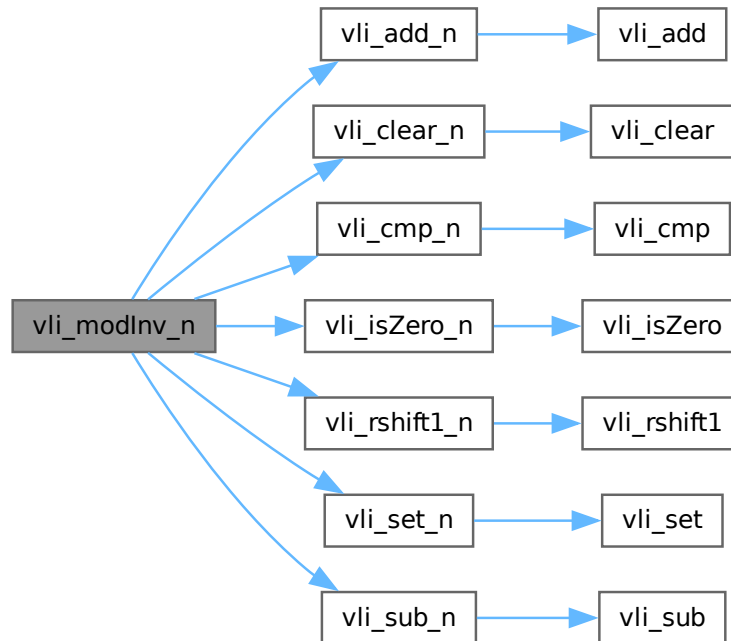
void vli_modInv_n (
    uECC_word_t * result,
    const uECC_word_t * input,
    const uECC_word_t * mod) [static]
  
```

Definition at line 2670 of file [uECC.c](#).

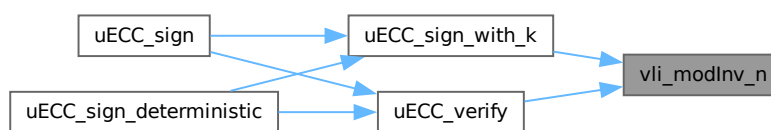
References [EVEN](#), [uECC_N_WORDS](#), [vli_add_n\(\)](#), [vli_clear_n\(\)](#), [vli_cmp_n\(\)](#), [vli_isZero_n\(\)](#), [vli_rshift1_n\(\)](#), [vli_set_n\(\)](#), and [vli_sub_n\(\)](#).

Referenced by [uECC_sign_with_k\(\)](#), and [uECC_verify\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.52 vli_modMult_fast()

```

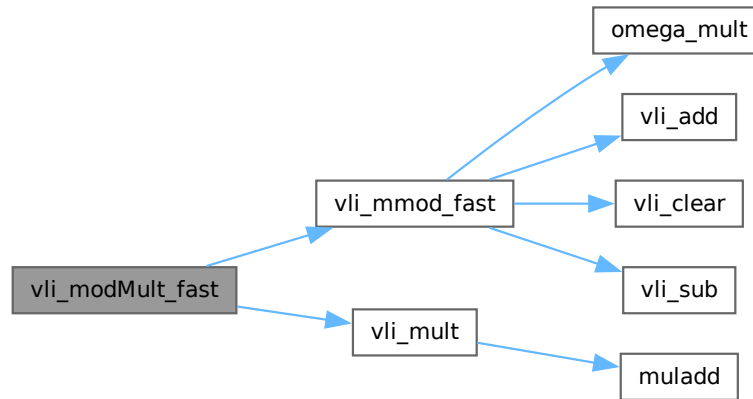
void vli_modMult_fast (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
  
```

Definition at line [1832](#) of file [uECC.c](#).

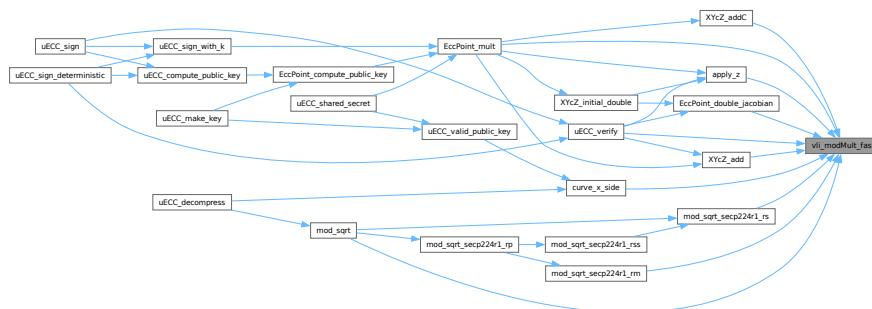
References [uECC_WORDS](#), [vli_mmod_fast\(\)](#), and [vli_mult\(\)](#).

Referenced by [apply_z\(\)](#), [curve_x_side\(\)](#), [EccPoint_double_jacobian\(\)](#), [EccPoint_mult\(\)](#), [mod_sqrt\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rs\(\)](#), [uECC_verify\(\)](#), [XYZ_add\(\)](#), and [XYZ_addC\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.53 vli_modMult_n()

```

void vli_modMult_n (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right) [static]

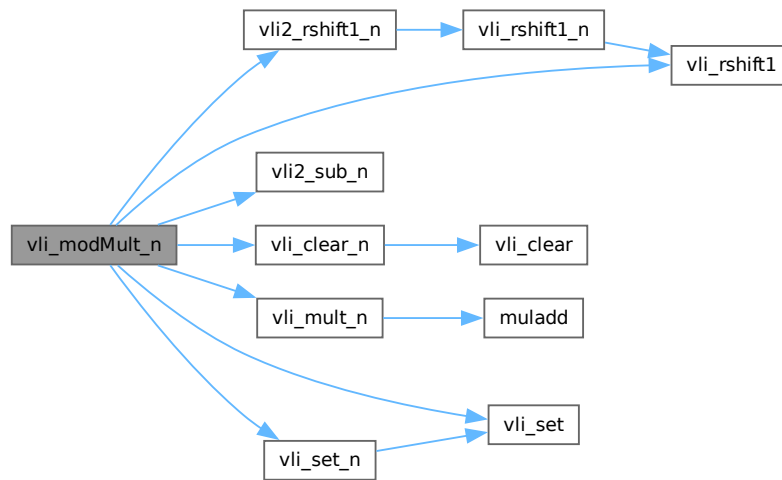
```

Definition at line 2785 of file [uECC.c](#).

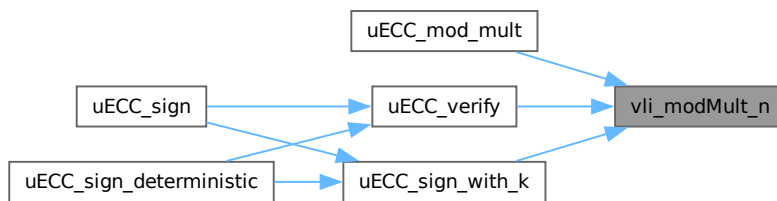
References [curve_n](#), [uECC_N_WORDS](#), [vli2_rshift1_n\(\)](#), [vli2_sub_n\(\)](#), [vli_clear_n\(\)](#), [vli_mult_n\(\)](#), [vli_rshift1\(\)](#), [vli_set\(\)](#), and [vli_set_n\(\)](#).

Referenced by [uECC_mod_mult\(\)](#), [uECC_sign_with_k\(\)](#), and [uECC_verify\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.54 vli_modSub()

```

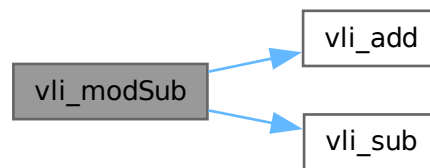
void vli_modSub (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right,
    const uECC_word_t * mod) [static]
  
```

Definition at line 918 of file `uECC.c`.

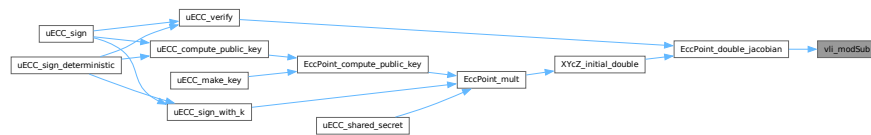
References `vli_add()`, and `vli_sub()`.

Referenced by `EccPoint_double_jacobian()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.55 vli_mult()

```

void vli_mult (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right) [static]
  
```

Definition at line 775 of file `uECC.c`.

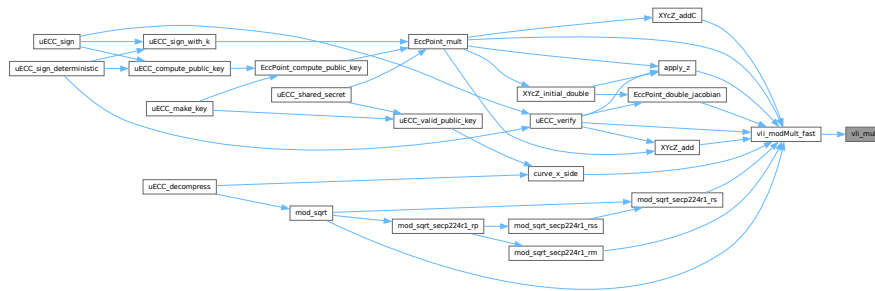
References `muladd()`, and `uECC_WORDS`.

Referenced by `vli_modMult_fast()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.56 vli_mult_n()

```

void vli_mult_n (
    uECC_word_t * result,
    const uECC_word_t * left,
    const uECC_word_t * right) [static]

```

Definition at line 2636 of file [uECC.c](#).

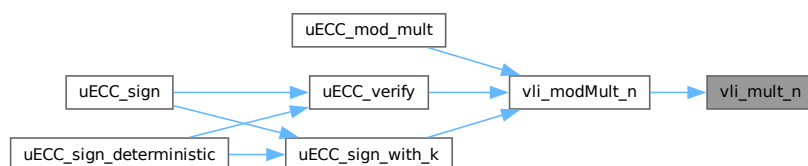
References [muladd\(\)](#), and [uECC_N_WORDS](#).

Referenced by [vli_modMult_n\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.57 vli_nativeToBytes()

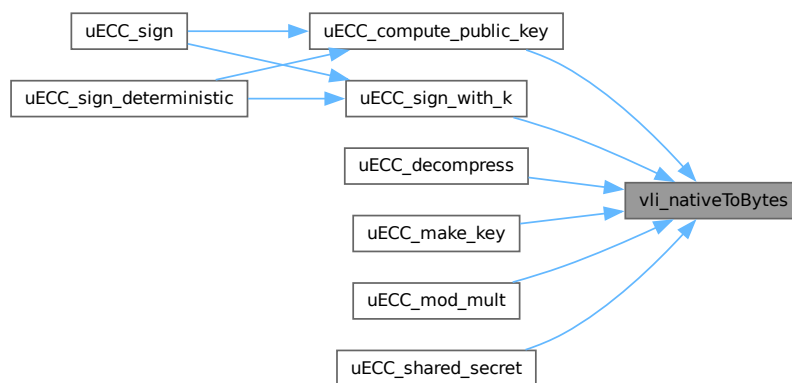
```
void vli_nativeToBytes (
    uint8_t * bytes,
    const uint64_t * native) [static]
```

Definition at line 2392 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [uECC_compute_public_key\(\)](#), [uECC_decompress\(\)](#), [uECC_make_key\(\)](#), [uECC_mod_mult\(\)](#), [uECC_shared_secret\(\)](#), and [uECC_sign_with_k\(\)](#).

Here is the caller graph for this function:



5.41.4.58 vli_numBits()

```
bitcount_t vli_numBits (
    const uECC_word_t * vli,
    wordcount_t max_words) [static]
```

Definition at line 610 of file [uECC.c](#).

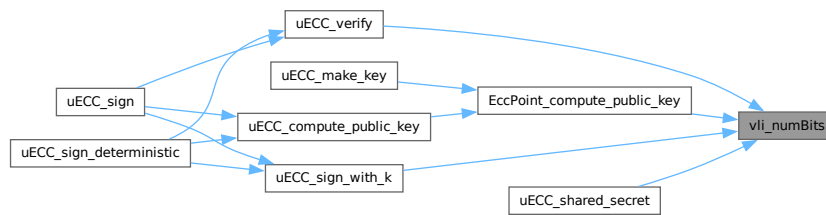
References [vli_numDigits\(\)](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign_with_k\(\)](#), and [uECC_verify\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



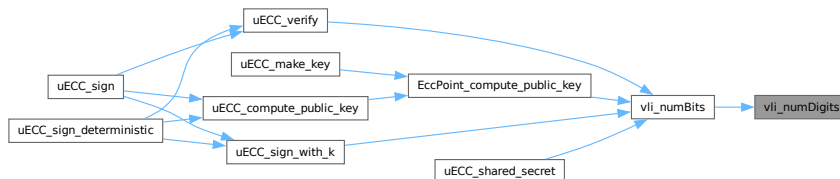
5.41.4.59 vli_numDigits()

```
wordcount_t vli_numDigits (
    const uECC_word_t * vli,
    wordcount_t max_words) [static]
```

Definition at line 596 of file [uECC.c](#).

Referenced by [vli_numBits\(\)](#).

Here is the caller graph for this function:



5.41.4.60 vli_rshift1()

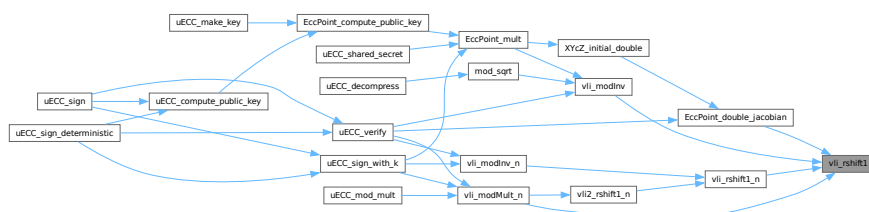
```
void vli_rshift1 (
    uECC_word_t * vli) [static]
```

Definition at line 676 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [EccPoint_double_jacobian\(\)](#), [vli_modInv\(\)](#), [vli_modMult_n\(\)](#), and [vli_rshift1_n\(\)](#).

Here is the caller graph for this function:



5.41.4.61 vli_rshift1_n()

```
void vli_rshift1_n (
    uECC_word_t * vli) [static]
```

Definition at line 2588 of file [uECC.c](#).

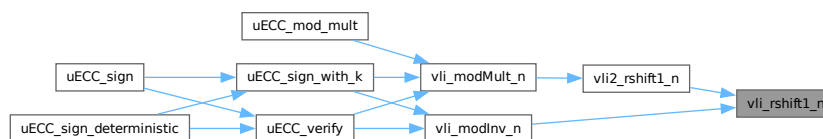
References [uECC_N_WORDS](#), and [vli_rshift1\(\)](#).

Referenced by [vli2_rshift1_n\(\)](#), and [vli_modInv_n\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.62 vli_set()

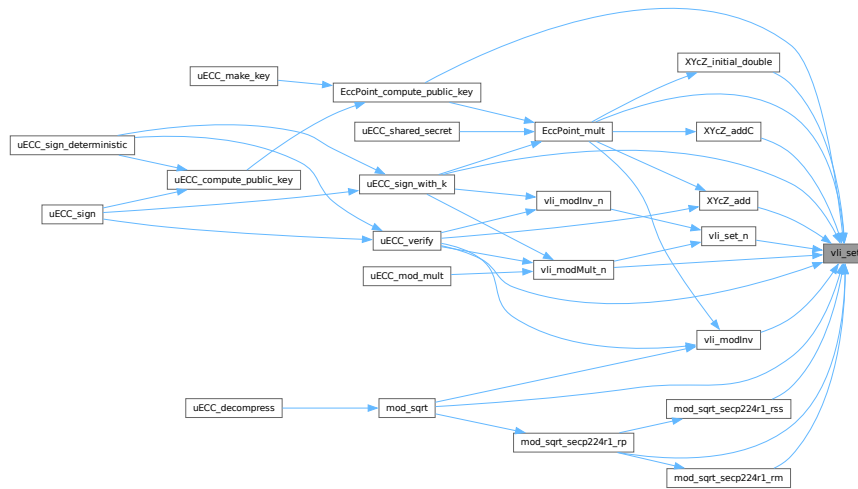
```
void vli_set (
    uECC_word_t * dest,
    const uECC_word_t * src) [static]
```

Definition at line 633 of file [uECC.c](#).

References [uECC_WORDS](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [EccPoint_mult\(\)](#), [mod_sqrt\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rp\(\)](#), [mod_sqrt_secp224r1_rss\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), [vli_modInv\(\)](#), [vli_modMult_n\(\)](#), [vli_set_n\(\)](#), [XYcZ_add\(\)](#), [XYcZ_addC\(\)](#), and [XYcZ_initial_double\(\)](#).

Here is the caller graph for this function:



5.41.4.63 vli_set_n()

```
void vli_set_n (
    uECC_word_t * dest,
    const uECC_word_t * src) [static]
```

Definition at line 2569 of file [uECC.c](#).

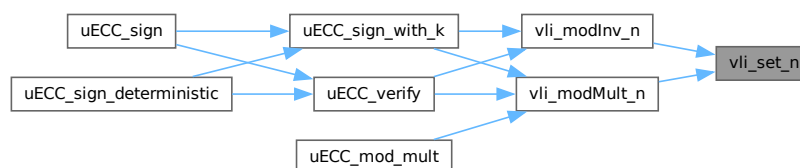
References [uECC_N_WORDS](#), and [vli_set\(\)](#).

Referenced by [vli_modInv_n\(\)](#), and [vli_modMult_n\(\)](#).

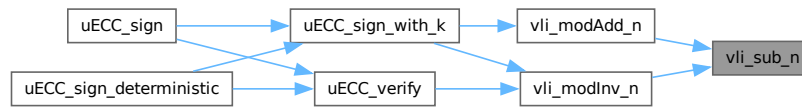
Here is the call graph for this function:



Here is the caller graph for this function:



Here is the caller graph for this function:



5.41.4.66 vli_testBit()

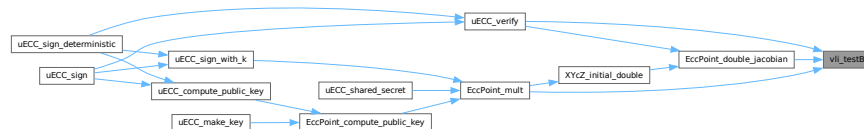
```

uECC_word_t vli_testBit (
    const uECC_word_t * vli,
    bitcount_t bit) [static]
  
```

Definition at line 587 of file [uECC.c](#).

Referenced by [EccPoint_double_jacobian\(\)](#), [EccPoint_mult\(\)](#), and [uECC_verify\(\)](#).

Here is the caller graph for this function:



5.41.4.67 XYcZ_add()

```

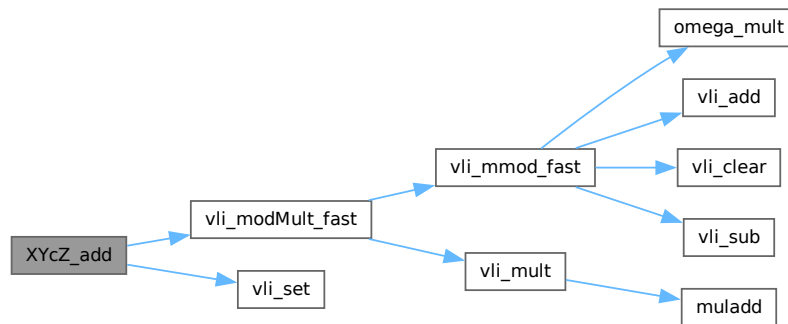
void XYcZ_add (
    uECC_word_t *RESTRICT X1,
    uECC_word_t *RESTRICT Y1,
    uECC_word_t *RESTRICT X2,
    uECC_word_t *RESTRICT Y2) [static]
  
```

Definition at line 2101 of file [uECC.c](#).

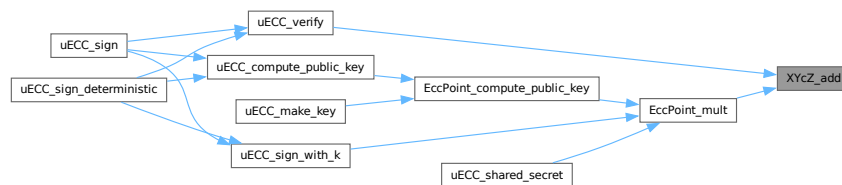
References [RESTRICT](#), [uECC_WORDS](#), [vli_modMult_fast\(\)](#), [vli_modSquare_fast\(\)](#), [vli_modSub_fast\(\)](#), and [vli_set\(\)](#).

Referenced by [EccPoint_mult\(\)](#), and [uECC_verify\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.68 XYcZ_addC()

```

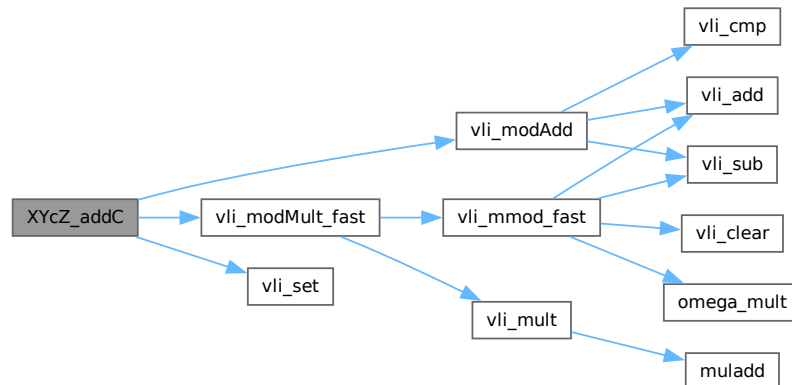
void XYcZ_addC (
    uECC_word_t *RESTRICT X1,
    uECC_word_t *RESTRICT Y1,
    uECC_word_t *RESTRICT X2,
    uECC_word_t *RESTRICT Y2) [static]
  
```

Definition at line 2129 of file [uECC.c](#).

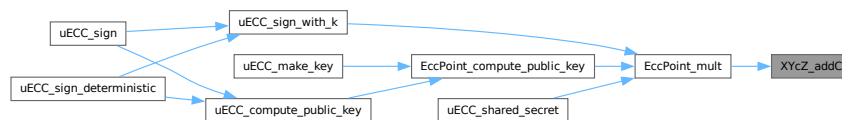
References [curve_p](#), [RESTRICT](#), [uECC_WORDS](#), [vli_modAdd\(\)](#), [vli_modMult_fast\(\)](#), [vli_modSquare_fast](#), [vli_modSub_fast](#), and [vli_set\(\)](#).

Referenced by [EccPoint_mult\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.4.69 XYcZ_initial_double()

```

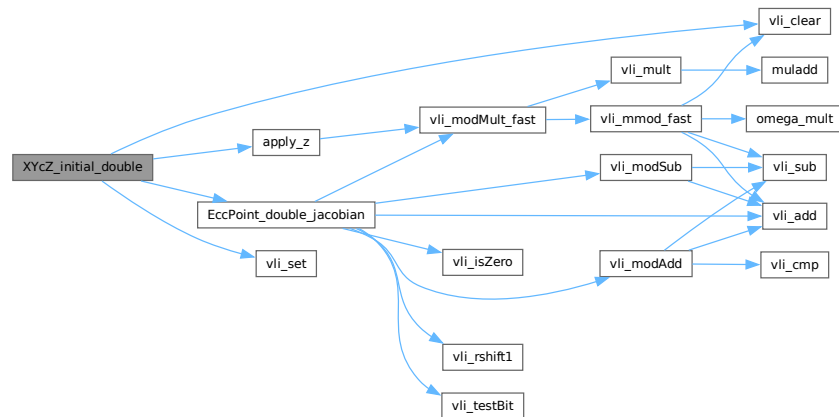
void XYcZ_initial_double (
    uECC_word_t *RESTRICT X1,
    uECC_word_t *RESTRICT Y1,
    uECC_word_t *RESTRICT X2,
    uECC_word_t *RESTRICT Y2,
    const uECC_word_t *RESTRICT initial_Z) [static]
  
```

Definition at line 2072 of file [uECC.c](#).

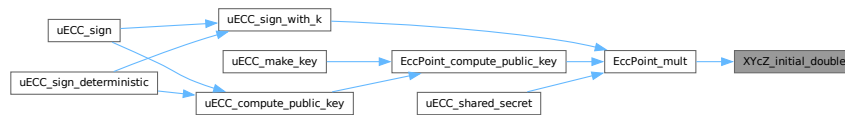
References [apply_z\(\)](#), [EccPoint_double_jacobian\(\)](#), [RESTRICT](#), [uECC_WORDS](#), [vli_clear\(\)](#), and [vli_set\(\)](#).

Referenced by [EccPoint_mult\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.41.5 Variable Documentation

5.41.5.1 curve_b

```
const uECC_word_t curve_b[uECC_WORDS] [static]
```

Initial value:

```
=
uECC_CONCAT (Curve_B_, uECC_CURVE)
```

Definition at line 407 of file [uECC.c](#).

Referenced by [curve_x_side\(\)](#).

5.41.5.2 curve_G

```
const EccPoint curve_G = uECC_CONCAT (Curve_G_, uECC_CURVE) [static]
```

Definition at line 409 of file [uECC.c](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_sign_with_k\(\)](#), and [uECC_verify\(\)](#).

5.41.5.3 curve_n

```
const uECC_word_t curve_n[uECC_N_WORDS] [static]
```

Initial value:

```
=
uECC_CONCAT (Curve_N_, uECC_CURVE)
```

Definition at line 410 of file [uECC.c](#).

Referenced by [EccPoint_compute_public_key\(\)](#), [uECC_sign_with_k\(\)](#), [uECC_verify\(\)](#), [vli_blindScalar\(\)](#), and [vli_modMult_n\(\)](#).

5.41.5.4 curve_p

```
const uECC_word_t curve_p[uECC_WORDS] [static]
```

Initial value:

```
=
uECC_CONCAT (Curve_P_, uECC_CURVE)
```

Definition at line 405 of file [uECC.c](#).

Referenced by [curve_x_side\(\)](#), [EccPoint_double_jacobian\(\)](#), [EccPoint_mult\(\)](#), [mod_sqrt\(\)](#), [mod_sqrt_secp224r1_rm\(\)](#), [mod_sqrt_secp224r1_rp\(\)](#), [mod_sqrt_secp224r1_rs\(\)](#), [uECC_decompress\(\)](#), [uECC_valid_public_key\(\)](#), [uECC_verify\(\)](#), [vli_mmod_fast\(\)](#), and [XYZC_addC\(\)](#).

5.41.5.5 g_rng_function

```
uECC_RNG_Function g_rng_function = &default_RNG [static]
```

Definition at line 539 of file [uECC.c](#).

Referenced by [__attribute__\(\)](#), [EccPoint_compute_public_key\(\)](#), [uECC_make_key\(\)](#), [uECC_shared_secret\(\)](#), [uECC_sign\(\)](#), [uECC_sign_with_k\(\)](#), and [vli_blindScalar\(\)](#).

5.42 uECC.c

[Go to the documentation of this file.](#)

```
00001 /* Copyright 2014, Kenneth MacKay. Licensed under the BSD 2-clause license. */
00002
00011
00012 #include "uECC.h"
00013
00014 #ifndef uECC_PLATFORM
00015 #if __AVR__
00016 #define uECC_PLATFORM uECC_avr
00017 #elif defined(__thumb2__) ||
00018     defined(_M_ARM) /* I think MSVC only supports Thumb-2 targets */
00019 #define uECC_PLATFORM uECC_arm_thumb2
00020 #elif defined(__thumb__)
00021 #define uECC_PLATFORM uECC_arm_thumb
00022 #elif defined(__arm__) || defined(_M_ARM)
00023 #define uECC_PLATFORM uECC_arm
00024 #elif defined(__i386__) || defined(_M_IX86) || defined(_X86_) ||
00025     defined(_I86_)
00026 #define uECC_PLATFORM uECC_x86
00027 #elif defined(__amd64__) || defined(_M_X64)
00028 #define uECC_PLATFORM uECC_x86_64
```

```

00029 #else
00030 #define uECC_PLATFORM uECC_arch_other
00031 #endif
00032 #endif
00033
00034 #ifndef uECC_WORD_SIZE
00035 #if uECC_PLATFORM == uECC_avr
00036 #define uECC_WORD_SIZE 1
00037 #elif (uECC_PLATFORM == uECC_x86_64)
00038 #define uECC_WORD_SIZE 8
00039 #else
00040 #define uECC_WORD_SIZE 4
00041 #endif
00042 #endif
00043
00044 #if (uECC_CURVE == uECC_secp160r1 || uECC_CURVE == uECC_secp224r1) && \
00045     (uECC_WORD_SIZE == 8)
00046 #undef uECC_WORD_SIZE
00047 #define uECC_WORD_SIZE 4
00048 #if (uECC_PLATFORM == uECC_x86_64)
00049 #undef uECC_PLATFORM
00050 #define uECC_PLATFORM uECC_x86
00051 #endif
00052 #endif
00053
00054 #if (uECC_WORD_SIZE != 1) && (uECC_WORD_SIZE != 4) && (uECC_WORD_SIZE != 8)
00055 #error "Unsupported value for uECC_WORD_SIZE"
00056 #endif
00057
00058 #if (uECC_ASM && (uECC_PLATFORM == uECC_avr) && (uECC_WORD_SIZE != 1))
00059 #pragma message("uECC_WORD_SIZE must be 1 when using AVR asm")
00060 #undef uECC_WORD_SIZE
00061 #define uECC_WORD_SIZE 1
00062 #endif
00063
00064 #if (uECC_ASM && \
00065     (uECC_PLATFORM == uECC_arm || uECC_PLATFORM == uECC_arm_thumb) && \
00066     (uECC_WORD_SIZE != 4))
00067 #pragma message("uECC_WORD_SIZE must be 4 when using ARM asm")
00068 #undef uECC_WORD_SIZE
00069 #define uECC_WORD_SIZE 4
00070 #endif
00071
00072 #if __STDC_VERSION__ >= 199901L
00073 #define RESTRICT restrict
00074 #else
00075 #define RESTRICT
00076 #endif
00077
00078 #if defined(__SIZEOF_INT128__) || \
00079     ((__clang_major__ * 100 + __clang_minor__) >= 302)
00080 #define SUPPORTS_INT128 1
00081 #else
00082 #define SUPPORTS_INT128 0
00083 #endif
00084
00085 #define MAX_TRIES 64
00086
00087 #if (uECC_WORD_SIZE == 1)
00088
00089 typedef uint8_t uECC_word_t;
00090 typedef uint16_t uECC_dword_t;
00091 typedef uint8_t wordcount_t;
00092 typedef int8_t swordcount_t;
00093 typedef int16_t bitcount_t;
00094 typedef int8_t cmpresult_t;
00095
00096 #define HIGH_BIT_SET 0x80
00097 #define uECC_WORD_BITS 8
00098 #define uECC_WORD_BITS_SHIFT 3
00099 #define uECC_WORD_BITS_MASK 0x07
00100
00101 #define uECC_WORDS_1 20
00102 #define uECC_WORDS_2 24
00103 #define uECC_WORDS_3 32
00104 #define uECC_WORDS_4 32
00105 #define uECC_WORDS_5 28
00106
00107 #define uECC_N_WORDS_1 21
00108 #define uECC_N_WORDS_2 24
00109 #define uECC_N_WORDS_3 32
00110 #define uECC_N_WORDS_4 32
00111 #define uECC_N_WORDS_5 28
00112
00113 #define Curve_P_1 \
00114     {0xFF, 0xFF, 0xFF, 0x7F, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, \
00115     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}

```

```

00116 #define Curve_P_2
00117     {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFE, 0xFF, 0xFF, 0xFF,
00118      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00119 #define Curve_P_3
00120     {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00121      0xFF, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00122      0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF}
00123 #define Curve_P_4
00124     {0x2F, 0xFC, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00125      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00126      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00127 #define Curve_P_5
00128     {0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00129      0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00130      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00131
00132 #define Curve_B_1
00133     {0x45, 0xFA, 0x65, 0xC5, 0xAD, 0xD4, 0xD4, 0x81, 0x9F, 0xF8,
00134      0xAC, 0x65, 0x8B, 0x7A, 0xBD, 0x54, 0xFC, 0xBE, 0x97, 0x1C}
00135 #define Curve_B_2
00136     {0xB1, 0xB9, 0x46, 0xC1, 0xEC, 0xDE, 0xB8, 0xFE, 0x49, 0x30, 0x24, 0x72,
00137      0xAB, 0xE9, 0xA7, 0x0F, 0xE7, 0x80, 0x9C, 0xE5, 0x19, 0x05, 0x21, 0x64}
00138 #define Curve_B_3
00139     {0x4B, 0x60, 0xD2, 0x27, 0x3E, 0x3C, 0xCE, 0x3B, 0xF6, 0xB0, 0x53,
00140      0xCC, 0xB0, 0x06, 0x1D, 0x65, 0xBC, 0x86, 0x98, 0x76, 0x55, 0xBD,
00141      0xEB, 0xB3, 0xE7, 0x93, 0x3A, 0xAA, 0xD8, 0x35, 0xC6, 0x5A}
00142 #define Curve_B_4
00143     {0x07, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00144      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
00145      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
00146 #define Curve_B_5
00147     {0xB4, 0xFF, 0x55, 0x23, 0x43, 0x39, 0x0B, 0x27, 0xBA, 0xD8,
00148      0xBF, 0xD7, 0xB7, 0xB0, 0x44, 0x50, 0x56, 0x32, 0x41, 0xF5,
00149      0xAB, 0xB3, 0x04, 0x0C, 0x85, 0x0A, 0x05, 0xB4}
00150
00151 #define Curve_G_1
00152     {{0x82, 0xFC, 0xCB, 0x13, 0xB9, 0x8B, 0xC3, 0x68, 0x89, 0x69,
00153      0x64, 0x46, 0x28, 0x73, 0xF5, 0x8E, 0x68, 0xB5, 0x96, 0x4A},
00154      {0x32, 0xFB, 0xC5, 0x7A, 0x37, 0x51, 0x23, 0x04, 0x12, 0xC9,
00155      0xDC, 0x59, 0x7D, 0x94, 0x68, 0x31, 0x55, 0x28, 0xA6, 0x23}}
00156
00157 #define Curve_G_2
00158     {{0x12, 0x10, 0xFF, 0x82, 0xFD, 0x0A, 0xFF, 0xF4, 0x00, 0x88, 0xA1, 0x43,
00159      0xEB, 0x20, 0xBF, 0x7C, 0xF6, 0x90, 0x30, 0xB0, 0x0E, 0xA8, 0x8D, 0x18},
00160      {0x11, 0x48, 0x79, 0x1E, 0xA1, 0x77, 0xF9, 0x73, 0xD5, 0xCD, 0x24, 0x6B,
00161      0xED, 0x11, 0x10, 0x63, 0x78, 0xDA, 0xC8, 0xFF, 0x95, 0x2B, 0x19, 0x07}}
00162
00163 #define Curve_G_3
00164     {{0x96, 0xC2, 0x98, 0xD8, 0x45, 0x39, 0xA1, 0xF4, 0xA0, 0x33, 0xEB,
00165      0x2D, 0x81, 0x7D, 0x03, 0x77, 0xF2, 0x40, 0xA4, 0x63, 0xE5, 0xE6,
00166      0xBC, 0xF8, 0x47, 0x42, 0x2C, 0xE1, 0xF2, 0xD1, 0x17, 0x6B},
00167      {0xF5, 0x51, 0xBF, 0x37, 0x68, 0x40, 0xB6, 0xCB, 0xCE, 0x5E, 0x31,
00168      0x6B, 0x57, 0x33, 0xCE, 0x2B, 0x16, 0x9E, 0x0F, 0x7C, 0x4A, 0xEB,
00169      0xE7, 0x8E, 0x9B, 0x7F, 0x1A, 0xFE, 0xE2, 0x42, 0xE3, 0x4F}}
00170
00171 #define Curve_G_4
00172     {{0x98, 0x17, 0xF8, 0x16, 0x5B, 0x81, 0xF2, 0x59, 0xD9, 0x28, 0xCE,
00173      0x2D, 0xDB, 0xFC, 0x9B, 0x02, 0x07, 0x0B, 0x87, 0xCE, 0x95, 0x62,
00174      0xA0, 0x55, 0xAC, 0xBB, 0xDC, 0xF9, 0x7E, 0x66, 0xBE, 0x79},
00175      {0xB8, 0xD4, 0x10, 0xFB, 0x8F, 0xD0, 0x47, 0x9C, 0x19, 0x54, 0x85,
00176      0xA6, 0x48, 0xB4, 0x17, 0xFD, 0xA8, 0x08, 0x11, 0x0E, 0xFC, 0xFB,
00177      0xA4, 0x5D, 0x65, 0xC4, 0xA3, 0x26, 0x77, 0xDA, 0x3A, 0x48}}
00178
00179 #define Curve_G_5
00180     {{0x21, 0x1D, 0x5C, 0x11, 0xD6, 0x80, 0x32, 0x34, 0x22, 0x11,
00181      0xC2, 0x56, 0xD3, 0xC1, 0x03, 0x4A, 0xB9, 0x90, 0x13, 0x32,
00182      0x7F, 0xBF, 0xB4, 0x6B, 0xBD, 0x0C, 0x0E, 0xB7},
00183      {0x34, 0x7E, 0x00, 0x85, 0x99, 0x81, 0xD5, 0x44, 0x64, 0x47,
00184      0x07, 0x5A, 0xA0, 0x75, 0x43, 0xCD, 0xE6, 0xDF, 0x22, 0x4C,
00185      0xFB, 0x23, 0xF7, 0xB5, 0x88, 0x63, 0x37, 0xBD}}
00186
00187 #define Curve_N_1
00188     {0x57, 0x22, 0x75, 0xCA, 0xD3, 0xAE, 0x27, 0xF9, 0xC8, 0xF4, 0x01,
00189      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01}
00190 #define Curve_N_2
00191     {0x31, 0x28, 0xD2, 0xB4, 0xB1, 0xC9, 0x6B, 0x14, 0x36, 0xF8, 0xDE, 0x99,
00192      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00193 #define Curve_N_3
00194     {0x51, 0x25, 0x63, 0xFC, 0xC2, 0xCA, 0xB9, 0xF3, 0x84, 0x9E, 0x17,
00195      0xA7, 0xAD, 0xFA, 0xE6, 0xBC, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00196      0xFF, 0xFF, 0x00, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF}
00197 #define Curve_N_4
00198     {0x41, 0x41, 0x36, 0xD0, 0x8C, 0x5E, 0xD2, 0xBF, 0x3B, 0xA0, 0x48,
00199      0xAF, 0xE6, 0xDC, 0xAE, 0xBA, 0xFE, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00200      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00201 #define Curve_N_5
00202     {0x3D, 0x2A, 0x5C, 0x5C, 0x45, 0x29, 0xDD, 0x13, 0x3E, 0xF0,

```

```

00203      0xB8, 0xE0, 0xA2, 0x16, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
00204      0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
00205
00206 #elif (uECC_WORD_SIZE == 4)
00207
00208 typedef uint32_t uECC_word_t;
00209 typedef uint64_t uECC_dword_t;
00210 typedef unsigned wordcount_t;
00211 typedef int swordcount_t;
00212 typedef int bitcount_t;
00213 typedef int cmpresult_t;
00214
00215 #define HIGH_BIT_SET 0x80000000
00216 #define uECC_WORD_BITS 32
00217 #define uECC_WORD_BITS_SHIFT 5
00218 #define uECC_WORD_BITS_MASK 0x01F
00219
00220 #define uECC_WORDS_1 5
00221 #define uECC_WORDS_2 6
00222 #define uECC_WORDS_3 8
00223 #define uECC_WORDS_4 8
00224 #define uECC_WORDS_5 7
00225
00226 #define uECC_N_WORDS_1 6
00227 #define uECC_N_WORDS_2 6
00228 #define uECC_N_WORDS_3 8
00229 #define uECC_N_WORDS_4 8
00230 #define uECC_N_WORDS_5 7
00231
00232 #define Curve_P_1 {0x7FFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00233 #define Curve_P_2
00234 {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00235 #define Curve_P_3
00236 {0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0x00000000,
00237 0x00000000, 0x00000000, 0x00000001, 0xFFFFFFFF}
00238 #define Curve_P_4
00239 {0xFFFFFC2F, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF,
00240 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00241 #define Curve_P_5
00242 {0x00000001, 0x00000000, 0x00000000, 0xFFFFFFFF,
00243 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00244
00245 #define Curve_B_1 {0xC565FA45, 0x81D4D4AD, 0x65ACF89F, 0x54BD7A8B, 0x1C97BEFC}
00246 #define Curve_B_2
00247 {0xC146B9B1, 0xFEB8DEEC, 0x72243049, 0x0FA7E9AB, 0xE59C80E7, 0x64210519}
00248 #define Curve_B_3
00249 {0x27D2604B, 0x3BCE3C3E, 0xCC53B0F6, 0x651D06B0,
00250 0x769886BC, 0xB3EBBD55, 0xAA3A93E7, 0x5AC635D8}
00251 #define Curve_B_4
00252 {0x00000007, 0x00000000, 0x00000000, 0x00000000,
00253 0x00000000, 0x00000000, 0x00000000, 0x00000000}
00254 #define Curve_B_5
00255 {0x2355FFB4, 0x270B3943, 0xD7BFD8BA, 0x5044B0B7,
00256 0xF5413256, 0x0C04B3AB, 0xB4050A85}
00257
00258 #define Curve_G_1
00259 {{0x13CBFC82, 0x68C38BB9, 0x46646989, 0x8EF57328, 0x4A96B568},
00260 {0x7AC5FB32, 0x04235137, 0x59DCC912, 0x3168947D, 0x23A62855}}
00261
00262 #define Curve_G_2
00263 {{0x82FF1012, 0xF4FF0AFD, 0x43A18800, 0x7CBF20EB, 0xB03090F6, 0x188DA80E},
00264 {0x1E794811, 0x73F977A1, 0x6B24CDD5, 0x631011ED, 0xFFC8DA78, 0x07192B95}}
00265
00266 #define Curve_G_3
00267 {{0xD898C296, 0xF4A13945, 0x2DEB33A0, 0x77037D81, 0x63A440F2, 0xF8BCE6E5,
00268 0xE12C4247, 0x6B17D1F2},
00269 {0x37BF51F5, 0xCBB64068, 0x6B315ECE, 0x2BCE3357, 0x7C0F9E16, 0x8EE7EB4A,
00270 0xFE1A7F9B, 0x4FE342E2}}
00271
00272 #define Curve_G_4
00273 {{0x16F81798, 0x59F2815B, 0x2DCE28D9, 0x029BFCDB, 0xCE870B07, 0x55A06295,
00274 0xF9DCBBAC, 0x79BE667E},
00275 {0xFB10D4B8, 0x9C47D08F, 0xA6855419, 0xFD17B448, 0x0E1108A8, 0x5DA4FBFC,
00276 0x26A3C465, 0x483ADA77}}
00277
00278 #define Curve_G_5
00279 {{0x115C1D21, 0x343280D6, 0x56C21122, 0x4A03C1D3, 0x321390B9, 0x6BB4BF7F,
00280 0xB70E0CBD},
00281 {0x85007E34, 0x44D58199, 0x5A074764, 0xCD4375A0, 0x4C22DFE6, 0xB5F723FB,
00282 0xBD376388}}
00283
00284 #define Curve_N_1
00285 {0xCA752257, 0xF927AED3, 0x0001F4C8, 0x00000000, 0x00000000, 0x00000001}
00286 #define Curve_N_2
00287 {0xB4D22831, 0x146BC9B1, 0x99DEF836, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00288 #define Curve_N_3
00289 {0xFC632551, 0xF3B9CAC2, 0xA7179E84, 0xBCE6FAAD,

```



```

00290     0xFFFFFFFF, 0xFFFFFFFF, 0x00000000, 0xFFFFFFFF}
00291 #define Curve_N_4
00292     {0xD0364141, 0xBF25E8C, 0xAF48A03B, 0xBAAEDCE6,
00293     0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00294 #define Curve_N_5
00295     {0x5C5C2A3D, 0x13DD2945, 0xE0B8F03E, 0xFFFF16A2,
00296     0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF}
00297
00298 #elif (uECC_WORD_SIZE == 8)
00299
00300 typedef uint64_t uECC_word_t;
00301 #if SUPPORTS_INT128
00302 typedef unsigned __int128 uECC_dword_t;
00303 #endif
00304 typedef unsigned wordcount_t;
00305 typedef int swordcount_t;
00306 typedef int bitcount_t;
00307 typedef int cmpresult_t;
00308
00309 #define HIGH_BIT_SET 0x8000000000000000ull
00310 #define uECC_WORD_BITS 64
00311 #define uECC_WORD_BITS_SHIFT 6
00312 #define uECC_WORD_BITS_MASK 0x03F
00313
00314 #define uECC_WORDS_1 3
00315 #define uECC_WORDS_2 3
00316 #define uECC_WORDS_3 4
00317 #define uECC_WORDS_4 4
00318 #define uECC_WORDS_5 4
00319
00320 #define uECC_N_WORDS_1 3
00321 #define uECC_N_WORDS_2 3
00322 #define uECC_N_WORDS_3 4
00323 #define uECC_N_WORDS_4 4
00324 #define uECC_N_WORDS_5 4
00325
00326 #define Curve_P_1
00327     {0xFFFFFFFFFFFFFFFFull, 0xFFFFFFFFFFFFFFFFull, 0x00000000FFFFFFFFull}
00328 #define Curve_P_2
00329     {0xFFFFFFFFFFFFFFFFull, 0xFFFFFFFFFFFFFFFFull, 0xFFFFFFFFFFFFFFFFull}
00330 #define Curve_P_3
00331     {0xFFFFFFFFFFFFFFFFull, 0x00000000FFFFFFFFull, 0x0000000000000000ull,
00332     0xFFFFFFFF00000001ull}
00333 #define Curve_P_4
00334     {0xFFFFFFFFFFFFFFFFC2ull, 0xFFFFFFFFFFFFFFFFull, 0xFFFFFFFFFFFFFFFFull,
00335     0xFFFFFFFFFFFFFFFFull}
00336 #define Curve_P_5
00337     {0x0000000000000000ull, 0xFFFFFFFF00000000ull, 0xFFFFFFFFFFFFFFFFull,
00338     0x00000000FFFFFFFFull}
00339
00340 #define Curve_B_1
00341     {0x81D4D4ADC565FA45ull, 0x54BD7A8B65ACF89Full, 0x000000001C97BEFCull}
00342 #define Curve_B_2
00343     {0xFEB8DEECC146B9B1ull, 0x0FA7E9AB72243049ull, 0x64210519E59C80E7ull}
00344 #define Curve_B_3
00345     {0x3BCE3C3E27D2604Bull, 0x651D06B0CC53B0F6ull, 0xB3EBBD55769886BCull,
00346     0x5AC635D8AA3A93E7ull}
00347 #define Curve_B_4
00348     {0x0000000000000007ull, 0x0000000000000000ull, 0x0000000000000000ull,
00349     0x0000000000000000ull}
00350 #define Curve_B_5
00351     {0x270B39432355FFB4ull, 0x5044B0B7D7BFD8BAull, 0x0C04B3ABF5413256ull,
00352     0x00000000B4050A85ull}
00353
00354 #define Curve_G_1
00355     {{0x68C38BB913CBFC82ull, 0x8EF5732846646989ull, 0x000000004A96B568ull},
00356     {0x042351377AC5FB32ull, 0x3168947D59DCC912ull, 0x00000000023A6285ull}}
00357
00358 #define Curve_G_2
00359     {{0xF4FF0AFD82FF1012ull, 0x7CBF20EB43A18800ull, 0x188DA80EB03090F6ull},
00360     {0x73F977A11E794811ull, 0x631011ED6B24CDD5ull, 0x07192B95FFC8DA78ull}}
00361
00362 #define Curve_G_3
00363     {{0xF4A13945D898C296ull, 0x77037D812DEB33A0ull, 0xF8BCE6E563A440F2ull,
00364     0x6B17D1F2E12C4247ull},
00365     {0xCBB6406837BF51F5ull, 0x2BCE33576B315ECEull, 0x8EE7EB4A7C0F9E16ull,
00366     0x4FE342E2FE1A7F9Bull}}
00367
00368 #define Curve_G_4
00369     {{0x59F2815B16F81798ull, 0x029BFCDB2DCE28D9ull, 0x55A06295CE870B07ull,
00370     0x79BE667EF9DCBBACull},
00371     {0x9C47D08FFB10D4B8ull, 0xFD17B448A6855419ull, 0x5DA4FBFC0E1108A8ull,
00372     0x483ADA7726A3C465ull}}
00373
00374 #define Curve_G_5
00375     {{0x343280D6115C1D21ull, 0x4A03C1D356C21122ull, 0x6BB4BF7F321390B9ull,
00376     0x00000000B70E0CBFull},

```

```

00377     {0x44D5819985007E34u1l, 0xCD4375A05A074764u1l, 0xB5F723FB4C22DFE6u1l, \
00378     0x00000000BD376388u1l}}
00379
00380 #define Curve_N_1 \
00381     {0xF927AED3CA752257u1l, 0x000000000001F4C8u1l, 0x0000000100000000u1l}
00382 #define Curve_N_2 \
00383     {0x146BC9B1B4D22831u1l, 0xFFFFFFFF99DEF836u1l, 0xFFFFFFFFFFFFFFFFu1l}
00384 #define Curve_N_3 \
00385     {0xF3B9CAC2FC632551u1l, 0xBCE6FAADA7179E84u1l, 0xFFFFFFFFFFFFFFFFu1l, \
00386     0xFFFFFFFFF0000000u1l}
00387 #define Curve_N_4 \
00388     {0xBFD25E8CD0364141u1l, 0xBAAEDCE6AF48A03Bu1l, 0xFFFFFFFFFFFFFFFFu1l, \
00389     0xFFFFFFFFFFFFFFFFu1l}
00390 #define Curve_N_5 \
00391     {0x13DD29455C5C2A3Du1l, 0xFFFF16A2E0B8F03Eu1l, 0xFFFFFFFFFFFFFFFFu1l, \
00392     0x00000000FFFFFFFFu1l}
00393
00394 #endif /* (uECC_WORD_SIZE == 8) */
00395
00396 #define uECC_WORDS uECC_CONCAT(uECC_WORDS_, uECC_CURVE)
00397 #define uECC_N_WORDS uECC_CONCAT(uECC_N_WORDS_, uECC_CURVE)
00398
00399 typedef struct EccPoint
00400 {
00401     uECC_word_t x[uECC_WORDS];
00402     uECC_word_t y[uECC_WORDS];
00403 } EccPoint;
00404
00405 static const uECC_word_t curve_p[uECC_WORDS] =
00406     uECC_CONCAT(Curve_P_, uECC_CURVE);
00407 static const uECC_word_t curve_b[uECC_WORDS] =
00408     uECC_CONCAT(Curve_B_, uECC_CURVE);
00409 static const EccPoint curve_G = uECC_CONCAT(Curve_G_, uECC_CURVE);
00410 static const uECC_word_t curve_n[uECC_N_WORDS] =
00411     uECC_CONCAT(Curve_N_, uECC_CURVE);
00412
00413 static void vli_clear(uECC_word_t *vli);
00414 static uECC_word_t vli_isZero(const uECC_word_t *vli);
00415 static uECC_word_t vli_testBit(const uECC_word_t *vli, bitcount_t bit);
00416 static bitcount_t vli_numBits(const uECC_word_t *vli, wordcount_t max_words);
00417 static void vli_set(uECC_word_t *dest, const uECC_word_t *src);
00418 static cmpresult_t vli_cmp(const uECC_word_t *left, const uECC_word_t *right);
00419 static cmpresult_t vli_equal(const uECC_word_t *left, const uECC_word_t *right);
00420 static void vli_rshift1(uECC_word_t *vli);
00421 static uECC_word_t vli_add(uECC_word_t *result, const uECC_word_t *left,
00422     const uECC_word_t *right);
00423 static uECC_word_t vli_sub(uECC_word_t *result, const uECC_word_t *left,
00424     const uECC_word_t *right);
00425 static void vli_mult(uECC_word_t *result, const uECC_word_t *left,
00426     const uECC_word_t *right);
00427 static void vli_modAdd(uECC_word_t *result, const uECC_word_t *left,
00428     const uECC_word_t *right, const uECC_word_t *mod);
00429 static void vli_modInv(uECC_word_t *result, const uECC_word_t *input,
00430     const uECC_word_t *mod);
00431 #if uECC_SQUARE_FUNC
00432 static void vli_square(uECC_word_t *result, const uECC_word_t *left);
00433 static void vli_modSquare_fast(uECC_word_t *result, const uECC_word_t *left);
00434 #endif
00435
00436 #if (uECC_CURVE == uECC_secp160r1)
00437 static void vli_clear_n(uECC_word_t *vli);
00438 static void vli_set_n(uECC_word_t *dest, const uECC_word_t *src);
00439 static cmpresult_t vli_cmp_n(const uECC_word_t *left, const uECC_word_t *right);
00440 static void vli_modMult_n(uECC_word_t *result, const uECC_word_t *left,
00441     const uECC_word_t *right);
00442 static void vli_modAdd_n(uECC_word_t *result, const uECC_word_t *left,
00443     const uECC_word_t *right, const uECC_word_t *mod);
00444 static void vli_modInv_n(uECC_word_t *result, const uECC_word_t *input,
00445     const uECC_word_t *mod);
00446 static void vli_rshift1_n(uECC_word_t *vli);
00447 static uECC_word_t vli_add_n(uECC_word_t *result, const uECC_word_t *left,
00448     const uECC_word_t *right);
00449 static uECC_word_t vli_sub_n(uECC_word_t *result, const uECC_word_t *left,
00450     const uECC_word_t *right);
00451 #else
00452 #define vli_clear_n vli_clear
00453 #define vli_set_n vli_set
00454 #define vli_cmp_n vli_cmp
00455 #define vli_modMult_n vli_modMult_generic
00456 #define vli_modAdd_n vli_modAdd
00457 #define vli_modInv_n vli_modInv
00458 #define vli_rshift1_n vli_rshift1
00459 #define vli_add_n vli_add
00460 #define vli_sub_n vli_sub
00461 #endif
00462
00463 static void vli_blindScalar(uECC_word_t *result, const uECC_word_t *scalar);

```

```

00464 static int EccPoint_compute_public_key(EccPoint *result, uECC_word_t *private);
00465
00466 #if (defined(_WIN32) || defined(_WIN64))
00467 /* Windows */
00468
00469 #define WIN32_LEAN_AND_MEAN
00470 #include <windows.h>
00471 #include <wincrypt.h>
00472
00473 static int default_RNG(uint8_t *dest, unsigned size)
00474 {
00475     HCRYPTPROV prov;
00476     if (!CryptAcquireContext(&prov, NULL, NULL, PROV_RSA_FULL,
00477                             CRYPT_VERIFYCONTEXT))
00478     {
00479         return 0;
00480     }
00481     CryptGenRandom(prov, size, (BYTE *) dest);
00482     CryptReleaseContext(prov, 0);
00483     return 1;
00484 }
00485 #endif
00486
00487 #elif defined(unix) || defined(__linux__) || defined(__unix__) || \
00488         defined(__unix) || (defined(__APPLE__) && defined(__MACH__)) || \
00489         defined(uECC_POSIX)
00490
00491 /* Some POSIX-like system with /dev/urandom or /dev/random. */
00492 #include <fcntl.h>
00493 #include <sys/types.h>
00494 #include <unistd.h>
00495
00496 #ifndef O_CLOEXEC
00497 #define O_CLOEXEC 0
00498 #endif
00499
00500 static int default_RNG(uint8_t *dest, unsigned size)
00501 {
00502     int fd = open("/dev/urandom", O_RDONLY | O_CLOEXEC);
00503     if (fd == -1)
00504     {
00505         fd = open("/dev/random", O_RDONLY | O_CLOEXEC);
00506         if (fd == -1)
00507         {
00508             return 0;
00509         }
00510     }
00511
00512     char *ptr = (char *) dest;
00513     size_t left = size;
00514     while (left > 0)
00515     {
00516         ssize_t bytes_read = read(fd, ptr, left);
00517         if (bytes_read <= 0)
00518         { // read failed
00519             close(fd);
00520             return 0;
00521         }
00522         left -= bytes_read;
00523         ptr += bytes_read;
00524     }
00525     close(fd);
00526     return 1;
00527 }
00528 #endif
00529
00530 #else /* Some other platform */
00531
00532 static int default_RNG(uint8_t *dest, unsigned size)
00533 {
00534     return 0;
00535 }
00536 #endif
00537
00538 static uECC_RNG_Function g_rng_function = &default_RNG;
00539
00540 __attribute__((used)) void uECC_set_rng(uECC_RNG_Function rng_function)
00541 {
00542     g_rng_function = rng_function;
00543 }
00544
00545 #ifdef __GNUC__ /* Only support GCC inline asm for now */
00546 #if (uECC_ASM && (uECC_PLATFORM == uECC_avr))
00547 #include "asm_avr.inc"
00548 #endif
00549 #endif
00550

```

```

00551 #if (uECC_ASM &&
00552      (uECC_PLATFORM == uECC_arm || uECC_PLATFORM == uECC_arm_thumb ||
00553       uECC_PLATFORM == uECC_arm_thumb2))
00554 #include "asm_arm.inc"
00555 #endif
00556 #endif
00557
00558 #if !asm_clear
00559 static void vli_clear(uECC_word_t *vli)
00560 {
00561     wordcount_t i;
00562     for (i = 0; i < uECC_WORDS; ++i)
00563     {
00564         vli[i] = 0;
00565     }
00566 }
00567 #endif
00568
00569 /* Returns 1 if vli == 0, 0 otherwise. */
00570 #if !asm_isZero
00571 static uECC_word_t vli_isZero(const uECC_word_t *vli)
00572 {
00573     wordcount_t i;
00574     for (i = 0; i < uECC_WORDS; ++i)
00575     {
00576         if (vli[i])
00577         {
00578             return 0;
00579         }
00580     }
00581     return 1;
00582 }
00583 #endif
00584
00585 /* Returns nonzero if bit 'bit' of vli is set. */
00586 #if !asm_testBit
00587 static uECC_word_t vli_testBit(const uECC_word_t *vli, bitcount_t bit)
00588 {
00589     return (vli[bit >> uECC_WORD_BITS_SHIFT] &
00590            ((uECC_word_t) 1 << (bit & uECC_WORD_BITS_MASK)));
00591 }
00592 #endif
00593
00594 /* Counts the number of words in vli. */
00595 #if !asm_numBits
00596 static wordcount_t vli_numDigits(const uECC_word_t *vli, wordcount_t max_words)
00597 {
00598     swordcount_t i;
00599     /* Search from the end until we find a non-zero digit.
00600      * We do it in reverse because we expect that most digits will be nonzero.
00601      */
00602     for (i = max_words - 1; i >= 0 && vli[i] == 0; --i)
00603     {
00604     }
00605     return (i + 1);
00606 }
00607 #endif
00608
00609 /* Counts the number of bits required to represent vli. */
00610 static bitcount_t vli_numBits(const uECC_word_t *vli, wordcount_t max_words)
00611 {
00612     uECC_word_t i;
00613     uECC_word_t digit;
00614
00615     wordcount_t num_digits = vli_numDigits(vli, max_words);
00616     if (num_digits == 0)
00617     {
00618         return 0;
00619     }
00620
00621     digit = vli[num_digits - 1];
00622     for (i = 0; digit; ++i)
00623     {
00624         digit >>= 1;
00625     }
00626
00627     return ((bitcount_t) (num_digits - 1) << uECC_WORD_BITS_SHIFT) + i;
00628 }
00629 #endif /* !asm_numBits */
00630
00631 /* Sets dest = src. */
00632 #if !asm_set
00633 static void vli_set(uECC_word_t *dest, const uECC_word_t *src)
00634 {
00635     wordcount_t i;
00636     for (i = 0; i < uECC_WORDS; ++i)
00637     {

```

```

00638     dest[i] = src[i];
00639 }
00640 }
00641 #endif
00642
00643 /* Returns sign of left - right. */
00644 #if !asm_cmp
00645 static cmpresult_t vli_cmp(const uECC_word_t *left, const uECC_word_t *right)
00646 {
00647     swordcount_t i;
00648     for (i = uECC_WORDS - 1; i >= 0; --i)
00649     {
00650         if (left[i] > right[i])
00651         {
00652             return 1;
00653         }
00654         else if (left[i] < right[i])
00655         {
00656             return -1;
00657         }
00658     }
00659     return 0;
00660 }
00661 #endif
00662
00663 static cmpresult_t vli_equal(const uECC_word_t *left, const uECC_word_t *right)
00664 {
00665     uECC_word_t result = 0;
00666     swordcount_t i;
00667     for (i = uECC_WORDS - 1; i >= 0; --i)
00668     {
00669         result |= (left[i] ^ right[i]);
00670     }
00671     return (result == 0);
00672 }
00673
00674 /* Computes vli = vli » 1. */
00675 #if !asm_rshift1
00676 static void vli_rshift1(uECC_word_t *vli)
00677 {
00678     uECC_word_t *end = vli;
00679     uECC_word_t carry = 0;
00680
00681     vli += uECC_WORDS;
00682     while (vli-- > end)
00683     {
00684         uECC_word_t temp = *vli;
00685         *vli = (temp » 1) | carry;
00686         carry = temp « (uECC_WORD_BITS - 1);
00687     }
00688 }
00689 #endif
00690
00691 /* Computes result = left + right, returning carry. Can modify in place. */
00692 #if !asm_add
00693 static uECC_word_t vli_add(uECC_word_t *result, const uECC_word_t *left,
00694                             const uECC_word_t *right)
00695 {
00696     uECC_word_t carry = 0;
00697     wordcount_t i;
00698     for (i = 0; i < uECC_WORDS; ++i)
00699     {
00700         uECC_word_t sum = left[i] + right[i] + carry;
00701         if (sum != left[i])
00702         {
00703             carry = (sum < left[i]);
00704         }
00705         result[i] = sum;
00706     }
00707     return carry;
00708 }
00709 #endif
00710
00711 /* Computes result = left - right, returning borrow. Can modify in place. */
00712 #if !asm_sub
00713 static uECC_word_t vli_sub(uECC_word_t *result, const uECC_word_t *left,
00714                             const uECC_word_t *right)
00715 {
00716     uECC_word_t borrow = 0;
00717     wordcount_t i;
00718     for (i = 0; i < uECC_WORDS; ++i)
00719     {
00720         uECC_word_t diff = left[i] - right[i] - borrow;
00721         if (diff != left[i])
00722         {
00723             borrow = (diff > left[i]);
00724         }
00725     }

```

```

00725         result[i] = diff;
00726     }
00727     return borrow;
00728 }
00729 #endif
00730
00731 #if (!asm_mult || (uECC_SQUARE_FUNC && !asm_square) || \
00732     uECC_CURVE == uECC_secp256k1)
00733 static void muladd(uECC_word_t a, uECC_word_t b, uECC_word_t *r0,
00734     uECC_word_t *r1, uECC_word_t *r2)
00735 {
00736     #if uECC_WORD_SIZE == 8 && !SUPPORTS_INT128
00737         uint64_t a0 = a & 0xfffffffffull;
00738         uint64_t a1 = a >> 32;
00739         uint64_t b0 = b & 0xfffffffffull;
00740         uint64_t b1 = b >> 32;
00741
00742         uint64_t i0 = a0 * b0;
00743         uint64_t i1 = a0 * b1;
00744         uint64_t i2 = a1 * b0;
00745         uint64_t i3 = a1 * b1;
00746
00747         uint64_t p0, p1;
00748
00749         i2 += (i0 >> 32);
00750         i2 += i1;
00751         if (i2 < i1)
00752             { // overflow
00753                 i3 += 0x100000000ull;
00754             }
00755
00756         p0 = (i0 & 0xfffffffffull) | (i2 << 32);
00757         p1 = i3 + (i2 >> 32);
00758
00759         *r0 += p0;
00760         *r1 += (p1 + (*r0 < p0));
00761         *r2 += ((*r1 < p1) || (*r1 == p1 && *r0 < p0));
00762     #else
00763         uECC_dword_t p = (uECC_dword_t) a * b;
00764         uECC_dword_t r01 = ((uECC_dword_t) (*r1) << uECC_WORD_BITS) | *r0;
00765         r01 += p;
00766         *r2 += (r01 < p);
00767         *r1 = r01 >> uECC_WORD_BITS;
00768         *r0 = (uECC_word_t) r01;
00769     #endif
00770 }
00771 #define muladd_exists 1
00772 #endif
00773
00774 #if !asm_mult
00775 static void vli_mult(uECC_word_t *result, const uECC_word_t *left,
00776     const uECC_word_t *right)
00777 {
00778     uECC_word_t r0 = 0;
00779     uECC_word_t r1 = 0;
00780     uECC_word_t r2 = 0;
00781     wordcount_t i, k;
00782
00783     /* Compute each digit of result in sequence, maintaining the carries. */
00784     for (k = 0; k < uECC_WORDS; ++k)
00785     {
00786         for (i = 0; i <= k; ++i)
00787         {
00788             muladd(left[i], right[k - i], &r0, &r1, &r2);
00789         }
00790         result[k] = r0;
00791         r0 = r1;
00792         r1 = r2;
00793         r2 = 0;
00794     }
00795     for (k = uECC_WORDS; k < uECC_WORDS * 2 - 1; ++k)
00796     {
00797         for (i = (k + 1) - uECC_WORDS; i < uECC_WORDS; ++i)
00798         {
00799             muladd(left[i], right[k - i], &r0, &r1, &r2);
00800         }
00801         result[k] = r0;
00802         r0 = r1;
00803         r1 = r2;
00804         r2 = 0;
00805     }
00806     result[uECC_WORDS * 2 - 1] = r0;
00807 }
00808 #endif
00809
00810 #if uECC_SQUARE_FUNC
00811

```

```

00812 #if !asm_square
00813 static void mul2add(uECC_word_t a, uECC_word_t b, uECC_word_t *r0,
00814                   uECC_word_t *r1, uECC_word_t *r2)
00815 {
00816     #if uECC_WORD_SIZE == 8 && !SUPPORTS_INT128
00817         uint64_t a0 = a & 0xfffffffffull;
00818         uint64_t a1 = a >> 32;
00819         uint64_t b0 = b & 0xfffffffffull;
00820         uint64_t b1 = b >> 32;
00821
00822         uint64_t i0 = a0 * b0;
00823         uint64_t i1 = a0 * b1;
00824         uint64_t i2 = a1 * b0;
00825         uint64_t i3 = a1 * b1;
00826
00827         uint64_t p0, p1;
00828
00829         i2 += (i0 >> 32);
00830         i2 += i1;
00831         if (i2 < i1)
00832             { // overflow
00833                 i3 += 0x100000000ull;
00834             }
00835
00836         p0 = (i0 & 0xfffffffffull) | (i2 << 32);
00837         p1 = i3 + (i2 >> 32);
00838
00839         *r2 += (p1 >> 63);
00840         p1 = (p1 << 1) | (p0 >> 63);
00841         p0 <<= 1;
00842
00843         *r0 += p0;
00844         *r1 += (p1 + (*r0 < p0));
00845         *r2 += ((*r1 < p1) || (*r1 == p1 && *r0 < p0));
00846     #else
00847         uECC_dword_t p = (uECC_dword_t) a * b;
00848         uECC_dword_t r01 = ((uECC_dword_t) (*r1) << uECC_WORD_BITS) | *r0;
00849         *r2 += (p >> (uECC_WORD_BITS * 2 - 1));
00850         p <<= 2;
00851         r01 += p;
00852         *r2 += (r01 < p);
00853         *r1 = r01 >> uECC_WORD_BITS;
00854         *r0 = (uECC_word_t) r01;
00855     #endif
00856 }
00857
00858 static void vli_square(uECC_word_t *result, const uECC_word_t *left)
00859 {
00860     uECC_word_t r0 = 0;
00861     uECC_word_t r1 = 0;
00862     uECC_word_t r2 = 0;
00863
00864     wordcount_t i, k;
00865
00866     for (k = 0; k < uECC_WORDS * 2 - 1; ++k)
00867     {
00868         uECC_word_t min = (k < uECC_WORDS ? 0 : (k + 1) - uECC_WORDS);
00869         for (i = min; i <= k && i <= k - i; ++i)
00870         {
00871             if (i < k - i)
00872             {
00873                 mul2add(left[i], left[k - i], &r0, &r1, &r2);
00874             }
00875             else
00876             {
00877                 muladd(left[i], left[k - i], &r0, &r1, &r2);
00878             }
00879             result[k] = r0;
00880             r0 = r1;
00881             r1 = r2;
00882             r2 = 0;
00883         }
00884     }
00885     result[uECC_WORDS * 2 - 1] = r0;
00886 }
00887 #endif
00888 #else /* uECC_SQUARE_FUNC */
00889 #define vli_square(result, left, size) \
00890     vli_mult((result), (left), (left), (size))
00891 #endif /* uECC_SQUARE_FUNC */
00892
00893 /* Computes result = (left + right) % mod.
00894    Assumes that left < mod and right < mod, and that result does not overlap

```

```

00899     mod. */
00900 #if !asm_modAdd
00901 static void vli_modAdd(uECC_word_t *result, const uECC_word_t *left,
00902                      const uECC_word_t *right, const uECC_word_t *mod)
00903 {
00904     uECC_word_t carry = vli_add(result, left, right);
00905     if (carry || vli_cmp(result, mod) >= 0)
00906     {
00907         /* result > mod (result = mod + remainder), so subtract mod to get
00908          * remainder. */
00909         vli_sub(result, result, mod);
00910     }
00911 }
00912 #endif
00913
00914 /* Computes result = (left - right) % mod.
00915  * Assumes that left < mod and right < mod, and that result does not overlap
00916  * mod. */
00917 #if !asm_modSub
00918 static void vli_modSub(uECC_word_t *result, const uECC_word_t *left,
00919                      const uECC_word_t *right, const uECC_word_t *mod)
00920 {
00921     uECC_word_t l_borrow = vli_sub(result, left, right);
00922     if (l_borrow)
00923     {
00924         /* In this case, result == -diff == (max int) - diff. Since -x % d == d
00925          * - x, we can get the correct result from result + mod (with overflow).
00926          */
00927         vli_add(result, result, mod);
00928     }
00929 }
00930 #endif
00931
00932 #if !asm_modSub_fast
00933 #define vli_modSub_fast(result, left, right) \
00934     vli_modSub((result), (left), (right), curve_p)
00935 #endif
00936
00937 #if !asm_mmod_fast
00938
00939 #if (uECC_CURVE == uECC_secp160r1 || uECC_CURVE == uECC_secp256k1)
00940 /* omega_mult() is defined farther below for the different curves / word sizes
00941 */
00942 static void omega_mult(uECC_word_t *RESTRICT result,
00943                      const uECC_word_t *RESTRICT right);
00944
00945 /* Computes result = product % curve_p
00946  * see http://www.isys.uni-klu.ac.at/PDF/2001-0126-MT.pdf page 354
00947
00948  * Note that this only works if log2(omega) < log2(p) / 2 */
00949 static void vli_mmod_fast(uECC_word_t *RESTRICT result,
00950                          uECC_word_t *RESTRICT product)
00951 {
00952     uECC_word_t tmp[2 * uECC_WORDS];
00953     uECC_word_t carry;
00954     uECC_word_t dummy[uECC_WORDS];
00955     uECC_word_t borrow_out;
00956     uECC_word_t must_sub;
00957     uECC_word_t mask;
00958     int i, j;
00959
00960     vli_clear(tmp);
00961     vli_clear(tmp + uECC_WORDS);
00962
00963     omega_mult(tmp, product + uECC_WORDS); /* (Rq, q) = q * c */
00964
00965     carry = vli_add(result, product, tmp); /* (C, r) = r + q */
00966     vli_clear(product);
00967     omega_mult(product, tmp + uECC_WORDS); /* Rq*c */
00968     carry += vli_add(result, result, product); /* (Cl, r) = r + Rq*c */
00969
00970     for (i = 0; i < 4; ++i)
00971     {
00972         borrow_out = vli_sub(dummy, result, curve_p);
00973
00974         must_sub = (carry > 0) | (borrow_out == 0);
00975         mask = 0 - (uECC_word_t) (must_sub & 1); // 0 or 0xFFFFFFFF
00976
00977         for (j = 0; j < uECC_WORDS; ++j)
00978         {
00979             result[j] = (result[j] & ~mask) | (dummy[j] & mask);
00980         }
00981
00982         carry -= (carry > 0);
00983     }
00984 }
00985

```



```

00986 #endif
00987
00988 #if uECC_CURVE == uECC_secp160r1
00989
00990 #if uECC_WORD_SIZE == 1
00991 static void omega_mult(uint8_t *RESTRICT result, const uint8_t *RESTRICT right)
00992 {
00993     uint8_t carry;
00994     uint8_t i;
00995
00996     /* Multiply by (2^31 + 1). */
00997     vli_set(result + 4, right); /* 2^32 */
00998     vli_rshift1(result + 4); /* 2^31 */
00999     result[3] = right[0] << 7; /* get last bit from shift */
01000
01001     carry = vli_add(result, result, right); /* 2^31 + 1 */
01002     for (i = uECC_WORDS; carry; ++i)
01003     {
01004         uint16_t sum = (uint16_t) result[i] + carry;
01005         result[i] = (uint8_t) sum;
01006         carry = sum >> 8;
01007     }
01008 }
01009 #elif uECC_WORD_SIZE == 4
01010 static void omega_mult(uint32_t *RESTRICT result,
01011                        const uint32_t *RESTRICT right)
01012 {
01013     uint32_t carry;
01014     unsigned i;
01015
01016     /* Multiply by (2^31 + 1). */
01017     vli_set(result + 1, right); /* 2^32 */
01018     vli_rshift1(result + 1); /* 2^31 */
01019     result[0] = right[0] << 31; /* get last bit from shift */
01020
01021     carry = vli_add(result, result, right); /* 2^31 + 1 */
01022     for (i = uECC_WORDS; carry; ++i)
01023     {
01024         uint64_t sum = (uint64_t) result[i] + carry;
01025         result[i] = (uint32_t) sum;
01026         carry = sum >> 32;
01027     }
01028 }
01029 #endif /* uECC_WORD_SIZE */
01030
01031 #elif uECC_CURVE == uECC_secp192r1
01032
01033 /* Computes result = product % curve_p.
01034 See algorithm 5 and 6 from http://www.isys.uni-klu.ac.at/PDF/2001-0126-MT.pdf
01035 */
01036 #if uECC_WORD_SIZE == 1
01037 static void vli_mmod_fast(uint8_t *RESTRICT result, uint8_t *RESTRICT product)
01038 {
01039     uint8_t tmp[uECC_WORDS];
01040     uint8_t carry;
01041
01042     vli_set(result, product);
01043
01044     vli_set(tmp, &product[24]);
01045     carry = vli_add(result, result, tmp);
01046
01047     tmp[0] = tmp[1] = tmp[2] = tmp[3] = tmp[4] = tmp[5] = tmp[6] = tmp[7] = 0;
01048     tmp[8] = product[24];
01049     tmp[9] = product[25];
01050     tmp[10] = product[26];
01051     tmp[11] = product[27];
01052     tmp[12] = product[28];
01053     tmp[13] = product[29];
01054     tmp[14] = product[30];
01055     tmp[15] = product[31];
01056     tmp[16] = product[32];
01057     tmp[17] = product[33];
01058     tmp[18] = product[34];
01059     tmp[19] = product[35];
01060     tmp[20] = product[36];
01061     tmp[21] = product[37];
01062     tmp[22] = product[38];
01063     tmp[23] = product[39];
01064     carry += vli_add(result, result, tmp);
01065
01066     tmp[0] = tmp[8] = product[40];
01067     tmp[1] = tmp[9] = product[41];
01068     tmp[2] = tmp[10] = product[42];
01069     tmp[3] = tmp[11] = product[43];
01070     tmp[4] = tmp[12] = product[44];
01071     tmp[5] = tmp[13] = product[45];
01072     tmp[6] = tmp[14] = product[46];

```

```

01073     tmp[7] = tmp[15] = product[47];
01074     tmp[16] = tmp[17] = tmp[18] = tmp[19] = tmp[20] = tmp[21] = tmp[22] =
01075         tmp[23] = 0;
01076     carry += vli_add(result, result, tmp);
01077
01078     while (carry || vli_cmp(curve_p, result) != 1)
01079     {
01080         carry -= vli_sub(result, result, curve_p);
01081     }
01082 }
01083 #elif uECC_WORD_SIZE == 4
01084 static void vli_mmod_fast(uint32_t *RESTRICT result, uint32_t *RESTRICT product)
01085 {
01086     uint32_t tmp[uECC_WORDS];
01087     int carry;
01088
01089     vli_set(result, product);
01090
01091     vli_set(tmp, &product[6]);
01092     carry = vli_add(result, result, tmp);
01093
01094     tmp[0] = tmp[1] = 0;
01095     tmp[2] = product[6];
01096     tmp[3] = product[7];
01097     tmp[4] = product[8];
01098     tmp[5] = product[9];
01099     carry += vli_add(result, result, tmp);
01100
01101     tmp[0] = tmp[2] = product[10];
01102     tmp[1] = tmp[3] = product[11];
01103     tmp[4] = tmp[5] = 0;
01104     carry += vli_add(result, result, tmp);
01105
01106     while (carry || vli_cmp(curve_p, result) != 1)
01107     {
01108         carry -= vli_sub(result, result, curve_p);
01109     }
01110 }
01111 #else
01112 static void vli_mmod_fast(uint64_t *RESTRICT result, uint64_t *RESTRICT product)
01113 {
01114     uint64_t tmp[uECC_WORDS];
01115     int carry;
01116
01117     vli_set(result, product);
01118
01119     vli_set(tmp, &product[3]);
01120     carry = vli_add(result, result, tmp);
01121
01122     tmp[0] = 0;
01123     tmp[1] = product[3];
01124     tmp[2] = product[4];
01125     carry += vli_add(result, result, tmp);
01126
01127     tmp[0] = tmp[1] = product[5];
01128     tmp[2] = 0;
01129     carry += vli_add(result, result, tmp);
01130
01131     while (carry || vli_cmp(curve_p, result) != 1)
01132     {
01133         carry -= vli_sub(result, result, curve_p);
01134     }
01135 }
01136 #endif /* uECC_WORD_SIZE */
01137
01138 #elif uECC_CURVE == uECC_secp256r1
01139
01140 /* Computes result = product % curve_p
01141    from http://www.nsa.gov/ia/\_files/nist-routines.pdf */
01142 #if uECC_WORD_SIZE == 1
01143 static void vli_mmod_fast(uint8_t *RESTRICT result, uint8_t *RESTRICT product)
01144 {
01145     uint8_t tmp[uECC_BYTES];
01146     int8_t carry;
01147
01148     /* t */
01149     vli_set(result, product);
01150
01151     /* s1 */
01152     tmp[0] = tmp[1] = tmp[2] = tmp[3] = 0;
01153     tmp[4] = tmp[5] = tmp[6] = tmp[7] = 0;
01154     tmp[8] = tmp[9] = tmp[10] = tmp[11] = 0;
01155     tmp[12] = product[44];
01156     tmp[13] = product[45];
01157     tmp[14] = product[46];
01158     tmp[15] = product[47];
01159     tmp[16] = product[48];

```

```
01160     tmp[17] = product[49];
01161     tmp[18] = product[50];
01162     tmp[19] = product[51];
01163     tmp[20] = product[52];
01164     tmp[21] = product[53];
01165     tmp[22] = product[54];
01166     tmp[23] = product[55];
01167     tmp[24] = product[56];
01168     tmp[25] = product[57];
01169     tmp[26] = product[58];
01170     tmp[27] = product[59];
01171     tmp[28] = product[60];
01172     tmp[29] = product[61];
01173     tmp[30] = product[62];
01174     tmp[31] = product[63];
01175     carry = vli_add(tmp, tmp, tmp);
01176     carry += vli_add(result, result, tmp);
01177
01178     /* s2 */
01179     tmp[12] = product[48];
01180     tmp[13] = product[49];
01181     tmp[14] = product[50];
01182     tmp[15] = product[51];
01183     tmp[16] = product[52];
01184     tmp[17] = product[53];
01185     tmp[18] = product[54];
01186     tmp[19] = product[55];
01187     tmp[20] = product[56];
01188     tmp[21] = product[57];
01189     tmp[22] = product[58];
01190     tmp[23] = product[59];
01191     tmp[24] = product[60];
01192     tmp[25] = product[61];
01193     tmp[26] = product[62];
01194     tmp[27] = product[63];
01195     tmp[28] = tmp[29] = tmp[30] = tmp[31] = 0;
01196     carry += vli_add(tmp, tmp, tmp);
01197     carry += vli_add(result, result, tmp);
01198
01199     /* s3 */
01200     tmp[0] = product[32];
01201     tmp[1] = product[33];
01202     tmp[2] = product[34];
01203     tmp[3] = product[35];
01204     tmp[4] = product[36];
01205     tmp[5] = product[37];
01206     tmp[6] = product[38];
01207     tmp[7] = product[39];
01208     tmp[8] = product[40];
01209     tmp[9] = product[41];
01210     tmp[10] = product[42];
01211     tmp[11] = product[43];
01212     tmp[12] = tmp[13] = tmp[14] = tmp[15] = 0;
01213     tmp[16] = tmp[17] = tmp[18] = tmp[19] = 0;
01214     tmp[20] = tmp[21] = tmp[22] = tmp[23] = 0;
01215     tmp[24] = product[56];
01216     tmp[25] = product[57];
01217     tmp[26] = product[58];
01218     tmp[27] = product[59];
01219     tmp[28] = product[60];
01220     tmp[29] = product[61];
01221     tmp[30] = product[62];
01222     tmp[31] = product[63];
01223     carry += vli_add(result, result, tmp);
01224
01225     /* s4 */
01226     tmp[0] = product[36];
01227     tmp[1] = product[37];
01228     tmp[2] = product[38];
01229     tmp[3] = product[39];
01230     tmp[4] = product[40];
01231     tmp[5] = product[41];
01232     tmp[6] = product[42];
01233     tmp[7] = product[43];
01234     tmp[8] = product[44];
01235     tmp[9] = product[45];
01236     tmp[10] = product[46];
01237     tmp[11] = product[47];
01238     tmp[12] = product[52];
01239     tmp[13] = product[53];
01240     tmp[14] = product[54];
01241     tmp[15] = product[55];
01242     tmp[16] = product[56];
01243     tmp[17] = product[57];
01244     tmp[18] = product[58];
01245     tmp[19] = product[59];
01246     tmp[20] = product[60];
```

```
01247     tmp[21] = product[61];
01248     tmp[22] = product[62];
01249     tmp[23] = product[63];
01250     tmp[24] = product[52];
01251     tmp[25] = product[53];
01252     tmp[26] = product[54];
01253     tmp[27] = product[55];
01254     tmp[28] = product[32];
01255     tmp[29] = product[33];
01256     tmp[30] = product[34];
01257     tmp[31] = product[35];
01258     carry += vli_add(result, result, tmp);
01259
01260     /* d1 */
01261     tmp[0] = product[44];
01262     tmp[1] = product[45];
01263     tmp[2] = product[46];
01264     tmp[3] = product[47];
01265     tmp[4] = product[48];
01266     tmp[5] = product[49];
01267     tmp[6] = product[50];
01268     tmp[7] = product[51];
01269     tmp[8] = product[52];
01270     tmp[9] = product[53];
01271     tmp[10] = product[54];
01272     tmp[11] = product[55];
01273     tmp[12] = tmp[13] = tmp[14] = tmp[15] = 0;
01274     tmp[16] = tmp[17] = tmp[18] = tmp[19] = 0;
01275     tmp[20] = tmp[21] = tmp[22] = tmp[23] = 0;
01276     tmp[24] = product[32];
01277     tmp[25] = product[33];
01278     tmp[26] = product[34];
01279     tmp[27] = product[35];
01280     tmp[28] = product[40];
01281     tmp[29] = product[41];
01282     tmp[30] = product[42];
01283     tmp[31] = product[43];
01284     carry -= vli_sub(result, result, tmp);
01285
01286     /* d2 */
01287     tmp[0] = product[48];
01288     tmp[1] = product[49];
01289     tmp[2] = product[50];
01290     tmp[3] = product[51];
01291     tmp[4] = product[52];
01292     tmp[5] = product[53];
01293     tmp[6] = product[54];
01294     tmp[7] = product[55];
01295     tmp[8] = product[56];
01296     tmp[9] = product[57];
01297     tmp[10] = product[58];
01298     tmp[11] = product[59];
01299     tmp[12] = product[60];
01300     tmp[13] = product[61];
01301     tmp[14] = product[62];
01302     tmp[15] = product[63];
01303     tmp[16] = tmp[17] = tmp[18] = tmp[19] = 0;
01304     tmp[20] = tmp[21] = tmp[22] = tmp[23] = 0;
01305     tmp[24] = product[36];
01306     tmp[25] = product[37];
01307     tmp[26] = product[38];
01308     tmp[27] = product[39];
01309     tmp[28] = product[44];
01310     tmp[29] = product[45];
01311     tmp[30] = product[46];
01312     tmp[31] = product[47];
01313     carry -= vli_sub(result, result, tmp);
01314
01315     /* d3 */
01316     tmp[0] = product[52];
01317     tmp[1] = product[53];
01318     tmp[2] = product[54];
01319     tmp[3] = product[55];
01320     tmp[4] = product[56];
01321     tmp[5] = product[57];
01322     tmp[6] = product[58];
01323     tmp[7] = product[59];
01324     tmp[8] = product[60];
01325     tmp[9] = product[61];
01326     tmp[10] = product[62];
01327     tmp[11] = product[63];
01328     tmp[12] = product[32];
01329     tmp[13] = product[33];
01330     tmp[14] = product[34];
01331     tmp[15] = product[35];
01332     tmp[16] = product[36];
01333     tmp[17] = product[37];
```

```

01334     tmp[18] = product[38];
01335     tmp[19] = product[39];
01336     tmp[20] = product[40];
01337     tmp[21] = product[41];
01338     tmp[22] = product[42];
01339     tmp[23] = product[43];
01340     tmp[24] = tmp[25] = tmp[26] = tmp[27] = 0;
01341     tmp[28] = product[48];
01342     tmp[29] = product[49];
01343     tmp[30] = product[50];
01344     tmp[31] = product[51];
01345     carry -= vli_sub(result, result, tmp);
01346
01347     /* d4 */
01348     tmp[0] = product[56];
01349     tmp[1] = product[57];
01350     tmp[2] = product[58];
01351     tmp[3] = product[59];
01352     tmp[4] = product[60];
01353     tmp[5] = product[61];
01354     tmp[6] = product[62];
01355     tmp[7] = product[63];
01356     tmp[8] = tmp[9] = tmp[10] = tmp[11] = 0;
01357     tmp[12] = product[36];
01358     tmp[13] = product[37];
01359     tmp[14] = product[38];
01360     tmp[15] = product[39];
01361     tmp[16] = product[40];
01362     tmp[17] = product[41];
01363     tmp[18] = product[42];
01364     tmp[19] = product[43];
01365     tmp[20] = product[44];
01366     tmp[21] = product[45];
01367     tmp[22] = product[46];
01368     tmp[23] = product[47];
01369     tmp[24] = tmp[25] = tmp[26] = tmp[27] = 0;
01370     tmp[28] = product[52];
01371     tmp[29] = product[53];
01372     tmp[30] = product[54];
01373     tmp[31] = product[55];
01374     carry -= vli_sub(result, result, tmp);
01375
01376     if (carry < 0)
01377     {
01378         do
01379         {
01380             carry += vli_add(result, result, curve_p);
01381         } while (carry < 0);
01382     }
01383     else
01384     {
01385         while (carry || vli_cmp(curve_p, result) != 1)
01386         {
01387             carry -= vli_sub(result, result, curve_p);
01388         }
01389     }
01390 }
01391 #elif uECC_WORD_SIZE == 4
01392 static void vli_mmod_fast(uint32_t *RESTRICT result, uint32_t *RESTRICT product)
01393 {
01394     uint32_t tmp[uECC_WORDS];
01395     int carry;
01396
01397     /* t */
01398     vli_set(result, product);
01399
01400     /* s1 */
01401     tmp[0] = tmp[1] = tmp[2] = 0;
01402     tmp[3] = product[11];
01403     tmp[4] = product[12];
01404     tmp[5] = product[13];
01405     tmp[6] = product[14];
01406     tmp[7] = product[15];
01407     carry = vli_add(tmp, tmp, tmp);
01408     carry += vli_add(result, result, tmp);
01409
01410     /* s2 */
01411     tmp[3] = product[12];
01412     tmp[4] = product[13];
01413     tmp[5] = product[14];
01414     tmp[6] = product[15];
01415     tmp[7] = 0;
01416     carry += vli_add(tmp, tmp, tmp);
01417     carry += vli_add(result, result, tmp);
01418
01419     /* s3 */
01420     tmp[0] = product[8];

```

```

01421     tmp[1] = product[9];
01422     tmp[2] = product[10];
01423     tmp[3] = tmp[4] = tmp[5] = 0;
01424     tmp[6] = product[14];
01425     tmp[7] = product[15];
01426     carry += vli_add(result, result, tmp);
01427
01428     /* s4 */
01429     tmp[0] = product[9];
01430     tmp[1] = product[10];
01431     tmp[2] = product[11];
01432     tmp[3] = product[13];
01433     tmp[4] = product[14];
01434     tmp[5] = product[15];
01435     tmp[6] = product[13];
01436     tmp[7] = product[8];
01437     carry += vli_add(result, result, tmp);
01438
01439     /* d1 */
01440     tmp[0] = product[11];
01441     tmp[1] = product[12];
01442     tmp[2] = product[13];
01443     tmp[3] = tmp[4] = tmp[5] = 0;
01444     tmp[6] = product[8];
01445     tmp[7] = product[10];
01446     carry -= vli_sub(result, result, tmp);
01447
01448     /* d2 */
01449     tmp[0] = product[12];
01450     tmp[1] = product[13];
01451     tmp[2] = product[14];
01452     tmp[3] = product[15];
01453     tmp[4] = tmp[5] = 0;
01454     tmp[6] = product[9];
01455     tmp[7] = product[11];
01456     carry -= vli_sub(result, result, tmp);
01457
01458     /* d3 */
01459     tmp[0] = product[13];
01460     tmp[1] = product[14];
01461     tmp[2] = product[15];
01462     tmp[3] = product[8];
01463     tmp[4] = product[9];
01464     tmp[5] = product[10];
01465     tmp[6] = 0;
01466     tmp[7] = product[12];
01467     carry -= vli_sub(result, result, tmp);
01468
01469     /* d4 */
01470     tmp[0] = product[14];
01471     tmp[1] = product[15];
01472     tmp[2] = 0;
01473     tmp[3] = product[9];
01474     tmp[4] = product[10];
01475     tmp[5] = product[11];
01476     tmp[6] = 0;
01477     tmp[7] = product[13];
01478     carry -= vli_sub(result, result, tmp);
01479
01480     if (carry < 0)
01481     {
01482         do
01483         {
01484             carry += vli_add(result, result, curve_p);
01485         } while (carry < 0);
01486     }
01487     else
01488     {
01489         while (carry || vli_cmp(curve_p, result) != 1)
01490         {
01491             carry -= vli_sub(result, result, curve_p);
01492         }
01493     }
01494 }
01495 #else
01496 static void vli_mmod_fast(uint64_t *RESTRICT result, uint64_t *RESTRICT product)
01497 {
01498     uint64_t tmp[uECC_WORDS];
01499     int carry;
01500
01501     /* t */
01502     vli_set(result, product);
01503
01504     /* s1 */
01505     tmp[0] = 0;
01506     tmp[1] = product[5] & 0xffffffff00000000ull;
01507     tmp[2] = product[6];

```

```

01508     tmp[3] = product[7];
01509     carry = vli_add(tmp, tmp, tmp);
01510     carry += vli_add(result, result, tmp);
01511
01512     /* s2 */
01513     tmp[1] = product[6] << 32;
01514     tmp[2] = (product[6] >> 32) | (product[7] << 32);
01515     tmp[3] = product[7] >> 32;
01516     carry += vli_add(tmp, tmp, tmp);
01517     carry += vli_add(result, result, tmp);
01518
01519     /* s3 */
01520     tmp[0] = product[4];
01521     tmp[1] = product[5] & 0xffffffff;
01522     tmp[2] = 0;
01523     tmp[3] = product[7];
01524     carry += vli_add(result, result, tmp);
01525
01526     /* s4 */
01527     tmp[0] = (product[4] >> 32) | (product[5] << 32);
01528     tmp[1] = (product[5] >> 32) | (product[6] & 0xffffffff00000000ull);
01529     tmp[2] = product[7];
01530     tmp[3] = (product[6] >> 32) | (product[4] << 32);
01531     carry += vli_add(result, result, tmp);
01532
01533     /* d1 */
01534     tmp[0] = (product[5] >> 32) | (product[6] << 32);
01535     tmp[1] = (product[6] >> 32);
01536     tmp[2] = 0;
01537     tmp[3] = (product[4] & 0xffffffff) | (product[5] << 32);
01538     carry -= vli_sub(result, result, tmp);
01539
01540     /* d2 */
01541     tmp[0] = product[6];
01542     tmp[1] = product[7];
01543     tmp[2] = 0;
01544     tmp[3] = (product[4] >> 32) | (product[5] & 0xffffffff00000000ull);
01545     carry -= vli_sub(result, result, tmp);
01546
01547     /* d3 */
01548     tmp[0] = (product[6] >> 32) | (product[7] << 32);
01549     tmp[1] = (product[7] >> 32) | (product[4] << 32);
01550     tmp[2] = (product[4] >> 32) | (product[5] << 32);
01551     tmp[3] = (product[6] << 32);
01552     carry -= vli_sub(result, result, tmp);
01553
01554     /* d4 */
01555     tmp[0] = product[7];
01556     tmp[1] = product[4] & 0xffffffff00000000ull;
01557     tmp[2] = product[5];
01558     tmp[3] = product[6] & 0xffffffff00000000ull;
01559     carry -= vli_sub(result, result, tmp);
01560
01561     if (carry < 0)
01562     {
01563         do
01564         {
01565             carry += vli_add(result, result, curve_p);
01566         } while (carry < 0);
01567     }
01568     else
01569     {
01570         while (carry || vli_cmp(curve_p, result) != 1)
01571         {
01572             carry -= vli_sub(result, result, curve_p);
01573         }
01574     }
01575 }
01576 #endif /* uECC_WORD_SIZE */
01577
01578 #elif uECC_CURVE == uECC_secp256k1
01579
01580 #if uECC_WORD_SIZE == 1
01581 static void omega_mult(uint8_t *RESTRICT result, const uint8_t *RESTRICT right)
01582 {
01583     /* Multiply by (2^32 + 2^9 + 2^8 + 2^7 + 2^6 + 2^4 + 1). */
01584     uECC_word_t r0 = 0;
01585     uECC_word_t r1 = 0;
01586     uECC_word_t r2 = 0;
01587     wordcount_t k;
01588
01589     /* Multiply by (2^9 + 2^8 + 2^7 + 2^6 + 2^4 + 1). */
01590     muladd(0xD1, right[0], &r0, &r1, &r2);
01591     result[0] = r0;
01592     r0 = r1;
01593     r1 = r2;
01594     /* r2 is still 0 */

```

```

01595
01596     for (k = 1; k < uECC_WORDS; ++k)
01597     {
01598         muladd(0x03, right[k - 1], &r0, &r1, &r2);
01599         muladd(0xD1, right[k], &r0, &r1, &r2);
01600         result[k] = r0;
01601         r0 = r1;
01602         r1 = r2;
01603         r2 = 0;
01604     }
01605     muladd(0x03, right[uECC_WORDS - 1], &r0, &r1, &r2);
01606     result[uECC_WORDS] = r0;
01607     result[uECC_WORDS + 1] = r1;
01608
01609     result[4 + uECC_WORDS] =
01610         vli_add(result + 4, result + 4, right); /* add the 2^32 multiple */
01611 }
01612 #elif uECC_WORD_SIZE == 4
01613 static void omega_mult(uint32_t *RESTRICT result,
01614                       const uint32_t *RESTRICT right)
01615 {
01616     /* Multiply by (2^9 + 2^8 + 2^7 + 2^6 + 2^4 + 1). */
01617     uint32_t carry = 0;
01618     wordcount_t k;
01619
01620     for (k = 0; k < uECC_WORDS; ++k)
01621     {
01622         uint64_t p = (uint64_t) 0x3D1 * right[k] + carry;
01623         result[k] = (p & 0xffffffff);
01624         carry = p >> 32;
01625     }
01626     result[uECC_WORDS] = carry;
01627
01628     result[1 + uECC_WORDS] =
01629         vli_add(result + 1, result + 1, right); /* add the 2^32 multiple */
01630 }
01631 #else
01632 static void omega_mult(uint64_t *RESTRICT result,
01633                       const uint64_t *RESTRICT right)
01634 {
01635     uECC_word_t r0 = 0;
01636     uECC_word_t r1 = 0;
01637     uECC_word_t r2 = 0;
01638     wordcount_t k;
01639
01640     /* Multiply by (2^32 + 2^9 + 2^8 + 2^7 + 2^6 + 2^4 + 1). */
01641     for (k = 0; k < uECC_WORDS; ++k)
01642     {
01643         muladd(0x1000003D1ull, right[k], &r0, &r1, &r2);
01644         result[k] = r0;
01645         r0 = r1;
01646         r1 = r2;
01647         r2 = 0;
01648     }
01649     result[uECC_WORDS] = r0;
01650 }
01651 #endif /* uECC_WORD_SIZE */
01652
01653 #elif uECC_CURVE == uECC_secp224r1
01654
01655 /* Computes result = product % curve_p
01656    from http://www.nsa.gov/ia/_files/nist-routines.pdf */
01657 #if uECC_WORD_SIZE == 1
01658 // TODO it may be faster to use the omega_mult method when fully asm optimized.
01659 void vli_mmod_fast(uint8_t *RESTRICT result, uint8_t *RESTRICT product)
01660 {
01661     uint8_t tmp[uECC_WORDS];
01662     int8_t carry;
01663
01664     /* t */
01665     vli_set(result, product);
01666
01667     /* s1 */
01668     tmp[0] = tmp[1] = tmp[2] = tmp[3] = 0;
01669     tmp[4] = tmp[5] = tmp[6] = tmp[7] = 0;
01670     tmp[8] = tmp[9] = tmp[10] = tmp[11] = 0;
01671     tmp[12] = product[28];
01672     tmp[13] = product[29];
01673     tmp[14] = product[30];
01674     tmp[15] = product[31];
01675     tmp[16] = product[32];
01676     tmp[17] = product[33];
01677     tmp[18] = product[34];
01678     tmp[19] = product[35];
01679     tmp[20] = product[36];
01680     tmp[21] = product[37];
01681     tmp[22] = product[38];

```



```

01682     tmp[23] = product[39];
01683     tmp[24] = product[40];
01684     tmp[25] = product[41];
01685     tmp[26] = product[42];
01686     tmp[27] = product[43];
01687     carry = vli_add(result, result, tmp);
01688
01689     /* s2 */
01690     tmp[12] = product[44];
01691     tmp[13] = product[45];
01692     tmp[14] = product[46];
01693     tmp[15] = product[47];
01694     tmp[16] = product[48];
01695     tmp[17] = product[49];
01696     tmp[18] = product[50];
01697     tmp[19] = product[51];
01698     tmp[20] = product[52];
01699     tmp[21] = product[53];
01700     tmp[22] = product[54];
01701     tmp[23] = product[55];
01702     tmp[24] = tmp[25] = tmp[26] = tmp[27] = 0;
01703     carry += vli_add(result, result, tmp);
01704
01705     /* d1 */
01706     tmp[0] = product[28];
01707     tmp[1] = product[29];
01708     tmp[2] = product[30];
01709     tmp[3] = product[31];
01710     tmp[4] = product[32];
01711     tmp[5] = product[33];
01712     tmp[6] = product[34];
01713     tmp[7] = product[35];
01714     tmp[8] = product[36];
01715     tmp[9] = product[37];
01716     tmp[10] = product[38];
01717     tmp[11] = product[39];
01718     tmp[12] = product[40];
01719     tmp[13] = product[41];
01720     tmp[14] = product[42];
01721     tmp[15] = product[43];
01722     tmp[16] = product[44];
01723     tmp[17] = product[45];
01724     tmp[18] = product[46];
01725     tmp[19] = product[47];
01726     tmp[20] = product[48];
01727     tmp[21] = product[49];
01728     tmp[22] = product[50];
01729     tmp[23] = product[51];
01730     tmp[24] = product[52];
01731     tmp[25] = product[53];
01732     tmp[26] = product[54];
01733     tmp[27] = product[55];
01734     carry -= vli_sub(result, result, tmp);
01735
01736     /* d2 */
01737     tmp[0] = product[44];
01738     tmp[1] = product[45];
01739     tmp[2] = product[46];
01740     tmp[3] = product[47];
01741     tmp[4] = product[48];
01742     tmp[5] = product[49];
01743     tmp[6] = product[50];
01744     tmp[7] = product[51];
01745     tmp[8] = product[52];
01746     tmp[9] = product[53];
01747     tmp[10] = product[54];
01748     tmp[11] = product[55];
01749     tmp[12] = tmp[13] = tmp[14] = tmp[15] = 0;
01750     tmp[16] = tmp[17] = tmp[18] = tmp[19] = 0;
01751     tmp[20] = tmp[21] = tmp[22] = tmp[23] = 0;
01752     tmp[24] = tmp[25] = tmp[26] = tmp[27] = 0;
01753     carry -= vli_sub(result, result, tmp);
01754
01755     if (carry < 0)
01756     {
01757         do
01758         {
01759             carry += vli_add(result, result, curve_p);
01760         } while (carry < 0);
01761     }
01762     else
01763     {
01764         while (carry || vli_cmp(curve_p, result) != 1)
01765         {
01766             carry -= vli_sub(result, result, curve_p);
01767         }
01768     }

```

```

01769 }
01770 #elif uECC_WORD_SIZE == 4
01771 void vli_mmod_fast(uint32_t *RESTRICT result, uint32_t *RESTRICT product)
01772 {
01773     uint32_t tmp[uECC_WORDS];
01774     int carry;
01775
01776     /* t */
01777     vli_set(result, product);
01778
01779     /* s1 */
01780     tmp[0] = tmp[1] = tmp[2] = 0;
01781     tmp[3] = product[7];
01782     tmp[4] = product[8];
01783     tmp[5] = product[9];
01784     tmp[6] = product[10];
01785     carry = vli_add(result, result, tmp);
01786
01787     /* s2 */
01788     tmp[3] = product[11];
01789     tmp[4] = product[12];
01790     tmp[5] = product[13];
01791     tmp[6] = 0;
01792     carry += vli_add(result, result, tmp);
01793
01794     /* d1 */
01795     tmp[0] = product[7];
01796     tmp[1] = product[8];
01797     tmp[2] = product[9];
01798     tmp[3] = product[10];
01799     tmp[4] = product[11];
01800     tmp[5] = product[12];
01801     tmp[6] = product[13];
01802     carry -= vli_sub(result, result, tmp);
01803
01804     /* d2 */
01805     tmp[0] = product[11];
01806     tmp[1] = product[12];
01807     tmp[2] = product[13];
01808     tmp[3] = tmp[4] = tmp[5] = tmp[6] = 0;
01809     carry -= vli_sub(result, result, tmp);
01810
01811     if (carry < 0)
01812     {
01813         do
01814         {
01815             carry += vli_add(result, result, curve_p);
01816         } while (carry < 0);
01817     }
01818     else
01819     {
01820         while (carry || vli_cmp(curve_p, result) != 1)
01821         {
01822             carry -= vli_sub(result, result, curve_p);
01823         }
01824     }
01825 }
01826 #endif /* uECC_WORD_SIZE */
01827
01828 #endif /* uECC_CURVE */
01829 #endif /* !asm_mmod_fast */
01830
01831 /* Computes result = (left * right) % curve_p. */
01832 static void vli_modMult_fast(uECC_word_t *result, const uECC_word_t *left,
01833                             const uECC_word_t *right)
01834 {
01835     uECC_word_t product[2 * uECC_WORDS];
01836     vli_mult(product, left, right);
01837     vli_mmod_fast(result, product);
01838 }
01839
01840 #if uECC_SQUARE_FUNC
01841
01842 /* Computes result = left^2 % curve_p. */
01843 static void vli_modSquare_fast(uECC_word_t *result, const uECC_word_t *left)
01844 {
01845     uECC_word_t product[2 * uECC_WORDS];
01846     vli_square(product, left);
01847     vli_mmod_fast(result, product);
01848 }
01849
01850 #else /* uECC_SQUARE_FUNC */
01851 #define vli_modSquare_fast(result, left) \
01852     vli_modMult_fast((result), (left), (left))
01853
01854 #endif /* uECC_SQUARE_FUNC */

```

```

01856
01857 #define EVEN(vli) (!(vli[0] & 1))
01858 /* Computes result = (1 / input) % mod. All VLIs are the same size.
01859    See "From Euclid's GCD to Montgomery Multiplication to the Great Divide"
01860    https://labs.oracle.com/techrep/2001/smlr_tr-2001-95.pdf */
01861 #if !asm_modInv
01862 static void vli_modInv(uECC_word_t *result, const uECC_word_t *input,
01863                      const uECC_word_t *mod)
01864 {
01865     uECC_word_t a[uECC_WORDS], b[uECC_WORDS], u[uECC_WORDS], v[uECC_WORDS];
01866     uECC_word_t carry;
01867     cmpresult_t cmpResult;
01868
01869     if (vli_isZero(input))
01870     {
01871         vli_clear(result);
01872         return;
01873     }
01874
01875     vli_set(a, input);
01876     vli_set(b, mod);
01877     vli_clear(u);
01878     u[0] = 1;
01879     vli_clear(v);
01880     while ((cmpResult = vli_cmp(a, b)) != 0)
01881     {
01882         carry = 0;
01883         if (EVEN(a))
01884         {
01885             vli_rshiftl(a);
01886             if (!EVEN(u))
01887             {
01888                 carry = vli_add(u, u, mod);
01889             }
01890             vli_rshiftl(u);
01891             if (carry)
01892             {
01893                 u[uECC_WORDS - 1] |= HIGH_BIT_SET;
01894             }
01895         }
01896         else if (EVEN(b))
01897         {
01898             vli_rshiftl(b);
01899             if (!EVEN(v))
01900             {
01901                 carry = vli_add(v, v, mod);
01902             }
01903             vli_rshiftl(v);
01904             if (carry)
01905             {
01906                 v[uECC_WORDS - 1] |= HIGH_BIT_SET;
01907             }
01908         }
01909         else if (cmpResult > 0)
01910         {
01911             vli_sub(a, a, b);
01912             vli_rshiftl(a);
01913             if (vli_cmp(u, v) < 0)
01914             {
01915                 vli_add(u, u, mod);
01916             }
01917             vli_sub(u, u, v);
01918             if (!EVEN(u))
01919             {
01920                 carry = vli_add(u, u, mod);
01921             }
01922             vli_rshiftl(u);
01923             if (carry)
01924             {
01925                 u[uECC_WORDS - 1] |= HIGH_BIT_SET;
01926             }
01927         }
01928         else
01929         {
01930             vli_sub(b, b, a);
01931             vli_rshiftl(b);
01932             if (vli_cmp(v, u) < 0)
01933             {
01934                 vli_add(v, v, mod);
01935             }
01936             vli_sub(v, v, u);
01937             if (!EVEN(v))
01938             {
01939                 carry = vli_add(v, v, mod);
01940             }
01941             vli_rshiftl(v);
01942             if (carry)

```

```

01943         {
01944             v[uECC_WORDS - 1] |= HIGH_BIT_SET;
01945         }
01946     }
01947 }
01948 vli_set(result, u);
01949 }
01950 #endif /* !asm_modInv */
01951
01952 /* ----- Point operations ----- */
01953
01954 /* Returns 1 if 'point' is the point at infinity, 0 otherwise. */
01955 static cmpresult_t EccPoint_isZero(const EccPoint *point)
01956 {
01957     return (vli_isZero(point->x) && vli_isZero(point->y));
01958 }
01959
01960 /* Point multiplication algorithm using Montgomery's ladder with co-Z
01961 coordinates. From http://eprint.iacr.org/2011/338.pdf
01962 */
01963
01964 /* Double in place */
01965 #if (uECC_CURVE == uECC_secp256k1)
01966 static void EccPoint_double_jacobian(uECC_word_t *RESTRICT X1,
01967                                     uECC_word_t *RESTRICT Y1,
01968                                     uECC_word_t *RESTRICT Z1)
01969 {
01970     /* t1 = X, t2 = Y, t3 = Z */
01971     uECC_word_t t4[uECC_WORDS];
01972     uECC_word_t t5[uECC_WORDS];
01973
01974     if (vli_isZero(Z1))
01975     {
01976         return;
01977     }
01978
01979     vli_modSquare_fast(t5, Y1); /* t5 = y1^2 */
01980     vli_modMult_fast(t4, X1, t5); /* t4 = x1*y1^2 = A */
01981     vli_modSquare_fast(X1, X1); /* t1 = x1^2 */
01982     vli_modSquare_fast(t5, t5); /* t5 = y1^4 */
01983     vli_modMult_fast(Z1, Y1, Z1); /* t3 = y1*z1 = z3 */
01984
01985     vli_modAdd(Y1, X1, X1, curve_p); /* t2 = 2*x1^2 */
01986     vli_modAdd(Y1, Y1, X1, curve_p); /* t2 = 3*x1^2 */
01987     if (vli_testBit(Y1, 0))
01988     {
01989         uECC_word_t carry = vli_add(Y1, Y1, curve_p);
01990         vli_rshift1(Y1);
01991         Y1[uECC_WORDS - 1] |= carry << (uECC_WORD_BITS - 1);
01992     }
01993     else
01994     {
01995         vli_rshift1(Y1);
01996     }
01997     /* t2 = 3/2*(x1^2) = B */
01998
01999     vli_modSquare_fast(X1, Y1); /* t1 = B^2 */
02000     vli_modSub(X1, X1, t4, curve_p); /* t1 = B^2 - A */
02001     vli_modSub(X1, X1, t4, curve_p); /* t1 = B^2 - 2A = x3 */
02002
02003     vli_modSub(t4, t4, X1, curve_p); /* t4 = A - x3 */
02004     vli_modMult_fast(Y1, Y1, t4); /* t2 = B * (A - x3) */
02005     vli_modSub(Y1, Y1, t5, curve_p); /* t2 = B * (A - x3) - y1^4 = y3 */
02006 }
02007 #else
02008 static void EccPoint_double_jacobian(uECC_word_t *RESTRICT X1,
02009                                     uECC_word_t *RESTRICT Y1,
02010                                     uECC_word_t *RESTRICT Z1)
02011 {
02012     /* t1 = X, t2 = Y, t3 = Z */
02013     uECC_word_t t4[uECC_WORDS];
02014     uECC_word_t t5[uECC_WORDS];
02015
02016     if (vli_isZero(Z1))
02017     {
02018         return;
02019     }
02020
02021     vli_modSquare_fast(t4, Y1); /* t4 = y1^2 */
02022     vli_modMult_fast(t5, X1, t4); /* t5 = x1*y1^2 = A */
02023     vli_modSquare_fast(t4, t4); /* t4 = y1^4 */
02024     vli_modMult_fast(Y1, Y1, Z1); /* t2 = y1*z1 = z3 */
02025     vli_modSquare_fast(Z1, Z1); /* t3 = z1^2 */
02026
02027     vli_modAdd(X1, X1, Z1, curve_p); /* t1 = x1 + z1^2 */
02028     vli_modAdd(Z1, Z1, Z1, curve_p); /* t3 = 2*z1^2 */
02029     vli_modSub_fast(Z1, X1, Z1); /* t3 = x1 - z1^2 */

```

```

02030     vli_modMult_fast(X1, X1, Z1);    /* t1 = x1^2 - z1^4 */
02031
02032     vli_modAdd(Z1, X1, X1, curve_p); /* t3 = 2*(x1^2 - z1^4) */
02033     vli_modAdd(X1, X1, Z1, curve_p); /* t1 = 3*(x1^2 - z1^4) */
02034     if (vli_testBit(X1, 0))
02035     {
02036         uECC_word_t l_carry = vli_add(X1, X1, curve_p);
02037         vli_rshift1(X1);
02038         X1[uECC_WORDS - 1] |= l_carry << (uECC_WORD_BITS - 1);
02039     }
02040     else
02041     {
02042         vli_rshift1(X1);
02043     }
02044     /* t1 = 3/2*(x1^2 - z1^4) = B */
02045
02046     vli_modSquare_fast(Z1, X1);    /* t3 = B^2 */
02047     vli_modSub_fast(Z1, Z1, t5);    /* t3 = B^2 - A */
02048     vli_modSub_fast(Z1, Z1, t5);    /* t3 = B^2 - 2A = x3 */
02049     vli_modSub_fast(t5, t5, Z1);    /* t5 = A - x3 */
02050     vli_modMult_fast(X1, X1, t5);    /* t1 = B * (A - x3) */
02051     vli_modSub_fast(t4, X1, t4);    /* t4 = B * (A - x3) - y1^4 = y3 */
02052
02053     vli_set(X1, Z1);
02054     vli_set(Z1, Y1);
02055     vli_set(Y1, t4);
02056 }
02057 #endif
02058
02059 /* Modify (x1, y1) => (x1 * z^2, y1 * z^3) */
02060 static void apply_z(uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1,
02061                    const uECC_word_t *RESTRICT Z)
02062 {
02063     uECC_word_t t1[uECC_WORDS];
02064
02065     vli_modSquare_fast(t1, Z);    /* z^2 */
02066     vli_modMult_fast(X1, X1, t1); /* x1 * z^2 */
02067     vli_modMult_fast(t1, t1, Z);  /* z^3 */
02068     vli_modMult_fast(Y1, Y1, t1); /* y1 * z^3 */
02069 }
02070
02071 /* P = (x1, y1) => 2P, (x2, y2) => P' */
02072 static void XYcZ_initial_double(uECC_word_t *RESTRICT X1,
02073                                 uECC_word_t *RESTRICT Y1,
02074                                 uECC_word_t *RESTRICT X2,
02075                                 uECC_word_t *RESTRICT Y2,
02076                                 const uECC_word_t *RESTRICT initial_Z)
02077 {
02078     uECC_word_t z[uECC_WORDS];
02079     if (initial_Z)
02080     {
02081         vli_set(z, initial_Z);
02082     }
02083     else
02084     {
02085         vli_clear(z);
02086         z[0] = 1;
02087     }
02088
02089     vli_set(X2, X1);
02090     vli_set(Y2, Y1);
02091
02092     apply_z(X1, Y1, z);
02093     EccPoint_double_jacobian(X1, Y1, z);
02094     apply_z(X2, Y2, z);
02095 }
02096
02097 /* Input P = (x1, y1, Z), Q = (x2, y2, Z)
02098    Output P' = (x1', y1', Z3), P + Q = (x3, y3, Z3)
02099    or P => P', Q => P + Q
02100 */
02101 static void XYcZ_add(uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1,
02102                     uECC_word_t *RESTRICT X2, uECC_word_t *RESTRICT Y2)
02103 {
02104     /* t1 = X1, t2 = Y1, t3 = X2, t4 = Y2 */
02105     uECC_word_t t5[uECC_WORDS];
02106
02107     vli_modSub_fast(t5, X2, X1); /* t5 = x2 - x1 */
02108     vli_modSquare_fast(t5, t5);  /* t5 = (x2 - x1)^2 = A */
02109     vli_modMult_fast(X1, X1, t5); /* t1 = x1*A = B */
02110     vli_modMult_fast(X2, X2, t5); /* t3 = x2*A = C */
02111     vli_modSub_fast(Y2, Y2, Y1); /* t4 = y2 - y1 */
02112     vli_modSquare_fast(t5, Y2);  /* t5 = (y2 - y1)^2 = D */
02113
02114     vli_modSub_fast(t5, t5, X1); /* t5 = D - B */
02115     vli_modSub_fast(t5, t5, X2); /* t5 = D - B - C = x3 */
02116     vli_modSub_fast(X2, X2, X1); /* t3 = C - B */

```

```

02117     vli_modMult_fast(Y1, Y1, X2); /* t2 = y1*(C - B) */
02118     vli_modSub_fast(X2, X1, t5); /* t3 = B - x3 */
02119     vli_modMult_fast(Y2, Y2, X2); /* t4 = (y2 - y1)*(B - x3) */
02120     vli_modSub_fast(Y2, Y2, Y1); /* t4 = y3 */
02121
02122     vli_set(X2, t5);
02123 }
02124
02125 /* Input P = (x1, y1, Z), Q = (x2, y2, Z)
02126    Output P + Q = (x3, y3, Z3), P - Q = (x3', y3', Z3)
02127    or P => P - Q, Q => P + Q
02128 */
02129 static void XYcZ_addC(uECC_word_t *RESTRICT X1, uECC_word_t *RESTRICT Y1,
02130                      uECC_word_t *RESTRICT X2, uECC_word_t *RESTRICT Y2)
02131 {
02132     /* t1 = X1, t2 = Y1, t3 = X2, t4 = Y2 */
02133     uECC_word_t t5[uECC_WORDS];
02134     uECC_word_t t6[uECC_WORDS];
02135     uECC_word_t t7[uECC_WORDS];
02136
02137     vli_modSub_fast(t5, X2, X1); /* t5 = x2 - x1 */
02138     vli_modSquare_fast(t5, t5); /* t5 = (x2 - x1)^2 = A */
02139     vli_modMult_fast(X1, X1, t5); /* t1 = x1*A = B */
02140     vli_modMult_fast(X2, X2, t5); /* t3 = x2*A = C */
02141     vli_modAdd(t5, Y2, Y1, curve_p); /* t5 = y2 + y1 */
02142     vli_modSub_fast(Y2, Y2, Y1); /* t4 = y2 - y1 */
02143
02144     vli_modSub_fast(t6, X2, X1); /* t6 = C - B */
02145     vli_modMult_fast(Y1, Y1, t6); /* t2 = y1 * (C - B) = E */
02146     vli_modAdd(t6, X1, X2, curve_p); /* t6 = B + C */
02147     vli_modSquare_fast(X2, Y2); /* t3 = (y2 - y1)^2 = D */
02148     vli_modSub_fast(X2, X2, t6); /* t3 = D - (B + C) = x3 */
02149
02150     vli_modSub_fast(t7, X1, X2); /* t7 = B - x3 */
02151     vli_modMult_fast(Y2, Y2, t7); /* t4 = (y2 - y1)*(B - x3) */
02152     vli_modSub_fast(Y2, Y2, Y1); /* t4 = (y2 - y1)*(B - x3) - E = y3 */
02153
02154     vli_modSquare_fast(t7, t5); /* t7 = (y2 + y1)^2 = F */
02155     vli_modSub_fast(t7, t7, t6); /* t7 = F - (B + C) = x3' */
02156     vli_modSub_fast(t6, t7, X1); /* t6 = x3' - B */
02157     vli_modMult_fast(t6, t6, t5); /* t6 = (y2 + y1)*(x3' - B) */
02158     vli_modSub_fast(Y1, t6, Y1); /* t2 = (y2 + y1)*(x3' - B) - E = y3' */
02159
02160     vli_set(X1, t7);
02161 }
02162
02163 static void EccPoint_mult(EccPoint *RESTRICT result,
02164                           const EccPoint *RESTRICT point,
02165                           const uECC_word_t *RESTRICT scalar,
02166                           const uECC_word_t *RESTRICT initialZ,
02167                           bitcount_t numBits)
02168 {
02169     /* R0 and R1 */
02170     uECC_word_t Rx[2][uECC_WORDS];
02171     uECC_word_t Ry[2][uECC_WORDS];
02172     uECC_word_t z[uECC_WORDS];
02173     bitcount_t i;
02174     uECC_word_t nb;
02175
02176     vli_set(Rx[1], point->x);
02177     vli_set(Ry[1], point->y);
02178
02179     XYcZ_initial_double(Rx[1], Ry[1], Rx[0], Ry[0], initialZ);
02180
02181     for (i = numBits - 2; i > 0; --i)
02182     {
02183         nb = !vli_testBit(scalar, i);
02184         XYcZ_addC(Rx[1 - nb], Ry[1 - nb], Rx[nb], Ry[nb]);
02185         XYcZ_add(Rx[nb], Ry[nb], Rx[1 - nb], Ry[1 - nb]);
02186     }
02187
02188     nb = !vli_testBit(scalar, 0);
02189     XYcZ_addC(Rx[1 - nb], Ry[1 - nb], Rx[nb], Ry[nb]);
02190
02191     /* Find final 1/Z value. */
02192     vli_modSub_fast(z, Rx[1], Rx[0]); /* X1 - X0 */
02193     vli_modMult_fast(z, z, Ry[1 - nb]); /* Yb * (X1 - X0) */
02194     vli_modMult_fast(z, z, point->x); /* xP * Yb * (X1 - X0) */
02195     vli_modInv(z, z, curve_p); /* 1 / (xP * Yb * (X1 - X0)) */
02196     vli_modMult_fast(z, z, point->y); /* yP / (xP * Yb * (X1 - X0)) */
02197     vli_modMult_fast(z, z, Rx[1 - nb]); /* Xb * yP / (xP * Yb * (X1 - X0)) */
02198     /* End 1/Z calculation */
02199
02200     XYcZ_add(Rx[nb], Ry[nb], Rx[1 - nb], Ry[1 - nb]);
02201     apply_z(Rx[0], Ry[0], z);
02202
02203     vli_set(result->x, Rx[0]);

```

```

02204     vli_set(result->y, Ry[0]);
02205 }
02206
02207 #if uECC_CURVE == uECC_secp224r1
02208
02209 /* Routine 3.2.4 RS; from http://www.nsa.gov/ia/_files/nist-routines.pdf */
02210 static void mod_sqrt_secp224r1_rs(uECC_word_t *d1, uECC_word_t *e1,
02211     uECC_word_t *f1, const uECC_word_t *d0,
02212     const uECC_word_t *e0, const uECC_word_t *f0)
02213 {
02214     uECC_word_t t[uECC_WORDS];
02215
02216     vli_modSquare_fast(t, d0); /* t <-- d0 ^ 2 */
02217     vli_modMult_fast(e1, d0, e0); /* e1 <-- d0 * e0 */
02218     vli_modAdd(d1, t, f0, curve_p); /* d1 <-- t + f0 */
02219     vli_modAdd(e1, e1, e1, curve_p); /* e1 <-- e1 + e1 */
02220     vli_modMult_fast(f1, t, f0); /* f1 <-- t * f0 */
02221     vli_modAdd(f1, f1, f1, curve_p); /* f1 <-- f1 + f1 */
02222     vli_modAdd(f1, f1, f1, curve_p); /* f1 <-- f1 + f1 */
02223 }
02224
02225 /* Routine 3.2.5 RSS; from http://www.nsa.gov/ia/_files/nist-routines.pdf */
02226 */
02227 static void mod_sqrt_secp224r1_rss(uECC_word_t *d1, uECC_word_t *e1,
02228     uECC_word_t *f1, const uECC_word_t *d0,
02229     const uECC_word_t *e0, const uECC_word_t *f0,
02230     const bitcount_t j)
02231 {
02232     bitcount_t i;
02233
02234     vli_set(d1, d0); /* d1 <-- d0 */
02235     vli_set(e1, e0); /* e1 <-- e0 */
02236     vli_set(f1, f0); /* f1 <-- f0 */
02237     for (i = 1; i <= j; i++)
02238     {
02239         mod_sqrt_secp224r1_rs(d1, e1, f1, d1, e1,
02240             f1); /* RS (d1,e1,f1,d1,e1,f1) */
02241     }
02242 }
02243
02244 /* Routine 3.2.6 RM; from http://www.nsa.gov/ia/_files/nist-routines.pdf */
02245 static void mod_sqrt_secp224r1_rm(uECC_word_t *d2, uECC_word_t *e2,
02246     uECC_word_t *f2, const uECC_word_t *c,
02247     const uECC_word_t *d0, const uECC_word_t *e0,
02248     const uECC_word_t *d1, const uECC_word_t *e1)
02249 {
02250     uECC_word_t t1[uECC_WORDS];
02251     uECC_word_t t2[uECC_WORDS];
02252
02253     vli_modMult_fast(t1, e0, e1); /* t1 <-- e0 * e1 */
02254     vli_modMult_fast(t1, t1, c); /* t1 <-- t1 * c */
02255     vli_modSub_fast(t1, curve_p, t1); /* t1 <-- p - t1 */
02256     vli_modMult_fast(t2, d0, d1); /* t2 <-- d0 * d1 */
02257     vli_modAdd(t2, t2, t1, curve_p); /* t2 <-- t2 + t1 */
02258     vli_modMult_fast(t1, d0, e1); /* t1 <-- d0 * e1 */
02259     vli_modMult_fast(e2, d1, e0); /* e2 <-- d1 * e0 */
02260     vli_modAdd(e2, e2, t1, curve_p); /* e2 <-- e2 + t1 */
02261     vli_modSquare_fast(f2, e2); /* f2 <-- e2^2 */
02262     vli_modMult_fast(f2, f2, c); /* f2 <-- f2 * c */
02263     vli_modSub_fast(f2, curve_p, f2); /* f2 <-- p - f2 */
02264     vli_set(d2, t2); /* d2 <-- t2 */
02265 }
02266
02267 /* Routine 3.2.7 RP; from http://www.nsa.gov/ia/_files/nist-routines.pdf */
02268 static void mod_sqrt_secp224r1_rp(uECC_word_t *d1, uECC_word_t *e1,
02269     uECC_word_t *f1, const uECC_word_t *c,
02270     const uECC_word_t *r)
02271 {
02272     wordcount_t i;
02273     wordcount_t pow2i = 1;
02274     uECC_word_t d0[uECC_WORDS];
02275     uECC_word_t e0[uECC_WORDS] = {1}; /* e0 <-- 1 */
02276     uECC_word_t f0[uECC_WORDS];
02277
02278     vli_set(d0, r); /* d0 <-- r */
02279     vli_modSub_fast(f0, curve_p, c); /* f0 <-- p - c */
02280     for (i = 0; i <= 6; i++)
02281     {
02282         mod_sqrt_secp224r1_rss(d1, e1, f1, d0, e0, f0,
02283             pow2i); /* RSS (d1,e1,f1,d0,e0,f0,2^i) */
02284         mod_sqrt_secp224r1_rm(d1, e1, f1, c, d1, e1, d0,
02285             e0); /* RM (d1,e1,f1,c,d1,e1,d0,e0) */
02286         vli_set(d0, d1); /* d0 <-- d1 */
02287         vli_set(e0, e1); /* e0 <-- e1 */
02288         vli_set(f0, f1); /* f0 <-- f1 */
02289         pow2i *= 2;
02290     }

```

```

02291 }
02292
02293 /* Compute a = sqrt(a) (mod curve_p). */
02294 /* Routine 3.2.8 mp_mod_sqrt_224; from
02295  * http://www.nsa.gov/ia/\_files/nist-routines.pdf */
02296 static void mod_sqrt(uECC_word_t *a)
02297 {
02298     bitcount_t i;
02299     uECC_word_t e1[uECC_WORDS];
02300     uECC_word_t f1[uECC_WORDS];
02301     uECC_word_t d0[uECC_WORDS];
02302     uECC_word_t e0[uECC_WORDS];
02303     uECC_word_t f0[uECC_WORDS];
02304     uECC_word_t d1[uECC_WORDS];
02305
02306     // s = a; using constant instead of random value
02307     mod_sqrt_secp224r1_rp(d0, e0, f0, a, a); /* RP (d0, e0, f0, c, s) */
02308     mod_sqrt_secp224r1_rs(d1, e1, f1, d0, e0,
02309                          f0); /* RS (d1, e1, f1, d0, e0, f0) */
02310     for (i = 1; i <= 95; i++)
02311     {
02312         vli_set(d0, d1); /* d0 <-- d1 */
02313         vli_set(e0, e1); /* e0 <-- e1 */
02314         vli_set(f0, f1); /* f0 <-- f1 */
02315         mod_sqrt_secp224r1_rs(d1, e1, f1, d0, e0,
02316                              f0); /* RS (d1, e1, f1, d0, e0, f0) */
02317         if (vli_isZero(d1))
02318             { /* if d1 == 0 */
02319                 break;
02320             }
02321     }
02322     vli_modInv(f1, e0, curve_p); /* f1 <-- 1 / e0 */
02323     vli_modMult_fast(a, d0, f1); /* a <-- d0 / e0 */
02324 }
02325
02326 #else /* uECC_CURVE */
02327
02328 /* Compute a = sqrt(a) (mod curve_p). */
02329 static void mod_sqrt(uECC_word_t *a)
02330 {
02331     bitcount_t i;
02332     uECC_word_t p1[uECC_WORDS] = {1};
02333     uECC_word_t l_result[uECC_WORDS] = {1};
02334
02335     /* Since curve_p == 3 (mod 4) for all supported curves, we can
02336      compute sqrt(a) = a^((curve_p + 1) / 4) (mod curve_p). */
02337     vli_add(p1, curve_p, p1); /* p1 = curve_p + 1 */
02338     for (i = vli_numBits(p1, uECC_WORDS) - 1; i > 1; --i)
02339     {
02340         vli_modSquare_fast(l_result, l_result);
02341         if (vli_testBit(p1, i))
02342             {
02343                 vli_modMult_fast(l_result, l_result, a);
02344             }
02345     }
02346     vli_set(a, l_result);
02347 }
02348 #endif /* uECC_CURVE */
02349
02350 #if uECC_WORD_SIZE == 1
02351
02352 static void vli_nativeToBytes(uint8_t *RESTRICT dest,
02353                               const uint8_t *RESTRICT src)
02354 {
02355     uint8_t i;
02356     for (i = 0; i < uECC_BYTES; ++i)
02357     {
02358         dest[i] = src[(uECC_BYTES - 1) - i];
02359     }
02360 }
02361
02362 #define vli_bytesToNative(dest, src) vli_nativeToBytes((dest), (src))
02363
02364 #elif uECC_WORD_SIZE == 4
02365
02366 static void vli_nativeToBytes(uint8_t *bytes, const uint32_t *native)
02367 {
02368     unsigned i;
02369     for (i = 0; i < uECC_WORDS; ++i)
02370     {
02371         uint8_t *digit = bytes + 4 * (uECC_WORDS - 1 - i);
02372         digit[0] = native[i] >> 24;
02373         digit[1] = native[i] >> 16;
02374         digit[2] = native[i] >> 8;
02375         digit[3] = native[i];
02376     }
02377 }

```



```

02378
02379 static void vli_bytesToNative(uint32_t *native, const uint8_t *bytes)
02380 {
02381     unsigned i;
02382     for (i = 0; i < uECC_WORDS; ++i)
02383     {
02384         const uint8_t *digit = bytes + 4 * (uECC_WORDS - 1 - i);
02385         native[i] = ((uint32_t) digit[0] << 24) | ((uint32_t) digit[1] << 16) |
02386             ((uint32_t) digit[2] << 8) | (uint32_t) digit[3];
02387     }
02388 }
02389
02390 #else
02391
02392 static void vli_nativeToBytes(uint8_t *bytes, const uint64_t *native)
02393 {
02394     unsigned i;
02395     for (i = 0; i < uECC_WORDS; ++i)
02396     {
02397         uint8_t *digit = bytes + 8 * (uECC_WORDS - 1 - i);
02398         digit[0] = native[i] >> 56;
02399         digit[1] = native[i] >> 48;
02400         digit[2] = native[i] >> 40;
02401         digit[3] = native[i] >> 32;
02402         digit[4] = native[i] >> 24;
02403         digit[5] = native[i] >> 16;
02404         digit[6] = native[i] >> 8;
02405         digit[7] = native[i];
02406     }
02407 }
02408
02409 static void vli_bytesToNative(uint64_t *native, const uint8_t *bytes)
02410 {
02411     unsigned i;
02412     for (i = 0; i < uECC_WORDS; ++i)
02413     {
02414         const uint8_t *digit = bytes + 8 * (uECC_WORDS - 1 - i);
02415         native[i] = ((uint64_t) digit[0] << 56) | ((uint64_t) digit[1] << 48) |
02416             ((uint64_t) digit[2] << 40) | ((uint64_t) digit[3] << 32) |
02417             ((uint64_t) digit[4] << 24) | ((uint64_t) digit[5] << 16) |
02418             ((uint64_t) digit[6] << 8) | (uint64_t) digit[7];
02419     }
02420 }
02421
02422 #endif /* uECC_WORD_SIZE */
02423
02424 int uECC_make_key(uint8_t public_key[uECC_BYTES * 2],
02425     uint8_t private_key[uECC_BYTES])
02426 {
02427     uECC_word_t private[uECC_WORDS];
02428     EccPoint public;
02429     uECC_word_t tries;
02430     for (tries = 0; tries < MAX_TRIES; ++tries)
02431     {
02432         if (g_rng_function((uint8_t *) private, sizeof(private)) &&
02433             EccPoint_compute_public_key(&public, private))
02434         {
02435             vli_nativeToBytes(private_key, private);
02436             vli_nativeToBytes(public_key, public.x);
02437             vli_nativeToBytes(public_key + uECC_BYTES, public.y);
02438             if (!uECC_valid_public_key(public_key))
02439             {
02440                 return 0;
02441             }
02442             return 1;
02443         }
02444     }
02445     return 0;
02446 }
02447
02448 void uECC_compress(const uint8_t public_key[uECC_BYTES * 2],
02449     uint8_t compressed[uECC_BYTES + 1])
02450 {
02451     wordcount_t i;
02452     for (i = 0; i < uECC_BYTES; ++i)
02453     {
02454         compressed[i + 1] = public_key[i];
02455     }
02456     compressed[0] = 2 + (public_key[uECC_BYTES * 2 - 1] & 0x01);
02457 }
02458
02459 /* Computes result = x^3 + ax + b. result must not overlap x. */
02460 static void curve_x_side(uECC_word_t *RESTRICT result,
02461     const uECC_word_t *RESTRICT x)
02462 {
02463     #if (uECC_CURVE == uECC_secp256k1)
02464         vli_modSquare_fast(result, x);          /* r = x^2 */

```

```

02465     vli_modMult_fast(result, result, x);          /* r = x^3 */
02466     vli_modAdd(result, result, curve_b, curve_p); /* r = x^3 + b */
02467 #else
02468     uECC_word_t _3[uECC_WORDS] = {3}; /* -a = 3 */
02469
02470     vli_modSquare_fast(result, x);          /* r = x^2 */
02471     vli_modSub_fast(result, result, _3);    /* r = x^2 - 3 */
02472     vli_modMult_fast(result, result, x);    /* r = x^3 - 3x */
02473     vli_modAdd(result, result, curve_b, curve_p); /* r = x^3 - 3x + b */
02474 #endif
02475 }
02476
02477 void uECC_decompress(const uint8_t compressed[uECC_BYTES + 1],
02478                     uint8_t public_key[uECC_BYTES * 2])
02479 {
02480     EccPoint point;
02481     vli_bytesToNative(point.x, compressed + 1);
02482     curve_x_side(point.y, point.x);
02483     mod_sqrt(point.y);
02484
02485     if ((point.y[0] & 0x01) != (compressed[0] & 0x01))
02486     {
02487         vli_sub(point.y, curve_p, point.y);
02488     }
02489
02490     vli_nativeToBytes(public_key, point.x);
02491     vli_nativeToBytes(public_key + uECC_BYTES, point.y);
02492 }
02493
02494 int uECC_valid_public_key(const uint8_t public_key[uECC_BYTES * 2])
02495 {
02496     uECC_word_t tmp1[uECC_WORDS];
02497     uECC_word_t tmp2[uECC_WORDS];
02498     EccPoint public;
02499
02500     vli_bytesToNative(public.x, public_key);
02501     vli_bytesToNative(public.y, public_key + uECC_BYTES);
02502
02503     // The point at infinity is invalid.
02504     if (EccPoint_isZero(&public))
02505     {
02506         return 0;
02507     }
02508
02509     // x and y must be smaller than p.
02510     if (vli_cmp(curve_p, public.x) != 1 || vli_cmp(curve_p, public.y) != 1)
02511     {
02512         return 0;
02513     }
02514
02515     vli_modSquare_fast(tmp1, public.y); /* tmp1 = y^2 */
02516     curve_x_side(tmp2, public.x);      /* tmp2 = x^3 + ax + b */
02517
02518     /* Make sure that y^2 == x^3 + ax + b */
02519     return (vli_cmp(tmp1, tmp2) == 0);
02520 }
02521
02522 int uECC_compute_public_key(const uint8_t private_key[uECC_BYTES],
02523                             uint8_t public_key[uECC_BYTES * 2])
02524 {
02525     uECC_word_t private[uECC_N_WORDS];
02526     EccPoint public;
02527
02528     vli_clear(private + uECC_WORDS);
02529     vli_bytesToNative(private, private_key);
02530
02531     if (!EccPoint_compute_public_key(&public, private))
02532     {
02533         return 0;
02534     }
02535
02536     vli_nativeToBytes(public_key, public.x);
02537     vli_nativeToBytes(public_key + uECC_BYTES, public.y);
02538     return 1;
02539 }
02540
02541 int uECC_bytes(void)
02542 {
02543     return uECC_BYTES;
02544 }
02545
02546 int uECC_curve(void)
02547 {
02548     return uECC_CURVE;
02549 }
02550
02551 /* ----- ECDSA code ----- */

```

```

02552
02553 #if (uECC_CURVE == uECC_secp160r1)
02554 static void vli_clear_n(uECC_word_t *vli)
02555 {
02556     vli_clear(vli);
02557     vli[uECC_N_WORDS - 1] = 0;
02558 }
02559
02560 static uECC_word_t vli_isZero_n(const uECC_word_t *vli)
02561 {
02562     if (vli[uECC_N_WORDS - 1])
02563     {
02564         return 0;
02565     }
02566     return vli_isZero(vli);
02567 }
02568
02569 static void vli_set_n(uECC_word_t *dest, const uECC_word_t *src)
02570 {
02571     vli_set(dest, src);
02572     dest[uECC_N_WORDS - 1] = src[uECC_N_WORDS - 1];
02573 }
02574
02575 static cmpresult_t vli_cmp_n(const uECC_word_t *left, const uECC_word_t *right)
02576 {
02577     if (left[uECC_N_WORDS - 1] > right[uECC_N_WORDS - 1])
02578     {
02579         return 1;
02580     }
02581     else if (left[uECC_N_WORDS - 1] < right[uECC_N_WORDS - 1])
02582     {
02583         return -1;
02584     }
02585     return vli_cmp(left, right);
02586 }
02587
02588 static void vli_rshift1_n(uECC_word_t *vli)
02589 {
02590     vli_rshift1(vli);
02591     vli[uECC_N_WORDS - 2] |= vli[uECC_N_WORDS - 1] << (uECC_WORD_BITS - 1);
02592     vli[uECC_N_WORDS - 1] = vli[uECC_N_WORDS - 1] >> 1;
02593 }
02594
02595 static uECC_word_t vli_add_n(uECC_word_t *result, const uECC_word_t *left,
02596                             const uECC_word_t *right)
02597 {
02598     uECC_word_t carry = vli_add(result, left, right);
02599     uECC_word_t sum = left[uECC_N_WORDS - 1] + right[uECC_N_WORDS - 1] + carry;
02600     if (sum != left[uECC_N_WORDS - 1])
02601     {
02602         carry = (sum < left[uECC_N_WORDS - 1]);
02603     }
02604     result[uECC_N_WORDS - 1] = sum;
02605     return carry;
02606 }
02607
02608 static uECC_word_t vli_sub_n(uECC_word_t *result, const uECC_word_t *left,
02609                             const uECC_word_t *right)
02610 {
02611     uECC_word_t borrow = vli_sub(result, left, right);
02612     uECC_word_t diff =
02613         left[uECC_N_WORDS - 1] - right[uECC_N_WORDS - 1] - borrow;
02614     if (diff != left[uECC_N_WORDS - 1])
02615     {
02616         borrow = (diff > left[uECC_N_WORDS - 1]);
02617     }
02618     result[uECC_N_WORDS - 1] = diff;
02619     return borrow;
02620 }
02621
02622 #if !muladd_exists
02623 static void muladd(uECC_word_t a, uECC_word_t b, uECC_word_t *r0,
02624                  uECC_word_t *r1, uECC_word_t *r2)
02625 {
02626     uECC_dword_t p = (uECC_dword_t) a * b;
02627     uECC_dword_t r01 = ((uECC_dword_t) (*r1) << uECC_WORD_BITS) | *r0;
02628     r01 += p;
02629     *r2 += (r01 < p);
02630     *r1 = r01 >> uECC_WORD_BITS;
02631     *r0 = (uECC_word_t) r01;
02632 }
02633 #define muladd_exists 1
02634 #endif
02635
02636 static void vli_mult_n(uECC_word_t *result, const uECC_word_t *left,
02637                      const uECC_word_t *right)
02638 {

```

```

02639     uECC_word_t r0 = 0;
02640     uECC_word_t r1 = 0;
02641     uECC_word_t r2 = 0;
02642     wordcount_t i, k;
02643
02644     for (k = 0; k < uECC_N_WORDS * 2 - 1; ++k)
02645     {
02646         wordcount_t min = (k < uECC_N_WORDS ? 0 : (k + 1) - uECC_N_WORDS);
02647         wordcount_t max = (k < uECC_N_WORDS ? k : uECC_N_WORDS - 1);
02648         for (i = min; i <= max; ++i)
02649         {
02650             muladd(left[i], right[k - i], &r0, &r1, &r2);
02651         }
02652         result[k] = r0;
02653         r0 = r1;
02654         r1 = r2;
02655         r2 = 0;
02656     }
02657     result[uECC_N_WORDS * 2 - 1] = r0;
02658 }
02659
02660 static void vli_modAdd_n(uECC_word_t *result, const uECC_word_t *left,
02661                          const uECC_word_t *right, const uECC_word_t *mod)
02662 {
02663     uECC_word_t carry = vli_add_n(result, left, right);
02664     if (carry || vli_cmp_n(result, mod) >= 0)
02665     {
02666         vli_sub_n(result, result, mod);
02667     }
02668 }
02669
02670 static void vli_modInv_n(uECC_word_t *result, const uECC_word_t *input,
02671                          const uECC_word_t *mod)
02672 {
02673     uECC_word_t a[uECC_N_WORDS], b[uECC_N_WORDS], u[uECC_N_WORDS],
02674                 v[uECC_N_WORDS];
02675     uECC_word_t carry;
02676     cmpresult_t cmpResult;
02677
02678     if (vli_isZero_n(input))
02679     {
02680         vli_clear_n(result);
02681         return;
02682     }
02683
02684     vli_set_n(a, input);
02685     vli_set_n(b, mod);
02686     vli_clear_n(u);
02687     u[0] = 1;
02688     vli_clear_n(v);
02689     while ((cmpResult = vli_cmp_n(a, b)) != 0)
02690     {
02691         carry = 0;
02692         if (EVEN(a))
02693         {
02694             vli_rshiftl_n(a);
02695             if (!EVEN(u))
02696             {
02697                 carry = vli_add_n(u, u, mod);
02698             }
02699             vli_rshiftl_n(u);
02700             if (carry)
02701             {
02702                 u[uECC_N_WORDS - 1] |= HIGH_BIT_SET;
02703             }
02704         }
02705         else if (EVEN(b))
02706         {
02707             vli_rshiftl_n(b);
02708             if (!EVEN(v))
02709             {
02710                 carry = vli_add_n(v, v, mod);
02711             }
02712             vli_rshiftl_n(v);
02713             if (carry)
02714             {
02715                 v[uECC_N_WORDS - 1] |= HIGH_BIT_SET;
02716             }
02717         }
02718         else if (cmpResult > 0)
02719         {
02720             vli_sub_n(a, a, b);
02721             vli_rshiftl_n(a);
02722             if (vli_cmp_n(u, v) < 0)
02723             {
02724                 vli_add_n(u, u, mod);
02725             }

```

```

02726         vli_sub_n(u, u, v);
02727         if (!EVEN(u))
02728         {
02729             carry = vli_add_n(u, u, mod);
02730         }
02731         vli_rshiftl_n(u);
02732         if (carry)
02733         {
02734             u[uECC_N_WORDS - 1] |= HIGH_BIT_SET;
02735         }
02736     }
02737     else
02738     {
02739         vli_sub_n(b, b, a);
02740         vli_rshiftl_n(b);
02741         if (vli_cmp_n(v, u) < 0)
02742         {
02743             vli_add_n(v, v, mod);
02744         }
02745         vli_sub_n(v, v, u);
02746         if (!EVEN(v))
02747         {
02748             carry = vli_add_n(v, v, mod);
02749         }
02750         vli_rshiftl_n(v);
02751         if (carry)
02752         {
02753             v[uECC_N_WORDS - 1] |= HIGH_BIT_SET;
02754         }
02755     }
02756 }
02757 vli_set_n(result, u);
02758 }
02759
02760 static void vli2_rshiftl_n(uECC_word_t *vli)
02761 {
02762     vli_rshiftl_n(vli);
02763     vli[uECC_N_WORDS - 1] |= vli[uECC_N_WORDS] << (uECC_WORD_BITS - 1);
02764     vli_rshiftl_n(vli + uECC_N_WORDS);
02765 }
02766
02767 static uECC_word_t vli2_sub_n(uECC_word_t *result, const uECC_word_t *left,
02768                               const uECC_word_t *right)
02769 {
02770     uECC_word_t borrow = 0;
02771     wordcount_t i;
02772     for (i = 0; i < uECC_N_WORDS * 2; ++i)
02773     {
02774         uECC_word_t diff = left[i] - right[i] - borrow;
02775         if (diff != left[i])
02776         {
02777             borrow = (diff > left[i]);
02778         }
02779         result[i] = diff;
02780     }
02781     return borrow;
02782 }
02783
02784 /* Computes result = (left * right) % curve_n. */
02785 static void vli_modMult_n(uECC_word_t *result, const uECC_word_t *left,
02786                           const uECC_word_t *right)
02787 {
02788     bitcount_t i;
02789     uECC_word_t product[2 * uECC_N_WORDS];
02790     uECC_word_t modMultiple[2 * uECC_N_WORDS];
02791     uECC_word_t tmp[2 * uECC_N_WORDS];
02792     uECC_word_t *v[2] = {tmp, product};
02793     uECC_word_t index = 1;
02794
02795     vli_mult_n(product, left, right);
02796     vli_clear_n(modMultiple);
02797     vli_set(modMultiple + uECC_N_WORDS + 1, curve_n);
02798     vli_rshiftl(modMultiple + uECC_N_WORDS + 1);
02799     modMultiple[2 * uECC_N_WORDS - 1] |= HIGH_BIT_SET;
02800     modMultiple[uECC_N_WORDS] = HIGH_BIT_SET;
02801
02802     for (i = 0; i <= (((bitcount_t) uECC_N_WORDS) << uECC_WORD_BITS_SHIFT) +
02803                    (uECC_WORD_BITS - 1));
02804         ++i)
02805     {
02806         uECC_word_t borrow = vli2_sub_n(v[1 - index], v[index], modMultiple);
02807         index = !(index ^ borrow); /* Swap the index if there was no borrow */
02808         vli2_rshiftl_n(modMultiple);
02809     }
02810     vli_set_n(result, v[index]);
02811 }
02812

```

```

02813 #else
02814
02815 static void vli2_rshift1_n(uECC_word_t *vli)
02816 {
02817     vli_rshift1(vli);
02818     vli[uECC_WORDS - 1] |= vli[uECC_WORDS] << (uECC_WORD_BITS - 1);
02819     vli_rshift1(vli + uECC_WORDS);
02820 }
02821
02822 static uECC_word_t vli2_sub_n(uECC_word_t *result, const uECC_word_t *left,
02823                               const uECC_word_t *right)
02824 {
02825     uECC_word_t borrow = 0;
02826     wordcount_t i;
02827     for (i = 0; i < uECC_WORDS * 2; ++i)
02828     {
02829         uECC_word_t diff = left[i] - right[i] - borrow;
02830         if (diff != left[i])
02831         {
02832             borrow = (diff > left[i]);
02833         }
02834         result[i] = diff;
02835     }
02836     return borrow;
02837 }
02838
02839 static void vli_modMult_generic(uECC_word_t *result, const uECC_word_t *left,
02840                                const uECC_word_t *right)
02841 {
02842     uECC_word_t product[2 * uECC_WORDS];
02843     uECC_word_t modMultiple[2 * uECC_WORDS];
02844     uECC_word_t tmp[2 * uECC_WORDS];
02845     uECC_word_t *v[2] = {tmp, product};
02846     bitcount_t i;
02847     uECC_word_t index = 1;
02848
02849     vli_mult(product, left, right);
02850     vli_set(modMultiple + uECC_WORDS, curve_n);
02851     vli_clear(modMultiple);
02852
02853     for (i = 0; i <= uECC_BYTES * 8; ++i)
02854     {
02855         uECC_word_t borrow = vli2_sub_n(v[1 - index], v[index], modMultiple);
02856         index = !(index ^ borrow);
02857         vli2_rshift1_n(modMultiple);
02858     }
02859     vli_set(result, v[index]);
02860 }
02861
02862 #endif /* (uECC_CURVE != uECC_secp160r1) */
02863
02864 static void vli_blindScalar(uECC_word_t *result, const uECC_word_t *scalar)
02865 {
02866     uECC_word_t r;
02867     uECC_word_t tries;
02868     for (tries = 0; tries < MAX_TRIES; ++tries)
02869     {
02870         if (g_rng_function((uint8_t *) &r, sizeof(r)) && r != 0)
02871         {
02872             break;
02873         }
02874     }
02875     if (tries == MAX_TRIES)
02876     {
02877         r = 1;
02878     }
02879
02880     uECC_dword_t carry = 0;
02881     wordcount_t i;
02882     for (i = 0; i < uECC_N_WORDS; ++i)
02883     {
02884         uECC_dword_t p = (uECC_dword_t) r * curve_n[i] + scalar[i] + carry;
02885         result[i] = (uECC_word_t) p;
02886         carry = p >> uECC_WORD_BITS;
02887     }
02888     result[uECC_N_WORDS] = (uECC_word_t) carry;
02889 }
02890
02891 static int EccPoint_compute_public_key(EccPoint *result, uECC_word_t *private)
02892 {
02893     uECC_word_t blinded[uECC_N_WORDS + 1];
02894     uECC_word_t random_Z[uECC_WORDS];
02895     uECC_word_t *initial_Z = 0;
02896     uECC_word_t scalar_n[uECC_N_WORDS];
02897
02898     vli_clear_n(scalar_n);
02899     vli_set(scalar_n, private);

```

```

02900
02901     if (vli_isZero(scalar_n) || vli_cmp_n(curve_n, scalar_n) != 1)
02902     {
02903         return 0;
02904     }
02905
02906     if (g_rng_function((uint8_t *) random_Z, sizeof(random_Z)) &&
02907         !vli_isZero(random_Z))
02908     {
02909         initial_Z = random_Z;
02910     }
02911
02912     vli_blindScalar(blinded, scalar_n);
02913     EccPoint_mult(result, &curve_G, blinded, initial_Z,
02914                   vli_numBits(blinded, uECC_N_WORDS + 1));
02915
02916     if (EccPoint_isZero(result))
02917     {
02918         return 0;
02919     }
02920     return 1;
02921 }
02922
02923 int uECC_shared_secret(const uint8_t public_key[uECC_BYTES * 2],
02924                       const uint8_t private_key[uECC_BYTES],
02925                       uint8_t secret[uECC_BYTES])
02926 {
02927     EccPoint public;
02928     EccPoint product;
02929     uECC_word_t private[uECC_N_WORDS];
02930     uECC_word_t blinded[uECC_N_WORDS + 1];
02931     uECC_word_t random[uECC_WORDS];
02932     uECC_word_t *initial_Z = 0;
02933     uECC_word_t tries;
02934
02935     for (tries = 0; tries < MAX_TRIES; ++tries)
02936     {
02937         if (g_rng_function((uint8_t *) random, sizeof(random)) &&
02938             !vli_isZero(random))
02939         {
02940             initial_Z = random;
02941             break;
02942         }
02943     }
02944
02945     vli_clear_n(private);
02946     vli_bytesToNative(private, private_key);
02947     vli_bytesToNative(public.x, public_key);
02948     vli_bytesToNative(public.y, public_key + uECC_BYTES);
02949
02950     vli_blindScalar(blinded, private);
02951     EccPoint_mult(&product, &public, blinded, initial_Z,
02952                   vli_numBits(blinded, uECC_N_WORDS + 1));
02953
02954     vli_nativeToBytes(secret, product.x);
02955     if (EccPoint_isZero(&product))
02956     {
02957         return 0;
02958     }
02959
02960     uint8_t check_point[uECC_BYTES * 2];
02961     vli_nativeToBytes(check_point, product.x);
02962     vli_nativeToBytes(check_point + uECC_BYTES, product.y);
02963     if (!uECC_valid_public_key(check_point))
02964     {
02965         return 0;
02966     }
02967
02968     return 1;
02969 }
02970
02971 static int uECC_sign_with_k(const uint8_t private_key[uECC_BYTES],
02972                             const uint8_t message_hash[uECC_BYTES],
02973                             uECC_word_t k[uECC_N_WORDS],
02974                             uint8_t signature[uECC_BYTES * 2])
02975 {
02976     uECC_word_t blinded[uECC_N_WORDS + 1];
02977     uECC_word_t tmp[uECC_N_WORDS];
02978     uECC_word_t s[uECC_N_WORDS];
02979     EccPoint p;
02980     uECC_word_t tries;
02981     uECC_word_t carry;
02982
02983     /* Make sure 0 < k < curve_n */
02984     if (vli_isZero(k) || vli_cmp_n(curve_n, k) != 1)
02985     {
02986         return 0;

```

```

02987     }
02988
02989     /* p = k * G */
02990     vli_blindScalar(blinded, k);
02991     EccPoint_mult(&p, &curve_G, blinded, 0,
02992                  vli_numBits(blinded, uECC_N_WORDS + 1));
02993
02994     /* r = x1 (mod n) */
02995     if (vli_cmp(curve_n, p.x) != 1)
02996     {
02997         vli_sub(p.x, p.x, curve_n);
02998     }
02999     if (vli_isZero(p.x))
03000     {
03001         return 0;
03002     }
03003
03004     // Attempt to get a random number to prevent side channel analysis of k.
03005     // If the RNG fails every time (eg it was not defined), we continue so
03006     // that deterministic signing can still work (with reduced security)
03007     // without an RNG defined.
03008     carry = 0; // use to signal that the RNG succeeded at least once.
03009     for (tries = 0; tries < MAX_TRIES; ++tries)
03010     {
03011         if (!g_rng_function((uint8_t *) tmp, sizeof(tmp)))
03012         {
03013             continue;
03014         }
03015         carry = 1;
03016         if (!vli_isZero(tmp))
03017         {
03018             break;
03019         }
03020     }
03021     if (!carry)
03022     {
03023         vli_clear(tmp);
03024         tmp[0] = 1;
03025     }
03026
03027     /* Prevent side channel analysis of vli_modInv() to determine
03028        bits of k / the private key by premultiplying by a random number */
03029     vli_modMult_n(k, k, tmp); /* k' = rand * k */
03030     vli_modInv_n(k, k, curve_n); /* k = 1 / k' */
03031     vli_modMult_n(k, k, tmp); /* k = 1 / k */
03032
03033     vli_nativeToBytes(signature, p.x); /* store r */
03034
03035     vli_clear_n(tmp);
03036     vli_bytesToNative(tmp, private_key); /* tmp = d */
03037     vli_clear_n(s);
03038     vli_set(s, p.x);
03039     vli_modMult_n(s, tmp, s); /* s = r*d */
03040
03041     vli_bytesToNative(tmp, message_hash);
03042     vli_modAdd_n(s, tmp, s, curve_n); /* s = e + r*d */
03043     vli_modMult_n(s, s, k); /* s = (e + r*d) / k */
03044     #if (uECC_CURVE == uECC_secp160r1)
03045     if (s[uECC_N_WORDS - 1])
03046     {
03047         return 0;
03048     }
03049     #endif
03050     vli_nativeToBytes(signature + uECC_BYTES, s);
03051     return 1;
03052 }
03053
03054 int uECC_sign(const uint8_t private_key[uECC_BYTES],
03055              const uint8_t message_hash[uECC_BYTES],
03056              uint8_t signature[uECC_BYTES * 2])
03057 {
03058     uECC_word_t k[uECC_N_WORDS];
03059     uECC_word_t tries;
03060
03061     for (tries = 0; tries < MAX_TRIES; ++tries)
03062     {
03063         if (g_rng_function((uint8_t *) k, sizeof(k)))
03064         {
03065             #if (uECC_CURVE == uECC_secp160r1)
03066             k[uECC_WORDS] &= 0x01;
03067             #endif
03068             if (uECC_sign_with_k(private_key, message_hash, k, signature))
03069             {
03070                 uint8_t public_key[uECC_BYTES * 2];
03071                 if (!uECC_compute_public_key(private_key, public_key))
03072                 {
03073                     return 0;

```



```

03074         }
03075         if (!uECC_verify(public_key, message_hash, signature))
03076         {
03077             return 0;
03078         }
03079         return 1;
03080     }
03081 }
03082 }
03083 return 0;
03084 }
03085
03086 /* Compute an HMAC using K as a key (as in RFC 6979). Note that K is always
03087    the same size as the hash result size. */
03088 static void HMAC_init(uECC_HashContext *hash_context, const uint8_t *K)
03089 {
03090     uint8_t *pad = hash_context->tmp + 2 * hash_context->result_size;
03091     unsigned i;
03092     for (i = 0; i < hash_context->result_size; ++i)
03093         pad[i] = K[i] ^ 0x36;
03094     for (; i < hash_context->block_size; ++i)
03095         pad[i] = 0x36;
03096
03097     hash_context->init_hash(hash_context);
03098     hash_context->update_hash(hash_context, pad, hash_context->block_size);
03099 }
03100
03101 static void HMAC_update(uECC_HashContext *hash_context, const uint8_t *message,
03102                        unsigned message_size)
03103 {
03104     hash_context->update_hash(hash_context, message, message_size);
03105 }
03106
03107 static void HMAC_finish(uECC_HashContext *hash_context, const uint8_t *K,
03108                        uint8_t *result)
03109 {
03110     uint8_t *pad = hash_context->tmp + 2 * hash_context->result_size;
03111     unsigned i;
03112     for (i = 0; i < hash_context->result_size; ++i)
03113         pad[i] = K[i] ^ 0x5c;
03114     for (; i < hash_context->block_size; ++i)
03115         pad[i] = 0x5c;
03116
03117     hash_context->finish_hash(hash_context, result);
03118
03119     hash_context->init_hash(hash_context);
03120     hash_context->update_hash(hash_context, pad, hash_context->block_size);
03121     hash_context->update_hash(hash_context, result, hash_context->result_size);
03122     hash_context->finish_hash(hash_context, result);
03123 }
03124
03125 /* V = HMAC_K(V) */
03126 static void update_V(uECC_HashContext *hash_context, uint8_t *K, uint8_t *V)
03127 {
03128     HMAC_init(hash_context, K);
03129     HMAC_update(hash_context, V, hash_context->result_size);
03130     HMAC_finish(hash_context, K, V);
03131 }
03132
03133 /* Deterministic signing, similar to RFC 6979. Differences are:
03134    * We just use (truncated) H(m) directly rather than bits2octets(H(m))
03135    (it is not reduced modulo curve_n).
03136    * We generate a value for k (aka T) directly rather than converting
03137    endianness.
03138
03139    Layout of hash_context->tmp: <K> | <V> | (1 byte overlapped 0x00 or 0x01)
03140    / <HMAC pad> */
03141 int uECC_sign_deterministic(const uint8_t private_key[uECC_BYTES],
03142                             const uint8_t message_hash[uECC_BYTES],
03143                             uECC_HashContext *hash_context,
03144                             uint8_t signature[uECC_BYTES * 2])
03145 {
03146     uint8_t *K = hash_context->tmp;
03147     uint8_t *V = K + hash_context->result_size;
03148     uECC_word_t tries;
03149     unsigned i;
03150     for (i = 0; i < hash_context->result_size; ++i)
03151     {
03152         V[i] = 0x01;
03153         K[i] = 0;
03154     }
03155
03156     // K = HMAC_K(V || 0x00 || int2octets(x) || h(m))
03157     HMAC_init(hash_context, K);
03158     V[hash_context->result_size] = 0x00;
03159     HMAC_update(hash_context, V, hash_context->result_size + 1);
03160     HMAC_update(hash_context, private_key, uECC_BYTES);

```

```

03161     HMAC_update(hash_context, message_hash, uECC_BYTES);
03162     HMAC_finish(hash_context, K, K);
03163
03164     update_V(hash_context, K, V);
03165
03166     // K = HMAC_K(V || 0x01 || int2octets(x) || h(m))
03167     HMAC_init(hash_context, K);
03168     V[hash_context->result_size] = 0x01;
03169     HMAC_update(hash_context, V, hash_context->result_size + 1);
03170     HMAC_update(hash_context, private_key, uECC_BYTES);
03171     HMAC_update(hash_context, message_hash, uECC_BYTES);
03172     HMAC_finish(hash_context, K, K);
03173
03174     update_V(hash_context, K, V);
03175
03176     for (tries = 0; tries < MAX_TRIES; ++tries)
03177     {
03178         uECC_word_t T[uECC_N_WORDS];
03179         uint8_t *T_ptr = (uint8_t *) T;
03180         unsigned T_bytes = 0;
03181         while (T_bytes < sizeof(T))
03182         {
03183             update_V(hash_context, K, V);
03184             for (i = 0; i < hash_context->result_size && T_bytes < sizeof(T);
03185                  ++i, ++T_bytes)
03186             {
03187                 T_ptr[T_bytes] = V[i];
03188             }
03189         }
03190         #if (uECC_CURVE == uECC_secp160r1)
03191             T[uECC_WORDS] &= 0x01;
03192         #endif
03193
03194         if (uECC_sign_with_k(private_key, message_hash, T, signature))
03195         {
03196             uint8_t public_key[uECC_BYTES * 2];
03197             if (!uECC_compute_public_key(private_key, public_key))
03198             {
03199                 return 0;
03200             }
03201             if (!uECC_verify(public_key, message_hash, signature))
03202             {
03203                 return 0;
03204             }
03205             return 1;
03206         }
03207
03208         // K = HMAC_K(V || 0x00)
03209         HMAC_init(hash_context, K);
03210         V[hash_context->result_size] = 0x00;
03211         HMAC_update(hash_context, V, hash_context->result_size + 1);
03212         HMAC_finish(hash_context, K, K);
03213
03214         update_V(hash_context, K, V);
03215     }
03216     return 0;
03217 }
03218
03219 static bitcount_t smax(bitcount_t a, bitcount_t b)
03220 {
03221     return (a > b ? a : b);
03222 }
03223
03224 int uECC_verify(const uint8_t public_key[uECC_BYTES * 2],
03225                 const uint8_t hash[uECC_BYTES],
03226                 const uint8_t signature[uECC_BYTES * 2])
03227 {
03228     uECC_word_t u1[uECC_N_WORDS], u2[uECC_N_WORDS];
03229     uECC_word_t z[uECC_N_WORDS];
03230     EccPoint public, sum;
03231     uECC_word_t rx[uECC_WORDS];
03232     uECC_word_t ry[uECC_WORDS];
03233     uECC_word_t tx[uECC_WORDS];
03234     uECC_word_t ty[uECC_WORDS];
03235     uECC_word_t tz[uECC_WORDS];
03236     const EccPoint *points[4];
03237     const EccPoint *point;
03238     bitcount_t numBits;
03239     bitcount_t i;
03240     uECC_word_t r[uECC_N_WORDS], s[uECC_N_WORDS];
03241
03242     vli_clear_n(r);
03243     vli_clear_n(s);
03244
03245     vli_bytesToNative(public.x, public_key);
03246     vli_bytesToNative(public.y, public_key + uECC_BYTES);
03247     vli_bytesToNative(r, signature);

```

```

03248     vli_bytesToNative(s, signature + uECC_BYTES);
03249
03250     if (vli_isZero(r) || vli_isZero(s))
03251     { /* r, s must not be 0. */
03252         return 0;
03253     }
03254
03255     #if (uECC_CURVE != uECC_secp160r1)
03256     if (vli_cmp_n(curve_n, r) != 1 || vli_cmp_n(curve_n, s) != 1)
03257     { /* r, s must be < n. */
03258         return 0;
03259     }
03260     #endif
03261
03262     /* Calculate u1 and u2. */
03263     vli_modInv_n(z, s, curve_n); /* Z = s^-1 */
03264     vli_clear_n(u1);
03265     vli_bytesToNative(u1, hash);
03266     vli_modMult_n(u1, u1, z); /* u1 = e/s */
03267     vli_modMult_n(u2, r, z); /* u2 = r/s */
03268
03269     /* Calculate sum = G + Q. */
03270     vli_set(sum.x, public.x);
03271     vli_set(sum.y, public.y);
03272     vli_set(tx, curve_G.x);
03273     vli_set(ty, curve_G.y);
03274     vli_modSub_fast(z, sum.x, tx); /* Z = x2 - x1 */
03275     XYcZ_add(tx, ty, sum.x, sum.y);
03276     vli_modInv(z, z, curve_p); /* Z = 1/Z */
03277     apply_z(sum.x, sum.y, z);
03278
03279     /* Use Shamir's trick to calculate u1*G + u2*Q */
03280     points[0] = 0;
03281     points[1] = &curve_G;
03282     points[2] = &public;
03283     points[3] = &sum;
03284     numBits =
03285         smax(vli_numBits(u1, uECC_N_WORDS), vli_numBits(u2, uECC_N_WORDS));
03286
03287     point = points[(!!vli_testBit(u1, numBits - 1)) |
03288         ((!!vli_testBit(u2, numBits - 1)) << 1)];
03289     vli_set(rx, point->x);
03290     vli_set(ry, point->y);
03291     vli_clear(z);
03292     z[0] = 1;
03293
03294     for (i = numBits - 2; i >= 0; --i)
03295     {
03296         uECC_word_t index;
03297         EccPoint_double_jacobian(rx, ry, z);
03298
03299         index = (!!vli_testBit(u1, i)) | ((!!vli_testBit(u2, i)) << 1);
03300         point = points[index];
03301         if (point)
03302         {
03303             vli_set(tx, point->x);
03304             vli_set(ty, point->y);
03305             apply_z(tx, ty, z);
03306             vli_modSub_fast(tz, rx, tx); /* Z = x2 - x1 */
03307             XYcZ_add(tx, ty, rx, ry);
03308             vli_modMult_fast(z, z, tz);
03309         }
03310     }
03311
03312     vli_modInv(z, z, curve_p); /* Z = 1/Z */
03313     apply_z(rx, ry, z);
03314
03315     /* v = x1 (mod n) */
03316     #if (uECC_CURVE != uECC_secp160r1)
03317     if (vli_cmp(curve_n, rx) != 1)
03318     {
03319         vli_sub(rx, rx, curve_n);
03320     }
03321     #endif
03322
03323     /* Accept only if v == r. */
03324     return vli_equal(rx, r);
03325 }
03326
03327 void uECC_mod_mult(uint8_t result[32], const uint8_t left[32],
03328     const uint8_t right[32])
03329 {
03330     uECC_word_t l[uECC_WORDS];
03331     uECC_word_t r[uECC_WORDS];
03332     uECC_word_t res[uECC_WORDS];
03333
03334     vli_bytesToNative(l, left);

```

```

03335     vli_bytesToNative(r, right);
03336
03337     vli_modMult_n(res, l, r);
03338
03339     vli_nativeToBytes(result, res);
03340 }

```

5.43 firmware/utils/derive_secrets.py File Reference

Code generator for HSM cryptographic identities and access control.

Data Structures

- class [derive_secrets.Permission](#)
- class [derive_secrets.PermissionList](#)

Namespaces

- namespace [derive_secrets](#)

Functions

- [derive_secrets.key_as_array](#) (int groupid, dict[str, dict[str, dict[str, bytes]]] [secrets](#), str perm, str sel)
- [derive_secrets.interrogate_as_array](#) (dict[str, dict[str, dict[str, bytes]]] [secrets](#))
- [derive_secrets.generateCaseList](#) (str permission, [PermissionList](#) permissionlist)
- [derive_secrets.generateSecrets](#) (str outfile, [PermissionList](#) permissionlist, dict[str, dict[str, dict[str, bytes]]] [secrets](#))
- [argparse.Namespace](#) [derive_secrets.arg_parse](#) ()

Variables

- tuple [derive_secrets.VALID_PERMS](#) = ("Write", "Read", "Send", "Recv")
- str [derive_secrets.DISCLAIMER](#)
- str [derive_secrets.DEFINITIONS](#)
- Callable [derive_secrets.HEADER_CONTENT](#)
- tuple [derive_secrets.GEN_WRITEKEY](#)
- tuple [derive_secrets.GEN_READKEY](#)
- tuple [derive_secrets.GEN_SENDKEY](#)
- tuple [derive_secrets.GEN_RECVKEY](#)
- tuple [derive_secrets.GEN_INTERROGATEKEY](#)
- Callable [derive_secrets.GEN_REPLYKEY](#)
- Callable [derive_secrets.GEN_STUB](#)
- Callable [derive_secrets.GEN_CASE_SMT](#)
- Callable [derive_secrets.GEN_CASE_STUB](#)
- Callable [derive_secrets.CASELIST](#)
- Callable [derive_secrets.RECV_GROUPS](#)
- [argparse.Namespace](#) [derive_secrets.args](#) = [arg_parse](#)()
- [derive_secrets.secrets](#) = [pickle.load](#)(args.secrets)
- [derive_secrets.permissions](#) = [PermissionList.deserialize](#)(args.permissions)
- [derive_secrets._](#)

5.43.1 Detailed Description

Code generator for HSM cryptographic identities and access control.

Author

Sumedh Girish

Overview

This script is the core provisioning utility for the SHIFT HSM. It takes a master secrets dictionary (generated during the system build phase) and a user-provided permission list to dynamically generate `secrets.c` and `secrets.h`.

Cryptographic Architecture

Instead of storing static keys that can be easily extracted from flash memory, this framework implements an asymmetric derivation scheme:

1. It compiles group-specific public/private keypairs into the firmware.
2. At runtime, handlers generate ephemeral keys, blinding them using the TRNG.
3. An ECDH shared secret is established between the static device key and the blinded ephemeral key.
4. This shared secret is fed into an ASCON-XOF Key Derivation Function (KDF) along with a domain string ("FS-ASCON-KDF") to produce the final 128-bit AEAD session key.

Usage Examples:

```
# Grant HSM Read/Write for group 0x1234, and Read/Recv for 0x4321
python3 derive_secrets.py global.secrets 123456 "1234=RW-:4321=R-C"
```

Definition in file [derive_secrets.py](#).

5.44 derive_secrets.py

[Go to the documentation of this file.](#)

```
00001
00031
00032 # flake8: noqa: E731
00033 # pyright: reportAny=false
00034 import argparse
00035 import pickle
00036 from dataclasses import dataclass
00037 from typing import Callable
00038
00039
00040 # ----- HELPERS -----
00041 def key_as_array(
00042     groupid: int, secrets: dict[str, dict[str, dict[str, bytes]]], perm: str, sel: str
00043 ):
00044     """
00045     @brief Format a specific secret as a C-style array initializer list.
00046
00047     @param groupid (int): The group ID to retrieve.
00048     @param secrets (dict): The secrets dictionary.
00049     @param sel (str): The selector ('internal' or 'external').
00050
00051     @return str: The formatted C array.
00052     """
00053     return ", ".join([f"0x{b:02X}" for b in secrets[f"{groupid:04x}"][perm][sel]])
00054
00055
```

```

00056 def interrogate_as_array(secrets: dict[str, dict[str, dict[str, bytes]]]):
00057     return ", ".join([f"0x{b:02X}" for b in secrets["global"]["interrogate"]["value"]])
00058
00059
00060 VALID_PERMS = ("Write", "Read", "Send", "Recv")
00061
00062 # ----- TEMPLATES -----
00063 DISCLAIMER = """\
00064 // -----
00065 //                                     DISCLAIMER
00066 // This file is generated by a utility script. Do not edit this file directly,
00067 // as it will be overwritten by the script. Instead, view the script to
00068 // understand how this file is generated and make changes to the script if
00069 // necessary.
00070 // -----
00071 """
00072
00073 DEFINITIONS = """\
00074 #include "ascon.h"
00075 #include "crypto_aead.h"
00076 #include "filesystem.h"
00077 #include "randombytes.h"
00078 #include "secrets.h"
00079 #include "stubs.h"
00080 #include "ti/devices/msp/msp.h"
00081 #include "uECC.h"
00082 #include <stdint.h>
00083 #include <stddef.h>
00084 #include <string.h>
00085
00086
00087 /**
00088  * @brief Performs modular multiplication of two scalars (mod n).
00089  *
00090  * Used for blinding scalars in the asymmetric master key scheme.
00091  *
00092  * @param result Destination buffer (32 bytes).
00093  * @param left First scalar (32 bytes).
00094  * @param right Second scalar (32 bytes).
00095  */
00096 void uECC_mod_mult(uint8_t result[ECC_PRIV_KEYSIZE],
00097                   const uint8_t left[ECC_PRIV_KEYSIZE],
00098                   const uint8_t right[ECC_PRIV_KEYSIZE]);
00099
00100 /**
00101  * @brief Adds a small, random timing delay to decorrelate power traces.
00102  *
00103  * Uses the hardware TRNG to wait for a random number of CPU cycles, making
00104  * it significantly harder for an attacker to align multiple power traces
00105  * for differential analysis.
00106  */
00107 static inline void apply_jitter(void) {
00108     uint32_t r = getrandom32() & 0x3F; // Up to 63 cycles
00109     for (volatile uint32_t i = 0; i < r; i++) {
00110         __asm__ volatile ("nop");
00111     }
00112 }
00113
00114 /**
00115  * @brief Derives an Ascon AEAD key from an ECC shared secret.
00116  *
00117  * This function uses a domain-separated KDF based on Ascon-XOF. It concatenates
00118  * a domain string ("FS-ASCON-KDF"), the raw shared secret, and a usage string
00119  * ("AEAD-KEY") before passing them through the XOF. This ensures that the
00120  * resulting key is cryptographically bound to its intended purpose.
00121  *
00122  * @param output Destination buffer for the 16-byte Ascon key.
00123  * @param sharedKey Raw ECC shared secret (typically 32 bytes).
00124  * @return KEYS_StatusCode KEYS_OK on success.
00125  */
00126 static inline void DeriveAsconKey(uint8_t output[ASCON_KEY_SIZE],
00127                                   uint8_t sharedKey[ECC_PRIV_KEYSIZE])
00128 {
00129     const uint8_t domain[] = "FS-ASCON-KDF";
00130     const uint8_t usage[] = "AEAD-KEY";
00131
00132     uint8_t input[sizeof(domain) - 1 + ECC_PRIV_KEYSIZE + sizeof(usage) - 1];
00133
00134     size_t offset = 0;
00135     memcpy(input + offset, domain, sizeof(domain) - 1);
00136     offset += sizeof(domain) - 1;
00137     memcpy(input + offset, sharedKey, ECC_PRIV_KEYSIZE);
00138     offset += ECC_PRIV_KEYSIZE;
00139     memcpy(input + offset, usage, sizeof(usage) - 1);
00140     offset += sizeof(usage) - 1;
00141
00142     ascon_xof_masked(output, ASCON_KEY_SIZE, input, offset);

```

```

00143
00144     /* Wipe intermediate sensitive state */
00145     memclear(input, offset, 0);
00146     memclear(sharedKey, ECC_PRIV_KEYSIZE, 0);
00147 }
00148
00149
00150 typedef struct {
00151     uint8_t nonce[NONCE_SIZE];
00152     uint8_t masterKey[MASTER_KEY_SIZE];
00153     uint8_t id[ID_SIZE];
00154 } KeyContext;
00155 ""
00156
00157 HEADER_CONTENT: Callable[[str, int], str] = lambda pin, n_groups: (
00158     f"""\
00159 #ifndef __SECRETS_H__
00160 #define __SECRETS_H__
00161
00162 #include <stdint.h>
00163 #include "status.h"
00164
00165 #define ECC_PRIV_KEYSIZE 32
00166 #define ECC_PUB_KEYSIZE 64
00167
00168 #define MASTER_PRIV_KEY_SIZE ECC_PRIV_KEYSIZE
00169 #define MASTER_PUB_KEY_SIZE ECC_PUB_KEYSIZE
00170 #define MASTER_KEY_SIZE 16
00171
00172 #define ASCON_KEY_SIZE 16
00173 #define ASCON_TAG_SIZE 16
00174
00175 #define NONCE_SIZE 16
00176 #define ID_SIZE 16
00177
00178 #define HSM_PIN "{pin}"
00179
00180 #define N_RECV_GROUPS {n_groups}
00181 extern const uint16_t recv_groups[{n_groups}];
00182
00183
00184 StatusCode SelectWriteKey(uint16_t groupid, uint8_t output[ASCON_KEY_SIZE]);
00185 StatusCode SelectReadKey(uint16_t groupid, uint8_t output[ASCON_KEY_SIZE]);
00186 StatusCode SelectSendKey(uint16_t groupid, uint8_t output[ASCON_KEY_SIZE]);
00187 StatusCode SelectRecvKey(uint16_t groupid, uint8_t output[ASCON_KEY_SIZE]);
00188
00189 StatusCode SelectInterrogateKey(uint8_t output[ASCON_KEY_SIZE]);
00190 StatusCode SelectReplyKey(uint8_t output[ASCON_KEY_SIZE]);
00191
00192 #endif
00193 ""
00194 )
00195
00196 GEN_WRITEKEY: Callable[[int, dict[str, dict[str, dict[str, bytes]]]], str] = (
00197     lambda groupid, secrets: (
00198         f"""\
00199 static StatusCode GenWriteKey_{groupid:04x}(uint8_t key[ASCON_KEY_SIZE])
00200 {{
00201     KeyContext ctx = {{0}};
00202     static const uint8_t master_pub[MASTER_PUB_KEY_SIZE] = {{
00203         {key_as_array(groupid, secrets, "write", "master")}
00204     }};
00205     static const uint8_t tweak_secret[MASTER_KEY_SIZE] = {{
00206         {key_as_array(groupid, secrets, "write", "tweak")}
00207     }};
00208
00209     randbytes(ctx.nonce, NONCE_SIZE);
00210     memcpy(ctx.id, (uint8_t *) &stage.as.file.metadata.fileid, ID_SIZE);
00211     memcpy(ctx.masterKey, tweak_secret, MASTER_KEY_SIZE);
00212
00213     __disable_irq();
00214     apply_jitter();
00215
00216     uint8_t s[ECC_PRIV_KEYSIZE];
00217     if (ascon_xof_masked(s, ECC_PRIV_KEYSIZE, (uint8_t *) &ctx, sizeof(ctx)) != 0)
00218     {{
00219         memclear((uint8_t *) &ctx, sizeof(ctx), 0x00);
00220
00221         __enable_irq();
00222         return KEYGENERROR;
00223     }}
00224
00225     apply_jitter();
00226     uint8_t eph_priv[ECC_PRIV_KEYSIZE];
00227     uint8_t eph_pub[ECC_PUB_KEYSIZE];
00228     if (uECC_make_key(eph_pub, eph_priv) != 1) {{
00229

```

```

00230     memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00231     memclear(s, ECC_PRIV_KEYSIZE, 0);
00232
00233     __enable_irq();
00234     return KEYGENERROR;
00235 }
00236
00237 apply_jitter();
00238 uint8_t k_blind[ECC_PRIV_KEYSIZE];
00239 uECC_mod_mult(k_blind, eph_priv, s);
00240
00241 uint8_t sharedSecret_raw[ECC_PRIV_KEYSIZE];
00242 if (uECC_shared_secret(master_pub, k_blind, sharedSecret_raw) != 1) {{
00243
00244     memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00245     memclear(s, ECC_PRIV_KEYSIZE, 0);
00246     memclear(eph_priv, ECC_PRIV_KEYSIZE, 0);
00247     memclear(eph_pub, ECC_PUB_KEYSIZE, 0);
00248     memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00249
00250     __enable_irq();
00251     return KEYGENERROR;
00252 }}
00253
00254 if (key != NULL)
00255     DeriveAsconKey(key, sharedSecret_raw);
00256
00257 memcpy((uint8_t *) &stage.as.file.metadata.key, eph_pub, ECC_PUB_KEYSIZE);
00258 memcpy((uint8_t *) &stage.as.file.metadata.nonce, ctx.nonce, NONCE_SIZE);
00259
00260 memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00261 memclear(s, ECC_PRIV_KEYSIZE, 0);
00262 memclear(eph_priv, ECC_PRIV_KEYSIZE, 0);
00263 memclear(eph_pub, ECC_PUB_KEYSIZE, 0);
00264 memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00265 memclear(sharedSecret_raw, ECC_PRIV_KEYSIZE, 0);
00266
00267 __enable_irq();
00268 return OPWRITE;
00269 }}
00270 ""
00271 )
00272 )
00273
00274
00275 GEN_READKEY: Callable[[int, dict[str, dict[str, dict[str, bytes]]]], str] = (
00276     lambda groupid, secrets: (
00277         f"""\
00278 static StatusCode GenReadKey_{groupid:04x}(uint8_t key[ASCON_KEY_SIZE])
00279 {{
00280     KeyContext ctx = {{0}};
00281     static const uint8_t master_priv[MASTER_PRIV_KEY_SIZE] = {{
00282         {key_as_array(groupid, secrets, "read", "master")}
00283     }};
00284     static const uint8_t tweak_secret[MASTER_KEY_SIZE] = {{
00285         {key_as_array(groupid, secrets, "read", "tweak")}
00286     }};
00287
00288     memcpy(ctx.nonce, (uint8_t *) &stage.as.file.metadata.nonce, NONCE_SIZE);
00289     memcpy(ctx.id, (uint8_t *) &stage.as.file.metadata.fileid, ID_SIZE);
00290     memcpy(ctx.masterKey, tweak_secret, MASTER_KEY_SIZE);
00291
00292     __disable_irq();
00293     apply_jitter();
00294
00295     uint8_t s[ECC_PRIV_KEYSIZE];
00296     if (ascon_xof_masked(s, ECC_PRIV_KEYSIZE, (uint8_t *) &ctx, sizeof(ctx)) != 0) {{
00297         memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00298
00299         __enable_irq();
00300         return KEYGENERROR;
00301     }}
00302
00303     apply_jitter();
00304     uint8_t k_blind[ECC_PRIV_KEYSIZE];
00305     uECC_mod_mult(k_blind, master_priv, s);
00306
00307     uint8_t sharedSecret_raw[ECC_PRIV_KEYSIZE];
00308     if (uECC_shared_secret(
00309         (uint8_t *) &stage.as.file.metadata.key, k_blind, sharedSecret_raw) != 1)
00310     {{
00311
00312         memclear((uint8_t *)&ctx, sizeof(ctx), 0);
00313         memclear(s, ECC_PRIV_KEYSIZE, 0);
00314         memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00315
00316         __enable_irq();

```



```

00317         return KEYGENERERROR;
00318     }}
00319
00320     if (key != NULL)
00321         DeriveAsconKey(key, sharedSecret_raw);
00322
00323     memclear((uint8_t*)&ctx, sizeof(ctx), 0);
00324     memclear(s, ECC_PRIV_KEYSIZE, 0);
00325     memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00326     memclear(sharedSecret_raw, ECC_PRIV_KEYSIZE, 0);
00327
00328     __enable_irq();
00329     return OPREAD;
00330 }}
00331 """
00332 )
00333 )
00334
00335
00336 GEN_SENDKEY: Callable[[int, dict[str, dict[str, dict[str, bytes]]]], str] = (
00337     lambda groupid, secrets: (
00338         f"""\
00339 static StatusCode GenSendKey_{groupid:04x}(uint8_t key[ASCON_KEY_SIZE])
00340 {{
00341     KeyContext ctx = {{0}};
00342     static const uint8_t peer_pub[MASTER_PUB_KEY_SIZE] = {{
00343         {key_as_array(groupid, secrets, "send", "master")}
00344     }};
00345     static const uint8_t tweak_secret[MASTER_KEY_SIZE] = {{
00346         {key_as_array(groupid, secrets, "send", "tweak")}
00347     }};
00348
00349     randombytes(ctx.nonce, NONCE_SIZE);
00350     randombytes(ctx.id, ID_SIZE);
00351     memcpy(ctx.masterKey, tweak_secret, MASTER_KEY_SIZE);
00352
00353     __disable_irq();
00354     apply_jitter();
00355
00356     uint8_t s[ECC_PRIV_KEYSIZE];
00357     if (ascon_xof_masked(s, ECC_PRIV_KEYSIZE, (uint8_t *) &ctx, sizeof(ctx)) != 0) {{
00358
00359         memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00360
00361         __enable_irq();
00362         return KEYGENERERROR;
00363     }}
00364
00365     apply_jitter();
00366     uint8_t eph_priv[ECC_PRIV_KEYSIZE];
00367     uint8_t eph_pub[ECC_PUB_KEYSIZE];
00368     if (uECC_make_key(eph_pub, eph_priv) != 1) {{
00369
00370         memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00371         memclear(s, ECC_PRIV_KEYSIZE, 0);
00372
00373         __enable_irq();
00374         return KEYGENERERROR;
00375     }}
00376
00377     apply_jitter();
00378     uint8_t k_blind[ECC_PRIV_KEYSIZE];
00379     uECC_mod_mult(k_blind, eph_priv, s);
00380
00381     uint8_t sharedSecret_raw[ECC_PRIV_KEYSIZE];
00382     if (uECC_shared_secret(peer_pub, k_blind, sharedSecret_raw) != 1) {{
00383
00384         memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00385         memclear(s, ECC_PRIV_KEYSIZE, 0);
00386         memclear(eph_priv, ECC_PRIV_KEYSIZE, 0);
00387         memclear(eph_pub, ECC_PUB_KEYSIZE, 0);
00388         memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00389
00390         __enable_irq();
00391         return KEYGENERERROR;
00392     }}
00393
00394     if (key != NULL)
00395         DeriveAsconKey(key, sharedSecret_raw);
00396
00397     memcpy((uint8_t *) &stage.preamble.key, eph_pub, ECC_PUB_KEYSIZE);
00398     memcpy((uint8_t *) &stage.preamble.nonce, ctx.nonce, NONCE_SIZE);
00399     memcpy((uint8_t *) &stage.preamble.mesgid, ctx.id, ID_SIZE);
00400
00401     memclear((uint8_t *) &ctx, sizeof(ctx), 0);
00402     memclear(s, ECC_PRIV_KEYSIZE, 0);
00403     memclear(eph_priv, ECC_PRIV_KEYSIZE, 0);

```

```

00404     memclear(eph_pub, ECC_PUB_KEYSIZE, 0);
00405     memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00406     memclear(sharedSecret_raw, ECC_PRIV_KEYSIZE, 0);
00407
00408     __enable_irq();
00409     return OPSEND;
00410 }
00411 """
00412 )
00413 )
00414
00415
00416 GEN_RECVKEY: Callable[[int, dict[str, dict[str, dict[str, bytes]]]], str] = (
00417     lambda groupid, secrets: (
00418         f"""\
00419 static StatusCode GenRecvKey_{groupid:04x}(uint8_t key[ASCON_KEY_SIZE])
00420 {{
00421     KeyContext ctx = {{0}};
00422     static const uint8_t local_priv[MASTER_PRIV_KEY_SIZE] = {{
00423         {key_as_array(groupid, secrets, "recv", "master")}
00424     }};
00425     static const uint8_t tweak_secret[MASTER_KEY_SIZE] = {{
00426         {key_as_array(groupid, secrets, "recv", "tweak")}
00427     }};
00428
00429     memcpy(ctx.nonce, (uint8_t *) &stage.preamble.nonce, NONCE_SIZE);
00430     memcpy(ctx.id, (uint8_t *) &stage.preamble.mesgid, ID_SIZE);
00431     memcpy(ctx.masterKey, tweak_secret, MASTER_KEY_SIZE);
00432
00433     __disable_irq();
00434     apply_jitter();
00435
00436     uint8_t s[ECC_PRIV_KEYSIZE];
00437     if (ascon_xof_masked(s, ECC_PRIV_KEYSIZE, (uint8_t *) &ctx, sizeof(ctx)) != 0) {{
00438
00439         memclear((uint8_t*)&ctx, sizeof(ctx), 0);
00440
00441         __enable_irq();
00442         return KEYGENERROR;
00443     }}
00444
00445     apply_jitter();
00446     uint8_t k_blind[ECC_PRIV_KEYSIZE];
00447     uECC_mod_mult(k_blind, local_priv, s);
00448
00449     uint8_t sharedSecret_raw[ECC_PRIV_KEYSIZE];
00450     if (uECC_shared_secret(
00451         (uint8_t *) &stage.preamble.key, k_blind, sharedSecret_raw) != 1
00452     )
00453     {{
00454
00455         memclear((uint8_t*)&ctx, sizeof(ctx), 0);
00456         memclear(s, ECC_PRIV_KEYSIZE, 0);
00457         memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00458
00459         __enable_irq();
00460         return KEYGENERROR;
00461     }}
00462
00463     if (key != NULL)
00464         DeriveAsconKey(key, sharedSecret_raw);
00465
00466     memclear((uint8_t*)&ctx, sizeof(ctx), 0);
00467     memclear(s, ECC_PRIV_KEYSIZE, 0);
00468     memclear(k_blind, ECC_PRIV_KEYSIZE, 0);
00469     memclear(sharedSecret_raw, ECC_PRIV_KEYSIZE, 0);
00470
00471     __enable_irq();
00472     return OPRECEIVE;
00473 }}
00474 """
00475 )
00476 )
00477
00478 GEN_INTERROGATEKEY: Callable[[dict[str, dict[str, dict[str, bytes]]]], str] = (
00479     lambda secrets: (
00480         f"""\
00481 StatusCode SelectInterrogateKey(uint8_t output[ASCON_KEY_SIZE])
00482 {{
00483     KeyContext derivation_ctx = {{0}};
00484     static const uint8_t master_secret[MASTER_KEY_SIZE] = {{
00485         {interrogate_as_array(secrets)}
00486     }};
00487
00488     __disable_irq();
00489     apply_jitter();
00490

```

```

00491     memcpy(derivation_ctx.masterKey, master_secret, MASTER_KEY_SIZE);
00492     memcpy(derivation_ctx.nonce, (uint8_t *) &stage.preamble.nonce, NONCE_SIZE);
00493     memcpy(derivation_ctx.id, (uint8_t *) &stage.preamble.mesgid, ID_SIZE);
00494
00495     if (ascon_xof_masked(
00496         output, ASCON_KEY_SIZE, (uint8_t*)&derivation_ctx, sizeof(derivation_ctx)
00497     ) != 0
00498     )
00499     {{
00500         memclear((uint8_t *) &derivation_ctx, sizeof(KeyContext), 0);
00501
00502         __enable_irq();
00503         return KEYGENERERROR;
00504     }}
00505
00506     memclear(&derivation_ctx, sizeof(derivation_ctx), 0);
00507
00508     __enable_irq();
00509     return OPINTERROGATE;
00510 }}
00511 """
00512 )
00513 )
00514
00515 GEN_REPLYKEY: Callable[[dict[str, dict[str, dict[str, bytes]]]], str] = lambda secrets: (
00516     f"""
00517     StatusCode SelectReplyKey(uint8_t output[ASCON_KEY_SIZE])
00518     {{
00519         KeyContext derivation_ctx = {{0}};
00520         static const uint8_t master_secret[MASTER_KEY_SIZE] = {{
00521             {interrogate_as_array(secrets)}
00522         }};
00523
00524         __disable_irq();
00525         apply_jitter();
00526
00527         memcpy(derivation_ctx.masterKey, master_secret, MASTER_KEY_SIZE);
00528         memcpy(derivation_ctx.nonce, (uint8_t *) &stage.preamble.nonce, NONCE_SIZE);
00529         memcpy(derivation_ctx.id, (uint8_t *) &stage.preamble.mesgid, ID_SIZE);
00530
00531         if (ascon_xof_masked(
00532             output, ASCON_KEY_SIZE, (uint8_t*)&derivation_ctx, sizeof(derivation_ctx)
00533         ) != 0
00534         )
00535         {{
00536             memclear((uint8_t *) &derivation_ctx, sizeof(KeyContext), 0);
00537
00538             __enable_irq();
00539             return KEYGENERERROR;
00540         }}
00541
00542         memclear(&derivation_ctx, sizeof(derivation_ctx), 0);
00543         __enable_irq();
00544         return OPREPLY;
00545     }}
00546 """
00547 )
00548
00549 GEN_STUB: Callable[[], str] = lambda: (
00550     f"""
00551     static StatusCode GenStub(void)
00552     {{
00553         return PERMISSIONERROR;
00554     }}
00555 """
00556 )
00557
00558 GEN_CASE_SMT: Callable[[int, str], str] = lambda groupid, permstr: (
00559     f"""
00560     case 0x{groupid:04x}:
00561         return Gen{permstr}Key_{groupid:04x}(key);
00562     """
00563 )
00564
00565 GEN_CASE_STUB: Callable[[int], str] = lambda groupid: (
00566     f"""
00567     case 0x{groupid:04x}:
00568         return GenStub();
00569     """
00570 )
00571
00572 CASELIST: Callable[[str, str], str] = lambda permstr, cases: (
00573     f"""
00574     StatusCode Select{permstr}Key(uint16_t groupid, uint8_t key[ASCON_KEY_SIZE])
00575     {{
00576         switch (groupid) {{
00577     {cases}
00577 }}

```

```

00578     default:
00579         return GenStub();
00580     }}
00581 }}
00582 """
00583 )
00584
00585 RECV_GROUPS: Callable[[list[int]], str] = lambda groups: (
00586     f"""\
00587 const uint16_t recv_groups[{len(groups)}] = {{
00588     {"", ".join(sorted([f'0x{g:04x}" for g in groups]))}
00589 }};
00590 """
00591 )
00592
00593
00594 # ----- CLI -----
00595 @dataclass
00596 class Permission:
00597     """
00598     @brief Represents permissions for a specific group.
00599
00600     @param group_id (int): The 16-bit group identifier.
00601     @param read (bool): Permission to read files in this group.
00602     @param write (bool): Permission to write files in this group.
00603     @param receive (bool): Permission to receive files from peers for this group.
00604     """
00605
00606     group_id: int = 0
00607     read: bool = False
00608     write: bool = False
00609     receive: bool = False
00610
00611     @classmethod
00612     def deserialize(cls, perms: str):
00613         group_id, perm_string = perms.split("=")
00614
00615         perm_obj = cls(
00616             int(group_id, 16),
00617             read=perm_string[0] == "R",
00618             write=perm_string[1] == "W",
00619             receive=perm_string[2] == "C",
00620         )
00621         return perm_obj
00622
00623     def serialize(self):
00624         ret = f"{self.group_id:04x}="
00625         prepr: dict[str, str] = {"read": "R", "write": "W", "receive": "C"}
00626         for perm, shorthand in prepr.items():
00627             ret += shorthand if getattr(self, perm) else "-"
00628         return ret
00629
00630
00631 class PermissionList(list[Permission]):
00632     """
00633     @brief A collection of Permission objects with serialization support.
00634     """
00635
00636     def __init__(self, *args: Permission):
00637         super().__init__()
00638         for item in args:
00639             self.append(item)
00640
00641     @classmethod
00642     def deserialize(cls, perms: str):
00643         ret = cls()
00644         permissions_strings = perms.split(":")
00645         for entry in permissions_strings:
00646             perm_obj = Permission.deserialize(entry)
00647             ret.append(perm_obj)
00648         return ret
00649
00650     def serialize(self):
00651         return ":".join(perm.serialize() for perm in self)
00652
00653
00654 # ----- GEN -----
00655
00656
00657 def generateCaseList(permission: str, permissionlist: PermissionList):
00658     """
00659     @brief Generates a C switch-case block for selecting the correct key derivation
00660     routine based on group ID and the requested operation.
00661
00662     @param permission (str): The permission type (Read, Write, etc.).
00663     @param permissionlist (PermissionList): The list of permissions.
00664

```

```

00665     @return str: The generated C code.
00666     """
00667     assert permission in VALID_PERMS, "Invalid permission passed to genCaseList"
00668
00669     validgen: Callable[[int], str] = lambda groupid: GEN_CASE_SMT(groupid, permission)
00670     stubgen: Callable[[int], str] = GEN_CASE_STUB
00671
00672     genmap: Callable[[Permission], dict[str, bool]] = lambda p: dict(
00673         zip(VALID_PERMS, (p.write, p.read, True, p.receive))
00674     )
00675     cases = [
00676         validgen(p.group_id) if genmap(p)[permission] else stubgen(p.group_id)
00677         for p in permissionlist
00678     ]
00679
00680     return CASELIST(permission, "\n".join(cases))
00681
00682
00683 def generateSecrets(
00684     outfilename: str,
00685     permissionlist: PermissionList,
00686     secrets: dict[str, dict[str, dict[str, bytes]]],
00687 ):
00688     """
00689     @brief Compiles all derived key routines and lookup tables into a single C source file.
00690
00691     @param outfilename (str): The output file path.
00692     @param permissionlist (PermissionList): The list of permissions.
00693     @param secrets (dict): The master secrets.
00694     """
00695     builder = [
00696         DISCLAIMER,
00697         DEFINITIONS,
00698         RECV_GROUPS([p.group_id for p in permissionlist if p.receive]),
00699         GEN_STUB(),
00700     ]
00701
00702     for p in permissionlist:
00703         if p.write:
00704             builder.append(GEN_WRITEKEY(p.group_id, secrets))
00705         if p.read:
00706             builder.append(GEN_READKEY(p.group_id, secrets))
00707         if p.receive:
00708             builder.append(GEN_RECVKEY(p.group_id, secrets))
00709             builder.append(GEN_SENDKEY(p.group_id, secrets))
00710
00711     builder.append(GEN_INTERROGATEKEY(secrets))
00712     builder.append(GEN_REPLYKEY(secrets))
00713
00714     builder.append(generateCaseList("Write", permissionlist))
00715     builder.append(generateCaseList("Read", permissionlist))
00716     builder.append(generateCaseList("Recv", permissionlist))
00717     builder.append(generateCaseList("Send", permissionlist))
00718
00719     with open(outfilename, "w") as outfile:
00720         _ = outfile.write("\n\n".join(builder))
00721
00722
00723 def arg_parse() -> argparse.Namespace:
00724     """
00725     @brief Defines and parses command line arguments for secret derivation.
00726
00727     @return argparse.Namespace: The parsed arguments.
00728     """
00729     parser = argparse.ArgumentParser()
00730
00731     _ = parser.add_argument(
00732         "secrets", type=argparse.FileType("rb"), help="Path to secrets file"
00733     )
00734     _ = parser.add_argument("hsm_pin", type=str, help="User PIN for the HSM")
00735     _ = parser.add_argument(
00736         "permissions",
00737         type=str,
00738         help="List of colon-separated permissions. E.g., '1234=R--:4321=RWC'",
00739     )
00740
00741     return parser.parse_args()
00742
00743
00744 if __name__ == "__main__":
00745     args = arg_parse()
00746
00747     secrets = pickle.load(args.secrets)
00748     permissions = PermissionList.deserialize(args.permissions)
00749
00750     assert len(args.hsm_pin) == 6, "Invalid pin length"
00751     generateSecrets("secrets/secrets.c", permissions, secrets)

```

```
00752
00753     with open("secrets/secrets.h", "w") as headerfile:
00754         _ = headerfile.write(
00755             DISCLAIMER
00756             + "\n\n"
00757             + HEADER_CONTENT(args.hsm_pin, len([g for g in permissions if g.receive]))
00758         )
00759
00760
00761 # -----
```

5.45 firmware/README.md File Reference

5.46 firmware/utils/README.md File Reference

5.47 README.md File Reference

Index

- - derive_secrets, [12](#)
- __PROGRAM_START
 - startup_mspm0l222x_ticlang.c, [176](#)
- __STACK_END
 - startup_mspm0l222x_ticlang.c, [177](#)
- __attribute__
 - filesystem.c, [125](#)
 - flash.c, [129](#)
 - uECC.c, [207](#)
- __init__
 - derive_secrets.PermissionList, [33](#)
- _pad
 - FS_FileEntry, [22](#)
- _padding
 - FS_MetadataEntryFlash, [25](#)
- apply_z
 - uECC.c, [207](#)
- arg_parse
 - derive_secrets, [10](#)
- args
 - derive_secrets, [12](#)
- as
 - FS_Stage, [29](#)
- asBytes
 - FAT_TableTypeFlash, [18](#)
 - FS_MetadataEntryFlash, [25](#)
- BlinkLED
 - hsm.c, [147](#)
- block_size
 - uECC_HashContext, [37](#)
- CASELIST
 - derive_secrets, [12](#)
- checkpin
 - stubs.c, [180](#)
 - stubs.h, [82](#)
- CONFIG_MSPM0L2228
 - dl_config.h, [41](#)
- CONFIG_MSPM0L222X
 - dl_config.h, [41](#)
- content
 - FS_File, [20](#)
- count
 - UART_RingBuffer, [36](#)
- CPUCLK_FREQ
 - dl_config.h, [41](#)
- curve_b
 - uECC.c, [258](#)
- curve_G
 - uECC.c, [258](#)
- curve_n
 - uECC.c, [258](#)
- curve_p
 - uECC.c, [259](#)
- curve_x_side
 - uECC.c, [208](#)
- data
 - FS_FiledataFlash, [21](#)
 - FS_MetadataEntryFlash, [25](#)
 - UART_RingBuffer, [36](#)
- DECRYPTIONERROR
 - status.h, [80](#)
- Default_Handler
 - startup_mspm0l222x_ticlang.c, [176](#)
- default_RNG
 - uECC.c, [209](#)
- DEFINITIONS
 - derive_secrets, [12](#)
- derive_secrets, [9](#)
 - _, [12](#)
 - arg_parse, [10](#)
 - args, [12](#)
 - CASELIST, [12](#)
 - DEFINITIONS, [12](#)
 - DISCLAIMER, [12](#)
 - GEN_CASE_SMT, [12](#)
 - GEN_CASE_STUB, [13](#)
 - GEN_INTERROGATEKEY, [13](#)
 - GEN_READKEY, [13](#)
 - GEN_RECVKEY, [13](#)
 - GEN_REPLYKEY, [14](#)
 - GEN_SENDKEY, [14](#)
 - GEN_STUB, [14](#)
 - GEN_WRITEKEY, [14](#)
 - generateCaseList, [10](#)
 - generateSecrets, [10](#)
 - HEADER_CONTENT, [15](#)
 - interrogate_as_array, [11](#)
 - key_as_array, [11](#)
 - permissions, [15](#)
 - RECV_GROUPS, [15](#)
 - secrets, [15](#)
 - VALID_PERMS, [15](#)
- derive_secrets.Permission, [30](#)
 - deserialize, [30](#)
 - group_id, [31](#)

- read, [31](#)
- receive, [31](#)
- serialize, [30](#)
- write, [31](#)
- derive_secrets.PermissionList, [32](#)
- __init__, [33](#)
- deserialize, [33](#)
- serialize, [33](#)
- deserialize
 - derive_secrets.Permission, [30](#)
 - derive_secrets.PermissionList, [33](#)
- dirty
 - UART_RingBuffer, [36](#)
- DISCLAIMER
 - derive_secrets, [12](#)
- dl_config.c
 - gUART_0ClockConfig, [120](#)
 - gUART_0Config, [120](#)
 - gUART_1ClockConfig, [121](#)
 - gUART_1Config, [121](#)
 - SYSCFG_DL_GPIO_init, [116](#)
 - SYSCFG_DL_init, [116](#)
 - SYSCFG_DL_initPower, [117](#)
 - SYSCFG_DL_SYSCTL_init, [117](#)
 - SYSCFG_DL_TRNG_init, [118](#)
 - SYSCFG_DL_UART_0_init, [118](#)
 - SYSCFG_DL_UART_1_init, [119](#)
 - TRNG_SecureWarmup, [119](#)
- dl_config.h
 - CONFIG_MSPM0L2228, [41](#)
 - CONFIG_MSPM0L222X, [41](#)
 - CPUCLK_FREQ, [41](#)
 - GPIO_UART_0_IOMUX_RX, [41](#)
 - GPIO_UART_0_IOMUX_RX_FUNC, [41](#)
 - GPIO_UART_0_IOMUX_TX, [42](#)
 - GPIO_UART_0_IOMUX_TX_FUNC, [42](#)
 - GPIO_UART_0_RX_PIN, [42](#)
 - GPIO_UART_0_RX_PORT, [42](#)
 - GPIO_UART_0_TX_PIN, [42](#)
 - GPIO_UART_0_TX_PORT, [42](#)
 - GPIO_UART_1_IOMUX_RX, [42](#)
 - GPIO_UART_1_IOMUX_RX_FUNC, [43](#)
 - GPIO_UART_1_IOMUX_TX, [43](#)
 - GPIO_UART_1_IOMUX_TX_FUNC, [43](#)
 - GPIO_UART_1_RX_PIN, [43](#)
 - GPIO_UART_1_RX_PORT, [43](#)
 - GPIO_UART_1_TX_PIN, [43](#)
 - GPIO_UART_1_TX_PORT, [43](#)
 - LEDS_PORT, [44](#)
 - LEDS_STATUS_LED_IOMUX, [44](#)
 - LEDS_STATUS_LED_PIN, [44](#)
 - POWER_STARTUP_DELAY, [44](#)
 - SYSCFG_DL_GPIO_init, [47](#)
 - SYSCFG_DL_init, [47](#)
 - SYSCFG_DL_initPower, [48](#)
 - SYSCFG_DL_SYSCTL_init, [48](#)
 - SYSCFG_DL_TRNG_init, [49](#)
 - SYSCFG_DL_UART_0_init, [49](#)
 - SYSCFG_DL_UART_1_init, [50](#)
 - SYSCONFIG_WEAK, [44](#)
 - UART_0_BAUD_RATE, [44](#)
 - UART_0_FBRD_32_MHZ_115200_BAUD, [45](#)
 - UART_0_IBRD_32_MHZ_115200_BAUD, [45](#)
 - UART_0_INST, [45](#)
 - UART_0_INST_FREQUENCY, [45](#)
 - UART_0_INST_INT_IRQN, [45](#)
 - UART_0_INST_IRQHandler, [45](#)
 - UART_1_BAUD_RATE, [45](#)
 - UART_1_FBRD_32_MHZ_115200_BAUD, [46](#)
 - UART_1_IBRD_32_MHZ_115200_BAUD, [46](#)
 - UART_1_INST, [46](#)
 - UART_1_INST_FREQUENCY, [46](#)
 - UART_1_INST_INT_IRQN, [46](#)
 - UART_1_INST_IRQHandler, [46](#)
 - DrainPushBytes
 - uart.c, [186](#)
 - ecc_rng
 - hsm.c, [147](#)
 - EccPoint, [17](#)
 - uECC.c, [207](#)
 - x, [17](#)
 - y, [17](#)
 - EccPoint_compute_public_key
 - uECC.c, [209](#)
 - EccPoint_double_jacobian
 - uECC.c, [210](#)
 - EccPoint_isZero
 - uECC.c, [211](#)
 - EccPoint_mult
 - uECC.c, [212](#)
 - ENCRYPTIONERROR
 - status.h, [80](#)
 - entry
 - FAT_TableTypeFlash, [18](#)
 - FS_File, [20](#)
 - EVEN
 - uECC.c, [204](#)
 - fat
 - FS_Stage, [29](#)
 - FAT_Table
 - filesystem.h, [55](#)
 - FAT_TableTypeFlash, [18](#)
 - asBytes, [18](#)
 - entry, [18](#)
 - file
 - FS_Stage, [29](#)
 - fileaddr
 - FS_FileEntry, [22](#)
 - Filedata_Table
 - filesystem.h, [55](#)
 - fileid
 - FS_Metadata, [23](#)
 - filename
 - FS_Metadata, [23](#)
 - FS_Slot, [27](#)

- filesize
 - FS_FileEntry, 22
- filesystem.c
 - __attribute__, 125
 - stage, 126
 - StoreFile, 125
- filesystem.h
 - FAT_Table, 55
 - FiledData_Table, 55
 - FS_FileData, 54
 - LoadFile, 54
 - MAX_FILE_SIZE, 53
 - Metadata_Table, 56
 - NUM_SLOTS, 53
 - RAMFUNC, 54
 - stage, 56
 - StoreFile, 54
 - SystemStatus, 56
- finish_hash
 - uECC_HashContext, 37
- firmware Directory Reference, 3
- firmware/include Directory Reference, 4
- firmware/include/dl_config.h, 39, 51
- firmware/include/filesystem.h, 52, 56
- firmware/include/flash.h, 58, 61
- firmware/include/handler.h, 62, 68
- firmware/include/parser.h, 69, 75
- firmware/include/randombytes.h, 75, 78
- firmware/include/status.h, 79, 80
- firmware/include/stubs.h, 81, 85
- firmware/include/uart.h, 85, 93
- firmware/include/uECC.h, 94, 113
- firmware/README.md, 308
- firmware/src Directory Reference, 5
- firmware/src/dl_config.c, 114, 122
- firmware/src/filesystem.c, 124, 127
- firmware/src/flash.c, 127, 132
- firmware/src/handler.c, 133, 140
- firmware/src/hsm.c, 143, 155
- firmware/src/parser.c, 158, 168
- firmware/src/randombytes.c, 172, 174
- firmware/src/startup_mspm0l222x_ticlang.c, 175, 177
- firmware/src/stubs.c, 179, 183
- firmware/src/uart.c, 184, 198
- firmware/src/uECC.c, 201, 259
- firmware/utls Directory Reference, 6
- firmware/utls/derive_secrets.py, 298, 299
- firmware/utls/README.md, 308
- flash.c
 - __attribute__, 129
 - FLASH_ProcessSector, 129
 - FLASH_ReadSector, 130
 - FLASH_Write, 130
- flash.h
 - FLASH_Erase, 60
 - FLASH_PAGE_SIZE, 59
 - FLASH_Write, 60
 - RAMFUNC, 59
- FLASH_Erase
 - flash.h, 60
- FLASH_PAGE_SIZE
 - flash.h, 59
- FLASH_ProcessSector
 - flash.c, 129
- FLASH_ReadSector
 - flash.c, 130
- FLASH_Write
 - flash.c, 130
 - flash.h, 60
- FLASHERASEERROR
 - status.h, 80
- FLASHWRITEERROR
 - status.h, 80
- FS_FAT, 19
 - numEntries, 19
 - slots, 19
- FS_File, 20
 - content, 20
 - entry, 20
 - metadata, 20
- FS_FileData
 - filesystem.h, 54
- FS_FiledataFlash, 21
 - data, 21
- FS_FileEntry, 21
 - _pad, 22
 - fileaddr, 22
 - filesize, 22
 - uuid, 22
- FS_Metadata, 22
 - fileid, 23
 - filename, 23
 - groupid, 23
 - key, 23
 - nonce, 23
 - tag, 24
- FS_MetadataEntryFlash, 24
 - _padding, 25
 - asBytes, 25
 - data, 25
 - id, 25
 - magic, 25
 - status, 25
- FS_Preamble, 26
 - key, 26
 - mesgid, 26
 - nonce, 26
 - tag, 27
- FS_Slot, 27
 - filename, 27
 - groupid, 27
 - slot, 27
- FS_Stage, 28
 - as, 29
 - fat, 29
 - file, 29

- preamble, 29
- FS_SystemStatusFlash, 29
 - timeout, 30
- g_rng_function
 - uECC.c, 259
- GEN_CASE_SMT
 - derive_secrets, 12
- GEN_CASE_STUB
 - derive_secrets, 13
- GEN_INTERROGATEKEY
 - derive_secrets, 13
- GEN_READKEY
 - derive_secrets, 13
- GEN_RECVKEY
 - derive_secrets, 13
- GEN_REPLYKEY
 - derive_secrets, 14
- GEN_SENDKEY
 - derive_secrets, 14
- GEN_STUB
 - derive_secrets, 14
- GEN_WRITEKEY
 - derive_secrets, 14
- generateCaseList
 - derive_secrets, 10
- generateSecrets
 - derive_secrets, 10
- getrandom32
 - randombytes.c, 173
 - randombytes.h, 77
- GPIO_UART_0_IOMUX_RX
 - dl_config.h, 41
- GPIO_UART_0_IOMUX_RX_FUNC
 - dl_config.h, 41
- GPIO_UART_0_IOMUX_TX
 - dl_config.h, 42
- GPIO_UART_0_IOMUX_TX_FUNC
 - dl_config.h, 42
- GPIO_UART_0_RX_PIN
 - dl_config.h, 42
- GPIO_UART_0_RX_PORT
 - dl_config.h, 42
- GPIO_UART_0_TX_PIN
 - dl_config.h, 42
- GPIO_UART_0_TX_PORT
 - dl_config.h, 42
- GPIO_UART_1_IOMUX_RX
 - dl_config.h, 42
- GPIO_UART_1_IOMUX_RX_FUNC
 - dl_config.h, 43
- GPIO_UART_1_IOMUX_TX
 - dl_config.h, 43
- GPIO_UART_1_IOMUX_TX_FUNC
 - dl_config.h, 43
- GPIO_UART_1_RX_PIN
 - dl_config.h, 43
- GPIO_UART_1_RX_PORT
 - dl_config.h, 43
- GPIO_UART_1_TX_PIN
 - dl_config.h, 43
- GPIO_UART_1_TX_PORT
 - dl_config.h, 43
- group_id
 - derive_secrets.Permission, 31
- groupid
 - FS_Metadata, 23
 - FS_Slot, 27
- gUART_0ClockConfig
 - dl_config.c, 120
- gUART_0Config
 - dl_config.c, 120
- gUART_1ClockConfig
 - dl_config.c, 121
- gUART_1Config
 - dl_config.c, 121
- handler.c
 - InterrogateHandler, 134
 - isReceivable, 134
 - ListHandler, 135
 - ReadHandler, 135
 - ReceiveHandler, 136
 - ReplyHandler, 137
 - SendHandler, 138
 - WriteHandler, 139
- handler.h
 - InterrogateHandler, 63
 - ListHandler, 63
 - ReadHandler, 64
 - ReceiveHandler, 65
 - ReplyHandler, 66
 - SendHandler, 67
 - WriteHandler, 67
- HandleRequest
 - hsm.c, 148
- head
 - UART_RingBuffer, 36
- HEADER_CONTENT
 - derive_secrets, 15
- HMAC_finish
 - uECC.c, 213
- HMAC_init
 - uECC.c, 213
- HMAC_update
 - uECC.c, 214
- host
 - uart.c, 198
 - uart.h, 93
- host_magic_byte
 - hsm.c, 155
- hsm.c
 - BlinkLED, 147
 - ecc_rng, 147
 - HandleRequest, 148
 - host_magic_byte, 155
 - HSM_PIN_SIZE, 144
 - init, 149

- INTERROGATE_METADATA_SIZE, 144
- LIST_METADATA_SIZE, 145
- LISTEN_METADATA_SIZE, 145
- main, 150
- OP_ERROR, 145
- OP_INTERROGATE, 145
- OP_LIST, 145
- OP_LISTEN, 145
- OP_READ, 146
- OP_RECEIVE, 146
- OP_WRITE, 146
- ParseOpcode, 151
- READ_METADATA_SIZE, 146
- RECEIVE_METADATA_SIZE, 146
- SendError, 152
- SendHeader, 152
- SendResponse, 153
- ValidateBodySize, 154
- WRITE_METADATA_SIZE, 146
- HSM_PIN_SIZE
 - hsm.c, 144
- id
 - FS_MetadadataEntryFlash, 25
- init
 - hsm.c, 149
- init_hash
 - uECC_HashContext, 38
- interrogate_as_array
 - derive_secrets, 11
- INTERROGATE_METADATA_SIZE
 - hsm.c, 144
- InterrogateHandler
 - handler.c, 134
 - handler.h, 63
- InterrogateParser
 - parser.c, 160
 - parser.h, 70
- INVALIDBODYSIZE
 - status.h, 80
- INVALIDSLOT
 - status.h, 80
- IRQ_Handler
 - uart.c, 186
- isReceivable
 - handler.c, 134
- key
 - FS_Metadata, 23
 - FS_Preamble, 26
- key_as_array
 - derive_secrets, 11
- KEYGENERERROR
 - status.h, 80
- LEDS_PORT
 - dl_config.h, 44
- LEDS_STATUS_LED_IOMUX
 - dl_config.h, 44
- LEDS_STATUS_LED_PIN
 - dl_config.h, 44
- LIST_METADATA_SIZE
 - hsm.c, 145
- LISTEN_METADATA_SIZE
 - hsm.c, 145
- ListenParser
 - parser.c, 161
 - parser.h, 70
- ListHandler
 - handler.c, 135
 - handler.h, 63
- ListParser
 - parser.c, 161
 - parser.h, 71
- LoadFile
 - filesystem.h, 54
- magic
 - FS_MetadadataEntryFlash, 25
- main
 - hsm.c, 150
- MAX_DRAIN_SIZE
 - uart.h, 87
- MAX_FILE_SIZE
 - filesystem.h, 53
- MAX_TRIES
 - uECC.c, 204
- memclear
 - stubs.c, 181
 - stubs.h, 82
- mesgid
 - FS_Preamble, 26
- metadata
 - FS_File, 20
- Metadata_Table
 - filesystem.h, 56
- mod_sqrt
 - uECC.c, 214
- mod_sqrt_secp224r1_rm
 - uECC.c, 215
- mod_sqrt_secp224r1_rp
 - uECC.c, 216
- mod_sqrt_secp224r1_rs
 - uECC.c, 217
- mod_sqrt_secp224r1_rss
 - uECC.c, 217
- muladd
 - uECC.c, 218
- muladd_exists
 - uECC.c, 205
- nonce
 - FS_Metadata, 23
 - FS_Preamble, 26
- NUM_SLOTS
 - filesystem.h, 53
- numEntries
 - FS_FAT, 19

- omega_mult
 - uECC.c, [219](#)
- OP_ERROR
 - hsm.c, [145](#)
 - parser.c, [160](#)
- OP_INTERROGATE
 - hsm.c, [145](#)
 - parser.c, [160](#)
- OP_LIST
 - hsm.c, [145](#)
- OP_LISTEN
 - hsm.c, [145](#)
- OP_READ
 - hsm.c, [146](#)
- OP_RECEIVE
 - hsm.c, [146](#)
 - parser.c, [160](#)
- OP_WRITE
 - hsm.c, [146](#)
- OPINTERROGATE
 - status.h, [80](#)
- OPLIST
 - status.h, [80](#)
- OPLISTEN
 - status.h, [80](#)
- OPREAD
 - status.h, [80](#)
- OPRECEIVE
 - status.h, [80](#)
- OPREPLY
 - status.h, [80](#)
- OPSEND
 - status.h, [80](#)
- OPWRITE
 - status.h, [80](#)
- ParseOpcode
 - hsm.c, [151](#)
- parser.c
 - InterrogateParser, [160](#)
 - ListenParser, [161](#)
 - ListParser, [161](#)
 - OP_ERROR, [160](#)
 - OP_INTERROGATE, [160](#)
 - OP_RECEIVE, [160](#)
 - peer_magic_byte, [167](#)
 - ReadParser, [162](#)
 - ReceiveParser, [163](#)
 - ReplyParser, [164](#)
 - ResolveError, [165](#)
 - SendParser, [165](#)
 - WriteParser, [166](#)
- parser.h
 - InterrogateParser, [70](#)
 - ListenParser, [70](#)
 - ListParser, [71](#)
 - ReadParser, [72](#)
 - ReceiveParser, [73](#)
 - WriteParser, [74](#)
- peer
 - uart.c, [198](#)
 - uart.h, [93](#)
- peer_magic_byte
 - parser.c, [167](#)
- PEERERROR
 - status.h, [80](#)
- PERMISSIONERROR
 - status.h, [80](#)
- permissions
 - derive_secrets, [15](#)
- POWER_STARTUP_DELAY
 - dl_config.h, [44](#)
- preamble
 - FS_Stage, [29](#)
- progress
 - UART_RingBuffer, [36](#)
- RAMFUNC
 - filesystem.h, [54](#)
 - flash.h, [59](#)
 - stubs.c, [180](#)
- randombytes
 - randombytes.c, [173](#)
 - randombytes.h, [77](#)
- randombytes.c
 - getrandom32, [173](#)
 - randombytes, [173](#)
- randombytes.h
 - getrandom32, [77](#)
 - randombytes, [77](#)
- read
 - derive_secrets.Permission, [31](#)
- READ_METADATA_SIZE
 - hsm.c, [146](#)
- ReadHandler
 - handler.c, [135](#)
 - handler.h, [64](#)
- README.md, [308](#)
- ReadParser
 - parser.c, [162](#)
 - parser.h, [72](#)
- receive
 - derive_secrets.Permission, [31](#)
- RECEIVE_METADATA_SIZE
 - hsm.c, [146](#)
- ReceiveHandler
 - handler.c, [136](#)
 - handler.h, [65](#)
- ReceiveParser
 - parser.c, [163](#)
 - parser.h, [73](#)
- RECV_GROUPS
 - derive_secrets, [15](#)
- ReplyHandler
 - handler.c, [137](#)
 - handler.h, [66](#)
- ReplyParser
 - parser.c, [164](#)

- Reset_Handler
 - startup_mspm0l222x_ticlang.c, 176
- ResolveError
 - parser.c, 165
- RESTRICT
 - uECC.c, 205
- result_size
 - uECC_HashContext, 38
- rx
 - UART_Channel, 35
- secrets
 - derive_secrets, 15
- securecmp
 - stubs.c, 182
 - stubs.h, 83
- SendError
 - hsm.c, 152
- SendHandler
 - handler.c, 138
 - handler.h, 67
- SendHeader
 - hsm.c, 152
- SendHostACK
 - uart.c, 186
- SendParser
 - parser.c, 165
- SendPeerACK
 - uart.c, 187
- SendResponse
 - hsm.c, 153
- serialize
 - derive_secrets.Permission, 30
 - derive_secrets.PermissionList, 33
- SHIFT (Secure Hardware Interface for File Transfer), 1
- slot
 - FS_Slot, 27
- SLOTEEMPTY
 - status.h, 80
- slots
 - FS_FAT, 19
- smax
 - uECC.c, 219
- stage
 - filesystem.c, 126
 - filesystem.h, 56
- startup_mspm0l222x_ticlang.c
 - __PROGRAM_START, 176
 - __STACK_END, 177
 - Default_Handler, 176
 - Reset_Handler, 176
- status
 - FS_MetadataEntryFlash, 25
- status.h
 - DECRYPTIONERROR, 80
 - ENCRYPTIONERROR, 80
 - FLASHERASEERROR, 80
 - FLASHWRITEERROR, 80
 - INVALIDBODYSIZE, 80
 - INVALIDSLOT, 80
 - KEYGENERERROR, 80
 - OPINTERROGATE, 80
 - OPLIST, 80
 - OPLISTEN, 80
 - OPREAD, 80
 - OPRECEIVE, 80
 - OPREPLY, 80
 - OPSEND, 80
 - OPWRITE, 80
 - PEERERROR, 80
 - PERMISSIONERROR, 80
 - SLOTEEMPTY, 80
 - StatusCode, 79
 - UNKNOWNOP, 80
- StatusCode
 - status.h, 79
- StoreFile
 - filesystem.c, 125
 - filesystem.h, 54
- stubs.c
 - checkpin, 180
 - memclear, 181
 - RAMFUNC, 180
 - securecmp, 182
- stubs.h
 - checkpin, 82
 - memclear, 82
 - securecmp, 83
- SUPPORTS_INT128
 - uECC.c, 205
- SYSCFG_DL_GPIO_init
 - dl_config.c, 116
 - dl_config.h, 47
- SYSCFG_DL_init
 - dl_config.c, 116
 - dl_config.h, 47
- SYSCFG_DL_initPower
 - dl_config.c, 117
 - dl_config.h, 48
- SYSCFG_DL_SYCTL_init
 - dl_config.c, 117
 - dl_config.h, 48
- SYSCFG_DL_TRNG_init
 - dl_config.c, 118
 - dl_config.h, 49
- SYSCFG_DL_UART_0_init
 - dl_config.c, 118
 - dl_config.h, 49
- SYSCFG_DL_UART_1_init
 - dl_config.c, 119
 - dl_config.h, 50
- SYSCONFIG_WEAK
 - dl_config.h, 44
- SystemStatus
 - filesystem.h, 56
- tag
 - FS_Metadata, 24

- FS_Preamble, [27](#)
- tail
 - UART_RingBuffer, [36](#)
- timeout
 - FS_SystemStatusFlash, [30](#)
- tmp
 - uECC_HashContext, [38](#)
- TRNG_SecureWarmup
 - dl_config.c, [119](#)
- tx
 - UART_Channel, [35](#)
- uart.c
 - DrainPushBytes, [186](#)
 - host, [198](#)
 - IRQ_Handler, [186](#)
 - peer, [198](#)
 - SendHostACK, [186](#)
 - SendPeerACK, [187](#)
 - UART0_IRQHandler, [187](#)
 - UART1_IRQHandler, [188](#)
 - UART_ClearBuffer, [188](#)
 - UART_ClearChannel, [189](#)
 - UART_init, [190](#)
 - UART_PopByte, [190](#)
 - UART_PushByte, [191](#)
 - UART_RecvBytes, [192](#)
 - UART_RecvUntil, [193](#)
 - UART_SendBytes, [194](#)
 - UART_TransmitAll, [195](#)
 - WaitForHostACK, [196](#)
 - WaitForPeerACK, [197](#)
- uart.h
 - host, [93](#)
 - MAX_DRAIN_SIZE, [87](#)
 - peer, [93](#)
 - UART0_IRQHandler, [88](#)
 - UART1_IRQHandler, [88](#)
 - UART_BUFFER_SIZE, [87](#)
 - UART_HOST, [87](#)
 - UART_init, [89](#)
 - UART_PEER, [87](#)
 - UART_RecvBytes, [89](#)
 - UART_RecvUntil, [91](#)
 - UART_SendBytes, [92](#)
- UART0_IRQHandler
 - uart.c, [187](#)
 - uart.h, [88](#)
- UART1_IRQHandler
 - uart.c, [188](#)
 - uart.h, [88](#)
- UART_0_BAUD_RATE
 - dl_config.h, [44](#)
- UART_0_FBRD_32_MHZ_115200_BAUD
 - dl_config.h, [45](#)
- UART_0_IBRD_32_MHZ_115200_BAUD
 - dl_config.h, [45](#)
- UART_0_INST
 - dl_config.h, [45](#)
- UART_0_INST_FREQUENCY
 - dl_config.h, [45](#)
- UART_0_INST_INT_IRQN
 - dl_config.h, [45](#)
- UART_0_INST_IRQHandler
 - dl_config.h, [45](#)
- UART_1_BAUD_RATE
 - dl_config.h, [45](#)
- UART_1_FBRD_32_MHZ_115200_BAUD
 - dl_config.h, [46](#)
- UART_1_IBRD_32_MHZ_115200_BAUD
 - dl_config.h, [46](#)
- UART_1_INST
 - dl_config.h, [46](#)
- UART_1_INST_FREQUENCY
 - dl_config.h, [46](#)
- UART_1_INST_INT_IRQN
 - dl_config.h, [46](#)
- UART_1_INST_IRQHandler
 - dl_config.h, [46](#)
- UART_BUFFER_SIZE
 - uart.h, [87](#)
- UART_Channel, [34](#)
 - rx, [35](#)
 - tx, [35](#)
- UART_ClearBuffer
 - uart.c, [188](#)
- UART_ClearChannel
 - uart.c, [189](#)
- UART_HOST
 - uart.h, [87](#)
- UART_init
 - uart.c, [190](#)
 - uart.h, [89](#)
- UART_PEER
 - uart.h, [87](#)
- UART_PopByte
 - uart.c, [190](#)
- UART_PushByte
 - uart.c, [191](#)
- UART_RecvBytes
 - uart.c, [192](#)
 - uart.h, [89](#)
- UART_RecvUntil
 - uart.c, [193](#)
 - uart.h, [91](#)
- UART_RingBuffer, [35](#)
 - count, [36](#)
 - data, [36](#)
 - dirty, [36](#)
 - head, [36](#)
 - progress, [36](#)
 - tail, [36](#)
- UART_SendBytes
 - uart.c, [194](#)
 - uart.h, [92](#)
- UART_TransmitAll
 - uart.c, [195](#)

uECC.c

- [__attribute__, 207](#)
- [apply_z, 207](#)
- [curve_b, 258](#)
- [curve_G, 258](#)
- [curve_n, 258](#)
- [curve_p, 259](#)
- [curve_x_side, 208](#)
- [default_RNG, 209](#)
- [EccPoint, 207](#)
- [EccPoint_compute_public_key, 209](#)
- [EccPoint_double_jacobian, 210](#)
- [EccPoint_isZero, 211](#)
- [EccPoint_mult, 212](#)
- [EVEN, 204](#)
- [g_rng_function, 259](#)
- [HMAC_finish, 213](#)
- [HMAC_init, 213](#)
- [HMAC_update, 214](#)
- [MAX_TRIES, 204](#)
- [mod_sqrt, 214](#)
- [mod_sqrt_secp224r1_rm, 215](#)
- [mod_sqrt_secp224r1_rp, 216](#)
- [mod_sqrt_secp224r1_rs, 217](#)
- [mod_sqrt_secp224r1_rss, 217](#)
- [muladd, 218](#)
- [muladd_exists, 205](#)
- [omega_mult, 219](#)
- [RESTRICT, 205](#)
- [smax, 219](#)
- [SUPPORTS_INT128, 205](#)
- [uECC_bytes, 220](#)
- [uECC_compress, 220](#)
- [uECC_compute_public_key, 220](#)
- [uECC_curve, 221](#)
- [uECC_decompress, 222](#)
- [uECC_make_key, 222](#)
- [uECC_mod_mult, 223](#)
- [uECC_N_WORDS, 205](#)
- [uECC_PLATFORM, 205](#)
- [uECC_shared_secret, 224](#)
- [uECC_sign, 225](#)
- [uECC_sign_deterministic, 226](#)
- [uECC_sign_with_k, 227](#)
- [uECC_valid_public_key, 229](#)
- [uECC_verify, 230](#)
- [uECC_WORD_SIZE, 206](#)
- [uECC_WORDS, 206](#)
- [update_V, 232](#)
- [vli2_rshift1_n, 232](#)
- [vli2_sub_n, 233](#)
- [vli_add, 233](#)
- [vli_add_n, 234](#)
- [vli_blindScalar, 235](#)
- [vli_bytesToNative, 235](#)
- [vli_clear, 236](#)
- [vli_clear_n, 236](#)
- [vli_cmp, 237](#)

- [vli_cmp_n, 238](#)
- [vli_equal, 238](#)
- [vli_isZero, 239](#)
- [vli_isZero_n, 239](#)
- [vli_mmod_fast, 240](#)
- [vli_modAdd, 241](#)
- [vli_modAdd_n, 242](#)
- [vli_modInv, 243](#)
- [vli_modInv_n, 244](#)
- [vli_modMult_fast, 245](#)
- [vli_modMult_n, 246](#)
- [vli_modSquare_fast, 206](#)
- [vli_modSub, 247](#)
- [vli_modSub_fast, 206](#)
- [vli_mult, 248](#)
- [vli_mult_n, 249](#)
- [vli_nativeToBytes, 249](#)
- [vli_numBits, 250](#)
- [vli_numDigits, 251](#)
- [vli_rshift1, 251](#)
- [vli_rshift1_n, 251](#)
- [vli_set, 252](#)
- [vli_set_n, 253](#)
- [vli_square, 206](#)
- [vli_sub, 253](#)
- [vli_sub_n, 254](#)
- [vli_testBit, 255](#)
- [XYcZ_add, 255](#)
- [XYcZ_addC, 256](#)
- [XYcZ_initial_double, 257](#)

uECC.h

- [uECC_arch_other, 97](#)
- [uECC_arm, 97](#)
- [uECC_arm_thumb, 97](#)
- [uECC_arm_thumb2, 97](#)
- [uECC_ASM, 98](#)
- [uECC_asm_fast, 98](#)
- [uECC_asm_none, 98](#)
- [uECC_asm_small, 98](#)
- [uECC_avr, 98](#)
- [uECC_BYTES, 98](#)
- [uECC_bytes, 102](#)
- [uECC_compress, 102](#)
- [uECC_compute_public_key, 102](#)
- [uECC_CONCAT, 99](#)
- [uECC_CONCAT1, 99](#)
- [uECC_CURVE, 99](#)
- [uECC_curve, 103](#)
- [uECC_decompress, 104](#)
- [uECC_HashContext, 102](#)
- [uECC_make_key, 104](#)
- [uECC_RNG_Function, 102](#)
- [uECC_secp160r1, 99](#)
- [uECC_secp192r1, 99](#)
- [uECC_secp224r1, 100](#)
- [uECC_secp256k1, 100](#)
- [uECC_secp256r1, 100](#)
- [uECC_set_rng, 105](#)

- uECC_shared_secret, 106
- uECC_sign, 107
- uECC_sign_deterministic, 108
- uECC_size_1, 100
- uECC_size_2, 100
- uECC_size_3, 100
- uECC_size_4, 101
- uECC_size_5, 101
- uECC_SQUARE_FUNC, 101
- uECC_valid_public_key, 109
- uECC_verify, 110
- uECC_x86, 101
- uECC_x86_64, 101
- uECC_arch_other
 - uECC.h, 97
- uECC_arm
 - uECC.h, 97
- uECC_arm_thumb
 - uECC.h, 97
- uECC_arm_thumb2
 - uECC.h, 97
- uECC_ASM
 - uECC.h, 98
- uECC_asm_fast
 - uECC.h, 98
- uECC_asm_none
 - uECC.h, 98
- uECC_asm_small
 - uECC.h, 98
- uECC_avr
 - uECC.h, 98
- uECC_BYTES
 - uECC.h, 98
- uECC_bytes
 - uECC.c, 220
 - uECC.h, 102
- uECC_compress
 - uECC.c, 220
 - uECC.h, 102
- uECC_compute_public_key
 - uECC.c, 220
 - uECC.h, 102
- uECC_CONCAT
 - uECC.h, 99
- uECC_CONCAT1
 - uECC.h, 99
- uECC_CURVE
 - uECC.h, 99
- uECC_curve
 - uECC.c, 221
 - uECC.h, 103
- uECC_decompress
 - uECC.c, 222
 - uECC.h, 104
- uECC_HashContext, 37
 - block_size, 37
 - finish_hash, 37
 - init_hash, 38
 - result_size, 38
 - tmp, 38
- uECC.h, 102
- update_hash, 38
- uECC_make_key
 - uECC.c, 222
 - uECC.h, 104
- uECC_mod_mult
 - uECC.c, 223
- uECC_N_WORDS
 - uECC.c, 205
- uECC_PLATFORM
 - uECC.c, 205
- uECC_RNG_Function
 - uECC.h, 102
- uECC_secp160r1
 - uECC.h, 99
- uECC_secp192r1
 - uECC.h, 99
- uECC_secp224r1
 - uECC.h, 100
- uECC_secp256k1
 - uECC.h, 100
- uECC_secp256r1
 - uECC.h, 100
- uECC_set_rng
 - uECC.h, 105
- uECC_shared_secret
 - uECC.c, 224
 - uECC.h, 106
- uECC_sign
 - uECC.c, 225
 - uECC.h, 107
- uECC_sign_deterministic
 - uECC.c, 226
 - uECC.h, 108
- uECC_sign_with_k
 - uECC.c, 227
- uECC_size_1
 - uECC.h, 100
- uECC_size_2
 - uECC.h, 100
- uECC_size_3
 - uECC.h, 100
- uECC_size_4
 - uECC.h, 101
- uECC_size_5
 - uECC.h, 101
- uECC_SQUARE_FUNC
 - uECC.h, 101
- uECC_valid_public_key
 - uECC.c, 229
 - uECC.h, 109
- uECC_verify
 - uECC.c, 230
 - uECC.h, 110
- uECC_WORD_SIZE
 - uECC.c, 206

- uECC_WORDS
 - uECC.c, [206](#)
- uECC_x86
 - uECC.h, [101](#)
- uECC_x86_64
 - uECC.h, [101](#)
- UNKNOWNOP
 - status.h, [80](#)
- update_hash
 - uECC_HashContext, [38](#)
- update_V
 - uECC.c, [232](#)
- uuid
 - FS_FileEntry, [22](#)
- VALID_PERMS
 - derive_secrets, [15](#)
- ValidateBodySize
 - hsm.c, [154](#)
- vli2_rshift1_n
 - uECC.c, [232](#)
- vli2_sub_n
 - uECC.c, [233](#)
- vli_add
 - uECC.c, [233](#)
- vli_add_n
 - uECC.c, [234](#)
- vli_blindScalar
 - uECC.c, [235](#)
- vli_bytesToNative
 - uECC.c, [235](#)
- vli_clear
 - uECC.c, [236](#)
- vli_clear_n
 - uECC.c, [236](#)
- vli_cmp
 - uECC.c, [237](#)
- vli_cmp_n
 - uECC.c, [238](#)
- vli_equal
 - uECC.c, [238](#)
- vli_isZero
 - uECC.c, [239](#)
- vli_isZero_n
 - uECC.c, [239](#)
- vli_mmod_fast
 - uECC.c, [240](#)
- vli_modAdd
 - uECC.c, [241](#)
- vli_modAdd_n
 - uECC.c, [242](#)
- vli_modInv
 - uECC.c, [243](#)
- vli_modInv_n
 - uECC.c, [244](#)
- vli_modMult_fast
 - uECC.c, [245](#)
- vli_modMult_n
 - uECC.c, [246](#)
- vli_modSquare_fast
 - uECC.c, [206](#)
- vli_modSub
 - uECC.c, [247](#)
- vli_modSub_fast
 - uECC.c, [206](#)
- vli_mult
 - uECC.c, [248](#)
- vli_mult_n
 - uECC.c, [249](#)
- vli_nativeToBytes
 - uECC.c, [249](#)
- vli_numBits
 - uECC.c, [250](#)
- vli_numDigits
 - uECC.c, [251](#)
- vli_rshift1
 - uECC.c, [251](#)
- vli_rshift1_n
 - uECC.c, [251](#)
- vli_set
 - uECC.c, [252](#)
- vli_set_n
 - uECC.c, [253](#)
- vli_square
 - uECC.c, [206](#)
- vli_sub
 - uECC.c, [253](#)
- vli_sub_n
 - uECC.c, [254](#)
- vli_testBit
 - uECC.c, [255](#)
- WaitForHostACK
 - uart.c, [196](#)
- WaitForPeerACK
 - uart.c, [197](#)
- write
 - derive_secrets.Permission, [31](#)
- WRITE_METADATA_SIZE
 - hsm.c, [146](#)
- WriteHandler
 - handler.c, [139](#)
 - handler.h, [67](#)
- WriteParser
 - parser.c, [166](#)
 - parser.h, [74](#)
- x
 - EccPoint, [17](#)
- XYcZ_add
 - uECC.c, [255](#)
- XYcZ_addC
 - uECC.c, [256](#)
- XYcZ_initial_double
 - uECC.c, [257](#)
- y
 - EccPoint, [17](#)